

The background is a dark grey or black field filled with various geometric patterns. On the left, there are several sets of parallel, slightly curved lines that create a sense of depth and movement. On the right, a large, complex shape is formed by many thin, parallel lines that fan out from a point, resembling a stylized 'V' or a series of overlapping planes. Scattered throughout the design are thin, light blue and pink lines that form rectangles, L-shapes, and other geometric motifs. Some of these lines end in small dots of the same color. The overall aesthetic is modern, minimalist, and tech-oriented.

BAZE PODATAKA

Tin Kramberger
Sanja Duk
Renata Kovačević

PRIRUČNICI TEHNIČKOG VELEUČILIŠTA U ZAGREBU
MANUALIA POLYTECHNICI STUDIORUM ZAGRABIENSIS

Tin Kramberger, struč. spec. ing. techn. inf.

Sanja Duk, dipl. ing.

Renata Kovačević, struč. spec. ing. techn. inf.

Baze podataka

Udžbenik za kolegije 'Baze podataka' i 'Napredne baze podataka' koji se održavaju prema nastavnom planu na preddiplomskim studijima informatike i računarstva Tehničkog veleučilišta u Zagrebu

Zagreb, 2018.



**TEHNIČKO VELEUČILIŠTE U
ZAGREBU ZAGREB UNIVERSITY
OF APPLIED SCIENCES**

izdavač Tehničko veleučilište u Zagrebu
 Vrbik 8, Zagreb

za izdavača Slavica Ćosović Bajić

autori Tin Kramberger, struč. spec. ing. techn. inf.
 Sanja Duk, dipl. ing.
 Renata Kovačević, struč. spec. ing. techn. inf.

recenzenti Izv. prof. dr. sc. Boris Vrdoljak, Sveučilište u
 Zagrebu, Fakultet elektrotehnike i računarstva
 Tomislav Gligora, spec. ing. techn. inf., pred.,
 Veleučilište Velika Gorica

vrsta djela udžbenik

 Objavljivanje je odobrilo Stručno vijeće
 Tehničkog veleučilišta u Zagrebu
 odlukom broj: 2630-5 od 27. veljače 2018.

ISBN 978-953-7048-70-9

CIP zapis Dostupan u računalnom katalogu
 Nacionalne i sveučilišne knjižnice u Zagrebu
 pod brojem _____

Sadržaj

1. UVOD U BAZE PODATAKA	1
1.1. ŠTO JE TO PODATAK, A ŠTO INFORMACIJA?	1
1.2. ŠTO JE BAZA PODATAKA?	1
1.3. KRATKA POVIJEST BAZA PODATAKA	2
1.4. SUSTAV ZA UPRAVLJANJE BAZOM PODATAKA	3
1.5. MODEL BAZE PODATAKA	6
2. IZRADA BAZE PODATAKA.....	9
2.1. TIJEK IZRADE BAZE PODATAKA.....	9
2.2. CILJEVI IZRADE BAZE PODATAKA.....	11
2.3. OBLIKOVANJE MODELA ENTITETI-VEZE (ER MODELA)	12
2.4. ALATI ZA IZRADU MODELA BAZE PODATAKA	18
3. OBLIKOVANJE SCHEME RELACIJSKE BAZE PODATAKA.....	20
3.1. REDUNDANCIA I ANOMALIJE PODATAKA	20
3.2. FUNKCIJSKA ZAVISNOST.....	21
3.3. TRANSFORMACIJSKA PRAVILA FUNKCIJSKIH ZAVISNOSTI (ARMSTRONGOVI AKSIOMI)	24
3.4. KONAČNI SKUP FUNKCIJSKIH ZAVISNOSTI.....	27
3.5. NORMALIZACIJA BAZE PODATAKA	28
4. TIPOVI PODATAKA.....	38
4.1. NUMERIČKI TIPOVI	38
4.2. TIPOVI PODATAKA ZA POHRANU DATUMA I VREMENA.....	41
4.3. TEKSTUALNI TIPOVI PODATAKA	43
5. OSNOVE SQL-A	47
5.1. SQL STANDARDI	47
5.2. PODJELA NAREDBI U SQL JEZIKU	48
5.3. DDL NAREDBE	49
5.4. DML NAREDBE	54
6. UVJETI I WHERE KLAUZULA.....	59
6.1. WHERE U SELECT NAREDBI.....	59
6.2. WHERE U UPDATE I DELETE NAREDBI.....	67
7. OPERATORI I FUNKCIJE	68
7.1. OPERATORI	68
7.2. FUNKCIJE.....	71
8. NULL VRIJEDNOSTI	85
8.1. UNOS NULL VRIJEDNOSTI	85
8.2. PROMJENA NULL VRIJEDNOSTI	85
8.3. DOHVAT NULL VRIJEDNOSTI.....	85
9. SORTIRANJE PODATAKA	87
9.1. SORTIRANJE NULL VRIJEDNOSTI	89
10. DISTINCT IZJAVA	91
11. KARTEZIJEV PRODUKT I PRIRODNO SPAJANJE.....	93
11.1. KARTEZIJEV PRODUKT	93
11.2. PRIRODNO SPAJANJE	95

12.	UNIJA	104
12.2.	UNION	105
12.3.	UNION ALL	106
13.	JOIN KLAUZULA.....	108
13.1.	INNER JOIN.....	108
13.2.	NATURAL JOIN	113
13.3.	CROSS JOIN	114
13.4.	LEFT OUTER JOIN.....	115
13.5.	RIGHT OUTER JOIN	119
13.6.	FULL OUTER JOIN	123
14.	ALIASI.....	127
14.1.	ALIASI NA NAZIV ATRIBUTA	127
14.2.	ALIASI NA NAZIV TABLICA.....	128
15.	LIMIT.....	132
16.	GROUP BY I HAVING KLAUZULE	134
16.1.	GROUP BY	134
16.2.	HAVING	136
17.	PODUPITI	139
17.1.	USPOREDBA IZRAZA IZ VANJSKOG UPITA S PODUPITOM	139
17.2.	ISPITIVANJE JE LI NEKI IZRAZ IZ VANJSKOG UPITA SADRŽAN U REZULTATU PODUPITA	142
17.3.	ISPITIVANJE POJAVLJUJE LI SE KAO REZULTAT PODUPITA BAREM JEDNA N-TORKA	142
17.4.	KORELIRANI UPITI	143
17.5.	PODUPITI U SELECT KLAUZULI	144
17.6.	UGNJEŽĐIVANJE PODUPITA U PODUPITU	145
17.7.	UPDATE I DELETE NAREDBE.....	146
17.8.	SMJERNICE PRILIKOM KORIŠTENJA PODUPITA	147
18.	PRIMARNI I STRANI KLJUČEVI	149
18.1.	PRIMARNI KLJUČ	149
18.2.	STRANI KLJUČ.....	150
19.	INDEKSI	153
19.1.	RAD S INDEKSIMA	153
19.2.	TIPOVI INDEKSA.....	156
19.3.	OPĆENITO O INDEKSIMA.....	162
19.4.	INDEKSI ZA PRETRAGU TEKSTA	164
20.	TRANSAKCIJE	171
20.1.	DEFINICIJA I OSNOVNO NAČELO TRANSAKCIJA	171
20.2.	GRANICE TRANSAKCIJE	172
20.3.	PREDUVJETI ZA KORIŠTENJE TRANSAKCIJA.....	173
20.4.	PRIMJERI.....	174
20.5.	STANJA TRANSAKCIJA.....	178
20.6.	ACID	179
20.7.	OGRANIČENJA TRANSAKCIJA.....	181
20.8.	TOČKE POHRANJIVANJA.....	182
21.	POHRANJENI ZADATCI	184
21.1.	PREDNOSTI POHRANJENIH ZADATAKA.....	184
21.2.	NEDOSTATCI POHRANJENIH ZADATAKA.....	185
21.3.	PODJELA POHRANJENIH ZADATAKA	186
22.	VARIJABLE.....	187

22.1.	VRSTE VARIJABLI	187
22.2.	DODJELJIVANJE VRIJEDNOSTI VARIJABLAMA	188
23.	POHRANJENE RUTINE	190
23.1.	KORACI IZRADA RUTINE	190
23.2.	ODABIR GRANIČNIKA	190
24.	POHRANJENE PROCEDURE	192
24.1.	IZRADA PROCEDURE	192
24.2.	POZIVANJE PROCEDURE.....	193
24.3.	IZMJENA PROCEDURE.....	193
24.4.	BRISANJE PROCEDURE.....	194
24.5.	PRIMJER	194
24.6.	DEFINIRANJE PARAMETARA.....	195
24.7.	PRIMJERI.....	196
25.	POHRANJENE FUNKCIJE	200
25.1.	IZRADA FUNKCIJE	200
25.2.	POZIVANJE FUNKCIJE	201
25.3.	IZMJENA FUNKCIJE	201
25.4.	BRISANJE FUNKCIJE	202
25.5.	DEFINIRANJE PARAMETARA.....	202
25.6.	PRIMJERI.....	202
26.	OKIDAČI	205
26.1.	VRSTE OKIDAČA.....	205
26.2.	IZRADA OKIDAČA.....	206
26.3.	BRISANJE OKIDAČA.....	207
26.4.	IZMJENA OKIDAČA.....	207
26.5.	REFERENCIRANJE NA STARE / NOVE VRIJEDNOSTI.....	207
26.6.	UPOTREBA BEFORE I AFTER OKIDAČA	210
26.7.	OGRANIČENJA OKIDAČA	212
26.8.	OKIDAČI I EVIDENTIRANJA PROMJENA U BAZI PODATAKA	213
27.	DOGAĐAJI	216
27.1.	IZRADA DOGAĐAJA	216
27.2.	OMOGUĆAVANJE DOGAĐAJA ZA DBMS	217
27.3.	IZMJENA DOGAĐAJA	218
27.4.	BRISANJE DOGAĐAJA	218
27.5.	PRIMJER	218
28.	POGLEDI.....	222
28.1.	PREDNOSTI KORIŠTENJA POGLEDA	223
28.2.	NEDOSTATCI KORIŠTENJA POGLEDA	223
28.3.	IZRADA POGLEDA	223
28.4.	PREGLED, BRISANJE I IZMJENA POGLEDA	225
28.5.	PRIMJERI.....	225
28.6.	UNAPRIJEDIVI POGLEDI	228
29.	UPRAVLJANJE TOKOM POHRANJENIH ZADATAKA	231
29.1.	KONTROLA TOKA	231
29.2.	PETLJE	234
29.3.	REKURZIJE U POHRANJENIM PROCEDURAMA.....	237
30.	KURSORI	241
30.1.	KADA KORISTITI KURSOR?	241
30.2.	KADA NE KORISTITI KURSOR?.....	241

30.3.	RAD S KURSOM.....	241
31.	CONDITIONI I HANDLERI	250
31.1.	RAD S CONDITIONIMA.....	250
31.2.	RAD S HANDLERIMA	252
31.3.	PRIMJERI DEKLARACIJE I UPOTREBE HANDLERA.....	254
31.4.	PRIMJENA CONDITIONA I HANDLERA U KURSORIMA.....	256
32.	PRIVREMENE TABLICE U POHRANJENIM ZADATCIMA.....	263
32.1.	IZRADA PRIVREMENE TABLICE	263
32.2.	BRISANJE TABLICE	264
32.3.	PRIMJERI.....	264
33.	ZAŠTITA OD NEOVLAŠTENOG PRISTUPA PODATCIMA.....	270
33.1.	ZAŠTITA BAZE PODATAKA.....	270
33.2.	STVARANJE KORISNIKA	270
33.3.	DODJELJIVANJE DOZVOLA	273
33.4.	RAZINE DODJELJIVANJA DOZVOLA.....	275
33.5.	UKIDANJE DODJELJENIH DOZVOLA	276
33.6.	PREGLED POSTOJEĆIH DOZVOLA.....	277
33.7.	PROMJENA KORISNIČKOG IMENA.....	277
34.	ZAKLJUČAVANJE PODATAKA.....	279
34.1.	ISTOVREMENI PRISTUP PODACIMA.....	279
34.2.	POTENCIJALNI PROBLEMI KOD PARALELNOG PRISTUPANJA PODACIMA.....	280
34.3.	ZAKLJUČAVANJE PODATAKA U BAZI PODATAKA	283
34.4.	PROTOKOL DVOFAZNOG ZAKLJUČAVANJA (2PL)	284
34.5.	VRSTE KLJUČEVA	284
34.6.	NEPOTPUNI I POTPUNI ZASTOJ	285
34.7.	GRANULACIJA (ZRNATOST) ZAKLJUČAVANJA PODATAKA	288
34.8.	RAZINE IZOLIRANOSTI	292
35.	IZRADA SIGURNOSNE KOPIJE I OBNAVLJANJE BAZE PODATAKA	295
35.1.	POTREBE ZA IZRADOM	295
35.2.	MEHANIZMI POHRANJIVANJA NA MYSQL-U	298
35.3.	SIGURNOSNE KOPIJE I OBNAVLJANJE NA MYSQL-U	300
36.	OPTIMIZACIJA BAZE PODATAKA	303
36.1.	OPTIMIZACIJA DIZAJNA BAZE PODATAKA	303
36.2.	OPTIMIZACIJA NA RAZINI UPITA.....	307
37.	NOSQL	316
37.1.	CAP TEOREM.....	316
37.2.	KLASIFIKACIJA NoSQL BAZA PODATAKA	318
37.3.	ODABIR POGODNE BAZE PODATAKA.....	323
38.	LITERATURA	324

Predgovor

Ovaj udžbenik je namijenjen studentima Tehničkog veleučilišta u Zagrebu i kao takav pokriva područje baza podataka koje se izvodi kroz dva kolegija „Baze podataka“ i „Napredne baze podataka“ na stručnim preddiplomskom studijima računarstva i informatike. Udžbenik je nastao višegodišnjom izradom i spajanjem nastavnih materijala na oba kolegija.

Udžbenik je kreiran tako da korištenjem što jednostavnijeg jezika uz obilje primjera pruži studentima opsežna znanja o bazama podataka. Sa svojim opisima, teorijskom podlogom i primjerima prati stvarne situacije s kojima se susreću razvojni inženjeri i na taj način priprema studente za posao u struci i znanosti. Teorijski i praktični dijelovi popraćeni su ilustracijama i tablicama kako bi se što jednostavnije shvatila materija koju je potrebno izučiti.

Osim za studente Tehničkog veleučilišta u Zagrebu, udžbenik je idealan i za početnike jer čitatelja upoznaje s relacijskim bazama podataka, od samih osnova baza podataka do naprednog korištenja. Sustav za upravljanjem baza podataka na kojem su napravljeni svi primjeri i na kojem se udžbenik zasniva je MySQL koji se odlikuje svojom otvorenosti koda, jednostavnošću instalacije, korištenja i brojnim dodatnim alatima koji su razvijeni od strane zajednice otvorenog koda. Odlikuje ga i široka zajednica korisnika što pojednostavnjuje pretragu interneta u slučaju složenijih problema.

Želimo zahvaliti svima koji su sudjelovali u izradi ovog udžbenika, kao i studentima koji su pomogli tražiti pogreške te savjetima poboljšati kolegije, a samim time i udžbenik.

Autori, 2018.

1. Uvod u baze podataka

1.1. ŠTO JE TO PODATAK, A ŠTO INFORMACIJA?

Često se pojmovi podatak i informacija poistovjećuju pa je na početku potrebno napraviti odgovarajuću razliku. **Podatak** (engl. *Data*) je skup simbola ili znakova koji je pohranjen u memoriji. To je „sirovi“ građevni element koji je unesen u računalo i koje računalo obrađuje. Podatak izvan konteksta nema značenje, on dobiva značenje tek u određenom kontekstu odnosno kada se interpretira i primjereno poveže. Tada iz podatka proizlazi informacija.

Informacija (engl. *Information*) je obrađeni podatak. To je znanje koje primatelju opisuje nove činjenice, otklanja neizvjesnost i služi kao podloga za odlučivanje. To znanje se materijalizira u obliku podatka, simbola koji služe za prikaz informacija u svrhu pohrane, prijenosa i obrade. Informaciju, u pravilu, smatramo nematerijalnim elementom, što je samo donekle točno. Iako je za informaciju bitan samo sadržaj, a ne način na koji je informacija prikazana, no informacija uvijek ima neku materijalnu podlogu bilo da je pohranjena u memoriji računala, našoj glavi, zapisana ili izgovorena. Informacija je također i prikaz **semantičkih svojstava** podataka.

Uobičajeno je govoriti o bazi podataka, obradi podataka, a ne o bazi informacija ili obradi informacija. Obradeni podatak svojim sadržajem (semantičkim svojstvima) predstavlja informaciju.

Podatak „5“ izvan konteksta nema značenja, no ako znamo da je to ocjena iz kolegija Baze podataka za studenticu Ivanu Horvat, tada „5“ predstavlja informaciju.

Korisne informacije dobivaju se obradom podataka, pa samim time ako podaci nisu ispravni, ni informacija neće biti ispravna. U poslovnom okruženju kvalitetna informacija je vrlo vrijedna prilikom donošenja odluka što je ključ preživljavanja neke organizacije.

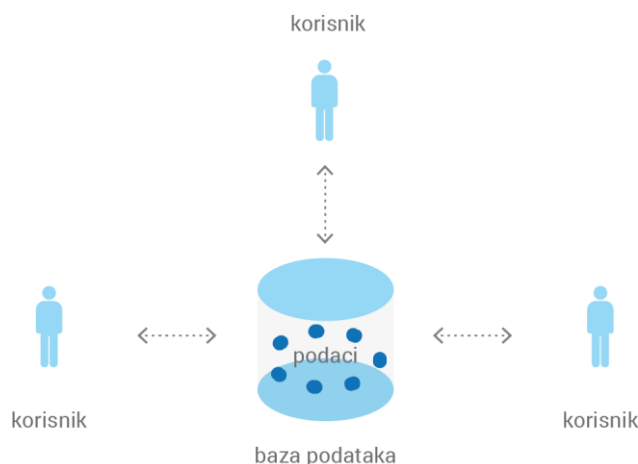
1.2. ŠTO JE BAZA PODATAKA?

Informacijski sustav (engl. *Information System*) je sustav koji prikuplja, pohranjuje, čuva, obrađuje i isporučuje informacije važne za organizaciju i društvo, tako da budu dostupne i uporabljive za svakog tko ih želi koristiti, uključujući vodstvo neke organizacije, klijente, osoblje i ostale. Može se reći i da je to dio svakog organizacijskog sustava čija je svrha prikupljanje, obrada, pohranjivanje i distribucija informacija, koje su potrebne za upravljanje organizacijskim sustavom. Središnji dio informacijskog sustava je **baza podataka** (engl. *Database*). Baze podataka pohranjuju podatke poslovanja neke organizacije i kao rezultat pohrane sadrže veliki broj podataka koje je potrebno dugoročno skladištiti.

U nastavku su dane dvije najčešće korištene definicije baze podataka.

Baza podataka je skup međusobno povezanih podataka, pohranjenih zajedno, uz isključenje bespotrebne zalihosti (redundancije), koji mogu zadovoljiti različite primjene. Podaci su pohranjeni na način neovisan o programima koji ih koriste. Prilikom dodavanja novih podataka, mijenjanja i pretraživanja postojećih podataka primjenjuje se zajednički i kontrolirani pristup. Podaci su strukturirani tako da služe kao osnova za razvoj budućih primjena. (J. Martin, 1979.).

Baza podataka je skup podataka koji su pohranjeni i organizirani tako da mogu zadovoljiti zahtjeve korisnika. (M. Vetter, 1981.)



Slika 1. Pojednostavnjen pogled na bazu podataka

Baza podataka sadrži podatke o entitetima. **Entitet** (engl. *Entity*) je bilo kakav stvarni ili apstraktni objekt, bilo što, što ima suštinu ili bit i posjeduje značajke s pomoću kojih se može razlučiti od svoje okoline. To je bilo što o čemu želimo prikupljati i pohranjivati podatke. Entitet mogu biti osobe, bića, objekti kao na primjer vozila, zgrade, mjesta, apstraktni pojmovi kao na primjer boje ili kolegiji, povezanost među objektima, osobama i slično.

Svaki entitet posjeduje određena svojstva odnosno **atribute** (engl. *Attribute*) koji ga karakteriziraju. Odabir atributa za entitet ovisi o svrsi informacijskog sustava. Za entitet osobe koja je radnik u određenoj tvrtki biti će potrebni atributi kao što su adresa, iznos plaće, podatak o odjelu u kojem radi, dok će se za osobu koja je student na nekom fakultetu izraditi atributi kao što su JMBAG studenta, naziv studija na kojem studira, prethodno završena škola i slično.

1.3. KRATKA POVIJEST BAZA PODATAKA

Od samih početaka razvoja programa i računarstva u svim granama primjene kao što je znanost, poslovna primjena, pravo, ekonomija, školstvo i ostalim granama javlja se potreba za pohranom podataka. U početku su se koristile datoteke za pohranu, ali se zbog konzistentnosti ne pokazuju praktičnim. Šezdesetih godina prošlog stoljeća zbog brzorastućih performansi računala i pojavljivanja sve više raznih softverskih rješenja pojavljuju se prve ozbiljnije baze podataka. Najveći problem bio je određivanje standarda, te Charles Bachman osniva „Database Task Group“ koja je kreirala prvi standard za programski jezik COBOL i navigacijsku bazu podataka. IBM je šezdesetih godina također imao svoj standard navigacijske baze podataka „IMS“ koji je poznat po projektu „Appolo“ u kojem se koristila za pohranjivanje popisa proizvoda za raketu „Saturn V“.

Većina današnjih baza podataka pripada skupini relacijskih baza podataka koje s vanjskim svijetom (programom ili korisnikom) komuniciraju pomoću **SQL** (engl. *Structured Query Language*) jezika.

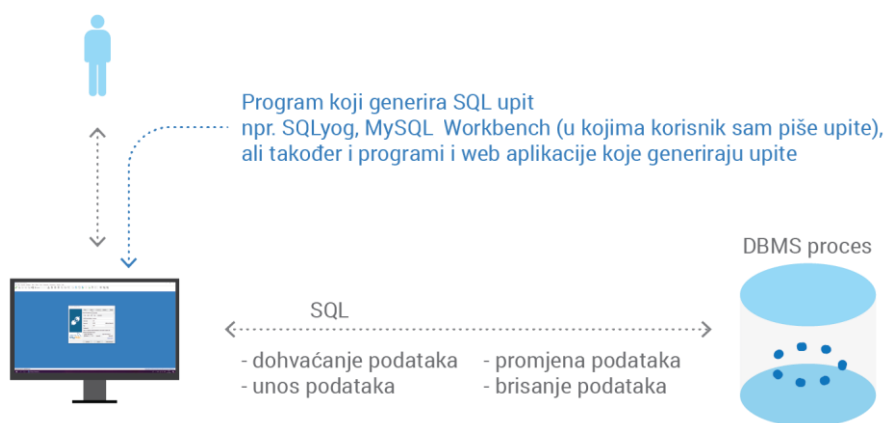
SQL standard napravljen je po uzoru na prvi relacijski model baze podataka kojeg je Edgar Frank Codd 1970.-te godine opisao u svojem članku „*A Relational Model of Data for Large Shared Data Banks*“. Prvu verziju SQL-a razvili su početkom sedamdesetih godina prošlog stoljeća Donald D. Chamberlin i Raymond F. Boycea u IBM-u. Njihova verzija se inicijalno zvala SEQUEL (engl. *Structured English Query Language*), a bila je izrađena za manipulaciju podacima pohranjenim u IBM-ovim polu-relacijskim sustavom baze podataka „System R“. Prva prava relacijska baza podataka nastala je na MIT-u (Massachusetts Institute of Technology) ranih 70-ih.

Prvu komercijalni sustav za upravljanje (relacijskim) bazama podataka koja koristi SQL na tržište je plasirao Oracle 1979.-te godine, a zvala se Oracle V2.

Zadnja revizija SQL standarda je SQL:2016 koju u potpunosti ne ispunjava ni jedan DBMS već se uglavnom koristi SQL-92.

1.4. SUSTAV ZA UPRAVLJANJE BAZOM PODATAKA

Sustav za upravljanjem baze podataka (engl. *Database Management System*, DBMS) je programski sustav opće namjene koji osigurava osnovne funkcije odabranog modela podataka u postupku kreiranja i korištenja baze podataka. Može se reći i da je DBMS poslužitelj (engl. *Server*) baze podataka, čije su glavne dužnosti osiguravanje dostupnosti, integriteta, neovisnosti i sigurnosti podataka. To je skup aplikacija i programa koji se koriste za pristup, ažuriranje i općenito upravljanje podacima.



Slika 2. Pristup podacima preko DBMS-a

DBMS se brine o pohrani i pristupanju podacima ostavljajući samo aplikacijski dio zahtjeva aplikacijama. Primjerice aplikacija koja koristi neki DBMS ne brine o konzistentnosti podataka

prilikom nestanka struje, već se o tome brine DBMS. Identičan DBMS (primjerice MySQL, PostgreSQL, SQL Server, Oracle) može se koristiti za informacijski sustav trgovine, knjižnice, videoteke, sveučilišta i slično.

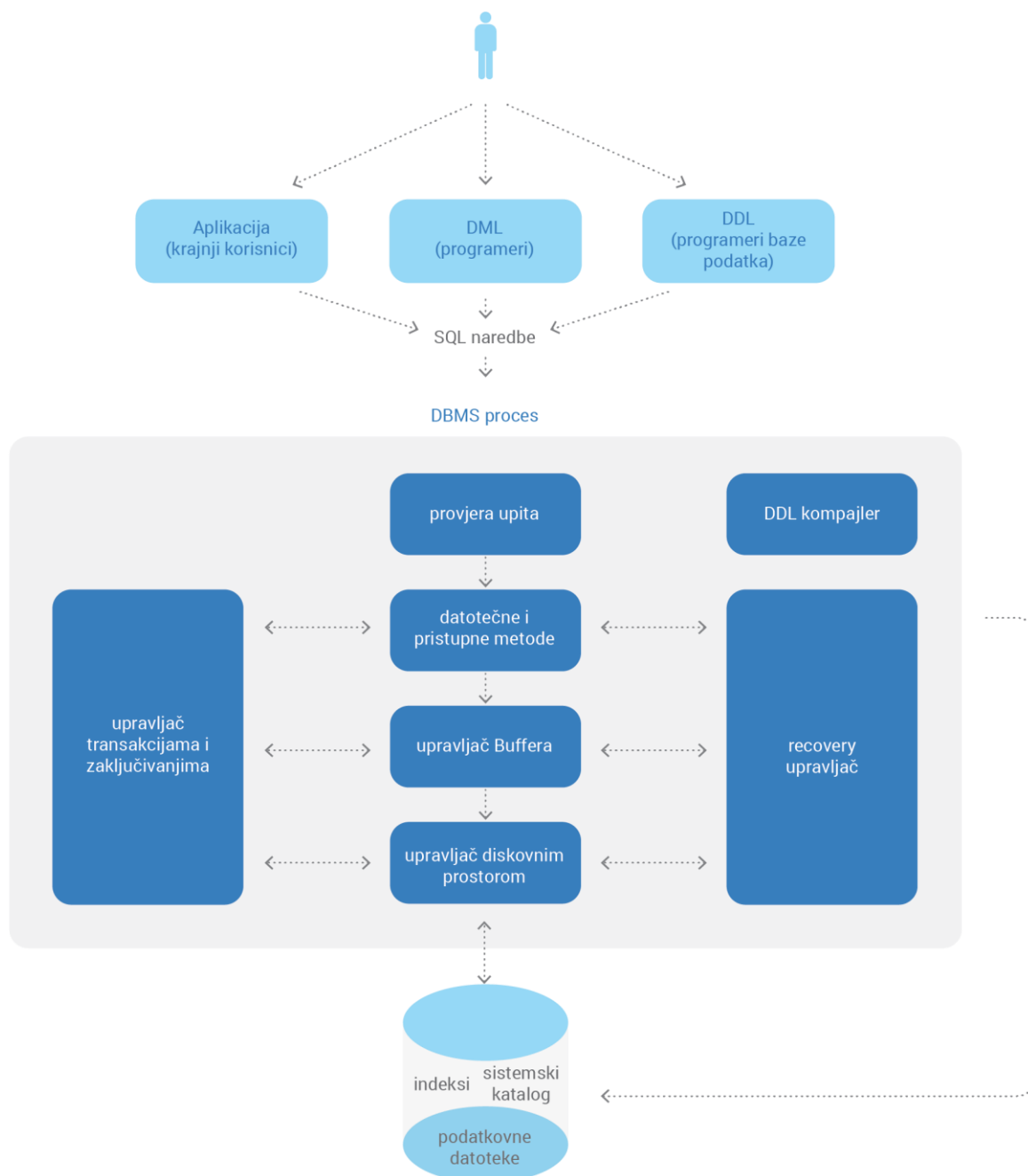
Zadaće DBMS-a su da:

1. Sakriva od korisnika detalje fizičke pohrane podataka,
2. Osigura logičku i fizičku nezavisnost podataka, jer fizička pohrana na disku ne mora ovisiti o logičkoj organizaciji podataka,
3. Omogućuje definiciju i rukovanje podacima korištenjem DDL (engl. *Data Definition Language*) i DML (engl. *Data Manipulation Language*) naredbi,
4. Obavlja funkciju zaštite podataka:
 - **Dostupnost podataka** – podaci trebaju biti dostupni širokom spektru korisnika u smislenom formatu po razumnoj cijeni kako bi korisnici što lakše pristupili podacima.
 - **Integritet** – Odnosi se na točnost podataka u bazi, podaci u bazi podataka trebaju biti pouzdani i ispravni.
 - **Sigurnost** – Samo ovlašteni korisnici smiju pristupati podacima odnosno određenoj grupi podataka. Ako dva ili više korisnika pristupaju istim podacima, DBMS ne smije dozvoliti konfliktne promjene.
 - **Neovisnost** – DBMS treba omogućiti korisnicima da efikasno i brzo pohranjuju, mijenjaju i dohvaćaju podatke. DBMS omogućava „apstraktan pogled“ na način kako se podaci pohranjuju u bazi podataka. Kako bi se efikasno pohranila informacija koriste se kompleksne strukture podataka.
5. Obavlja optimizaciju upita.

Tablica 1. Najvažniji proizvođači DBMS-a i njihovi proizvodi

Proizvođač	Proizvod
IBM	DB2
	Informix
Microsoft	Access
	SQL Server
	SQL Server compact
Open Source	MySQL
	PostgreSQL
Oracle	Oracle DBMS
	RDB

Sybase	Adaptive Server Enterprise Adaptive Server Anywhere Watcom
--------	--



Slika 3. Važnije komponente DBMS-a

1.5. MODEL BAZE PODATAKA

Modeliranje stvarnog svijeta predstavlja preslikavanje stvarnog svijeta u oblik pogodan za računalnu obradu. Nije ga moguće prikazati sa svim detaljima već se prikazuje s pojednostavljenim modelom koji sadrži podatke relevantne za sustav obrade. **Model podataka** (engl. *Data Model*) je formalni sustav koji se koristi za modeliranje baza podataka, a sastoji se od:

- **skupa objekata** - osnovnih elemenata baze podataka,
- **skupa operacija** koje se provode nad objektima,
- **skupa općih pravila integriteta** (engl. *integrity constraints*) koji definiraju skup konzistentnih stanja podataka, promjena stanja, ili oboje.

Postoje različiti modeli podataka, a u narednim poglavljima opisivati će se **relacijski model podataka**, koji se u povijesnom redoslijedu razvoju modela podataka smješta kao u nastavku:

- Hijerarhijski model
- Mrežni model
- Relacijski model
- ER model
- Objektni model
- Objektno-relacijski model

Model baze podataka pomaže prilikom opisa strukture podataka na logičkom nivou opisujući strukturu baze podataka. Svaki DBMS ima svoj način zapisivanja podataka na datotečni sustav. MySQL za svaku tablicu kreira novu datoteku unutar koje se pohranjuju podaci (ekstenzije ovisno o mehanizmu pohrane) te dodatno datoteku s meta podacima (.frm), a SQL Server ima najčešće dvije datoteke .mdf i .log.

1.5.1. Relacijski model podataka

Autor relacijskog modela podataka je E. F. Codd, a njegova je osnovna karakteristika jednostavnost i jaka teorijska osnova koja omogućava precizno definiranje struktura podataka i njihovih međusobnih odnosa. **Relacijski model** se oslanja na dobro razvijenu matematičku teoriju relacija i logiku prvog reda. Upravo su te matematičke osnove bile i glavni razlog daljnjeg uspješnog razvoja modela i njegove primjene u praksi. Razni koncepti normalizacije rezultat su razvoja upravo relacijskog modela podataka.

Kako je navedeno u prethodnom poglavlju, model podataka sastoji se od skupa objekata, skupa operacija i općih pravila integriteta. U nastavku će navedeno biti primijenjeno na relacijski model podataka.

1.5.1.1. Skup objekata

Element skupa objekata u relacijskom modelu su **relacije** (engl. *Relation*). Relacija je imenovana dvodimenzionalna tablica a sastoji se od atributa i **n-torki** (engl. *Tuples*) koje opisuju pojedinačne entitete. **Atribut** je imenovani stupac relacije, a n-torka je redak relacije.

Shema relacije obuhvaća naziv relacijske sheme (nastavnici) i skup atributa: (jmbg, ime, prezime, adresa, titulaIspred).

	jmbg	ime	prezime	adresa	titulaIspred
	0305988391019	Lucija	Brčić	Naserov trg 1	
	0404964361006	Danijel	Tkalčić	Nodilova 96	dr.sc.
	0501953210249	Anto	Bošnjak	Unska ulica 25	
	0509959360058	Leona	Vidović	Ulica svetog Duje 106	prof. dr.sc.
	0606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.

Slika 4. Relacija nastavnici

1.5.1.2. Skup operacija

Skup operacija temelji se na **relacijskoj algebri** a odnosi se na operacije kao što su unija, razlika, presjek, projekcija i ostalo.

Primjerice nad relacijom *nastavnici* moguće je izvršiti operaciju selekcije te prikazati samo one n-torke koje se odnose na nastavnice koji imaju atribut *titulaIspred* jednak „prof. dr.sc.“. U SQL-u navedeno će se realizirati WHERE klauzulom u SELECT upitu.

jmbg	ime	prezime	adresa	titulaIspred
0606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.
0610975181955	Renato	Topić	Krešimirova 98	prof. dr.sc.

Slika 5. Operacija selekcije nad relacijom *nastavnici*

Operacija projekcije postiže se ispisom određenog skupa atributa neke relacije.

1.5.1.3. Skup općih pravila integriteta

U relacijskom modelu podataka postoje četiri pravila integriteta:

- Opća pravila integriteta
 - Pravilo entitetskog integriteta
 - Pravilo referencijskog integriteta
- Korisnička pravila integriteta
 - Pravilo domenskog integriteta
 - Pravilo odnosnog integriteta

Primjerice nad relacijom *nastavnici* postoji pravilo domenskog integriteta koje je određeno domenom atributa *jmbg*. Navedeni atribut sastoji se od 13 znamenki te vrijednosti tog atributa mogu biti iz intervala [1, 9999999999999].

1.5.2. Model entiteti-veze (ER model)

Model entiteti-veze (engl. *Entity-Relationship Model*) poznat i kao **ER model** je postrelacijski model baze podataka i zadržava dobra svojstva relacijskog modela. Osnovni elementi strukture ER modela su:

- entiteti,
- njihove međusobne veze i
- odgovarajući atributi.

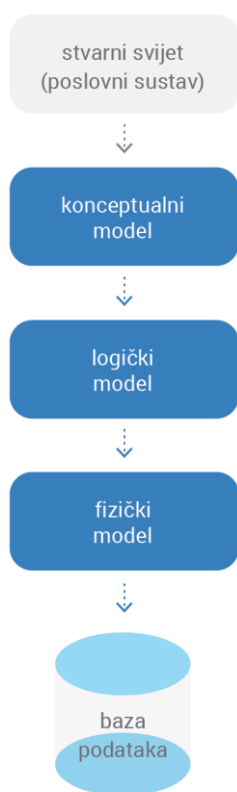
Ovaj model omogućuje eksplicitni prikaz veza koje u sebi sadrže važne semantičke informacije.

2. Izrada baze podataka

Izrada baze podataka je proces stvaranja modela baze podataka. Model mora objedinjavati relevantne podatke na način da se mogu obraditi pomoću mehanizma za pohranu podataka. Baza podataka neke organizacije je repozitorij podataka o samoj organizaciji i njenom poslovanju.

2.1. TIJEK IZRADE BAZE PODATAKA

Izrada baze podataka je kompleksan **proces** zbog potrebe za dobrim poznavanjem poslovnih procesa same organizacije za koju se baza podataka kreira. Kompleksnost raste s brojem individualnih komponenata poslovanja organizacije zato što raste broj entiteta i veza koje moraju zadržati traženu funkcionalnost. U većini slučajeva model baze podataka izrađuju osobe s ekspertizom u području modeliranja odnosno izrade modela baze, a ne području domene baze podataka kao što su primjerice, financije, bankarstvo, specifično poslovanje tvrtke i slično. Iz tog razloga model se najčešće oblikuje u suradnji s osobom koja ima ekspertizu u domeni. Koraci u oblikovanju modela baze podataka nalaze se na sljedećoj slici:



Slika 6. Pojednostavljeni prikaz tijeka izrade baze podataka

2.1.1. Prikupljanje zahtjeva i analiza

Prilikom prikupljanja zahtjeva i analize potrebno je odrediti koji podaci se moraju pohranjivati i kako će se isti koristiti. Osobe koje će koristiti bazu podataka moraju biti uključene u ovu akciju kako bi se prikupile što relevantnije informacije. Pretpostavke o korištenju baze u smislu entiteta i veze među njima moraju biti dobro postavljene.

Ako se primjerice razvija sustav za pohranjivanje podataka o pacijentima, potrebno je dobro razjasniti neke bitne informacije poput: Može li pacijent imati više od jednog doktora? Može li pacijent biti primljen u više bolnica? Ima li pacijent dopunsko zdravstveno osiguranje?

2.1.2. Oblikovanje modela

Oblikovanje modela podrazumijeva postupak analize i organizacije podataka u obliku pogodnom za podršku poslovnim potrebama te omogućava korištenje strateških tehnologija.

Oblikovanje modela baze podataka moguće je u literaturi pronaći i pod pojmovima **modeliranje** pa i **dizajniranje** baze podataka. Obzirom da se postupak svodi prvenstveno na osmišljavanje i izradu modela, ispravnije je koristiti izraz oblikovanje modela baze podataka.

Logički model baze podataka se stvara pomoću DDL naredbi SQL jezika, a DBMS sustav se brine o **fizičkoj pohrani** logičke strukture i samih podataka na tvrdom disku. Svaki DBMS sustav ima svoju implementaciju načina pohrane logičke strukture i podataka na diskovnu jedinicu.

Prilikom modeliranja postoje tri faze:

- **konceptualno,**
- **logičko,**
- **fizičko.**

2.1.2.1. Konceptualno oblikovanje modela

Glavni rezultat prve faze oblikovanja jest **konceptualna shema** cijele baze, sastavljena od entiteta, atributa i veza. Treba opisivati sadržaj baze i načine povezivanja podataka u njoj. Prikaz je jezgrovit, neformalan i pogodan ljudima za razumijevanje, no još je nedovoljno razrađen da bi omogućio izravnu implementaciju.

2.1.2.2. Logičko oblikovanje modela

Glavni rezultat druge faze oblikovanja je **logička shema** koja je u slučaju relacijskog modela sastavljena od relacija (tablica). Sastavni dio logičkog oblikovanja je i normalizacija, gdje se primjenom posebnih pravila nastoji popraviti logička struktura samih relacija.

2.1.2.3. Fizičko oblikovanje modela

Glavni rezultat treće faze oblikovanja je **fizička shema** cijele baze, dakle opis njezine fizičke građe. U slučaju uporabe DBMS-a zasnovanog na jeziku SQL, pojam fizičke razine treba shvatiti uvjetno. Fizička shema je niz SQL (DDL) naredbi kojima se relacije iz logičke sheme

realiziraju kao tablice. Pritom se dodaju pomoćne strukture i mehanizmi za postizavanje traženih performansi te čuvanje integriteta i sigurnosti podataka.

2.1.3. Implementacija

Implementacija baze podataka uključuje razvoj i pisanje samog programskog koda za kreiranje baze podataka i pohranjenih upita, kao i instalacija DBMS sustava.

2.2. CILJEVI IZRADE BAZE PODATAKA

Ciljevi izrade baze podataka obično variraju kod različitih implementacija i potreba korisnika. Najvažniji faktori o kojima treba voditi brigu prilikom izrade baze podataka su: efikasnost, integritet, privatnost podataka, sigurnost, implementacija i fleksibilnost.

2.2.1. Efikasnost

Efikasnost je najvažniji faktor prilikom izrade baze podataka. Baza podataka bi trebala koristiti sve moguće resurse na zadanom hardveru i DBMS softveru. Baza podataka mora biti izrađena tako da korisnici ne osjete vremensku zadržku prilikom pristupanja podacima.

2.2.2. Integritet

Pojam integriteta kod baze podataka znači da baza podataka mora što točnije pohranjivati željene podatke kako bi se pomoću upita dobila korisna informacija. Problem očuvanja integriteta podataka moguće je sagledati s dvije razine. S niže razine problem očuvanja podataka je mogućnost korupcije podataka od strane hardvera i softvera (na primjer kvar tvrdog diska, loš odabir datotečnog sustava, uništenje podataka od strane DBMS-a prilikom nestanka struje i slično). S više razine problem očuvanja podataka se svodi na točnost predstavljana podataka stvarnog svijeta, odnosno točnost podataka u bazi.

2.2.3. Privatnost

Baza podataka ne smije dozvoliti neovlašten pristup datotekama i podacima. Važnost privatnosti podataka dodatno je istaknuta kod baza koje sadrže financijske podatke ili podatke poslovanja neke organizacije koji ne smiju izaći u javnost. Primjerice stanje računa u banci smije vidjeti samo korisnik na koga se stanje odnosi, a nikako svi korisnici.

2.2.4. Sigurnost

Baza podataka kada je jednom učitana, mora biti sigurna od strane fizičke korupcije bez obzira radi li se o hardverskom kvaru, softverskom kvaru, neautoriziranom pristupu, neželjenom programu na računalu, preopterećenju ili programskoj grešci. Ovo je opći uvjet većine baza podataka. Većina DBMS programa ovaj problem rješava pomoću kontrole pristupa, nadgledanje akcija svakog korisnika (engl. *auditing*), ovjere autentičnosti (engl. *authentication*), enkripcije, kontrole integriteta, izrade sigurnosnih kopija i sigurnosti aplikacije.

Baze podataka su generalno poprilično sigurne protiv hakerskih napada preko mreže pomoću vatrozida (engl. *firewall*) i sustava za otkrivanje napada.

2.2.5. Implementacija

Konceptualni model baze podataka treba biti što jednostavniji i efikasniji kako bi mapiranje s konceptualnog modela na logički model bilo što jednostavnije. Prilikom izrade baze podataka potrebno je voditi brigu da aplikacija ili sustav koji se veže na bazu podataka bude efikasan u radu s bazom.

2.2.6. Fleksibilnost

Baza podataka se ne bi smjela implementirati na krut način koji pretpostavlja da će proces ostati identičan zauvijek. Promjene kao što su promjena stope poreza ili uvođenje novog načina poslovanja, će se tijekom rada događati i baza podataka ih mora moći pratiti. Osim što je potrebno implementirati bazu podataka da ju se s lakoćom može promijeniti, ispravan model baze podataka također mora osiguravati da ne postoji redundancija podataka.

2.3. OBLIKOVANJE MODELA ENTITETI-VEZE (ER MODELA)

Peter Chen je 1976.-te godine prvi predložio oblikovanje modela baze podataka koristeći grafičku tehniku koja je ljudima lako shvatljiva. Pomoću njegovog grafičkog modela jednostavnije je shvatiti entitete i njihove atribute u stvarnom svijetu kao i relacije koje povezuju entitete. Dizajner baze podataka u današnje vrijeme može koristiti mnoge popularne alate za oblikovanje modela baza podataka kao što su MySQL Workbench, Toad, SQL Server management studio i mnoge druge.

2.3.1. Građevni elementi ER modela

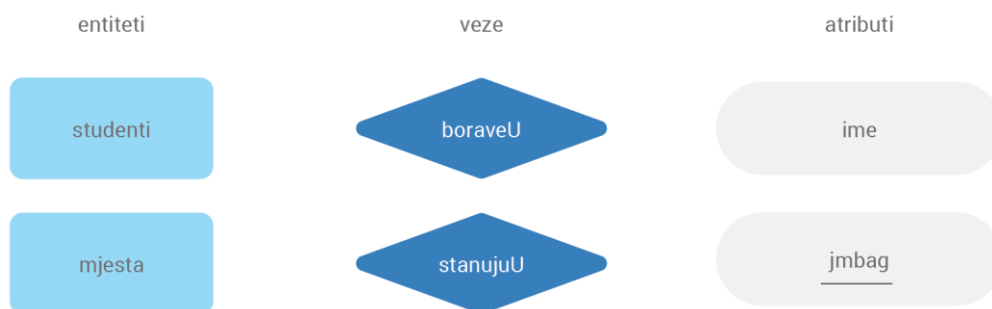
Entitet je bilo koji objekt iz stvarnog života koji se razlikuje od drugih objekata i unikatan je. Entiteti mogu biti *studenti* i *mjesta*, a entitet *mjesta* može sadržavati podatke o svim mjestima prebivanja ili stanovanja studenata. Broj entiteta koje povezuje određena veza definira stupanj veze. Broj entiteta koje povezuje određena veza definira stupanj veze.

Atribut ili svojstvo opisuje entitete. Svaki entitet mora se sastojati od barem jednog atributa. Atributi entiteta "studenti" mogu biti: jmbag, ime, prezime, datumUpisa, postBrPrebivanje, postBrStanovanja i datumUpisa. Atributi entiteta "mjesta" mogu biti postBr i naziv.

Entiteti su međusobno povezani **vezama**. Svaki student ima svoje mjesto prebivanja, a također ima i mjesto stanovanja, pa tako moraju postajati dvije veze između entiteta *studenti* i entiteta *mjesta*.

U grafičkom prikazu:

- **entitet** se prikazuje **pravokutnikom** unutar kojeg se nalazi ime entiteta,
- **veza** se prikazuje **rombom** unutar kojeg se nalazi ime veze,
- **atribut** se prikazuje **ovalom**, **elipsom** ili **krugom** unutar kojeg se nalazi ime atributa,
- atributi koji su **ključni** (kasnije će biti dio primarnog ključa), dodatno se **podcrtavaju**



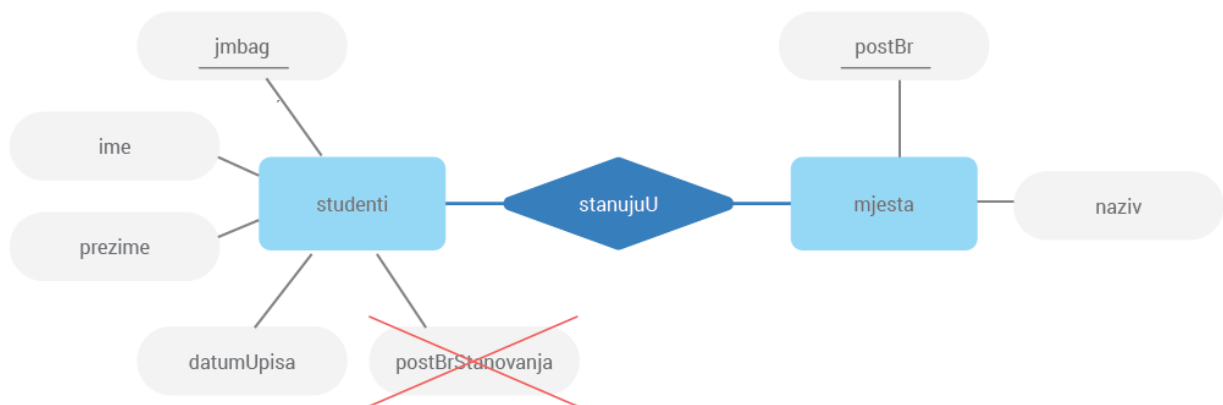
Slika 7. Grafički prikaz elemenata ER dijagrama

Vezano uz nomenklaturu, uobičajeno je nazive entiteta, atributa i veza pisati u jednini. S druge strane, učestalo je korištenje naziva entiteta u množini jer će se svaki entitet sastojati od većeg broja n-torki, te se u slučaju entiteta *studenti*, neće pohranjivati podatci o jednom studentu već o studentima. Radni okviri za razne programske jezike predlažu korištenje naziva entiteta (koji postaju relacije odnosno tablice), pa se u sklopu ovog udžbenika i baze podataka na kojoj su pokazivani primjeri, entitetima daju nazivi koji su u množini.

2.3.2. Grafički prikaz ER modela

Izradom ER modela dobiva se zapravo konceptualni model visokog nivoa.

Vlastiti atributi entiteta su oni koji opisuju svojstva entiteta koja se pripisuju isključivo samom entitetu i nemaju veze s drugim entitetima. Tako na dijagramu koji prikazuje entitete *studenti* i *mjesti*, *postBrStanovanja* nije ispravan atribut entiteta *studenti*.



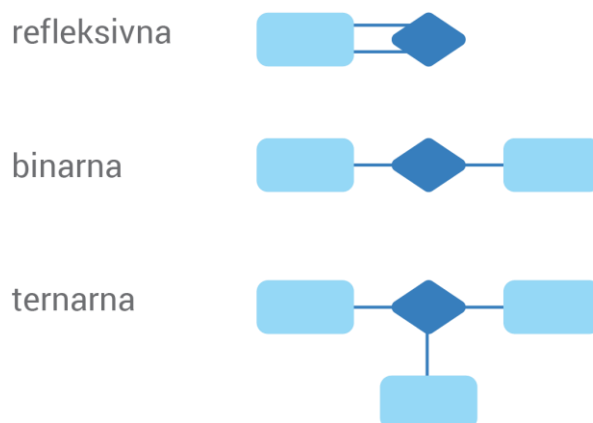
Slika 8. ER dijagram entiteta studenti i mjesta, veze između njih te nekih njihovih atributa

U trenutku kada entitet ima previše atributa koji ne stanu pregledno na dijagram, moguće je uz dijagram priložiti shemu entiteta. U nastavku je shema entiteta *studenti*.

studenti = jmbag, ime, prezime, datumUpisa

PK = {jmbag}

Broj entiteta koje povezuje određena veza definira **stupanj veze**. Veza može biti unarna (refleksivna), binarna, ternarna i tako dalje.



Slika 9. Stupnjevi veza

Kardinalnost veze opisuje ograničenje preslikavanja pojedinačnih entiteta koje veza povezuje. Mogu biti:

- Jedan (one)

- Više (many)

Pri tome se koriste oznake 1, N ili rasponi, na primjer 0..1, 1..N, 1..2, i slično.

Primjeri:



Slika 10. Primjer kardinalnosti

Gornji primjer čita se kao:

„Jedan student studira na jednom smjeru, na jednom smjeru studira N studenata.“



Slika 11. Primjer kardinalnosti

Gornji primjer čita se kao:

„Jedan student studira na nula ili jednom smjeru, na jednom smjeru studira nula ili više studenata.“

Odnos ograničenja preslikavanja pojedinačnih entiteta može biti primjerice:

- Jedan na jedan (1:1)
- Jedan na više (1:N)
- Više na jedan (N:1)
- Više na više (N:N)

Primjer kardinalnosti 1:N:



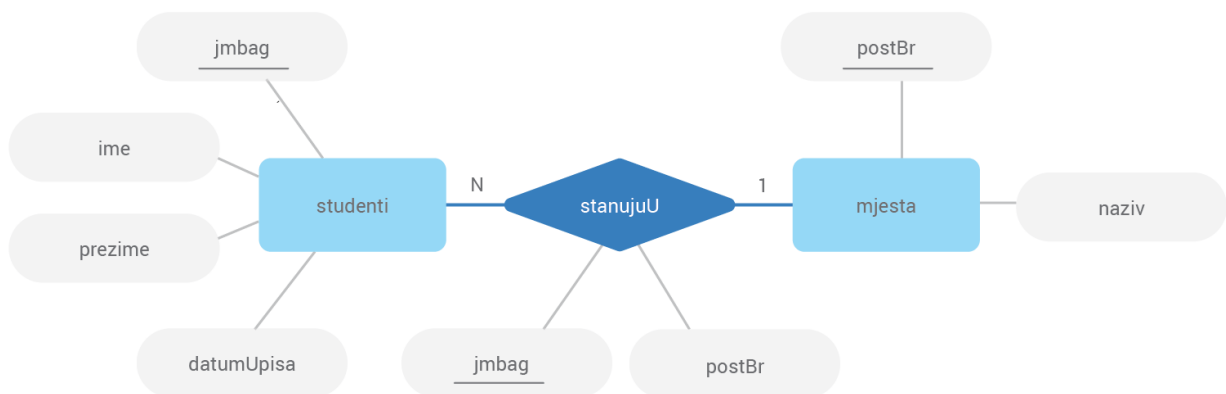
Slika 12. Primjer kardinalnosti

Primjer kardinalnosti N:N (pod pretpostavkom da student može studirati na više smjerova primjerice različitih ustanova):



Slika 13. Primjer kardinalnosti

Veza može sadržavati atribut kao i ključeve. Atribut veze se grafički prikazuje ovalom unutar kojeg se upisuje ime atributa, odnosno ključ se podcrtava kao i u slučaju ključa entiteta. Ključevi veza definiraju se pomoću ključeva entiteta koje povezuju te njihovih kardinalnosti.



Slika 14. Primjer kardinalnosti

Sheme entiteta:

STUDENTI = jmbag, ime, prezime, datumUpisa

MJESTA = postBr, naziv

Shema veze:

stanujeU = jmbag, postBr

2.4. PRESLIKAVANJE ER MODELA U RELACIJSKI MODEL

2.4.1. Preslikavanje entiteta

Svaki entitet prikazuje se jednom relacijom, pri čemu atributi entiteta postaju atributi relacije. Jedan primjerak entiteta prikazuje se jednom n-tokom. Primarni ključ entiteta postaje i primarni ključ relacije.

2.4.2. Preslikavanje veza

Sheme entiteta preslikavaju se u relacijske sheme, pri čemu i veze postaju entiteti pa nastaju relacijske sheme i od entiteta. Nad dobivenim relacijskim shemama radi se **unija s jednakim ključevima**. U slučaju veze 1:N to znači da će veza nestati, odnosno njen ključ će pripasti relaciji na koju se odnosi.

2.4.2.1. Preslikavanje veze 1:N

Razmatramo slučaj entiteta *studenti* i *mjesta* te veze *stanujuU* prema ranije prikazanom primjeru.



Slika 15. Preslikavanje veze 1:N

Sheme entiteta:

STUDENTI = jmbag, ime, prezime, datumUpisa

MJESTA = postBr, naziv

Shema veze:

stanujeU = jmbag, postBr

Unija gornjih shema entiteta s jednakim ključevima dati će rezultat s dvije relacijske sheme:

STUDENTI = jmbag, ime, prezime, datumUpisa, postBr

MJESTA = postBr, naziv

2.4.2.2. Preslikavanje veze N:N

Razmatramo slučaj entiteta *studenti* i *smjerovi* te veze *studiraNa* prema ranije prikazanom primjeru u kardinalnosti N:N.



Slika 16. Preslikavanje veze N:N

Sheme entiteta:

STUDENTI = jmbag, ime, prezime, datumUpisa

smjerovi = idSmjer, naziv

Shema veze:

studiraNa = jmbag, idSmjera

Obzirom da ne postoje dvije relacijske sheme s jednakim ključevima, ne radi se unija već shema veze koja je postala entitet, prilikom preslikavanja u relacijsku shemu ostaje takvom. Dakle preslikavanjem veze N:N iz modela entiteti veze u relacijski model, stvara se nova relacija odnosno dva entiteta kardinalnosti N:N preslikana su u tri relacije.

Relacijske sheme:

STUDENTI = jmbag, ime, prezime, datumUpisa

smjerovi = idSmjer, naziv

studiraNa = jmbag, idSmjera

2.5. ALATI ZA IZRADU MODELA BAZE PODATAKA

Nakon što su ciljevi baze podataka poznati, potrebno je znati koji će se alati za izradu modela baze podataka koristiti. Programi, odnosno ispravnije reći, alati za izradu modela baze podataka služe za grafičko modeliranje baze podataka nakon čega generiraju SQL DDL upite koji kreiraju logički model baze podataka. Alati za izradu modela baze podataka povećavaju produktivnost zato što se klasični ručno pisani programski kod automatizira i smanjuje mogućnost pogreške što povećava kvalitetu krajnjeg proizvoda. Također dozvoljava osobi koja modelira bazu podataka da više razmišlja o samom modelu baze podataka nego o pisanju SQL upita što ubrzava cijeli proces oblikovanja modela.

2.5.1. Poželjne funkcionalnosti alata za izradu baze podataka

Alati za izradu modela baze podataka bi trebali pomoći razvojnom inženjeru da oblikuje model baze podataka pravovremeno. Neke od mogućnosti alata za dizajn baze podataka slijede u nastavku:

- Mogućnost modeliranja toka podataka prema pravilima organizacije
- Mogućnost modeliranja entiteta te njihovih atributa i veza
- Generiranje DDL jezika za kreiranje objekata baze podataka

2.5.2. Prednosti korištenja alata za izradu modela baze podataka

Neke od prednosti prilikom korištenja ovakvih alata su sljedeće:

- Količina napisanog programskog koda se reducira
- Trajanje procesa oblikovanja modela baze podataka je reducirana
- Mogućnost pogreške je smanjena zato što ne postoji više „ručno“ pisanje programskog koda
- Poslovni model je jednostavno pretvoriti u radni model baze podataka
- Jednostavno je provjeriti da su svi poslovni zahtjevi zadovoljeni
- Proizvod je više kvalitete i točnosti

2.5.3. Nedostaci korištenja alata za izradu modela baze podataka

Neki nedostaci prilikom korištenja ovakvih alata su sljedeći:

- Mogućnost većih troškova (zbog cijene samog alata)
- Razvojni inženjeri možda moraju odraditi dodatno školovanje za korištenje alata

2.5.4. Komercijalni alati za izradu modela baze podataka

U nastavku su navedeni neki od alata za izradu modela baze podataka koji su komercijalno dostupni na tržištu:

- DeZign for Databases
- DBDesigner 4
- ER/Studio XE3
- Happy Fish Database Designer
- MySQL Workbench
- Oracle Designer
- Power designer
- SQL Server Management Studio

- Toad Data Modeler
- Web Objects
- xCase

3. Oblikovanje sheme relacijske baze podataka

3.1. REDUNDANCIJA I ANOMALIJE PODATAKA

Redundantni podaci znače da se isti podaci pohranjuju na više od jednog mjesta unutar baze podataka. Ako su podaci redundantni, iste je moguće ukloniti bez gubitka informacije. Redundancija može dovesti do anomalije podataka koja se može dogoditi prilikom unosa, promjene i brisanja podataka.

3.1.1. Problem redundancije

Redundancija može uzrokovati probleme prilikom normalnog rada baze podataka. Na primjer, prilikom unosa podatka u bazu podataka, podatak se mora duplicirati bez obzira postoji li isti u bazi podataka ili ne. Također, prilikom promjene podatka, isti je potrebno promijeniti na više mjesta kako bi se pohranile željene promjene.

3.1.2. Anomalija prilikom unosa, brisanja i promjene

Tablična anomalija je struktura zbog koje normalne operacije nad bazom podataka nisu moguće bez gubitka podataka ili potpunog pretraživanja tablice u bazi podataka. Tablične anomalije se mogu klasificirati u grupe: anomalija prilikom unosa, anomalija prilikom brisanja i anomalija prilikom izmjene.

Tablica 2. Primjer tablica studenata s anomalijama

Studenti					
Br. Stud.	Smjer	Br. odjel	Naziv Odjela	Grad stud.	Predmeti
100	elektrotehnika	10	Voltino	Zagreb	elektrotehnika
101	građevina	20	Holjevcéva	Zagreb	statika, fizika, Engleski
102	informatika	30	Vrbik	Split	baze, programiranje
103	informatika	30	Vrbik	Zagreb	fizika, baze, OOP

Anomalija prilikom unosa – primjer:

Nemoguće je u tablicu studenata unijeti novi odjel bez da se unese i student koji studira na tom odjelu.

Anomalija prilikom promjene podatka – primjer:

Moguće je promijeniti naziv odjela na kojem studira student broj 102, a da se studentu broj 103 ne promjeni naziv odjela.

Anomalija prilikom brisanja podatka – primjer:

Kada se ukloni student broj 100, uklonit će se i svi podaci o smjeru i odjelu.

3.1.3. Ponavljajuća grupa

Ponavljajuća grupa je atribut (ili skupina atributa) koja može imati više od jedne vrijednosti za primarni ključ. Kako bi se bolje shvatio koncept ponavljajuće grupe potrebno je proučiti primjer tablice studenata s anomalijama. Student može imati više od jednog upisanog predmeta. Kako bi se zadovoljila pravila normalizacije relacijske baze za svaki novoupisani predmet, trebalo bi dodati još cijeli jedan redak u tablicu s brojem studenta, odjelom i slično. U tom slučaju sve n-torke bi trebale sačinjavati primarni ključ. Takav dizajn dovodi do povećane potrebe za pohranom podataka. Ovako kako su ponavljajuće grupe napisane u primjeru nikako ne smije se koristiti u relacijskoj bazi podataka zato što vrijednosti nisu atomske.

3.2. FUNKCIJSKA ZAVISNOST

Funkcijske zavisnosti su odnosi između atributa unutar relacije. Funkcijske zavisnosti omogućuju mehanizam za izražavanje ograničenja između atributa.

Neka je r relacija sa shemom R i neka su A i B skupovi atributa, $A \subseteq R$, $B \subseteq R$. **Funkcijska zavisnost** $A \rightarrow B$ vrijedi na shemi R ako u svim dopuštenim stanjima relacije $r(R)$ svaki par n -torki t_1 i t_2 koje imaju jednake A -vrijednosti, također imaju jednake B -vrijednosti, odnosno:

$$t_1(A) = t_2(A) \Rightarrow t_1(B) = t_2(B)$$

Ako atribut (ili skup atributa) B funkcijski zavisi o atributu A , tada se za svaku instancu atributa A zna i vrijednost atributa B . Atribut B je funkcijski zavisan o atributu A (ili o skupini atributa) ako vrijednost atributa A (ili skupine atributa) određuju vrijednost atributa B . Funkcijska zavisnost na neki način pomaže uočavanju odnosa među atributima.

Notacija:

Funkcijska zavisnost se obilježava sa: $A \rightarrow B$

Značenje notacije je:

- A određuje B
- B je funkcijski zavisan o A
- A se naziva **determinanta**
- B se naziva **objekt determinante**

Primjeri:

StudentJmbag $\rightarrow$ **Prosjek**. Znači da je prosjek moguće odrediti po JMBAG-u studenta.

Dijete $\rightarrow$ **Majka**. Svako dijete ima točno jednu majku. Atribut majke je funkcijski zavisn o atributu dijete. Ako se specificira dijete, samo je jedna moguća vrijednost majke. Funkcijsku zavisnost $A \rightarrow B$ je trivijalno odrediti ako $B \subseteq A$ (B je podskup ili jednak A).

Kada je više od jednog atributa potrebno kako bi se odredio drugi atribut unutar entiteta, ta determinanta se naziva **složena determinanta**.

Primjer:

Ocjena iz pismenog ispita i usmenog ispita koje postigne student tvore krajnju ocjenu iz nekog kolegija.

Pismeni ispit, usmeni ispit $\rightarrow$ **Ocjena iz kolegija**.

Zato što je više od jednog atributa potrebno da se odredi atribut krajnje ocjene, to se zove složena determinanta.

3.2.1. Potpuna funkcijska zavisnost

Skup atributa B **potpuno je funkcijski zavisn** o skupu atributa A relacijske sheme R ako:

- B funkcijski ovisi o A,
i
- ne postoji pravi podskup od A koji funkcijski određuje B.

Atribut je potpuno funkcijski zavisn o drugom atributu samo isključivo ako je funkcijski ovisan o drugom atributu, ali ne i još nekom drugom atributu ili podskupu drugih atributa.

Primjer:

Zadan je skup FZ $F = \{ XYZ \rightarrow MN, N \rightarrow F \}$.

Je li $\{ M, N \}$ potpuno funkcijski ovisan o $\{ X, Y, Z \}$?

Da, jer ne postoji skup $S \subseteq \{ X, Y, Z \}$ takav da $S \rightarrow \{ M, N \}$

3.2.2. Parcijalna (nepotpuna) funkcijska zavisnost

Neka je zadana relacijska shema R i skupovi atributa A i B iz R , to jest $A \subseteq R$, $B \subseteq R$. Neka u R vrijedi FZ $A \rightarrow B$.

FZ $A \rightarrow B$ je **nepotpuna** ako postoji skup atributa C koji je pravi podskup od A , za koji vrijedi $C \rightarrow B$.

Atribut je parcijalno funkcijski zavisn ako je potrebno koristiti podskup atributa unutar tablice kao složenu determinantu kako bi se identificirao atribut koji je zavisn.

Primjer:

Zadan je skup FZ $F = \{ XYZ \rightarrow M, YZ \rightarrow N, N \rightarrow M \}$.

Je li $\{ M \}$ potpuno funkcijski ovisan o $\{ X, Y, Z \}$?

Ne, jer postoji skup $\{ Y, Z \} \subseteq \{ X, Y, Z \}$ takav da $\{ Y, Z \} \rightarrow \{ M \}$

3.2.3. Tranzitivna zavisnost

Tranzitivna zavisnost postoji ako postoji posredna funkcijska zavisnost.

Neka je zadana relacijska shema R , skupovi atributa $A \subseteq R$, $B \subseteq R$, $C \subseteq R$. Skup atributa C je **tranzitivno ovisan o A** ako vrijedi:

$$A \rightarrow B, B \nrightarrow A \text{ i } B \rightarrow C$$

Primjer

Zadana je relacijska shema $R = \{ X, Y, Z, M, N, O, P \}$ i skup FZ

$$F = \{ XY \rightarrow ZM, M \rightarrow NO, ZM \rightarrow XYP \}$$

NO je tranzitivno funkcijski ovisan o XY , jer

$$XY \rightarrow M, M \rightarrow NO, M \rightarrow NO$$

P nije tranzitivno funkcijski ovisan o XY , jer iako

$$XY \rightarrow ZM, ZM \rightarrow P$$

nije zadovoljen uvjet $ZM \nrightarrow XY$

Primjer

Neka postoji tablica s atributima kao u nastavku:

Tablica 3. Primjer tablice Profesori

Profesori		
OIB	Radno mjesto	Plaća

Atributi unutar tablice su OIB koji je unikatan za svakog profesora, Radno mjesto profesora kao što je Asistent, predavač, viši predavač, redovni profesor i sl. Zadnji atribut je plaća pojedinog profesora.

Potrebno je napomenuti da OIB određuje radno mjesto. Radno mjesto očito određuje plaću. Npr. Viši predavač ima veću plaću od asistenta. Ali također OIB određuje plaću. Dakle, ispada sljedeće:

- OIB $\rightarrow$ Radno mjesto
- Radno mjesto $\rightarrow$ Plaća
- Plaća $\nrightarrow$ OIB
- OIB $\rightarrow$ Plaća

Dakle, postoji tranzitivna ovisnost između OIB-a i plaće.

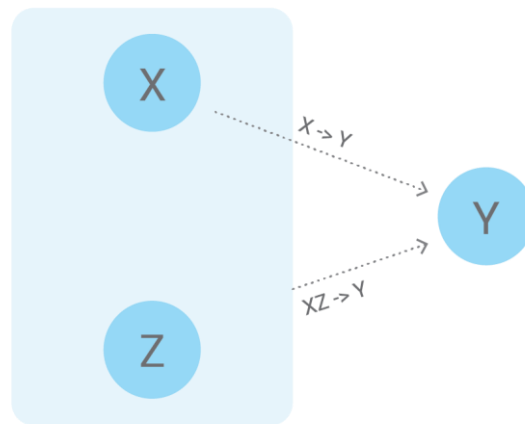
3.3. TRANSFORMACIJSKA PRAVILA FUNKCIJSKIH ZAVISNOSTI (ARMSTRONGOVI AKSIOMI)

3.3.1. Refleksivnost

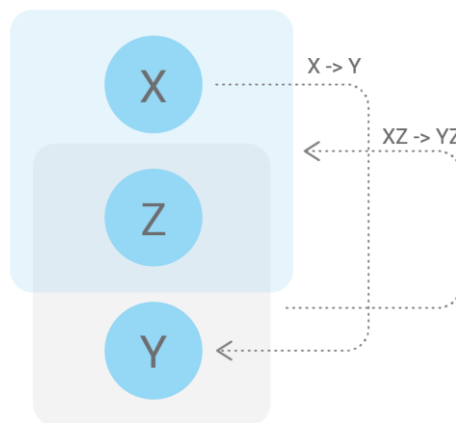
Ako je $Y \subseteq X$ (Y je podskup ili jednak X), tada je $X \rightarrow Y$. Aksiom refleksivnosti pokazuje da dani skup atributa sam sebi funkcijski određuje sve svoje podskupove.

3.3.2. Povećanje

Ako je $X \rightarrow Y$ i Z je podskup tablice R (Z je bilo koji skup atributa unutar tablice R), tada vrijedi $XZ \rightarrow YZ$. Aksiom povećanja ukazuje na to da je moguće lijevu stranu funkcijske zavisnosti ili obje strane jednostavno s jednim ili više atributa. Aksiom povećanja ne dozvoljava povećanje samo desne strane. Pravilo povećanja moguće je prikazati sljedećim izrazima:



Slika 17. Ako $X \rightarrow Y$ tada $XZ \rightarrow Y$



Slika 18. Ako $X \rightarrow Y$ tada $XZ \rightarrow YZ$

3.3.3. Tranzitivnost

Ako $X \rightarrow Y$ i $Y \rightarrow Z$, tada je $X \rightarrow Z$. Aksiom tranzitivnosti prikazuje da ako jedan atribut unikatno određuje drugi atribut i drugi atribut unikatno određuje treći atribut, tada prvi atribut određuje treći atribut. Otprilike bi bilo slično da postoje tri pravca X, Y i Z. Pravac X je paralelan s pravcem Y. Pravac Y je paralelan s pravcem Z. Tada je logično da je pravac X također paralelan s pravcem Z.

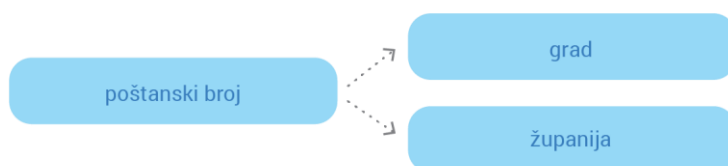
3.3.4. Prividna tranzitivnost

Ako $X \rightarrow Y$ i $YW \rightarrow Z$, tada je $XW \rightarrow Z$. Tranzitivnost je poseban slučaj prividne tranzitivnosti kada je W *NULL* vrijednost. Aksiom prividne tranzitivnosti je generalizacija aksioma tranzitivnosti.

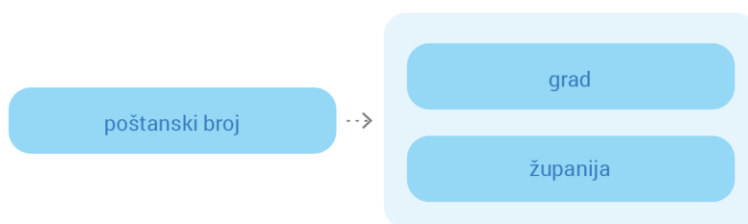
3.3.5. Unija

Ako je $X \rightarrow Y$ i $X \rightarrow Z$, tada je $X \rightarrow YZ$. Aksiom unije prikazuje da ako postoje dvije funkcijske zavisnosti s istom determinantom moguće je kreirati novu funkcijsku zavisnost koja zadržava determinantu i ima na svojoj desnoj strani uniju desnih strana dvije funkcijske zavisnosti.

Uniju je moguće prikazati na primjeru poštanskog broja i županije. Poštanski broj se koristi kako bi se identificirao grad, ali pomoću istog je također moguće identificirati i županiju. Ta tvrdnja implicira da poštanski broj određuje i grad i županiju.



Slika 19. Unija: Ako je Poštanski broj $\rightarrow$ Grad i Poštanski broj $\rightarrow$ Županija

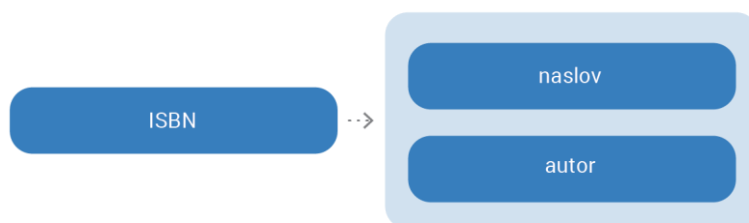


Slika 20. Unija: Tada je Poštanski broj $\rightarrow$ Grad, Županija

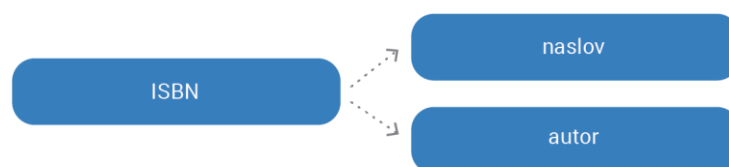
3.3.6. Dekompozicija

Ako je $X \rightarrow YZ$ tada je $X \rightarrow Y$ i $X \rightarrow Z$, odnosno obrnuto od unije. Aksiom dekompozicije prikazuje da determinanta bilo koje funkcijske zavisnosti može unikatno odrediti bilo koji atribut ili bilo koju kombinaciju atributa desne strane funkcijske zavisnosti.

Dekompoziciju je također moguće prikazati slično kao i uniju. Ovaj puta za primjer je uzet ISBN knjige (koji unikatno određuje knjigu). ISBN unikatno određuje naslov knjige i autora knjige (slično kao $X \rightarrow YZ$) što implicira da ISBN određuje naslov knjige ($X \rightarrow Y$) i ISBN određuje autora knjige ($X \rightarrow Z$).



Slika 21. Dekompozicija: Ako je $ISBN \rightarrow \text{Naslov, Autor}$



Slika 22. Dekompozicija: Tada je $ISBN \rightarrow \text{Naslov}$ i $ISBN \rightarrow \text{Autor}$

3.4. KONAČNI SKUP FUNKCIJSKIH ZAVISNOSTI

Dan je skup F svih funkcijskih zavisnosti za relaciju (tablicu) R . F^+ , konačnica od F je set svih funkcijskih zavisnosti koje su logično implicirane od strane F . Armstrongovi aksiomi su dovoljni da se izračuna sve iz F^+ , što znači da ako se primjene pravila Armstrongovih aksioma, moguće je naći sve funkcijske zavisnosti u F^+ .

Funkcijska zavisnost se može koristiti za testiranje relacije kako bi se uvidjelo jesu li relacije pravilne prilikom određenog seta funkcijskih zavisnosti. Ako je relacija (tablica) R pravilna prilikom korištenja seta F funkcijskih zavisnosti, tada relacija R zadovoljava F .

Funkcijska zavisnost specificira ograničenja (engl. *constraint*) na skupu pravilnih relacija.

3.5. NORMALIZACIJA BAZE PODATAKA

Normalizacija baze podataka je proces organiziranja podataka unutar baze. Taj proces uključuje modeliranje podataka sljedeći pravila koja su kreirana kako bi se podaci zaštitili i kako bi baza podataka bila što efikasnija pomoću eliminacije dva faktora: redundancije podataka i nedosljedne zavisnosti. Redundantni podaci rasipaju diskovni prostor i kreiraju probleme prilikom održavanja baze podataka. Ako podatak koji se nalazi na više mjesta mora biti promijenjen, podatak mora biti promijenjen na isti način na svim mjestima gdje je pohranjen. Nekonzistentne zavisnosti mogu onemogućiti ispravan pristup podacima jer putanja do podatka nedostajati.

Normalizacija je analiza funkcijskih zavisnosti između atributa i proces rastavljanja relacija s anomalijama kako bi se kreirale dobro strukturirane relacije. Dobro strukturirane relacije sadrže minimum redundancije i dozvoljavaju unos, izmjenu i brisanje podataka bez greške i nedosljednosti. Normalizacija je postupak za odlučivanje koji atributi trebaju biti grupirani zajedno u relaciju. To je postupak kojim se nostrificira (engl. *validate*) i unaprijeđuje logički model kako bi zadovoljio određena ograničenja kojima se izbjegava nepotrebno dupliranje podataka.

Teorija normalizacije baze podataka se svodi na koncept normalnih formi. Relacija je u nekoj određenoj normalnoj formi ako zadovoljava set pravila. Trenutno postoji šest definiranih normalnih formi. Normalizacijom baze podataka se ne smije uništiti integritet podataka, već samo ukloniti redundanciju i nekonzistentne zavisnosti. Normalizacijom se generalno govoreći generira više tablica s manje atributa.

3.5.1. Svrha normalizacije

Dobre strane normalizacije:

- Minimiziraju se anomalije prilikom unosa, izmjene i brisanja (očuvanje konzistentnosti i referencijskog integriteta baze)
- Izbjegavanje redundantnosti podataka na način da se jedan podatak pohranjuje na jedno mjesto unutar baze
- Pohranjivanje podataka na način koji je prikladniji za precizno izmjenjivanje
- Izbjegavanje anomalija prilikom promjene podataka
- Olakšavanje provođenja ograničenja nad podacima
- Izbjegavanje nepotrebnog pisanja programskog koda. Dodatno pisanje okidača, pohranjenih procedura i funkcija može biti potrebno kod ne normalizirane baze podataka te može biti razlog lošijim performansama baze.

Loše strane normalizacije:

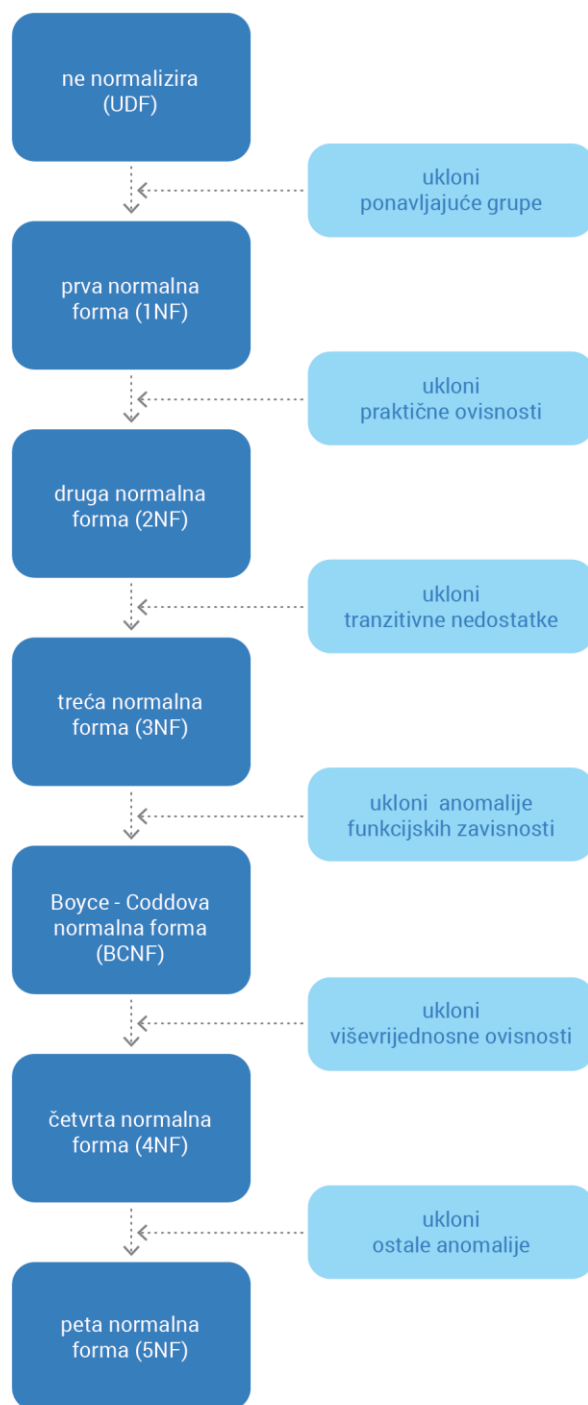
- Više relacija u bazi

- Više relacija može značiti lošije performanse zbog upita koji će morati spajati relacije
- Zahtjeva detaljnu analizu problema i modela baze podataka

3.5.2. Postupci normalizacije

Razina normalizacije baze podataka određena je normalnim formama. Što je viša normalna forma, potrebno je zadovoljiti više uvjeta. U sklopu udžbenika će biti obrađene: prva normalna forma (1NF), druga normalna forma (2NF) i treća normalna forma (3NF). Također postoje još i Boyce-Codd normalna forma, četvrta normalna forma (4NF), peta normalna forma (5NF) i šesta normalna forma (6NF) koja je razvijena 2002. godine.

Svaka normalna forma je skup predefiniranih uvjeta koje garantiraju neka određena svojstva koja se odnose na redundanciju podataka i anomalije prilikom promjene podataka. Treća normalna forma se smatra dovoljno dobrom. U nekim slučajevima, bazu podataka je potrebno denormalizirati kako bi se neki upiti izvršavali u kraćem vremenskom roku.



Slika 23. Prikaz normalizacije po normalnim formama

3.5.3. Prva normalna forma (1NF)

Relacijska shema R je u prvoj normalnoj formi (1NF) ako:

- svi atributi sadrže atomske (nedjeljive) vrijednosti,

- vrijednost svakog atributa je samo jedna vrijednost iz domene tog atributa,
- neključni atributi relacije funkcijski ovise o ključu relacije.

3.5.4. Druga normalna forma (2NF)

Relacijska shema R je u drugoj normalnoj formi (2NF) kada je:

- u prvoj normalnoj formi (1NF) i
- svaki atribut iz zavisnog dijela potpuno funkcijski ovisan o svakom ključu relacije.

3.5.5. Treća normalna forma (3NF)

Relacijska shema R je u trećoj normalnoj formi (3NF) kada je:

- u prvoj normalnoj formi (1NF) i
- niti jedan atribut iz zavisnog dijela nije tranzitivno funkcijski ovisan o bilo kojem ključu relacije.

3.5.6. Boyce-Coddova normalna forma

Relacijska shema R je u BCNF ako je:

- u prvoj normalnoj formi (1NF) i
- ako niti jedan atribut nije tranzitivno funkcijski ovisan o bilo kojem ključu relacije.

Tablica je u Boyce-Coddovoj normalnoj formi kada je u trećoj normalnoj formi i kada je svaka determinanta ključ kandidat. Ključ kandidat je podskup super ključa. Ključ kandidat je jedan atribut ili kombinacija atributa koja unikatno određuje svaku n-torku u tablici. Najmanje kombinacija atributa razlikuje ključ kandidat od super ključa. Kandidat ključ ne smije sadržavati NULL vrijednosti.

Iako je BCNF stroža od 3NF, rijetki su slučajevi da je relacijska shema u 3NF, a da istovremeno nije i u BCNF. Normalizaciju na BCNF nije nužno provoditi. Smatra se da shema baze podataka ima dobra svojstva ako zadovoljava 3NF.

3.5.7. Pretvorba nenormalizirane tablice u prvu normalnu formu (1NF)

Relacija u nastavku nije normalizirana jer ima ponavljajuću grupu („Lokacija“), što znači da vrijednosti nisu atomske. Križanje retka sa stupcem uvijek mora imati jednu vrijednost.

Tablica 4. Nenormalizirana relacija

Broj odjela	Odjel	Lokacija
1	Računarstvo	{Vrbik, Strojarsvo}
2	Informatika	{Vrbik, Voltino}
3	Elektrotehnika	Voltino
4	Građevina	Holjevćeva

Postoje tri načina pomoću kojih je ovu relaciju moguće prebaciti u prvu normalnu formu (1NF).

Prvi način:

Stupac „Lokacija“ sadrži jednu ili dvije vrijednosti. Stupac je moguće razdijeliti na dva stupca: „Lokacija1“ i „Lokacija2“ kao što je prikazano na slici dolje:

Tablica 5. 1NF – prvo rješenje

Broj odjela	Odjel	Lokacija1	Lokacija2
1	Računarstvo	Vrbik	Strojarsvo
2	Informatika	Vrbik	Voltino
3	Elektrotehnika	Voltino	NULL
4	Građevina	Holjevćeva	NULL

Nedostaci prvog rješenja:

- Možda će u budućnosti biti potrebno dodati više od dvije lokacije
- Prilikom SELECT naredbe potrebno je voditi računa o dvije lokacije
- NULL vrijednosti unutar druge lokacije

Drugi način:

Dodavanje ključa na stupce „Broj odjela“ i „Lokacija“ i dodavanje novih n-torki za svaku lokaciju kao što je prikazano na tablici:

Tablica 6. 1NF – drugo rješenje

Broj odjela	Odjel	Lokacija
1	Računarstvo	Vrbik
1	Računarstvo	Strojarstvo
2	Informatika	Vrbik
2	Informatika	Voltino
3	Elektrotehnika	Voltino
4	Građevina	Holjevska

Nedostaci drugog rješenja:

- Dolazi do ponavljanja vrijednosti
- Višak redaka u relaciji
- Veći trošak diskovnog prostora

Treći način:

Dekompozicija postojeće relacije na dvije relacije kao u nastavku:

Tablica 7. 1NF – treće rješenje

Broj odjela	Odjel	Broj odjela	Lokacija
1	Računarstvo	1	Vrbik
2	Informatika	1	Strojarstvo
3	Elektrotehnika	2	Vrbik
4	Građevina	2	Voltino
		3	Voltino
		4	Holjevska

Ovo rješenje zadovoljava i drugu normalnu formu (2NF).

3.5.8. Pretvorba tablice iz 1NF u 2NF

U nastavku je relacija *zaposlenici* koja je u 1NF. Kako bi zadovoljila 1NF, ključ joj je {Matbr, ImeDj}.

Tablica 8. Relacija u 1NF

Matbr	Prezime	Ime	ImeDj	DatRodDj
1234	Novak	Mario	Jasna	21.03.1991
1234	Novak	Mario	Tomislav	13.12.1995
1345	Marić	Ivana	Ivan	23.03.1992
1456	Horvat	Hrvoje	Petar	22.02.1991
1456	Horvat	Hrvoje	Ana	19.09.1994
1456	Horvat	Hrvoje	Ivan	05.11.1997

Rješenje pretvorbe dane relacije je podjela relacije na dvije nove zato što 2NF nalaže da svaki element koji nije ključ mora biti u potpunosti zavisn o primarnom ključu.

Tablica 9. 2NF rješenje

Matbr	Prezime	Ime
1234	Novak	Mario
1345	Marić	Ivana
1456	Horvat	Hrvoje

Matbr	ImeDj	DatRodDj
1234	Jasna	21.03.1991
1234	Tomislav	13.12.1995
1345	Ivan	23.03.1992
1456	Petar	22.02.1991
1456	Ana	19.09.1994
1456	Ivan	05.11.1997

$K_{Zaposlenik} = \{\text{Matbr}\}$
 $K_{DIJETE} = \{\text{Matbr}, \text{ImeDj}\}$

3.5.9. Pretvorba tablice iz 2NF u 3NF

Prvi primjer:

Kao što je već ranije napomenuto, tablica je u 2NF ako je u 1NF i ako svi ne ključni atributi ovise o ključu, to jest nema funkcijskih zavisnosti. Za ovaj primjer uzeta je relacija koja sadrži podatke o sobama studentskog doma i pripadajućih cijena smještaja ovisno o građevini (to jest zgradi) u kojoj se nalazi. ID je primarni ključ relacije „StudentskeSobe“. Relacija je u 2NF a ne u 3NF zato što ima tranzitivnu zavisnost $ID \rightarrow \text{Građevina}$, $\text{Građevina} \nrightarrow ID$, $\text{Građevina} \rightarrow \text{Cijena}$, odnosno $ID \rightarrow \text{Građevina} \rightarrow \text{Cijena}$. Zbog tranzitivne zavisnosti moguće su anomalije. Na primjer, ako se ukloni redak s vrijednošću stupca ID 101 iz tablice, briše se i činjenica da soba u dodatnoj zgradi košta 500 (nečega, primjerice kuna).

Tablica 10. Relacija u 2NF

ID	Građevina	Cijena
100	Glavna	600
101	Dodatna	500
102	Nova	650

Rješenje pretvorbe u ovom primjeru je da se relacija „StudentskeSobe” razdvoji u dvije relacije. Razdvajanjem relacije u dvije zasebne relacije uklanja se i tranzitivna zavisnost te postoje samo zavisnosti ID→Građevina u jednoj relaciji i Građevina→Cijena u drugoj.

Tablica 11. 3NF – rješenje

ID	Građevina	Grad	Cijena
100	Glavna	Glavna	600
101	Dodatna	Dodatna	500
102	Nova	Nova	650

Drugi primjer:

U bazi podataka postoji relacije „Prodaja” koja ima attribute KupacID, Ime kupca, Prodavač i Grad kupnje. U relaciji KupacID određuje ime kupca, prodavača, i grad kupnje.

Tablica 12. Tablica u 2NF

KupacID	Ime kupca	Prodavač	Grad
1	Štef	Ivan	Krapina
2	Zec	Vuk	Šuma XYZ
3	Joža	Marko	Krapina
4	Bart	Apu	Springfield
5	Mali medo	Vuk	Šuma XYZ
6	Homer	Apu	Springfield

Funkcijske zavisnosti unutar relacije su kao u nastavku:

- KupacID→ Ime kupca
- KupacID→ Prodavač
- KupacID→ Grad kupnje

U relaciji je moguće primijetiti da osim navedenih funkcijskih zavisnosti postoji još jedna zavisnost, a to je da prodavač određuje grad (što je i logično jer prodavači žive u nekom određenom mjestu), a istovremeno prodavač ne određuje atribut KupacID. Dakle, postoji tranzitivna zavisnost KupacID→Prodavač→Grad.

Kao što je predviđeno trećom normalnom formom, relacija ne smije sadržavati tranzitivne zavisnosti, tako da je jedini način izbjegavanja iste podjela relacije na dvije nove:

Tablica 13. 3NF - rješenje

KupacID	Ime kupca	Prodavač	Prodavač	Grad
1	Štef	Ivan	Ivan	Krapina
2	Zec	Vuk	Marko	Krapina
3	Joža	Marko	Apu	Springfield
4	Bart	Apu	Vuk	Šuma XYZ
5	Mali medo	Vuk		
6	Homer	Apu		

3.5.10. Pretvorba 1NF tablice do 3NF

U sljedećem je primjeru relacija najмова u 1NF zato što ima primarni ključ {Klijent id, Stan id, Vlasnik id}. Da to nije slučaj, relacija bi bila ne normalizirana!

Tablica 14. Tablica najмова u 1NF

KlijentID	Prezime klijenta	StanID	Adresa	Početak najma	Kraj najma	Najam iznos	VlasnikId	PrezimeVlasnika
CR76	Matić	PG4	THB 5	01.07.00	31.08.01	350	CO40	Ivić
CR76	Matić	PG16	THB 6	01.09.01	01.09.02	450	CO93	Mornar
CR56	Perić	PG4	THB 5	01.09.99	10.06.00	350	CO40	Ivić
CR56	Perić	PG36	THB 4	10.10.00	01.12.01	375	CO93	Mornar
CR56	Perić	PG16	THB 6	01.11.02	10.08.03	450	CO93	Mornar

Kako bi se relaciju normaliziralo u 2NF normalnu formu potrebno je prvo uvidjeti sve funkcijske zavisnosti. Relacija je u 2NF samo ako je u 1NF i ne postoji ni jedna druga funkcijska zavisnost između atributa osim između primarnog ključa i atributa.

Tablica 15. Razdioba relacije najмова kako bi se zadovoljila 2NF

Klijent id	Prezime Klijenta	Klijent id	Stan ID	Početak najma	Kraj najma
CR76	Matić	CR76	PG4	01.07.00	31.08.01
CR56	Perić	CR76	PG16	01.09.01	01.09.02
		CR56	PG4	01.09.99	10.06.00
		CR56	PG36	10.10.00	01.12.01
		CR56	PG16	01.11.02	10.08.03

K={klijent id}

K={klijent id, Stan ID}

Stan ID	Adresa	Najam iznos	Vlasnik id	PrezimeVlasnika
PG4	THB 5	350	CO40	Ivić
PG16	THB 6	450	CO93	Mornar
PG36	THB 4	375	CO93	Mornar

K={Stan ID, Vlasnik id}

Kako bi se trenutne relacije dodatno normaliziralo u 3NF normalnu formu potrebno je prvo uvidjeti postoje li tranzitivne zavisnosti jer je njih potrebno izbaciti ako se želi zadovoljiti 3NF.

Tablica 16. Razdioba relacije najмова kako bi se zadovoljila 3NF

Klijent id	Prezime Klijenta
CR76	Matić
CR56	Perić

K={klijent id}

Klijent id	Stan ID	Početak najma	Kraj najma
CR76	PG4	01.07.00	31.08.01
CR76	PG16	01.09.01	01.09.02
CR56	PG4	01.09.99	10.06.00
CR56	PG36	10.10.00	01.12.01
CR56	PG16	01.11.02	10.08.03

K={klijent id, Stan ID}

Stan ID	Adresa	Najam iznos	Vlasnik id
PG4	THB 5	350	CO40
PG16	THB 6	450	CO93
PG36	THB 4	375	CO93

K={Stan ID, Vlasnik id}

Vlasnik id	PrezimeVlasnika
CO40	Ivić
CO93	Mornar
CO93	Mornar

K={Vlasnik id}

4. Tipovi podataka

Kao što se moglo pročitati u prošlom poglavlju osim naziva atributa jedno od najvažnijih njegovih svojstva je tip podatka koji će pohraniti. Tipovi podataka su se kroz standarde i godine razvoja i unapređivanja baza podataka mijenjali i nadopunjavali. Najveći problem je taj što su proizvođači DBMS sustava kreirali mnoge vlastite tipove podataka koji naravno nisu kompatibilni s drugim proizvođačima. U ovom poglavlju biti će opisani tipovi podataka za MySQL bazu podataka.

4.1. NUMERIČKI TIPOVI

Numerički tipovi podataka koriste se prilikom pohranjivanja bročanih vrijednosti i bitova u bazu podataka. Postoje cjelobrojni tipovi, tipovi s fiksnom decimalnim zarezmom, tipovi s pomičnim decimalnim zarezmom i bit vrijednosti.

4.1.1. Cjelobrojni tipovi

MySQL baza podataka podržava standardne cjelobrojne tipove podataka INTEGER (ili INT) i SMALLINT. Osim standardnih cjelobrojnih tipova MySQL podržava TINYINT, MEDIUMINT i BIGINT. U nastavku slijedi usporedba cjelobrojnih tipova podataka:

Tablica 17. Usporedba cjelobrojnih tipova

Tip	Veličina (Bytes)	Minimalna moguća vrijednost (signed/unsigned)	Maksimalna moguća vrijednost (signed/unsigned)
TINYINT	1	-128 / 0	127 / 255
SMALLINT	2	-32768 / 0	32767 / 65535
MEDIUMINT	3	-8388608 / 0	8388607 / 16777215
INTEGER ili INT	4	-2147483648 / 0	2147483647 / 4294967295
BIGINT	8	-9223372036854775808 / 0	9223372036854775807 / 18446744073709551615

4.1.2. Tipovi s fiksnim decimalnim zarezmom

MySQL podržava DECIMAL (ili DEC) tj. NUMERIC tip s fiksnim decimalnim zarezmom. Znači DECIMAL i NUMERIC tipovi su zapravo identični te ne postoji nikakva razlika (u danjem tekstu DECIMAL). DECIMAL tip podatka koristi se kada je potrebno pohraniti točnu preciznost broja (npr. financijski podaci). Do MySQL verzije 5.0.3. vrijednost DECIMAL tipa bio je pohranjen u obliku stringa, a nakon toga u binarnom formatu jer zauzima manje mjesta. Prilikom deklaracije DECIMAL tipa podatka potrebno je odrediti skalu i preciznost. Ako je primjerice

potrebno zapisati podatak koji ima ukupno 7 brojeva, a zadnja dva su iza decimalnog zareza odnosno u rang od -99999,99-99999,99, potrebno je definirati tip atributa kao u nastavku:

```
nazivStupca DECIMAL (7, 2)
```

Do verzije MySQL-a 5.0.3. za isti podatak je trebalo definirati jednu decimalu više zato što je predznak ulazio u definiciju podatka.

4.1.3. Tipovi s pomičnim decimalnim zarezom

Tipove s pomičnim decimalnim zarezom zastupaju FLOAT i DOUBLE (ili DOUBLE PRECISION ili REAL) tipovi podataka. FLOAT tip podatka se pohranjuje s 4 bajta, dok se DOUBLE pohranjuje s 8 bajtova. Ovi tipovi podataka sadrže aproksimativne vrijednosti i nisu pogodni za pohranjivanje preciznih podataka u financijskim podacima. MySQL također dopušta nestandardnu sintaksu sličnu kao kod DECIMAL tipa podatka. Kada je potrebno zapisati podatak koji ima ukupno 7 brojeva, a zadnja 4 su iza decimalnog zareza moguće je koristiti sintaksu kao u nastavku:

```
nazivStupca FLOAT (7, 4)
```

Prilikom takve definicije stupca potrebno je obratiti pozornost na zaokruživanje prilikom pohrane brojeva. Ako se pokuša unijeti broj 999,00009 u stupac tipa FLOAT(7,4), doći će do zaokruživanja prilikom pohrane, te će vrijednost naposljetku biti 999,0001. Također je potrebno obratiti pozornost prilikom pisanja upita i filtriranja vrijednosti na FLOAT i DOUBLE tipovima podataka jer nisu pohranjene kao točne vrijednosti već aproksimacije.

Tablica 18. Usporedba tipova podataka s pomičnim zarezom

Tip	Veličina (bytes)	Minimalne i maksimalne vrijednosti
FLOAT	4	-3.402823466E+38 do -1.175494351E-38, 0, 1.175494351E-38 do 3.402823466E+38
DOUBLE ili DOUBLE PRECISION ili REAL	8	-1.7976931348623157E+308 do 2.2250738585072014E-308, 0, 2.2250738585072014E-308 do 1.7976931348623157E+308

4.1.4. BIT tip podatka

Tip podatka BIT (BOOL ili BOOLEAN) je zaslužan za pohranu jednog ili više bitova u bazu podataka. BIT je moguće definirati na način da pohranjuje od 1 – 64 bitova u jedno polje. Ako je primjerice potrebno pohraniti samo jedan bit, atribut bi se definirao na sljedeći način:

```
nazivStupca BIT
```

Ako je potrebno pohraniti npr. 6 bitova, koristi se sljedeća sintaksa:

```
nazivStupca BIT(6)
```

Kako bi se pohranila BIT vrijednost u bazu podataka prilikom INSERT naredbe potrebno je koristiti sljedeću sintaksu:

```
INSERT INTO tablica (bitVrijednost) VALUES (b'101010');
```

4.1.5. Atributi numeričkih tipova podataka

Prikaz numeričkih podataka

MySQL podržava dodatne opcije za prikaz numeričkih tipova podataka. Sve cjelobrojne tipove je osim na klasičan način moguće definirati da prikazuju određen broj znamenki. Ako je potrebno da INT tip podatka prikazuje 4 znamenke, atribut je potrebno definirati na sljedeći način:

```
nazivStupca INT(4)
```

Na identičan način je moguće definirati prikaz svih ostalih cjelobrojnih tipova. Širina prikaza podataka neće zabraniti unos većeg podatka od navedenog, a također neće zabraniti prikaz većih vrijednosti. Širina prikaza služi aplikaciji koja se spaja na bazu podataka kako bi jednostavnije prikazala vrijednosti jer se širina vraća zajedno s rezultatima upita u meta podacima.

Atribut ZEROFILL

Dodatak na širinu tj. prikaz određenog broja znamenki je atribut ZEROFILL. On omogućava da se prazna polja određena širinom ispune nulama. Ako je primjerice potrebno broj 5 pokazati prilikom ispisa kao „0005” moguće je definirati atribut kao u nastavku:

```
nazivStupca INT(4) ZEROFILL
```

Atribut ZEROFILL se ignorira ako se podatak koristi u UNION upitu ili izrazu. Čim se atribut ZEROFILL dodjeli nekom numeričkom tipu, on automatski postaje i UNSIGNED.

Atribut UNSIGNED

Kako bi se koristile vrijednosti bez predznaka (samo pozitivne) potrebno je koristiti atribut UNSIGNED. Npr:

```
nazivStupca INT UNSIGNED
```

U ovom slučaju INTEGER će umjesto vrijednosti -2147483648 do 2147483647 moći poprimiti vrijednosti 0 – 4294967295.

Atribut AUTO_INCREMENT

Ako je potrebno da se cjelobrojn timer tipu podatka ili tipu podatka s pomičnim decimalnim zarezom automatski dodjeljuje vrijednost, potrebno je koristiti atribut AUTO_INCREMENT.

AUTO_INCREMENT će za svaki novi redak umjesto korisnika dodijeliti sam vrijednost veću za 1 od zadnjeg unesenog retka. Atribut AUTO_INCREMENT može se koristiti samo na stupcu tj. atributu koji je primarni ključ tablice. O primarnim ključevima biti će više govora u narednim poglavljima. Primjer definiranja stupca s AUTO_INCREMENT atributom:

```
sifStupca INT AUTO_INCREMENT KEY
```

4.2. TIPOVI PODATAKA ZA POHRANU DATUMA I VREMENA

4.2.1. DATE

DATE tip podatka se koristi kada je potrebno pohraniti datum ali ne i vrijeme. MySQL prikazuje datum u obliku 'GGGG-MM-DD' formatu (na primjer ako je datum 28.04.2016, datum će biti prikazan kao '2016-04-28'). Datum smije biti u rangu od '1000-01-01' do '9999-12-31'. Dozvoljen je „ležeran“ unos podataka u obliku stringa, što bi značilo da se dvotočka može koristiti kao graničnik (engl. *delimiter*) za vrijeme ali i za datum. Ako se radi o DATE DATE tipu podatka, vrijednost '10:11:12' će biti interpretirana kao '2010-11-12'. Ako se unosi vrijednost koju nije moguće interpretirati kao što je '10:45:15', zapisat će se vrijednost '0000-00-00' zato što 45 nije legalan mjesec u godini.

Ovo navodi do zaključka da je moguće unositi godine s dvije znamenke. Dakle ispravno je napisati datum u obliku '16-04-28' te će on biti interpretiran kao '2016-04-28'). Prilikom unosa datuma u formatu gdje godina sadrži dvije znamenke kao '10-03-10', potrebno je voditi računa o interpretaciji godine. MySQL interpretira dvoznamenkaste godine tako da su vrijednosti od 00 do 69 prevedene u godine 2000-2069, a vrijednosti 70-99 su prevedene u godine 1970-1999. Prilikom unosa datuma, MySQL će također provjeriti ispravnosti datuma, tako da je. nemoguće unijeti datum 30.02.2012. jer on ne postoji). Pokušaj unosa neispravnog datuma vratiti će upozorenje korisniku te ispuniti datumsko polje sa vrijednosti '0000-00-00'.

4.2.2. DATETIME

Tip podatka DATETIME koristi se kada je potrebno pohraniti vrijednosti datuma i vremena u jednom stupcu. MySQL prikazuje DATETIME vrijednosti u obliku 'GGGG-MM-DD SS-mm-ss'. DATETIME može prihvatiti vrijednosti od '1000-01-01 00:00:00' do '9999-12-31 23:59:59'.

4.2.3. TIMESTAMP

TIMESTAMP tip podatka sličan je DATETIME tipu podatka te se također koristi prilikom pohranjivanja datuma i vremena u istom polju. Atribut definiran kao TIMESTAMP može pohraniti vrijednosti u vremenskom periodu od '1970-01-01 00:00:01' do '2038-01-19 03:14:07'. TIMESTAMP pohranjuje vrijeme u UTC (engl. *Cordinated Universal Time*), koje je zapravo nasljednik srednjeg vremena po Greenwichu. Prilikom prikaza vremena, MySQL ponovo pretvara vrijeme iz UTC-a u lokalnu vremensku zonu. To je iz razloga što se

TIMESTAMP polje koristi prilikom pregledavanja kada je neki zapis posljednji puta promijenjen. Zato nije potrebno popunjavati vrijednost TIMESTAMP tipa podatka, već će se on svaki puta automatski promijeniti. Ako je ipak nužno promijeniti postavke, to se može napraviti s `DEFAULT CURRENT_TIMESTAMP` i `ON UPDATE CURRENT_TIMESTAMP` atributima. Ako je potrebno da se prilikom svakog pohranjivanja ili promjene podatka promijeni vrijednost TIMESTAMP polja potrebno je definirati atribut na jedan od načina:

```
nazivStupca TIMESTAMP
```

ili

```
nazivStupca TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP
```

Ako je potrebno da se TIMESTAMP vrijednost mijenja samo prilikom unosa nove n-torke u tablicu, treba ga definirati na sljedeći način:

```
nazivStupca TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Ako je potrebno da se prilikom unosa nove n-torke ne upisuje automatska vrijednost TIMESTAMP polja, ali je potrebno da se mijenja prilikom promjene postojećeg podatka, definiciju je potrebno napisati na sljedeći način:

```
nazivStupca DEFAULT 0 ON UPDATE CURRENT_TIMESTAMP
```

Naposlijetku, ako se ne želi dodjeljivati ikakva automatska vrijednost TIMESTAMP polju, potrebno ga je definirati na sljedeći način:

```
nazivStupca TIMESTAMP DEFAULT 0
```

4.2.4. TIME

Tip podatka TIME koristi se za pohranjivanje vremena. Vrijeme se pohranjuje u obliku 'SS:MM:ss' ili 'SSS:MM:ss' za velike vrijednosti. Unutar TIME tipa podatka mogu se pohraniti vrijednosti od '-838:59:59' do '838:59:59' zato što je TIME polje namijenjeno ne samo pohraniti vremena unutar dana već i vremenskih intervala između dva događaja. Posebnu pozornost potrebno je obratiti prilikom unosa vrijednosti u TIME polje. Ako je u polje potrebno pohraniti '11:12', MySQL će navedenu vrijednost interpretirati kao '11:12:00' a ne '00:11:12'. Također ako se pokuša pohraniti vrijednost '1112', MySQL će ju interpretirati kao '00:11:12' umjesto kao '11:12:00'. Ako se pokuša unijeti vrijednost veća od dopuštene, kao na primjer '860:00:00' MySQL će je pohraniti kao '838:59:59'. Slično tome događa se i s najmanjom vrijednošću, odnosno ako se pokuša unijeti '-860:00:00', pohranit će se vrijednost '-838:59:59'.

4.2.5. YEAR

Tip podatka YEAR služi za pohranu godina u bazi podataka. YEAR je moguće definirati kao YEAR(2) i YEAR(4), a razlika je u tome da YEAR(2) prihvaća dvije znamenke za godinu, dok YEAR(4) prihvaća četiri znamenke. Raspon vrijednosti koje je moguće pohraniti u YEAR(4) je 1901 do 2155 i 0000. Ako se radi o YEAR(2) formatu, moguće je pohraniti vrijednosti od 00-99 s naznakom da 00-69 kada se pretvore u godinu s četiri znamenke

označavaju 2000-2069 godinu, dok 70-99 označavaju 1970-1999 godinu. Svaki krivo uneseni podatak prilikom pohranjivanja će se pretvoriti u 0000.

4.2.6. Prikaz djelića sekunde

Od verzije MySQL-a 5.6.4. (trenutno nije dostupna u završnoj verziji) dostupna je mogućnost dodavanja djelića sekunde (do mikro sekunde) na tipove podatka TIME, DATETIME i TIMESTAMP. Ovisno o tome kolika je preciznost potrebna, ovi tipovi podataka mogu se definirati s bročanim atributom. Na primjer ako je potrebna preciznost u mikro sekundu za tip podatka DATETIME, potrebno ga je definirati na slijedeći način:

```
nazivStupca DATETIME (6)
```

4.3. TEKSTUALNI TIPOVI PODATAKA

4.3.1. CHAR i VARCHAR

Tipovi podataka CHAR i VARCHAR služe za pohranjivanje teksta te su slični, ali se poprilično razlikuju na način kako se pohranjuju i povlače iz baze podataka. Također su različiti po maksimalnoj duljini. CHAR može pohraniti 0-255 znakova, a VARCHAR može pohraniti 0-65.535 znakova. CHAR tip podatka ima fiksnu duljinu znakova, te će se svaki puta iskoristiti jednak broj bajtova po n-torci bez obzira na to je li zapisan podatak ili nije. VARCHAR je varijabilne veličine, te zauzeće prostora na disku ovisi o količini pohranjenih znakova u polju.

Tablica 19. Usporedba CHAR i VARCHAR tipova s obzirom na zauzeće na disku

Unesena vrijednost	CHAR(4)	Zauzeće na disku (bytes)	VARCHAR(4)	Zauzeće na disku (bytes)
' '	' '	4 bytes	' '	1 byte
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

Primjer definiranja VARCHAR tipa podatka:

```
nazivStupca VARCHAR(100) /*VARCHAR sa 100 mogućih mjesta*/
```

Primjer definiranja CHAR tipa podatka:

```
nazivStupca CHAR(100) /*CHAR sa 100 mogućih mjesta*/
```

CHAR i VARCHAR tipovima podataka moguće je ručno dodijeliti znakovni set (engl. *Character set*) u kojem će biti zapisan podatak i kolotaciju. Znakovni set je kodiranje znakova

i označava koji ASCII kod zamjenjuje koje slovo. Kolotacija (engl. *collation*) je način usporedbe znakova, odnosno sortiranja znakova po redu, ne po ASCII vrijednostima. Najvažniji znakovni setovi:

- latin2 (iso8859-2) - ISO standard za naše znakove
- cp1250 - Microsoft standard za naše znakove
- latin1 - obično osnovni standard

Najvažnije kolotacije:

- latin1_swedish_ci - obično osnovni standard
- cp1250_croatian_ci - po MS standardu
- latin2_general_ci - po ISO standardu

Primjer definiranja VARCHAR tipa podatka s kolotacijom i znakovnim setom:

```
nazivStupca VARCHAR(6) CHARACTER SET cp1250 COLLATE  
cp1250_croatian_ci
```

Sve raspoložive kolotacije moguće je prikazati pomoću naredbe:

```
SHOW COLLATION
```

Sve raspoložive znakovne setove moguće je prikazati pomoću naredbe:

```
SHOW CHARACTER SET
```

Atribut NATIONAL

Ako je potrebno koristiti veći raspon znakova od onog koliko znakovni set omogućava (na primjer jedna n-torka ima tekst na Kineskom, druga na Hrvatskom i slično) moguće je koristiti atribut NATIONAL. Prilikom korištenja atributa NATIONAL jako je bitno voditi računa o stvarnoj potrebi za istim jer umjesto jednog byte-a po znaku koristi dva. Također, na MySQL-u u slučaju korištenja atributa NATIONAL ne rade kolotacije. Primjer definiranja:

```
nazivStupca NATIONAL VARCHAR(100)
```

4.3.2. BINARY i VARBINARY

BINARY i VARBINARY su slični tipovi podataka CHAR i VARCHAR tipovima podataka, ali služe za pohranjivanje binarnih oblika podataka umjesto tekstualnih. Kao i kod CHAR i VARCHAR tipova, duljina im je ista, odnosno 0-255 za BINARY i 0-65.535 za VARBINARY. Naravno BINARY i VARBINARY nemaju znakovne setove i kolotacije. Primjer definiranja BINARY atributa:

```
nazivStupca BINARY(100) /*100 bajtova*/
```

Primjer definiranja VARBINARY atributa:

```
nazivStupca VARBINARY (100) /*100 bajtova*/
```

4.3.3. BLOB i TEXT

BLOB stoji za „binary large object“ i može pohraniti varijabilne količine podataka. Postoji četiri podtipa BLOB-a: TINYBLOB, BLOB, MEDIUMBLOB i LONGBLOB. Oni se razlikuju samo po tome koliko podataka mogu pohraniti. Identično je s TEXT-om. Postoji TINYTEXT, TEXT, MEDIUMTEXT i LONGTEXT. Usporedba veličina slijedi u tablici:

Tablica 20. Usporedba BLOB i TEXT tipova podataka

Tip podatka	Maksimalna veličina zapisa
TINYBLOB, TINYTEXT	256 bytes
BLOB, TEXT	65536 bytes
MEDIUMBLOB, MEDIUMTEXT	16 MB
LONGBLOB, LONGTEXT	4 GB

Za razliku od BLOB tipova podataka, TEXT kao i kod CHAR i VARCHAR tipova podataka ima znakovni set i kolotaciju. Iz razloga što su TEXT i BLOB tipovi podataka izuzetno dugački, tj. veliki prilikom sortiranja koristi se prvih 1024 znaka, te se indeksiranje ne koristi nad tim stupcima zbog performansi cijelog sustava. To je zapravo i najveća razlika između TEXT, BLOB tipova podataka i CHAR, VARCHAR, BINARY i VARBINARY tipova podataka. Primjer definiranja BLOB i TEXT atributa:

```
nazivStupca LONGTEXT CHARACTER SET cp1250 COLLATE
```

```
cp1250_croatian_ci/*tekst od 4GB s znakovnim setom i kolotacijom*/
```

BLOB:

```
nazivStupca BLOB /*binarni podaci veličine 65536 bytes*/
```

4.3.4. ENUM i SET

ENUM i SET tipovi podataka su nestandardni tekstualni tipovi podataka te bi ih se trebalo koristiti u najmanjoj mogućoj mjeri. Umjesto njih bi trebalo koristiti osnovne značajke baze podataka a to je referencijski integritet i relacije.

ENUM tip podatka zapravo funkcionira kao *one-to-many* relacija. Može poprimiti maksimalno 65.535 različitih elemenata koje je moguće odabrati prilikom INSERT ili UPDATE naredbe. Primjer kreiranja enumeracije:

```
nazivStupca ENUM('prvaVrijednost', 'drugaVrijednost',  
'nekaVrijednost')
```

Prilikom takve definicije ENUM atributa prilikom INSERT ili UPDATE naredbe moguće je unijeti vrijednosti „prvaVrijednost“, „drugaVrijednost“ ili „nekaVrijednost“.

SET tip podatka funkcionira kao *many-to-many* veza između dvije tablice koja se ostvaruje preko još jedne pomoćne tablice, o čemu će biti govora u danjim poglavljima. SET tipu podatka također treba dodijeliti početnu listu vrijednosti koje može poprimiti.

```
nazivStupca SET('prvaVrijednost', 'drugaVrijednost',  
'nekaVrijednost')
```

INSERT ili UPDATE je potrebno izvršiti na sljedeći način:

```
INSERT INTO test (sifra, nazivStupca)  
VALUES (115, ('prvaVrijednost', 'drugaVrijednost'))
```

Vrijednosti 'prvaVrijednost', 'drugaVrijednost' su u dodatnoj zagradi jer je to zapravo jedna vrijednost koja se upisuje u SET tip podatka.

5. Osnove SQL-a

SQL (izgovara se /'s kju: 'd/ ili /,si:kwəl/) je jezik relacijskih baza podataka koji služi za pristupanje podacima. To je neproceduralni odnosno **deklarativni jezik**, a nastao je kao potreba da se teorija napisana od strane Codd-a prenese u praksu. SQL je skraćenica koja stoji za engl. *Structured Query Language* odnosno strukturirani jezik za upite. To je jezik četvrte generacije što znači da je izuzetno lagan i orijentiran na poslovnu logiku i model same baze podataka. Prvi komercijalni sustav za upravljanje bazom podataka (DMBS) koji je podržavao je Oracle (1979. godina).

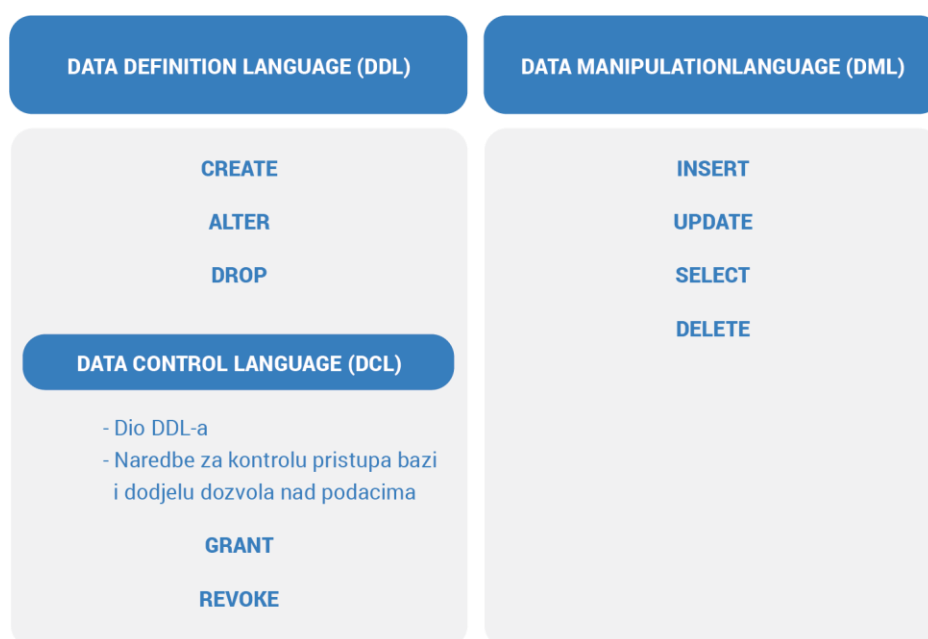
5.1. SQL STANDARDI

SQL se zbog odlične inicijalne ideje i arhitekture mijenjao poprilično sporo za razliku od ostalih programskih jezika i standarda. Postoji nekoliko SQL standarda:

1. ANSI X3.135-1986 (SQL-86) – prva verzija standarda objavljena 1986. godine. Verzija sadrži velik dio upitnog jezika, ali sa znatno ograničenim mogućnostima za definiciju podataka.
2. ANSI X3.135-1989 (SQL-89, SQL-1) – objavljen je 1989. godine. Uz mnoga proširenja, najvažnije je bilo uvođenje referencijskog integriteta.
3. ANSI X3.135-1992 (SQL-92, SQL-2) – objavljen je 1992. godine. Donosi mnoge nove elemente jezika kao što su naredbe UNION i JOIN, novi tipovi podataka: DATE, TIME, BIT i VARCHAR. Ovaj standard je donio najveće promjene od 1986. godine.
4. SQL:1999 (SQL-3) – objavljen je 1999. godine. Donosi nove mogućnosti poput ugradnje aktivnih pravila i okidača, rekurzivnih operacija i novih tipova podataka.
5. SQL:2003 (SQL-4) – objavljen je 2003. godine. Donosi neke dodatke na SQL-99 standard kao što su rad s XML-om unutar same baze, proširenja CREATE TABLE naredbi i MERGE naredba.
6. SQL:2006 (SQL-5) – objavljen je 2006. godine. Proširene su mogućnosti rada s XML podacima, integrirani su upiti u SQL kod pomoću XQuery izraza, W3 konzorcij je izdao XML upitni jezik.
7. SQL:2008 (SQL-6) – objavljen je 2008. godine. Donosi neke novosti kod MERGE, DIAGNOSTIC i TRUNCATE TABLE klauzulama. Također donosi poboljšani sustav za pretragu podataka pomoću regularnih izraza i XQuery izraza.
8. SQL:2011 (SQL-7) – objavljen je 2011. godine. Donosi nove elemente vezane uz temporalne baze podataka, tablice aplikativnih vremenskih intervala odnosno tablice ispravnosti, sistemski verzionirane tablice (transakcijske tablice).
9. SQL:2016 (SQL-8) – objavljen je 2016. godine. Donosi nove elemente kao primjerice funkcionalnosti u radu s JSON dokumentima, pretragu redaka regularnim izrazima, LISTAGG funkciju i tip podatka DECFLOAT.

Iako je zadnja revizija standarda SQL:2016, najrašireniji je i dalje SQL-92 standard koji zbog svoje složenosti nije u potpunosti prihvaćen ni u jednoj bazi podataka. Proizvođači komercijalnih DBMS sustava također ugrađuju svoje nestandardne DDL i DML naredbe pa tako postoji T-SQL (Microsoft), PL-SQL (Oracle, IBM) i mnogi drugi standardi. Nestandardne naredbe dovode do otežavanja oko suglasnosti budućih standarda i programski kod je neprenosiv između različitih DBMS sustava.

5.2. PODJELA NAREDBI U SQL JEZIKU



Slika 24. Podjela naredbi u SQL jeziku

SQL jezik moguće je podijeliti na dvije veće skupine:

1. **DDL** (engl. *Data Definition Language*) je dio SQL jezika za definiranje struktura baze podataka. DDL omogućava kreiranje, mijenjanje i brisanje entiteta (tablica), atributa (stupaca), baza podataka, relacija, procedura i funkcija.
2. **DML** (engl. *Data Manipulation Language*) je dio SQL jezika za manipulaciju podacima koji se nalaze u strukturi kreiranoj od strane DDL-a. DML omogućava umetanje podataka u tablice, ispravak podataka, brisanje podataka te projekcije podataka.

Unutar skupine DDL naredbi nalaze se **DCL** (engl. *Data Control Language*) naredbe. To su naredbe za kontrolu pristupa bazi odnosno dodjelu dozvola nad podacima, a predstavnici su **GRANT** i **REVOKE** naredbe. Detaljnije o njima biti će u poglavlju o zaštiti od neovlaštenog pristupa podacima.

5.3. DDL NAREDBE

Najvažnije DDL naredbe su:

1. **CREATE** – omogućava kreiranje novih struktura (baze podataka, tablica, funkcija...)
2. **ALTER** – omogućava promjenu prethodno kreiranih struktura
3. **RENAME** – mijenja naziv entitetu ili bazi podataka
4. **DROP** – briše strukturu i sve podatke unutar iste

5.3.1. CREATE

Za sada je dovoljno znati da se **CREATE** naredba koristi prilikom kreiranja baze podataka i prilikom kreiranja entiteta odnosno tablice.

5.3.2. CREATE DATABASE:

Osnovna sintaksa je:

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
[create_specification]
create_specification:
[DEFAULT] CHARACTER SET [=] charset_name
| [DEFAULT] COLLATE [=] collation_name
```

Jednostavan primjer SQL upita za kreiranja baze podataka:

```
CREATE DATABASE NazivBaze;
```

Naziv baze može biti u potpunosti proizvoljan. Preporuča se da se prilikom izrade baze napiše naredba kojom će se bazi odrediti skupine znakova za hrvatski jezik:

```
CREATE DATABASE NazivBaze
CHARACTER SET = cp1250
COLLATE = cp1250_croatian_ci;
```

5.3.3. CREATE TABLE

Osnovna sintaksa:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
(create_definition,...) [table_options]
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
[(create_definition,...)] [table_options] select_statement
```

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name {LIKE
old_tbl_name | (LIKE old_tbl_name)}
```

create_definition:

```
column_definition | [CONSTRAINT [symbol]] PRIMARY KEY [index_type]
(index_col_name,...) | {INDEX|KEY} [index_name] [index_type]
(index_col_name,...) | [CONSTRAINT [symbol]] UNIQUE [INDEX|KEY]
[index_name] [index_type] (index_col_name,...) |
{FULLTEXT|SPATIAL} [INDEX|KEY] [index_name] (index_col_name,...) |
[CONSTRAINT [symbol]] FOREIGN KEY [index_name]
(index_col_name,...) [reference_definition] | CHECK (expr)
```

column_definition:

```
col_name data_type [NOT NULL | NULL] [DEFAULT default_value]
[AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY] [COMMENT 'string']
[reference_definition]
```

Jednostavan primjer CREATE TABLE upita:

```
CREATE TABLE mjestoBezKljuca (
postBr INT,
nazivMjesto VARCHAR(50)
);
```

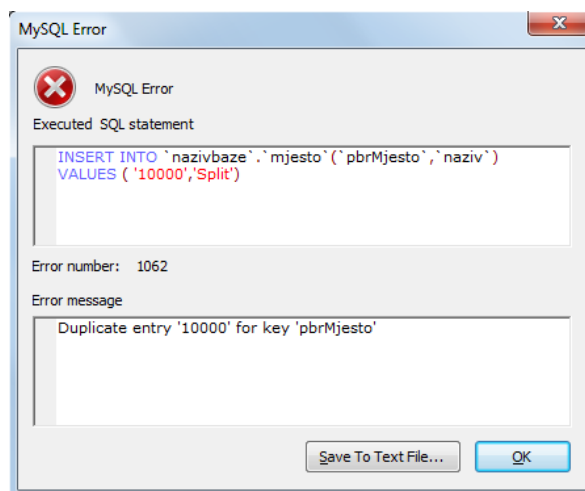
Problem kod tablice „mjestoBezKljuca“ jest da se ne može jednoznačno odrediti točan podatak unutar tablice jer svi stupci mogu sadržavati duplikate. Na taj način narušava se referencijski integritet podataka.

☐ pbrMjesto	naziv
☐ 10000	Zagreb
☐ 10000	Zagreb
☐ 10000	Split
☐ 42000	Zagreb
☐ 42000	Split
☐ 42000	Split

Primjer CREATE TABLE upita s primarnim ključem:

```
CREATE TABLE mjesta (
postBr INT,
nazivMjesto VARCHAR(50),
PRIMARY KEY postBr (postBr)
);
```

Proglašavajući stupac odnosno atribut primarnim ključem, nemoguće je unijeti dvije n-zorke s istim poštanskim brojem. Prilikom pokušaja unosa, DBMS će javiti grešku kao na idućoj slici.



Slika 25. Greška prilikom unosa iste vrijednosti primarnog ključa

Kako bi se stvorila relacija i održao referencijski integritet, osim primarnih ključeva potrebno je koristiti strane ključeve. Strani ključ neće dozvoliti da se unese podatak s vrijednosti koja ne postoji u referenciranoj tablici s primarnim ključem. To znači da ako se u tablicu „studenti“ pokuša unijeti vrijednost poštanskog broja „12345“, a on ne postoji u tablici „mjesto“, upis podatka neće se obaviti. Primjer kreiranja tablice sa stranim ključem je u nastavku:

```
CREATE TABLE studenti (
  jmbag CHAR(10) NOT NULL,
  ime VARCHAR(50) DEFAULT NULL,
  prezime VARCHAR(50) DEFAULT NULL,
  datumUpisa DATE NOT NULL,
  postBrPrebivanje INT(11) NOT NULL,
  PRIMARY KEY (jmbag),
  KEY fk_studenti_mjesto (postBrPrebivanje),
  CONSTRAINT fk_studenti_mjesto FOREIGN KEY (postBrPrebivanje)
REFERENCES mjesta (postbr) ON DELETE NO ACTION ON UPDATE NO ACTION
) ENGINE=INNODB DEFAULT CHARSET=cp1250 COLLATE=cp1250_croatian_ci
```

Osim dodavanja *constrainta* stranog ključa, u ovom primjeru mogu se primijetiti i neke druge opcije vezane uz stupce tj. attribute same tablice. Naredba COLLATE određuje jezik i postavke koje će se koristiti prilikom pohranjivanja i pretraživanja zapisa. Naredba DEFAULT NULL označava da ako se podatak ne upiše u stupac, on iznosi NULL vrijednost. O NULL vrijednostima će biti govora nešto kasnije u kolegiju.

5.3.4. ALTER

ALTER naredba se koristi prilikom mijenjanja već postojećeg entiteta odnosno tablice ili baze podataka.

ALTER DATABASE

Osnovna sintaksa:

```
ALTER {DATABASE | SCHEMA} [db_name]
    alter_specification ...

alter_specification:
    [DEFAULT] CHARACTER SET [=] charset_name
    | [DEFAULT] COLLATE [=] collation_name
```

Alter database u biti služi kako bi se promijenile postavke jezika i skupine znakova. Primjer takve promjene slijedi u nastavku:

```
ALTER DATABASE NazivBaze
CHARACTER SET = cp1250
COLLATE = cp1250_croatian_ci;
```

ALTER TABLE

Naredba ALTER TABLE služi kako bi se promijenio tip podatka atributa, brisali atributi, dodavali novi, kao i brisanju indeksa, ključeva, stranih ključeva i sl.

Osnovna sintaksa:

```
ALTER [ONLINE | OFFLINE] [IGNORE] TABLE tbl_name
    [alter_specification [, alter_specification] ...]

alter_specification:

ADD [COLUMN] col_name column_definition
    [FIRST | AFTER col_name ]
| ADD [COLUMN] (col_name column_definition,...)
| ADD {INDEX|KEY} [index_name]
    [index_type] (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    UNIQUE [INDEX|KEY] [index_name]
    [index_type] (index_col_name,...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    FOREIGN KEY [index_name] (index_col_name,...)
    reference_definition
| ALTER [COLUMN] col_name {SET DEFAULT literal | DROP DEFAULT}
| CHANGE [COLUMN] old_col_name new_col_name column_definition
    [FIRST|AFTER col_name]
| MODIFY [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
| DROP [COLUMN] col_name
| DROP PRIMARY KEY
| DROP {INDEX|KEY} index_name
| DROP FOREIGN KEY fk_symbol
```

```
| RENAME [TO] new_tbl_name
| CONVERT TO CHARACTER SET charset_name [COLLATE collation_name]
| [DEFAULT] CHARACTER SET [=] charset_name [COLLATE [=]
collation_name]
```

Primjer ALTER TABLE naredbe:

```
ALTER TABLE student
DROP COLUMN datUnos,
ADD COLUMN visina DECIMAL(5,2) DEFAULT NULL AFTER pbrRodjenja,
CHANGE COLUMN ime imeStudent VARCHAR(50) NOT NULL;
```

Gore napisana naredba radi sljedeće:

1. Odabire tablicu naziva „student“
2. Uklanja, odnosno briše atribut „datUnos“ kao i sve vrijednosti
3. Dodaje stupac odnosno atribut „visina“ koji je tipa „DECIMAL(5,2)“, dozvoljava da se smiju unositi NULL vrijednosti, a stupac dodaje nakon stupca „pbrRodjenja“
4. Mijenja stupac „ime“ u stupac „imeStudent“, mijenja tip podataka u „VARCHAR(50)“ i obvezuje unos podatka u stupac („NOT NULL“)

5.3.5. RENAME

RENAME naredba se koristi za mijenjanje naziva bazi podataka ili entitetu.

RENAME DATABASE

Osnovna sintaksa:

```
RENAME {DATABASE | SCHEMA} db_name TO new_db_name;
```

Primjer:

```
RENAME DATABASE NazivBaze TO NoviNazivBaze;
```

RENAME TABLE

Osnovna sintaksa:

```
RENAME TABLE tbl_name TO new_tbl_name
[, tbl_name2 TO new_tbl_name2] ...
```

Primjer:

```
RENAME TABLE studenti TO noviStudenti;
```

Moguće je promijeniti imena više tablica u istoj naredbi što je prikazano sljedećim primjerom:

```
RENAME TABLE studenti TO noviStudenti,  
mjesto TO novaMjesto;
```

5.3.6. DROP

Naredba drop se koristi prilikom brisanja entiteta, indeksa, funkcija, procedura, okidača, baze podataka i ostalih entiteta. Za sada će se obrađivati naredba koja briše entitet, tj. tablicu i bazu podataka.

DROP DATABASE

Naredba drop database se uvijek koristi u dolje navedenom obliku i nije ju moguće kombinirati s ostalim naredbama.

Osnovna sintaksa:

```
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name
```

Primjer:

```
DROP DATABASE NazivBaze;
```

DROP TABLE

Naredba drop table se uvijek koristi u dolje navedenom obliku i nije ju moguće kombinirati s ostalim naredbama.

Osnovna sintaksa:

```
DROP [TEMPORARY] TABLE [IF EXISTS]  
tbl_name [, tbl_name] ...
```

Primjer:

```
DROP TABLE NekaTablica;
```

5.4. DML NAREDBE

Popis DML naredbi:

1. INSERT – unos podataka u tablicu
2. UPDATE – izmjena postojećih podataka u tablici
3. SELECT – prikaz (ispis, dohvat) podataka
4. DELETE – brisanje podataka

5.4.1. INSERT

Naredba insert se koristi prilikom unosa nove n-torke u tablicu. Unos novih n-torki je moguće provesti na nekoliko načina ovisno o trenutnim potrebama i navikama samog programera baza podataka. Sintaksa naredbe slijedi u nastavku:

1. način:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [(col_name, ...)]
{VALUES | VALUE} ({expr | DEFAULT}, ...), (...), ...
[ ON DUPLICATE KEY UPDATE
  col_name=expr
  [, col_name=expr] ... ]
```

2. način:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
SET col_name={expr | DEFAULT}, ...
[ ON DUPLICATE KEY UPDATE
  col_name=expr
  [, col_name=expr] ... ]
```

3. način:

```
INSERT [LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [(col_name, ...)]
SELECT ...
[ ON DUPLICATE KEY UPDATE
  col_name=expr
  [, col_name=expr] ... ]
```

Primjeri:

1. način unosa n-torke u tablicu „mjesto“:

```
INSERT INTO mjesto VALUES (10153, 'Zagreb-Ravnice', 1);
```

Gornji primjer unosa nije optimalan ako se u međuvremenu promjeni ukupan broj stupaca u tablici ili ako se neke vrijednosti žele unijeti a neke ne. U tom slučaju potrebno je navesti koji se stupci žele popuniti. Primjer slijedi u nastavku:

```
INSERT INTO mjesto (postBr, nazivMjesto, idZupanija)
VALUES (10153, 'Zagreb-Ravnice', 1);
```

2. način unosa n-torke u tablicu „mjesto“:

```
INSERT INTO mjesto
SET postBr = 10153,
nazivMjesto = 'Zagreb-Ravnice',
idZupanija = 1;
```

3. način unosa n-torke u tablicu „mjesto“ pomoću SELECT naredbe:

```
INSERT INTO mjesto (postBr, nazivMjesto, idZupanija)
SELECT postBr + 1, nazivMjesto, idZupanija FROM mjesto
```

Naravno, gore prikazan primjer trenutno nema smisla jer su se pomoću SELECT naredbe n-torke praktički poduplale, ali SELECT naredba je vrlo moćna te se pomoću nje mogu dobiti raznorazne projekcije i presjeci između tablica što će biti prikazano u kasnijim poglavljima.

5.4.2. UPDATE

Naredba UPDATE koristi se prilikom izmjene jednog ili više redaka u tablici. Naredbu update je moguće kombinirati s ostalim klauzulama što će biti objašnjeno u sljedećim poglavljima.

Osnovna sintaksa UPDATE naredbe:

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference | table_references
  SET col_name1={expr1|DEFAULT} [, col_name2={expr2|DEFAULT}] ...
  [WHERE where_condition]
```

Primjeri:

```
UPDATE studenti
SET ime = 'Ivan',
prezime = 'Horvat';
```

Moguće je također mijenjati vrijednosti postojećim vrijednostima na temelju iste. Npr. Moguće je promijeniti sve ocjene da budu veće za vrijednost „1“:

```
UPDATE ocjene
SET ocjena = ocjena + 1;
```

Na ovaj način povećane su sve ocjene u tablici za vrijednost „1“ (čak i 5 na 6).

Ako je potrebno napisati upit koji će referencirati stupac prilikom promjene, a koju je također potrebno mijenjati, mora se paziti na redoslijed. Na primjer, potrebno je svim podacima promijeniti prvi atribut da bude veći za 2, a drugi atribut da bude kao promijenjeni prvi atribut umanjen za 3.

Trenutni podaci u tablici izgledaju kao na slici:

ID	prviAtribut	drugiAtribut
1	1	1
2	2	2
3	10	15
4	5	7
5	20	10

```
UPDATE test
SET prviAtribut = prviAtribut + 2,
```

```
drugiAtribut = prviAtribut - 3;
```

Nakon izvršavanja upita podaci moraju biti promijenjeni kao na slici:

☐	ID	prviAtribut	drugiAtribut
☐	1	3	0
☐	2	4	1
☐	3	12	9
☐	4	7	4
☐	5	22	19

Upit se ne smije napisati na sljedeći način jer će u tom slučaju prvo biti promijenjen drugi atribut, a potom prvi atribut što nije bilo traženo u zadatku.

```
UPDATE test
SET drugiAtribut = prviAtribut - 3,
    prviAtribut = prviAtribut + 2;
```

Podaci nakon izvršavanja upita izgledaju kao na slici:

☐	ID	prviAtribut	drugiAtribut
☐	1	3	-2
☐	2	4	-1
☐	3	12	7
☐	4	7	2
☐	5	22	17

5.4.3. SELECT

SELECT naredba je najmoćnija i najkorištenija naredba na svakoj relacijskoj bazi podataka. Ona omogućava projekcije podataka po želji, filtriranje podataka, spajanje više tablica u jednu projekciju, grupiranje podataka i tako dalje.

Osnovna sintaksa SELECT naredbe:

```
SELECT
    [ALL | DISTINCT | DISTINCTROW ]
    [HIGH PRIORITY]
    [STRAIGHT_JOIN]
    [SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
    [SQL_CACHE | SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
    select_expr [, select_expr ...]
    [FROM table_references]
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position}
    [ASC | DESC], ... [WITH ROLLUP]]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position}
    [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
    [PROCEDURE procedure_name(argument_list)]
    [INTO OUTFILE 'file_name'
    [CHARACTER SET charset_name]
```

```
export_options  
| INTO DUMPFILE 'file_name'  
| INTO var_name [, var_name]]  
[FOR UPDATE | LOCK IN SHARE MODE]]
```

Primjer SELECT naredbe kada je potrebno prikazati sve atribute tablice:

```
SELECT * FROM studenti;
```

ili striktno:

```
SELECT studenti.* FROM studenti;
```

Zvezdica u SELECT naredbi znači da se prikažu svi dostupni stupci iz tablice.

Ako je potrebno prikazati samo neke atribute pojedine tablice (npr. JMBAG, ime i prezime studenta):

```
SELECT jmbag, ime, prezime  
FROM studenti;
```

ili striktno:

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime  
FROM studenti;
```

Ovo je nekoliko najjednostavnijih primjera SELECT naredbe. Naredna poglavlja će se bazirati gotovo većinom na SELECT naredbi, te će se tada pokazati koliko je zapravo moćna.

5.4.4. DELETE

Naredba DELETE služi brisanju podataka iz tablice. Ne smije se nikako pomiješati s naredbom DROP koja se koristi za brisanje strukture. Dakle DELETE naredba briše podatke a strukturu ostavlja netaknutom, dok DROP naredba briše i podatke i strukturu zajedno. DELETE naredbom nije moguće brisati po stupcima već samo po recima. Po stupcima je moguće obrisati vrijednost pomoću UPDATE naredbe.

Osnovna sintaksa:

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name  
[WHERE where_condition]  
[ORDER BY ...]  
[LIMIT row_count]
```

Primjer brisanja svih podataka iz tablice ocjene:

```
DELETE FROM ocjene;
```

U praksi je jako rijetka potreba brisanja apsolutno svih podatke iz tablice, te je DELETE naredbu na ovaj način potrebno oprezno koristiti kako se podaci ne bi nepovratno obrisali. Naredba DELETE se najčešće koristi s WHERE klauzulom koja će biti objašnjena u kasnijim poglavljima.

6. Uvjeti i WHERE klauzula

Najčešće nije potrebno korisniku prikazati sve rezultate iz tablice, te najčešće nije potrebno promijeniti sve podatke unutar tablice. Prilikom filtriranja rezultata upita koristi se WHERE klauzula. Može se koristiti prilikom UPDATE, DELETE i SELECT naredbi. WHERE klauzula koristi operatore i funkcije kako bi filtrirala rezultate. Moguće je napisati bilo koju funkciju osim agregatne da bi se dobio željeni rezultat. Agregatne funkcije se ne smiju, tj. ne mogu koristiti unutar WHERE klauzule zato što one funkcioniraju na više rezultata, odnosno grupi rezultata, a WHERE će se izvršiti za svaku n-torku posebno.

6.1. WHERE U SELECT NAREDBI

Klauzulu WHERE u SELECT naredbi potrebno je uvijek smjestiti nakon FROM djela naredbe. Znači sintaksa treba izgledati ovako:

```
SELECT naziv_stupca|*[, naziv_stupca]...  
FROM ime tablice  
WHERE naziv_stupca operator vrijednost [AND|OR naziv_stupca  
operator vrijednost]...
```

Na primjer, potrebno je ispisati sva prezimena studenata čije je ime „Ivan“:

```
SELECT prezime  
FROM studenti  
WHERE ime = 'Ivan';
```

Rezultat izvršenog upita:

☐	prezime
☐	Horvat
☐	Turk
☐	Matić
☐	Zović
☐	Horvat
☐	Stojić
☐	Horvat

Prilikom pisanja upita može se primijetiti da je vrijednost „Ivan“ u navodnim znakovima. To je zato što se ovdje radi o stringu. Sve string vrijednosti potrebno je staviti u navodne znakove iz razloga što će ih SQL interpreter u protivnom zamijeniti ključnom riječi, te upit neće raditi. Ako je vrijednost broj, ne smije se postavljati u navodne znakove (iako MySQL to dopušta) zato što većina baza podataka neće prepoznati da se radi o bročanoj vrijednosti, već će pretpostaviti da se radi o stringu. U tom slučaju će najčešće vratiti grešku o usporedbi krivih tipova podataka.

Na primjer, potrebno je ispisati sve podatke o studentima koji imaju prebivanje u mjestu koje ima poštanski broj 40000:

```
SELECT *
FROM studenti
WHERE postBrPrebivanje = 40000;
```

Rezultati izvršenog upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0108065187	Tomislav	Obrovac	2017-09-26	40000	10000	7
☐	0108066602	Lorena	Fabijanić	2017-09-26	40000	22300	7
☐	0108067363	Filip	Kljaić	2016-09-28	40000	10000	8
☐	0313003717	Sladana	Kopačević	2017-09-27	40000	40000	7
☐	0313003860	Martina	Rak	2017-09-27	40000	47000	8
☐	0313004207	Luka	Skupnjak	2017-09-27	40000	47000	8
☐	0313004222	Marko	Keser	2013-09-25	40000	40000	8
☐	0313004277	Slavko	Bezeredi	2017-09-27	40000	34000	8
☐	0313004298	Ivan	Stojić	2017-09-27	40000	34000	7
☐	0313004387	Kristina	Copak	2017-09-27	40000	43000	7
☐	0313005214	Josip	Marin	2013-09-22	40000	10290	8
☐	0313006321	Nikola	Kupres	2014-09-25	40000	10290	7
☐	0313006332	Karla	Rolj	2013-09-24	40000	44000	8
☐	0313006854	Marko	Šarić	2014-09-24	40000	40000	7
☐	0313008547	Sandra	Tinjić	2013-09-23	40000	44000	8
☐	0313011025	Filip	Fadljević	2017-09-25	40000	44010	7

Osim mogućnosti točno određenog poštanskog broja, moguće je filtrirati podatke koji se nalaze u nekom rasponu vrijednosti. Na primjer, potrebno je prikazati sva mjesta koja imaju poštanski broj od 10000 do 20000:

```
SELECT *
FROM mjesta
WHERE postBr > 10000 AND postBr < 20000;
```

Osim pomoću dva uvjeta unutar WHERE klauzule, isti učinak je moguće postići pomoću BETWEEN operatora koji daje vrijednost „istina“ ako se broj nalazi unutar zadanih granica. BETWEEN uključuje granične vrijednosti u svoju pretragu, tj. ponaša se kao operator „>=“ ili „<=“. Dakle, potrebno je smanjiti granice za 1 ako se radi o prethodnom primjeru:

```
SELECT *
FROM mjesta
WHERE postBr BETWEEN 10001 AND 19999;
```

Rezultati izvršenog upita:

☐	postbr	nazivMjesto	idŽupanija
☐	10290	Zaprešić	1
☐	10295	Kupljenovo	2
☐	10310	Ivanić-Grad	2
☐	10315	Novoselec	2
☐	10340	Vrbovec	2
☐	10345	Gradec	2
☐	10410	Velika Gorica	2
☐	10430	Samobor	2

Ako je potrebno ispisati mjesta koja imaju poštanski broj koji nije između 10000 i 20000, to je moguće na nekoliko načina:

```
SELECT *
FROM mjesta
WHERE postBr NOT BETWEEN 10001 AND 19999;
```

ili

```
SELECT *
FROM mjesta
WHERE NOT(postBr > 10000 AND postBr < 20000);
```

ili

```
SELECT *
FROM mjesta
WHERE postBr <= 10000 OR postBr >= 20000;
```

Za ovaj primjer potrebno je ispisati sve studente koji se zovu „Marko“, „Ivan“ ili „Josip“:

```
SELECT *
FROM studenti
WHERE ime = 'Marko' OR ime = 'Ivan' OR ime = 'Josip';
```

Moguće je koristiti operator IN koji provjerava je li vrijednost određenog atributa svake pojedine n-torke u skupu vrijednosti:

```
SELECT *
FROM studenti
WHERE ime IN ('Marko', 'Ivan', 'Josip');
```

Rezultati izvršenog upita:

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 0016216326	Ivan	Horvat	2016-09-29	42000	10000	17
☐ 0035182233	Ivan	Turk	2013-09-26	10000	43000	3
☐ 0035187555	Marko	Krišto	2016-09-27	10410	10000	9
☐ 0036450215	Josip	Pavković	2014-09-24	10000	20000	14
☐ 0036460022	Ivan	Matić	2013-09-24	10000	10000	15
☐ 0036463254	Ivan	Zović	2015-09-27	10000	10000	14
☐ 0036463255	Ivan	Horvat	2015-09-28	10000	10410	14
☐ 0112047168	Marko	Erceg	2016-09-29	51000	10000	20
☐ 0246033365	Marko	Noskov	2013-09-24	10000	10000	4
☐ 0313004222	Marko	Keser	2013-09-25	40000	40000	8
☐ 0313004298	Ivan	Stojić	2017-09-27	40000	34000	7
☐ 0313005214	Josip	Marin	2013-09-22	40000	10290	8
☐ 0313006854	Marko	Šarić	2014-09-24	40000	40000	7
☐ 1191221499	Ivan	Horvat	2017-09-25	10000	10000	11

Ako je potrebno ispisati sve studente koji unutar svojeg prezimena sadrže niz znakova 'ba', to je moguće napraviti pomoći LIKE ili REGEXP operatora:

```
SELECT *
FROM studenti
WHERE prezime LIKE '%ba%';
```

ili s REGEXP operatorom:

```
SELECT *
FROM studenti
WHERE prezime REGEXP 'ba';
```

Rezultati izvršenog upita:

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 0036468547	Sandra	Labak	2008-09-23	10000	47000	15
☐ 0246022345	Ivana	Babić	2012-09-25	10000	10000	1
☐ 0246026699	Sonja	Batelić	2012-09-25	10000	53285	1
☐ 0283016670	Pavo	Baničević	2012-09-27	32000	31000	6
☐ 1191219570	Renato	Babojević	2012-09-25	10000	10000	11

Prilikom korištenja operatora LIKE i REGEXP potrebno je obratiti pozornost na ukupnu količinu podataka nad kojom se vrši filtriranje jer se za svaki podatak izvršava pretraga unutar stringa, te ona može trajati jako dugo, a cilj upita je da bude što kraći i efikasniji.

6.1.1. Korištenje funkcija u WHERE klauzuli:

Za ispis svih studenata iz tablice „studenti“ čije prezime počinje sa znakovnim nizom 'ba', moguće je koristiti nekoliko različitih funkcija:

```
SELECT *
FROM studenti
WHERE prezime LIKE 'ba%';
```

ili

```
SELECT *
FROM studenti
WHERE prezime REGEXP '^ba';
```

ili pomoću SUBSTRING funkcije

```
SELECT *
FROM studenti
WHERE SUBSTRING(prezime,1,2) = 'ba';
```

ili pomoću LEFT funkcije

```
SELECT *
FROM studenti
WHERE LEFT(prezime,2) = 'ba';
```

Rezultati izvršenog upita:

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 0246022345	Ivana	Babić	2012-09-25	10000	10000	1
☐ 0246026699	Sonja	Batelić	2012-09-25	10000	53285	1
☐ 0283016670	Pavo	Baničević	2012-09-27	32000	31000	6
☐ 1191219570	Renato	Babojević	2012-09-25	10000	10000	11

U WHERE klauzuli također je moguće koristiti datume i datumske funkcije. Na primjer, potrebno je dohvatiti podatak o studentu koji je upisan 28.09.2012. godine:

```
SELECT *
FROM studenti
WHERE datumUpisa = '2012-09-28';
```

Rezultati izvršenog upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
1191220001	Tina	Kostelić	2012-09-28	10000	32245	13

Potrebno je ispisati sve ustanove koje su osnovane u petom ili šestom mjesecu:

```
SELECT *
FROM ustanove
WHERE MONTH(datumOsnutka) = 5 OR MONTH(datumOsnutka) = 6;
```

ili

```
SELECT *
FROM ustanove
WHERE MONTH(datumOsnutka) IN (5, 6);
```

ili

```
SELECT *
FROM ustanove
WHERE datumOsnutka LIKE '%-05-%' OR
datumOsnutka LIKE '%-06-%';
```

Rezultati izvršenog upita:

oib	naziv	zRacun	adresa	datumOsnutka	postbr
08814003451	Tehničko Veleučilište u Zagrebu	2360000-1101350801	Vrbik 8	1998-05-27	10000
09032023114	Veleučilište Velika Gorica	2340009-1110122348	Zagrebačka cesta 5	2002-06-12	10410
28163265527	Prirodoslovno-matematički fakultet	2360000-1101522208	Horvatovac 102A	1946-06-08	10000
57029260362	Fakultet elektrotehnike i računarstva	2360000-1101218080	Unska 3	1956-06-08	10000
84477684422	Ekonomski fakultet	2390001-1100015220	Cvite Fiskovića 5	1974-06-15	21000

Na sličan način moguće je ispisati rezultate ustanova koja su osnovana tijekom 1974. godine:

```
SELECT *
FROM ustanove
WHERE datumOsnutka LIKE '1974%';
```

ili

```
SELECT *
FROM ustanove
WHERE YEAR(datumOsnutka) = 1974;
```

Rezultati izvršenog upita:

oib	naziv	zRacun	adresa	datumOsnutka	postbr
02024882310	Fakultet organizacije i informatike	2360000-1101741225	Pavlińska 2	1974-03-29	42000
84477684422	Ekonomski fakultet	2390001-1100015220	Cvite Fiskovića 5	1974-06-15	21000

Kada je potrebno ispisati sve studente koji su upisani 2012. godine, 24. dana u mjesecu, to je moguće napraviti na sljedeći način:

```
SELECT *
FROM studenti
WHERE YEAR(datumUpisa) = 2012 AND DAY(datumUpisa) = 24;
```

ili

```
SELECT *
FROM studenti
WHERE datumUpisa LIKE '2012%24';
```

Rezultati izvršenog upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0112055201	Nikola	Sušac	2012-09-24	51000	52210	20
☐	1191200784	Dubravko	Antunović	2012-09-24	10410	10410	10
☐	1191220782	Mare	Mistrić	2012-09-24	10000	10000	12
☐	1191220888	Nikolina	Orlović	2012-09-24	10410	10410	9
☐	1191221406	Ana	Andal	2012-09-24	10000	10000	11
☐	1191290785	Želimira	Ivček	2012-09-24	10000	31000	17

Na primjer, potrebno je ispisati sve studente koji su upisani između 24. i 26. dana u mjesecu uključujući i te dane:

```
SELECT *
FROM studenti
WHERE DAY(datumUpisa) BETWEEN 24 AND 26;
```

ili

```
SELECT *
FROM studenti
WHERE DAY(datumUpisa) >= 24 AND DAY(datumUpisa) <= 26;
```

Rezultati izvršenog upita (64 n-torke):

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0013020125	Matea	Omanović	2012-09-25	10410	44000	9
☐	0013025926	Antonio	Pavić	2012-09-25	32000	44000	5
☐	0016096972	Dora	Vidaković	2011-09-26	42000	44000	16
☐	0016233522	Nikola	Habijan	2011-09-26	42000	42000	16
☐	0016235684	Tomislav	Vukasović	2011-09-26	42000	23000	16
☐	0035166854	Krešo	Šišić	2011-09-26	10410	10410	10
☐	0035177990	Irena	Dabić	2012-09-26	10000	10000	1
☐	0035179953	Tanja	Sunek	2012-09-26	10000	10000	2
☐	0035180940	Sonja	Šimpraga	2011-09-26	10000	21000	4
☐	0035182233	Ivan	Turk	2008-09-26	10000	43000	3
☐	0036430456	Dragutina	Topić	2011-09-26	10000	43000	15
☐	0036440706	Vicka	Zečević	2011-09-26	10000	31000	14
☐	0036450215	Josip	Pavković	2009-09-24	10000	20000	14
☐	0036459870	Klaudija	Buljević	2009-09-25	10000	20000	15

Na primjer, potrebno je ispisati sve klijente koji su uneseni u horoskopskom znaku raka. (horoskopski znak raka je od 22.06. – 23.07.)

```
SELECT *
FROM ustanove
WHERE (MONTH(datumOsnutka)=6 AND DAY(datumOsnutka)>=22)
      OR (MONTH(datumOsnutka)=7 AND DAY(datumOsnutka)<=23);
```

ili

```
SELECT *
FROM ustanove
WHERE DATE_FORMAT(datumOsnutka, '%m%d') BETWEEN '0622'
AND '0723';
```

Rezultati izvođenja upita:

☐ oib	naziv	zRacun	adresa	datumOsnutka	postbr
☐ 2172082	Veleučilište "Ladislava Ružička"	2500009-110	županijska 50	2005-07-15	32000

Potrebno je ispisati sve nastavnike koji su rođeni u znaku raka. Pošto ne postoji datum rođenja, potrebno ga je preuzeti iz JMBG-a:

```
SELECT *
FROM nastavnici
WHERE (SUBSTRING(jmbg, 3, 2)='06' AND SUBSTRING(jmbg, 1, 2)>='22')
OR (SUBSTRING(jmbg, 3, 2)='07' AND SUBSTRING(jmbg, 1, 2)<='23');
```

ili

```
SELECT *
FROM nastavnici
WHERE STR_TO_DATE(jmbg, '%d%m') BETWEEN '0000-06-22' AND '0000-07-23';
```

Rezultati izvođenja upita:

☐ jmbg	ime	prezime	adresa	titu...	titulaIza	postBr	lozinka
☐ 0807977310602	Kristijan	Kolar	Jurija Dobrile		dipl.ing.r.	10000	25d7b6b412da198
☐ 0907987330057	Iva	Klarin	Gojranska ulica	dr.sc.	v. pred.	10000	8a92b02c4954bfb
☐ 1707963340011	Ankica	Čeh	Ulica kralja Pe		prof.	42000	0d94a5529152ca8
☐ 1907983383326	Jelena	Nikolić	Hvarska ulica 3	prof. dr		10000	dbc7336cf51d3cb
☐ 2207987301807	Josip	Dukić	Jurišićeva 21	mr.sc.		10000	9f50a136bc84fa1
☐ 2306978362104	Sonja	Duvnjak	Ulica žrtava fa	prof. dr		21000	dd4a0b6e486ded6
☐ 2806955260017	Branimira	Zagorec	Trg revolucije	prof. dr		22030	1fba6452ced5203

Na primjer, potrebno je ispisati sve nastavnike koji su rođeni u devetom, desetom ili jedanaestom mjesecu 1987. godine:

```
SELECT *
FROM nastavnici
WHERE SUBSTRING(jmbg, 5, 3) = '987'
AND SUBSTRING(jmbg, 3, 2) IN (9, 10, 11);
```

ili

```
SELECT *
FROM nastavnici
WHERE jmbg REGEXP '^([0-9]{3}[9,10,11]987.*$)';
```

ili

```
SELECT *
FROM nastavnici
WHERE jmbg LIKE '___9987%'
OR jmbg LIKE '___10987%'
OR jmbg LIKE '___11987%';
```

Rezultati izvođenja upita:

☐ jmbg	ime	prezime	adresa	titu...	titulaIza	postBr	lozinka
☐ 1409987320508	Zlatica	Vidakovi	Zvonimirova 32		prof.	10410	e046cb08b64f423.
☐ 2011987360005	Josip	Ivanovi	Zaharova ulica		prof.	10000	372ab64556ce24e
☐ 2610987300802	Karlo	Krsnik	Nodilova 7	dr.sc.		10000	c1586160da61466

Na primjer, potrebno je ispisati sve klijente kojima je danas rođendan:

```
SELECT *
FROM nastavnici
WHERE DATE_FORMAT(STR_TO_DATE(jmbg, "%d%m"), "%d%m") =
DATE_FORMAT(CURDATE(), "%d%m");
```

ili

```
SELECT *
FROM nastavnici
WHERE MONTH(STR_TO_DATE(jmbg, "%d%m")) = MONTH(CURDATE()) AND
DAY(STR_TO_DATE(jmbg, "%d%m")) = DAY(CURDATE());
```

ili

```
SELECT *
FROM nastavnici
WHERE LEFT(jmbg, 2) = DAY(CURDATE()) AND SUBSTRING(jmbg, 3, 2) =
MONTH(CURDATE());
```

ili

```
SELECT *
FROM nastavnici
WHERE LEFT(jmbg, 2) = DATE_FORMAT(CURDATE(), "%d") AND
SUBSTRING(jmbg, 3, 2) = DATE_FORMAT(CURDATE(), "%m");
```

ili

```
SELECT *
FROM nastavnici
WHERE jmbg REGEXP DATE_FORMAT(CURDATE(), "^%d%m.*$");
```

Napomena:

Potrebno je voditi računa prilikom korištenja funkcija u WHERE klauzuli. Funkcija će se izvršiti onoliko puta, koliko postoji n-torki bez WHERE klauzule. To znači da kada bi upit bez WHERE klauzule vraćao milijun redaka (što nije velika brojka za bazu podataka), funkcija će se izvršiti točno toliko puta. Pod pretpostavkom da je za izvršavanje funkcije potrebno jedna milisekunda, odnosno 0,001 sekunde po n-torci, upit će trajati preko 16,5 minuta samo za filtriranje podataka što u relacijskim bazama podataka najčešće nije prihvatljivo. Postoje sustavi za obradu podataka i transakcija kod kojih upiti dulje traju kako bi se dobili što

detaljniji izvještaji, te se ti upiti s duljim trajanjem najčešće obavljaju preko noći kada opterećenje na sustav nije toliko veliko.

6.2. WHERE U UPDATE I DELETE NAREDBI

WHERE klauzula u UPDATE i DELETE naredbi funkcionira identično kao i u SELECT naredbi, samo što će se umjesto prikaza podataka izvršiti zadano ažuriranje odnosno brisanje podataka.

Na primjer, potrebno je svim studentima imena „Marko“ promijeniti prezime u „Horvat“:

```
UPDATE studenti  
SET prezime = 'Horvat'  
WHERE ime = 'Marko';
```

Na primjer, studentu koji je na smjeru ID-a „2“, isti potrebno je promijeniti u „7“:

```
UPDATE studenti  
SET idSmjer = 7  
WHERE idSmjer = 2;
```

Na primjer, potrebno je obrisati sve studente imena „Marko“:

```
DELETE FROM studenti  
WHERE ime = 'Marko';
```

7. Operatori i funkcije

7.1. OPERATORI

Operatori se koriste u WHERE naredbi ili prilikom baratiranja varijablama. Popis operatora i funkcija koje se ponašaju kao operatori te njihova funkcionalnost slijedi u nastavku:

Tablica 21. Operatori

AND, &&	Logičko i
OR,	Logičko ili
=, >=, >, <, <=, != ili <>	Operatori kao u c-u ili Javi.
:=	Dodjeljivanje vrijednosti varijabli
BETWEEN ... AND ...	Provjera je li vrijednost unutar zadanih okvira
NOT BETWEEN AND	Negacija BETWEEN ... AND ...
DIV	Cjelobrojno dijeljenje
%, MOD	Ostatak cjelobrojnog dijeljenja
/	Operator dijeljenja
<=>	Znak jednakosti s mogućnošću rada s NULL vrijednostima
IS NOT NULL	Vrijednost koja nije NULL
IS NULL	Je li vrijednost NULL
IS	Provjerava boolean vrijednost
LIKE	Jednostavna usporedba stringa s uzorkom (engl. pattern)
NOT LIKE	Ne zadovoljava usporedbu stringa s uzorkom
SOUNDS LIKE	Koristi SOUNDEX funkciju za uspoređivanje riječi
NOT, !	Negira vrijednosti
REGEXP, RLIKE	Uspoređivanje vrijednosti pomoću regularnih izraza
GREATEST()	Vraća veći argument
IN	Je li vrijednost unutar skupine vrijednosti
INTERVAL()	Vraća indeks elementa koji je manji od prvog argumenta
ISNULL()	Vraća boolean vrijednost je li argument NULL
LEAST()	Vraća najmanji argument
STRCMP()	Vraća 0 ako su stringovi identični, -1 ako je prvi manji, 1 ako je drugi manji s obzirom na sortiranje.

Tablica 22. Operatori po prioritetu izvođenja

BINARY, COLLATE - pretvara nizove znakova u binarni niz ili kolotaciju
!
- (unarni minus), ~ (unarna inverzija bitova)
^ (XOR)
*, / ili DIV , % ili MOD
-, +
<<, >> & =, <=>, >=, >, <=, <, <> ili !=, IS , LIKE , REGEXP , IN
BETWEEN , CASE , WHEN , THEN , ELSE
NOT
&& ili AND
XOR
ili OR
:=

7.1.1. Operator LIKE i NOT LIKE

Operatori LIKE i NOT LIKE služe za usporedbu dva stringa (VARCHAR ili CHAR tipovi podataka). Ta usporedba nije osjetljiva na velika ili mala slova osim ako se striktno ne navede kolotacija koja je osjetljiva na velika ili mala slova. Popis kolotacija koje su osjetljive na velika i mala slova moguće je dobiti sljedećom naredbom:

```
SHOW COLLATION WHERE COLLATION LIKE '%_cs' /*cs je oznaka za case sensitive!*/
```

Kod operatora LIKE, ako se u uzorku za usporedbu (engl. *pattern*) koristi znak '%', on zamjenjuje **niz od jednog ili više znakova**. Na primjer:

- '10' LIKE '1%' -> istina
- '10abcdef' LIKE '1%' -> istina
- 'abc10def' LIKE '1%' -> laž
- 'abc10def' LIKE '%1%' -> istina (znak '%' na početku provjere treba izbjegavati zato što pretraga traje mnogo dulje)

Osim znaka '%' moguće je koristiti znak '_' koji zamjenjuje **jedan (bilo koji) znak**. Na primjer:

- '10' LIKE '1_' -> istina
- '10abcdef' LIKE '1_' -> laž
- '10abcdef' LIKE '1_____' -> istina
- 'abc10def' LIKE '___1%' -> istina

Operator NOT LIKE služi za istu funkcionalnost kao i LIKE, samo negirano. Dakle ono što je bilo istina za LIKE je laž za NOT LIKE i obrnuto.

7.1.2. REGEXP operator

Operator REGEXP je napredniji od LIKE zato što koristi regularne izraze za provjeru stringa s uzorkom. MySQL koristi POSIX ERE regularne izraze koji je poprilično jednostavan i rudimentaran za današnje standarde. Osnovni znakovi uzorka slijede u nastavku:

- ^ - početak niza,
- \$ -kraj niza,
- . -zamjenjuje jedan znak,
- a* - niz 0 do ∞ a znakova za redom,
- a+ -niz 1 do ∞ a znakova za redom,
- a? - a se pojavljuje ili ne pojavljuje,
- de | abc - ili se pojavljuje de ili abc niz,
- (abc)* - pojavljivanje niza abc 0 ili više puta

Na primjer:

- 'Bn' REGEXP '^Ba*n' -->istina
- 'Bn' REGEXP '^Ba+n' -->laž

Ostale posebne znakove i njihova objašnjenja moguće je naći na poveznici:

<http://www.cheatography.com/davechild/cheat-sheets/regular-expressions/>

7.2. FUNKCIJE

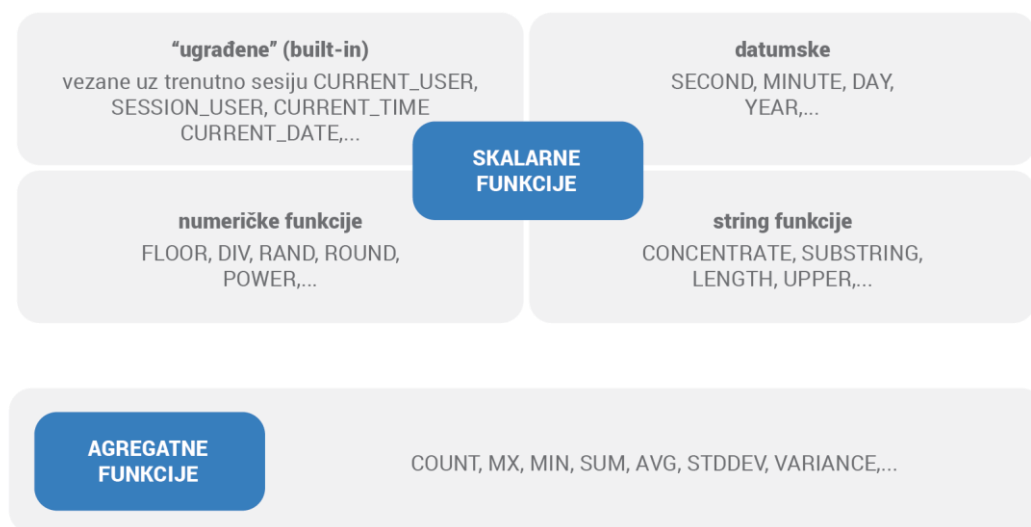
DBMS sustavi u sebi sadrže pohranjene i unaprijed definirane funkcije koje su velika pomoć prilikom rada s raznim oblicima podataka. Funkcije se mogu podijeliti na nekoliko različitih tipova s obzirom na tipove podataka s kojima rade i funkcionalnost koju pružaju. Osnovna podjela funkcija je na skalarne i agregatne, pri čemu vrijedi pravilo:

- **Skalarne funkcije** – rade nad jednom vrijednošću, rezultat je jedna vrijednost
- **Agregatne funkcije** – odnose se na operacije nad skupinom vrijednosti, za rezultat daju jednu vrijednost

Skalarne je funkcije moguće dodatno podijeliti na podgrupe:

- String funkcije
- Matematičke (numeričke) funkcije
- Datumske funkcije
- Ugrađene funkcije

Dodatno, postoje i **korisničke funkcije** koje nisu ugrađene u DBMS već iz izrađuju razvojni programeri u skladu s funkcionalnostima koje je potrebno nad podatcima obaviti. Mogu obrađivati jednu ili više vrijednosti, no uvijek će, kao i skalarne i agregatne funkcije, za rezultat vraćati samo jednu vrijednost.



Slika 26. Skalarne i agregatne funkcije

Sve funkcije moguće je koristiti u SELECT djelu upita. U WHERE djelu upita moguće je koristiti sve tipove funkcija osim agregatnih zato što one rade na određenom uzorku, tj. skupini podataka. Kao što je u SELECT naredbi moguće koristiti funkcije jednako ih je moguće koristiti u INSERT, UPDATE i DELETE naredbama.

Funkcije je moguće ugnježdjavati kako bi se dobila željena funkcionalnost. Također je moguće napisati vlastitu funkciju koja vraća neku računicu i u istoj koristiti ugrađene funkcije ili vlastite funkcije.

7.2.1. String funkcije

Služe za rad s tekstualnim tipovima podataka. Koriste za formatiranje, izrezivanje, mjerenje duljine teksta i slične operacije nad tekстом.

Tablica 23. Osnovne string funkcije

ASCII(str)	Vraća ASCII vrijednost znaka. Ako ih postoji više, vraća ASCII vrijednost prvog znaka.
BIN(n)	Vraća binarnu vrijednost broja u obliku stringa
BIT_LENGTH(str)	Vraća duljinu stringa u bitovima. (najčešće $8 \cdot$ ukupan broj znakova u stringu)
CHAR(number[, number])	Vraća string znak za ASCII vrijednost
CHAR_LENGTH(str) ili LENGTH(str)	Vraća broj znakova u string nizu
CONCAT(str1, str2,...)	Vraća string koji je spojen od argumenata funkcije
CONCAT_WS(separator, str1, str2, ...)	Vraća string koji je spojen od argumenata koji su odvojeni prvim argumentom koji predstavlja separator
FORMAT(X, D)	X je format kao npr: <code>###,###.##</code> , D je broj decimalnih mjesta
INSERT(str, pos, len, newstr)	U string str se ubacuje string newstr na mjesto pos duljine len
LCASE(str) ili LOWER(str)	Prebacuje sva slova stringa u mala slova.
UCASE(str) ili UPPER(str)	Prebacuje sva slova stringa u velika slova.
LEFT(str, len)	Vraća string sastavljen od len znakova s lijeve strane str.
RIGHT(str, len)	Vraća string sastavljen od len znakova s desne strane str.
REPLACE(str, old_str, new_str)	Mijenja string str na način da dio old_str zamjeni u new_str.

TRIM ([{ BOTH LEADING TRAILING }] [<i>remstr</i>] FROM str)	Uklanja višak znakova određen s remstr (ako ne piše podrazumijeva se razmak) sa lijeve (leading) , desne (trailing) ili obje(both) strane iz stringa str.
SUBSTRING (str,pos,len) ili SUBSTR (str, pos, len) ili MID (str, pos, len)	Iz stringa vadi string od pozicije pos za duljinu len. Duljina len je opcionalna, a ako se ne navede vraća se tekst do kraja originalnog stringa.

7.2.1.1. Najčešće korištene string funkcije

CONCAT() funkcija

Koristi se za spajanje dva ili više stringova u jednu cjelinu.

Primjer:

```
SELECT CONCAT('prva riječ, ', 'druga riječ, ', 'treća riječ.');
```

Upit će vratiti:

```
prva riječ, druga riječ, treća riječ
```

CONCAT_WS() funkcija

Koristi se za spajanje dva ili više stringova u jednu cjelinu s tim da je prvi argument separator.

Primjer:

```
SELECT CONCAT_WS(',', 'Upit je vratio 54', '57', '58', '65');
```

Upit će vratiti:

```
Upit je vratio 54, 57, 58, 65
```

LENGTH() funkcija

Koristi se za vraćanje ukupne duljine stringa.

Primjer:

```
SELECT LENGTH('1234567890')
```

Upit će vratiti:

```
10
```

LEFT() funkcija

Koristi se za izrezivanje stringa s lijeve strane. Prvi argument predstavlja string kojeg je potrebno izrezati, a drugi argument broj pozicija koji je potrebno izrezati.

Primjer:

```
SELECT LEFT('1234567890', 5)
```

Upit će vratiti:

```
12345
```

RIGHT() funkcija

Koristi se za izrezivanje stringa s desne strane. Prvi argument predstavlja string kojeg je potrebno izrezati, a drugi argument broj pozicija koji je potrebno izrezati.

Primjer:

```
SELECT RIGHT('1234567890', 5)
```

Upit će vratiti:

```
67890
```

INSERT() funkcija

Služi za umetanje jednog stringa u drugi. Prvi argument je string u kojeg je potrebno umetnuti tekst, drugi je argument pozicije na koji se želi umetnuti novi tekst, treći je broj pozicija koji će novi tekst preplaviti u prvom argumentu, a četvrti je tekst koji se umeće na u prvi argument.

Primjer:

```
SELECT INSERT('1234567890', 5, 2, 'Test')
```

Upit će vratiti:

```
1234Test7890
```

LCASE() ili LOWER() funkcije

LCASE i LOWER su sinonimne funkcije i rade identično. Služe za pretvorbu svih velikih slova u zadanom stringu u mala slova.

Primjer:

```
SELECT LCASE('TestNa ReČeNiCa 123.')
```

Upit će vratiti:

```
testna rečenica 123.
```

UCASE() ili UPPER() funkcije

UCASE i UPPER su sinonimne funkcije i rade identično. Služe za pretvorbu svih malih slova u zadanom stringu u velika slova.

Primjer:


```
SELECT UCASE('TestNa ReČeNiCa 123.')
```

Upit će vratiti:

```
TESTNA REČENICA 123.
```

REPLACE() funkcija

Služi za izmjenu određenog djela stringa u drugi dio stringa. Prvi argument je string kojeg je potrebno promijeniti, drugi argument je dio stringa prvog argumenta koji će se izmijeniti (ako postoji), treći argument je string koji će doći na mjesto drugog argumenta (ako je pronađen).

Primjer:

```
SELECT REPLACE('Danas je lijep dan', 'je lijep', 'nije ružan')
```

Upit će vratiti:

```
Danas nije ružan dan
```

Ako string nije pronađen:

```
SELECT REPLACE('Danas je lijep dan', 'xyz', 'nije ružan')
```

Upit će vratiti:

```
Danas je lijep dan
```

TRIM() funkcija

Funkcija služi za uklanjanje vodećih i pratećih razmaka na stringu. Jako često se koristi kod CHAR tipa podatka jer taj tip u bazu zapisuje prateće razmake nakon stringa. Moguće je i samostalno odrediti string kojeg je potrebno ukloniti.

Primjer:

```
SELECT TRIM(' Prva Druga ')
```

Upit će vratiti:

```
Prva Druga
```

TRIM() funkcija s određenim znakom:

```
SELECT TRIM(BOTH '.' FROM '...Prva.Druga...')
```

Upit će vratiti:

```
Prva.Druga
```

SUBSTRING() funkcija

Služi za izrezivanje podstringa (engl. substring) iz stringa. Prvi argument je string kojeg je potrebno izrezati, drugi argument je pozicija od koje se kreće, a treći argument je broj znakova od zadane pozicije.

Primjer:

```
SELECT SUBSTRING('Danas je lijep dan', 10, 5)
```

Upit će vratiti:

```
Lijep
```

7.2.2. Matematičke funkcije

Tablica 24. Popis matematičkih funkcija

ABS()	Vraća apsolutnu vrijednost
ACOS()	Vraća arc kosinus
ASIN()	Vraća arc sinus
ATAN2(), ATAN()	Vraća arc tangens
CEIL(), CEILING()	Vraća najmanju vrijednost cijelog broja, ne manju od argumenta
CONV()	Pretvara brojeve u različite baze (npr. 10, 16, 2)
COS()	Vraća kosinus
COT()	Vraća kotangens
CRC32()	Računa CRC32
DEGREES()	Pretvara radijane u stupnjeve
DIV	Cjelobrojno dijeljenje
EXP()	Eksponent
FLOOR()	Vraća najveći cijeli broj, ne veći od argumenta
LN(), LOG()	Vraća prirodni logaritam argumenta
LOG10()	Vraća logaritam baze 10 argumenta
LOG2()	Vraća logaritam baze 2 argumenta
% or MOD, MOD()	Vraća ostatak cjelobrojnog dijeljenja
OCT()	Vraća broj u oktalnom sustavu
PI()	Vraća vrijednost π

POW() , POWER()	Potenciranje
RADIANS()	Pretvara stupnjeve u radijane
RAND()	Vraća nasumičnu vrijednost
ROUND()	Zaokružuje argument
SIGN()	Vraća predznak argumenta
SIN()	Vraća sinus argumenta
SQRT()	Vraća korijen argumenta
TAN()	Vraća tangens argumenta
TRUNCATE()	Vraća broj odrezan za određeni broj decimalnih mjesta

7.2.3. Agregatne funkcije

Agregatne funkcije rade na uzorku, odnosno više n-torki. Najvažnije funkcije su:

Tablica 25. Popis agregatnih funkcija

AVG() ([DISTINCT])	Vraća prosječnu vrijednost atributa, distinct uzima u obzir samo različite attribute
COUNT() ([DISTINCT])	Prebrojava broj ne NULL vrijednosti, DISTINCT uzima u obzir samo različite vrijednosti
GROUP_CONCAT()	Grupira i spaja attribute
MAX()	Vraća najveću vrijednost
MIN()	Vraća minimalnu vrijednost
STD() , STDDEV()	Vraća standardnu devijaciju
SUM() ([DISTINCT])	Vraća sumu atributa

AVG() funkcija

Vraća prosječnu vrijednost nad svim obuhvaćenim atributima. Na primjer ako je potrebno prikazati prosječne ocjene svih studenata, najjednostavnije je napisati upit kao u nastavku:

```
SELECT AVG(ocjena) FROM ocjene
```

Upit će vratiti:

```
3,2500
```

COUNT() funkcija

Vraća ukupan broj svih obuhvaćenih atributa. Na primjer ako je potrebno prikazati koliki broj studenata ukupno postoji u bazi podataka, najjednostavnije je napisati upit kao u nastavku:

```
SELECT COUNT(jmbag) FROM studenti
```

Upit će vratiti:

```
162
```

MIN() i MAX() funkcije

Funkcije MIN() i MAX() koriste se za ispis najmanje odnosno najveće vrijednosti iz skupine podataka. Na primjer, ako je potrebno ispisati najveću ocjenu od svih ocjena, potrebno je napisati upit kao u nastavku:

```
SELECT MAX(ocjena) FROM ocjene
```

Upit će vratiti:

```
5
```

SUM() funkcija

Funkcija SUM() se koristi za sumiranje nad nekom skupinom ili grupom podataka. Na primjer, ako je potrebno izračunati koliki je zbroj svih upisanih ocjena potrebno je napisati sljedeći upit:

```
SELECT SUM(ocjena) FROM ocjene
```

Upit će vratiti:

```
780
```

7.2.4. Datumske funkcije

Tablica 26. Osnovne datumske funkcije i njihovi opisi

ADDDATE()	Dodaje datumu određen vremenski interval
ADDTIME()	Dodaje interval na DATETIME ili TIME podatak
CONVERT_TZ()	Pretvorba iz jedne vremenske zone u drugu
CURDATE(), CURRENT_DATE(), CURRENT_DATE	Vraća trenutni datum
CURTIME(), CURRENT_TIME(), CURRENT_TIME	Vraća trenutno vrijeme

DATE_ADD()	Dodaje vremenski interval na datum
DATE_FORMAT()	Formatira datum
DATE_SUB(),SUBDATE()	Oduzima vremenski interval od datuma
DATE()	Vraća datum iz DATETIME tipa podatka
DATEDIFF()	Oduzima dva datuma
DAYNAME()	Vraća naziv dana u tjednu
DAYOFMONTH(),DAY()	Vraća dan u mjesecu
DAYOFWEEK()	Vraća indeks dana u tjednu (1Nedjelja – 7Subota)
DAYOFYEAR()	Vraća broj dana godine (1-366)
EXTRACT()	Vraća određeni dio datuma
FROM_DAYS()	Pretvara broj dana u datum
FROM_UNIXTIME()	Formatira UNIX timestamp u datum
GET_FORMAT()	Vraća format datuma u stringu
HOUR()	Vraća sate iz DATETIME ili TIME podatka
LAST_DAY(date)	Vraća zadnji dan u mjesecu iz argumenta
MAKEDATE()	Kreira datum iz godine i dana u godini
MAKETIME()	Kreira vrijeme iz sata, minuta i sekundi
MICROSECOND()	Vraća mikrosekunde iz DATETIME ili TIME podataka
MINUTE()	Vraća minute iz TIME tipa podatka
MONTH()	Vraća mjesec iz datuma
MONTHNAME()	Vraća naziv mjeseca iz datuma
NOW(),CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP, LOCALTIME(), LOCALTIME, LOCALTIMESTAMP, LOCALTIMESTAMP()	Vraća trenutno vrijeme i datum
SEC_TO_TIME()	Pretvara sekunde u TIME tip podatka
SECOND()	Vraća sekunde iz TIME tipa podatka
STR_TO_DATE()	Pretvara string u datum
SUBTIME()	Oduzima vremenski interval od vremena
SYSDATE()	Vraća točno vrijeme kada se funkcija pozove

<code>TIME_FORMAT()</code>	Formatira vrijeme
<code>TIME_TO_SEC()</code>	Pretvara TIME tip podatka u sekunde
<code>TIME()</code>	Vraća vrijeme iz DATETIME podatka
<code>TIMEDIFF()</code>	Oduzima vremena
<code>TIMESTAMP()</code>	S jednim argumentom vraća TIMESTAMP tip podatka iz datuma, s dva argumenta vraća sumu argumenata.
<code>TIMESTAMPADD()</code>	Dodaje interval na DATETIME podatak
<code>TIMESTAMPDIFF()</code>	Oduzima dva DATETIME podatka
<code>TO_DAYS()</code>	Pretvara datum u broj dana od 0 godine
<code>UTC_DATE()</code>	Vraća trenutni UTC datum
<code>UTC_TIME()</code>	Vraća trenutno UTC vrijeme
<code>UTC_TIMESTAMP()</code>	Vraća trenutni UTC datum i vrijeme
<code>WEEK()</code>	Vraća broj tjedna iz datuma
<code>WEEKDAY()</code>	Vraća indeks dana u tjednu (0>Pon – 6>Ned)
<code>WEEKOFYEAR()</code>	Vraća tjedan u godini iz datuma (0-53)
<code>YEAR()</code>	Vraća godinu iz datuma

Tablica 27. Znakovi za formatiranje datuma i vremena

%a	Skraćeni dani u tjednu (Sun..Sat)
%b	Skraćeni nazivi mjeseca (Jan..Dec)
%c	Mjesec numerički (0..12)
%D	Dan u mjesecu s engleskim sufiksom (0th, 1st, 2nd, 3rd, ...)
%d	Dan u mjesecu numerički (dvije znamenke)(00..31)
%e	Dan u mjesecu numerički (jedna znamenka)(0..31)
%f	Mikrosekunde (000000..999999)
%H	Sat (00..23)
%h	Sat (01..12)
%I	Sat (01..12)
%i	Minute (00..59)
%j	Dan u godini (001..366)
%k	Sat (0..23)
%l	Sat (1..12)
%M	Naziv mjeseca (January..December)
%m	Mjesec numerički (dvije znamenke) (00..12)
%p	AM ili PM
%r	Vrijeme, 12 satni oblik (hh:mm:ss AM ili PM)
%S	Sekunde (00..59)
%s	Sekunde (00..59)
%T	Vrijeme, 24 satni oblik (hh:mm:ss)
%U	Tjedan (00..53), Nedjelja je prvi dan u tjednu
%u	Tjedan (00..53), Ponedjeljak je prvi dan u tjednu
%W	Naziv dana u tjednu (Sunday..Saturday)
%w	Dan u tjednu (0 =Nedjelja, 6 =Subota)
%Y	Godina, numerička, 4 znamenke
%y	Godina, numerička, 2 znamenke
%%	Doslovni "%" znak
%x	Za bilo koji znak

ADDDATE() funkcija

Dodaje datumu određeni vremenski interval. Na primjer, ako je potrebno ispisati datume upisa studenata iz tablice studenti uvećane za 30 dana, to je moguće napraviti na sljedeći način:

```
SELECT ADDDATE(datumUpisa, INTERVAL 30 DAY) FROM studenti
```

Kako se datum uveća za godinu, prikazano je u sljedećem primjeru:

```
SELECT ADDDATE(datumUpisa, INTERVAL 1 YEAR) FROM studenti
```

Kako uvećati datum za 3 mjeseca prikazuje sljedeći primjer:

```
SELECT ADDDATE(datumUpisa, INTERVAL 3 MONTH) FROM studenti
```

CURDATE() funkcija

CURDATE(), CURRENT_DATE() i CURRENT_DATE su sinonimi. Sve funkcije služe izvlačenju trenutnog datuma s poslužitelja. Ako je potrebno dohvatiti trenutni datum s poslužitelja, potrebno je napisati programski kod kao u nastavku:

```
SELECT CURDATE ()
```

CURTIME() funkcija

CURTIME(), CURRENT_TIME() i CURRENT_TIME su sinonimi. Sve funkcije služe izvlačenju trenutnog vremena s poslužitelja. Ako je potrebno dohvatiti trenutno vrijeme s poslužitelja, potrebno je napisati programski kod kao u nastavku:

```
SELECT CURTIME ()
```

DATEDIFF() funkcija

Funkcija DATEDIFF() vraća razliku između dva datuma u danima. Koliko je dana prošlo od svakog upisa studenta do današnjeg datuma prikazuje sljedeći primjer:

```
SELECT DATEDIFF(CURDATE(), datumUpisa)
FROM studenti;
```

Ovaj primjer također pokazuje da je funkcije moguće ugnježdjivati, tj. funkcije smiju biti jedna unutar druge kako bi se dobio željeni rezultat.

DAY(), MONTH() i YEAR() funkcije

Funkcije DAY(), MONTH() i YEAR() koriste se za izvlačenje pojedinog djela datuma. Na primjer, ako je datum upisa studenta potrebno prikazati u tri stupca u kojima prva predstavlja dan, druga mjesec, a treća godinu, to je moguće napraviti sa sljedećim programskim kodom:

```
SELECT DAY(datumUpisa), MONTH(datumUpisa), YEAR(datumUpisa) FROM
studenti;
```


HOUR(), MINUTE(), SECOND() funkcije

Funkcije HOUR(), MINUTE() i SECOND() koriste se za izvlačenje pojedinog djela vremena. Na primjer, potrebno je prikazati samo sekunde trenutnog vremena:

```
SELECT SECOND (CURTIME ( ) )
```

EXTRACT() funkcija

Funkcija EXTRACT() može se koristiti umjesto zasebnih DAY(), MONTH(), YEAR(), HOUR(), MINUTE() i SECOND() funkcija. Na primjer potrebno je prikazati samo dan u mjesecu trenutnog datuma:

```
SELECT EXTRACT (DAY FROM CURDATE ( ) )
```

DATE_FORMAT() funkcija

Funkcija DATE_FORMAT() služi pretvaranju datuma u string po točno navedenom formatu. Formatiranje se vrši pomoću znakova koji su navedeni u tablici „Znakovi za formatiranje datuma i vremena”. Na primjer, potrebno je ispisati datume upisa studenata u formatu „DD.MM.YYYY.”, odnosno na format hrvatskog jezika:

```
SELECT DATE_FORMAT (datumUpisa, '%d.%m.%Y. ')
FROM studenti;
```

STR_TO_DATE() funkcija

Funkcija STR_TO_DATE() radi obrnuto od DATE_FORMAT() funkcije. Ona pokušava od stringa stvoriti datum. Na primjer, potrebno je prikazati datume rođenja svih nastavnika. S obziroma na to da datum rođenja kao stupac ne postoji, potrebno ga je izvući iz JMBG atributa:

```
SELECT STR_TO_DATE (jmbg, '%d%m9%y'), jmbg
FROM nastavnici;
```

7.2.5. Ostale važne funkcije

Tablica 28. Ostale važne funkcije i njihovi opisi

AES_DECRYPT()	Dekriptiranje korištenjem AES algoritma
AES_ENCRYPT()	Kriptiranje korištenjem AES algoritma
DES_DECRYPT()	Dekriptiranje korištenjem DES algoritma
DES_ENCRYPT()	Kriptiranje korištenjem DES algoritma

MD5()	Izračunava MD5 hash
SHA1() , SHA()	Izračunava SHA-1 hash
SHA2()	Izračunava SHA-2 hash
CAST() , CONVERT()	Pretvara iz jednog tipa podatka u drugi
IFNULL()	Ako je vrijednost NULL, ispisuje neku drugu zadanu vrijednost

8. NULL vrijednosti

NULL vrijednost predstavlja nepostojeću vrijednost podatka. NULL vrijednost će se ponašati slično kao i NULL pokazivač prilikom programiranja. Znači zapis postoji, ali nema ništa u njemu: ni prazan string, ni nula. Ako je potrebno unijeti neki novi zapis ili promijeniti zapis u tablici za neki atribut koji ne mora nužno sadržavati vrijednost za taj atribut ako ne unesemo vrijednost on može poprimiti NULL vrijednost. NULL vrijednosti se tretiraju drugačije od ostalih vrijednosti, tj. tretiraju se kao iznimke koje odskaku od pravila. Primjerice nije moguće uspoređivati NULL sa vrijednosti nula. NULL vrijednosti se ne mogu uspoređivati uobičajenim operatorima kao što su „<, >, =, <>, LIKE“. NULL vrijednosti se uspoređuju koristeći operator IS NULL ako je potrebno provjeriti je li vrijednost jednaka NULL odnosno IS NOT NULL koja je zapravo negacija prvotnog operatora IS NULL.

8.1. UNOS NULL VRIJEDNOSTI

Na primjer, potrebno je unijeti novo mjesto, ali trenutno nije poznat naziv mjesta, već samo poštanski broj „10555“ i zna se da pripada županiji „Zagrebačka županija“, odnosno da mu je šifra županije „1“:

```
INSERT INTO mjesta  
VALUES (10555, NULL, 1);
```

ili

```
INSERT INTO mjesta (postBr, idZupanija)  
VALUES (10555, 1);
```

ili

```
INSERT INTO mjesta  
SET postBr = 10555,  
idZupanija = 1;
```

8.2. PROMJENA NULL VRIJEDNOSTI

Potrebno je postaviti mjestu s poštanskim brojem „10000“ naziv na NULL vrijednost:

```
UPDATE mjesta  
SET nazivMjesto = NULL  
WHERE postBr = 10000;
```

8.3. DOHVAT NULL VRIJEDNOSTI

Potrebno je ispisati sva mjesta kojima je naziv jednak NULL:

```
SELECT *  
FROM mjesta  
WHERE nazivMjesto IS NULL;
```

Primjer će vratiti prethodno unesena i promijenjena mjesta.

Potrebno je ispisati sva mjesta kojima naziv nije jednak NULL potrebno je napisati upit kao u nastavku:

```
SELECT *
FROM mjesta
WHERE nazivMjesto IS NOT NULL;
```

Dodatno, u primjeni je poželjno u ispisu korisniku prikazati vrijednost koja je smisljena za razliku od NULL, moguće je upit obogatiti korištenjem IFNULL() funkcije:

```
SELECT postBr, IFNULL(nazivMjesto, 'Nepoznat naziv mjesta!')
FROM mjesta;
```

Vrijednost koja je trebala biti NULL, sada je „Nepoznat naziv mjesta!“:

☐	postBr	IFNULL(nazivMjesto, 'Nepoznat naziv mjesta!')
☐	10000	Nepoznat naziv mjesta!
☐	10290	Zaprešić
☐	10295	Kupljenovo
☐	10310	Ivanić-Grad
☐	10315	Novoselec
☐	10340	Vrbovec
☐	10345	Gradec
☐	10410	Velika Gorica
☐	10430	Samobor

Na NULL vrijednosti je potrebno pripaziti kod matematičkih operacija. Kada se izvršava matematička operacija između dva ili više atributa od kojih jedan sadrži NULL vrijednost rezultat će uvijek biti NULL.

Funkciju IFNULL() je također moguće koristiti prilikom matematičkih operacija. Zamislimo da je potrebno izračunati umnožak dviju vrijednosti u testnoj tablici.

Testna tablica izgleda kao u nastavku:

☐	ID	prvaVrijednost	drugaVrijednost
☐	1	2	(NULL)
☐	2	3	3
☐	3	(NULL)	4
☐	4	(NULL)	(NULL)
☐	5	1	1

```
SELECT ID, prvaVrijednost*(IFNULL(drugaVrijednost,0))
FROM test;
```

9. Sortiranje podataka

Podatke je prilikom prikaza korisniku često potrebno sortirati po nekom redoslijedu. Za sortiranje se koristi ORDER BY klauzula koja može sortirati podatke uzlazno ili silazno po stupcima projekcije ili funkcijama.

Skraćena sintaksa SQL upita s ORDER BY klauzulom:

```
SELECT column_name(s)
FROM table_name
ORDER BY column_name(s) ASC|DESC
```

Na primjer, potrebno je ispisati sve studente sortirane po imenu uzlazno:

```
SELECT *
FROM studenti
ORDER BY ime ASC
```

Ili bez ASC (podrazumjeva se uzlazno sortiranje):

```
SELECT *
FROM studenti
ORDER BY ime
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0036440545	Aleksandra	Bedić	2011-09-29	32000	32000	6
☐	0036452014	Alen	Martić	2009-09-22	10000	10000	14
☐	0016201458	Ana	Radak	2011-09-27	42000	22000	16
☐	0112047238	Ana	Ujčić	2011-09-30	51000	51000	20
☐	0246025343	Ana	Miličević	2012-09-25	10000	52210	2
☐	1191221406	Ana	Anđal	2012-09-24	10000	10000	11
☐	0246022344	Andrej	Novak	2012-09-25	10000	52210	3
☐	0283018425	Anela	Pavelić	2012-09-26	32000	32000	5
☐	0067025920	Ante	Cindrić	2011-09-29	21000	10410	18
☐	0283018404	Ante	Jukić	2012-09-27	32000	31000	6
☐	0112053256	Anton	Žagar	2012-09-25	51000	23000	20
☐	0016099969	Antonia	Čoić	2011-09-28	42000	35000	17
☐	0013025626	Antonia	Bedić	2012-09-25	32000	44000	5

Kao što se može vidjeti iz rezultata, ORDER BY je sortirao studente samo po imenu, ali ne i po prezimenu. Na primjer, potrebno je sortirati popis studenata po imenu uzlazno, a zatim po prezimenu uzlazno:

```
SELECT *
FROM studenti
ORDER BY ime ASC, prezime ASC
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0036440545	Aleksandra	Bedić	2011-09-29	32000	32000	6
0036452014	Alen	Martić	2009-09-22	10000	10000	14
1191221406	Ana	Andal	2012-09-24	10000	10000	11
0246025343	Ana	Miličević	2012-09-25	10000	52210	2
0016201458	Ana	Radak	2011-09-27	42000	22000	16
0112047238	Ana	Ujčić	2011-09-30	51000	51000	20
0246022344	Andrej	Novak	2012-09-25	10000	52210	3
0283018425	Anela	Pavelić	2012-09-26	32000	32000	5
0067025920	Ante	Cindrić	2011-09-29	21000	10410	18
0283018404	Ante	Jukić	2012-09-27	32000	31000	6
0112053256	Anton	Žagar	2012-09-25	51000	23000	20
0016099969	Antonia	Čoić	2011-09-28	42000	35000	17

Na primjer, potrebno je ispisati sve studente sortirane po prezimenu silazno, a potom po imenu uzlazno:

```
SELECT *
FROM studenti
ORDER BY prezime DESC, ime ASC
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0128050002	Kristina	Žikić	2012-09-25	10000	34000	1
0016200033	Zvonimir	Žarković	2010-09-29	21000	20000	19
0112053256	Anton	Žagar	2012-09-25	51000	23000	20
0128051999	Marina	Zrno	2010-09-29	10000	22000	1
0036463254	Ivan	Zović	2010-09-27	10000	10000	14
0036450023	Darko	Zidanić	2009-09-23	10000	10000	15
0016220011	Danijel	Zečević	2010-09-30	21000	23000	19
0036440706	Vicka	Zečević	2011-09-26	10000	31000	14
0112050000	Filip	Zaležak	2012-09-27	51000	23000	21
0016098017	Marina	Zadro	2011-09-28	42000	47000	17
0036435111	Maja	Vuković	2011-09-27	10000	21300	14
0016235684	Tomislav	Vukasović	2011-09-26	42000	23000	16
0246031254	Petra	Vujić	2008-09-22	10000	34000	3

Kao i većina ostalih klauzula, ORDER BY može raditi s funkcijama. Na primjer, potrebno je ispisati sve studente sortirane po duljini prezimena silazno:

```
SELECT *, LENGTH(prezime) AS Duljina
FROM studenti
ORDER BY LENGTH(prezime) DESC
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	Duljina
1191230521	Any	Sardelić Wojcikiewicz	2010-09-27	10000	10410	13	21
0128052335	Matej	Levanić Kutni	2010-09-27	51000	22000	20	13
0128045526	Kristina	Košić-Kancir	2011-09-30	51000	22300	21	12
0035192565	Kristina	Matleković	2010-09-27	10000	43000	4	10
0246027955	Dino	Rosenzweig	2012-09-26	10410	10410	9	10
0016096972	Dora	Vidaković	2011-09-26	42000	44000	16	9
0016209876	Kristina	Pešerović	2011-09-29	42000	22300	17	9
0016235684	Tomislav	Vukasović	2011-09-26	42000	23000	16	9
0108065150	Vanesa	Kovačević	2009-09-22	32000	10290	6	9
0108065701	Nika	Kovačević	2009-09-23	10000	52210	14	9
0108066602	Lorena	Fabijanić	2012-09-26	40000	22300	7	9

Zato što ORDER BY klauzula koristi kolotaciju za uspoređivanje tekstualnih vrijednosti, oko iste je potrebno posebno voditi računa da bude na ispravnom jeziku.

Na primjer, ako se promijeni kolotacija prezimena svih studenata i pokuša ih se sortirati po prezimenu uzlazno, vidjet će se anomalije u sortiranju podataka.

Promjenu kolotacije moguće je izvršiti pomoću upita:

```
ALTER TABLE studenti
CHANGE COLUMN prezime prezime VARCHAR(50) COLLATE 'latin1_bin'
NULL DEFAULT NULL;
```

Nakon izvršavanja sljedećeg upita s „latin1_bin“ kolotacijom:

```
SELECT *
FROM studenti
ORDER BY prezime DESC
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0128050002	Kristina	Žikić	2012-09-25	10000	34000	1
0016200033	Zvonimir	Žarković	2010-09-29	21000	20000	19
0112053256	Anton	Žagar	2012-09-25	51000	23000	20
0067431720	Ivana	Šokć	2011-09-27	21000	21000	18
0067403663	Mihael	Škoda	2010-09-27	21000	21000	18
0035166854	Krešo	Šišić	2011-09-26	10410	10410	10
0035180940	Sonja	Šimpraga	2011-09-26	10000	21000	4
0128052105	Luka	Šimić	2012-09-25	10000	43000	2
0313006854	Marko	Šarić	2009-09-24	40000	40000	7
0016201125	Denis	Šapina	2011-09-27	42000	42000	17
0128051999	Marina	Zrno	2010-09-29	10000	22000	1
0036463254	Ivan	Zović	2010-09-27	10000	10000	14
0036450023	Darko	Zidanić	2009-09-23	10000	10000	15
0016220011	Danijel	Zečević	2010-09-30	21000	33000	19
0036440706	Vicko	Zečević	2011-09-26	10000	21000	14
0112050000	Filip	Zaležak	2012-09-27	51000	23000	21
0016098017	Marina	Zadro	2011-09-28	42000	47000	17
0036435111	Maja	Vuković	2011-09-27	10000	21300	14
0016235684	Tomislav	Vukasović	2011-09-26	42000	23000	16
0246031254	Petra	Vukić	2008-09-22	10000	34000	3

9.1. SORTIRANJE NULL VRIJEDNOSTI

Prilikom sortiranja podataka, NULL vrijednosti pojaviti će se:

- na početku niza - ako se sortira uzlazno,
- na kraju niza - ako se sortira silazno.

Najjednostavnija provjera ove tvrdnje jest da se nekim studentima promjeni ime u *NULL* vrijednost:

```
UPDATE studenti
SET ime = NULL
WHERE ime = 'Ivan';
```

Nakon promjene imena svih studenata sa „Ivan“ na *NULL*, prilikom izvršavanja sljedećeg upita dobivaju se rezultati kao u nastavku:

```
SELECT * FROM studenti
ORDER BY ime ASC;
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0016216326	(NULL)	Horvat	2011-09-29	42000	10000	17
☐	0035182233	(NULL)	Turk	2008-09-26	10000	43000	3
☐	0036460022	(NULL)	Matić	2008-09-24	10000	10000	15
☐	0036463254	(NULL)	Zović	2010-09-27	10000	10000	14
☐	0036463255	(NULL)	Horvat	2010-09-28	10000	10410	14
☐	0313004298	(NULL)	Stojić	2012-09-27	40000	34000	7
☐	1191221499	(NULL)	Horvat	2012-09-25	10000	10000	11
☐	0036440545	Aleksandra	Bedić	2011-09-29	32000	32000	6
☐	0036452014	Alen	Martić	2009-09-22	10000	10000	14
☐	0016201458	Ana	Radak	2011-09-27	42000	22000	16
☐	0112047238	Ana	Ujčić	2011-09-30	51000	51000	20
☐	0016203043	Ivan	Matić	2012-09-25	10000	50000	3

10. DISTINCT izjava

U rezultatu upita može se dogoditi da pojedina projekcija sadrži duplicirane vrijednosti. Izjava (engl. *statement*) **DISTINCT** se koristi kako bi se ispisale isključivo jedinstvene vrijednosti odnosno uklonili nepoželjni duplikati iz rezultata upita. Kako bi se dobile jedinstvene vrijednosti određene projekcije, **DISTINCT** izjavu je potrebno napisati nakon **SELECT** naredbe.

Na primjer, potrebno je dohvatiti sva različita imena studenata i poredati ih po abecedi:

```
SELECT DISTINCT ime FROM studenti ORDER BY ime;
```

Rezultati izvođenja upita:

☐	ime
☐	Aleksandra
☐	Alen
☐	Ana
☐	Andrej
☐	Anela
☐	Ante
☐	Anton
☐	Antonia
☐	Antonio
☐	Antun
☐	Anya
☐	Armin
☐	Bernarda
☐	Boris

Obzirom da se **DISTINCT** odnosi na cjelokupnu projekciju, za ispis jedinstvenih vrijednosti kombinacija imena i prezimena studenata može se koristiti sljedeći upit:

```
SELECT DISTINCT ime, prezime  
FROM studenti ORDER BY ime;
```

Rezultati izvođenja upita:

☐ ime	prezime
☐ Aleksandra	Bedić
☐ Ana	Andal
☐ Ana	Miličević
☐ Ana	Radak
☐ Ana	Ujčić
☐ Ante	Cindrić
☐ Ante	Jukić
☐ Antonia	Čoić
☐ Antonio	Pavić
☐ Antun	Mandić
☐ Anya	Sardelic Wojcikiewicz
☐ Armin	Čehajić

Isto ime, ali različito prezime što je različita n-torka!

DISTINCT je također moguće koristiti unutar agregatnih funkcija. U tom slučaju DISTINCT je potrebno napisati unutar agregatne funkcije. Na primjer, potrebno je ispisati ukupan broj različitih imena studenata:

```
SELECT COUNT(DISTINCT ime) FROM studenti;
```

Ako je potrebno ispisati ukupan broj studenata s različitim imenom i prezimenom, potrebno je napisati upit kao u nastavku (ovaj upit će funkcionirati samo s COUNT agregatnom funkcijom jer ona broji n-torke i može raditi s više stupaca istovremeno):

```
SELECT COUNT(DISTINCT ime, prezime) FROM studenti;
```

Ako je potrebno ispisati na koliko predmeta radi samo jedan nastavnik, odnosno u tablici „izvrsitelji“ koliko ID-eva kolegija se pojavljuje samo jednom, potrebno je napisati upit kao u nastavku:

```
SELECT COUNT(DISTINCT idKolegij) FROM izvrsitelji;
```

11. Kartezijev produkt i prirodno spajanje

11.1. KARTEZIJEV PRODUKT

Kartezijev produkt je umnožak relacija koji se u praksi bez uvjeta nikada ne koristi te je jedna od najčešćih grešaka programera. On služi kao priprema za prirodno spajanje relacija.

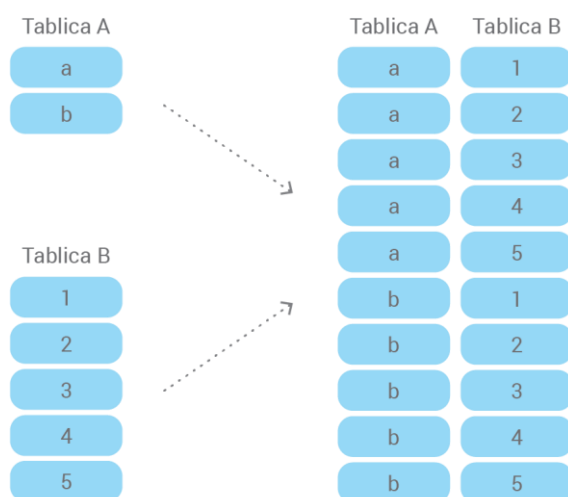
11.1.1. Relacijska algebra kartezijevog produkta

Za dvije relacije A i B, relacijska algebra prirodnog spajanja izgleda:

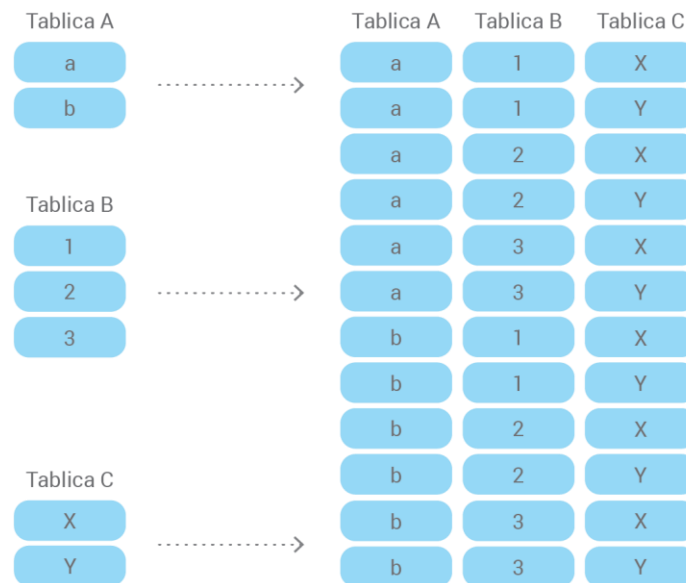
$$A \times B$$

Ako relacija A sadrži 2 n-torke, a relacija B 5 n-torki, broj n-torki dobivenih kartezijevim produktom je $2 \times 5 = 10$.

11.1.2. Izgled kartezijevog produkta



Slika 27. Kartezijev produkt dvije relacije



Slika 28. Kartezijev produkt tri relacije

11.1.3. Osnovna sintaksa:

```
SELECT naziv_stupca|*[, naziv_stupca]...
FROM tablica1, tablica2, tablica3...
WHERE tablica1.naziv_stupca operator vrijednost [AND|OR
tablica2.naziv_stupca operator vrijednost]
```

11.1.4. Primjeri:

Potrebno je kartezijevim produktom spojiti relacije *mjesta* i *zupanije*:

```
SELECT * FROM mjesta, zupanije
```

U rezultatima je moguće primijetiti da je svaka n-torka iz relacije *zupanije* spojena sa svakom n-torkom iz relacije *mjesta* (1428 n-torki):

postbr	nazivMjesto	idŽupanija	id	nazivŽupanija
10000	Zagreb	1	1	Grad Zagreb
10000	Zagreb	1	2	Zagrebačka županija
10000	Zagreb	1	3	Krapinsko-zagorska županija
10000	Zagreb	1	4	Sisačko-moslavačka županija
10000	Zagreb	1	5	Karlovačka županija
10000	Zagreb	1	6	Varaždinska županija
10000	Zagreb	1	7	Koprivničko-križevačka županija
10000	Zagreb	1	8	Bjelovarsko-bilogorska županija
10000	Zagreb	1	9	Primorsko-goranska županija
10000	Zagreb	1	10	Ličko-senjska županija
10000	Zagreb	1	11	Virovitičko-podravska županija
10000	Zagreb	1	12	Požeško-slavonska županija
10000	Zagreb	1	13	Brodsko-posavska županija
10000	Zagreb	1	14	Zadarska županija
10000	Zagreb	1	15	Osječko-baranjska županija
10000	Zagreb	1	16	Šibensko-kninska županija
10000	Zagreb	1	17	Vukovarsko-srijemska županija
10000	Zagreb	1	18	Splitsko-dalmatinska županija
10000	Zagreb	1	19	Istarska županija
10000	Zagreb	1	20	Dubrovačko-neretvanska županija
10000	Zagreb	1	21	Međimurska županija
10290	Zaprešić	1	1	Grad Zagreb
10290	Zaprešić	1	2	Zagrebačka županija
10290	Zaprešić	1	3	Krapinsko-zagorska županija
10290	Zaprešić	1	4	Sisačko-moslavačka županija

Naravno, kao što se vidi iz rezultata prilikom vršenja kartezijevog produkta, zapravo se radi umnožak dvije ili više tablica. Ako tablica mjesta sadrži 68 n-torki, a tablica županija 21 n-torku, ukupno će biti $68 * 21 = 1428$ n-torki.

Spajanje više tablica moguće je na isti način. Na primjer, potrebno je napraviti kartezijev produkt tablice mjesta, županije i studenti:

```
SELECT * FROM mjesta, zupanije, studenti
```

Ovaj upit vraća čak 231.336 redaka! Zbog mogućnosti velikog seta rezultata koji upit može vratiti potrebno je biti izuzetno oprezan prilikom izvođenja kartezijevog produkta.

11.2. PRIRODNO SPAJANJE

Prirodno spajanje je kartezijev produkt s definiranim uvjetima prema kojima se spajaju tablice. Uvjeti se u prirodnom spajanju dodaju u WHERE klauzulu. Minimalan broj uvjeta za prirodno spajanje je **ukupan broj tablica – 1**.

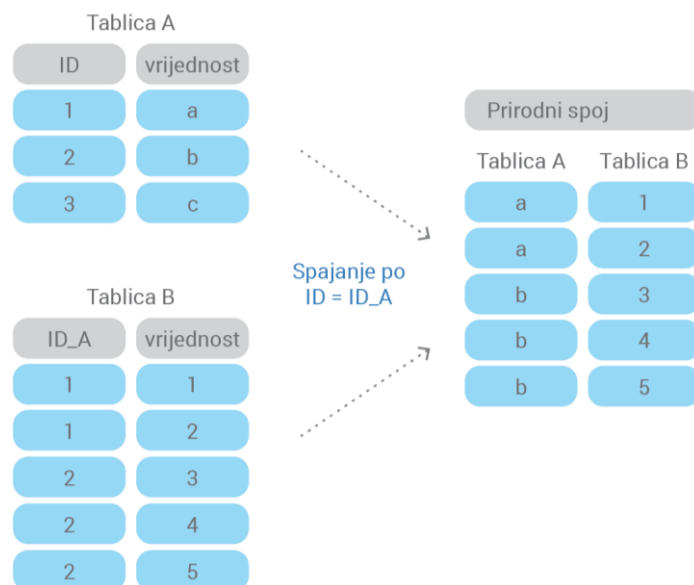
11.2.1. Relacijska algebra prirodnog spajanja

Za dvije tablice (relacije) A i B, relacijska algebra prirodnog spajanja izgleda:

$A \bowtie B$

Broj n-torki dobivenih prirodnim spajanjem nije moguće odrediti bez uvida u stvarne podatke.

11.2.2. Izgled prirodnog spoja



Slika 29. Prirodni spoj dvije tablice

11.2.3. Sintaksa:

```
SELECT naziv_stupca|*[, naziv_stupca]...
FROM tablica1, tablica2, tablica3
WHERE tablica1.naziv_stupca = tablica2.naziv_stupca AND
      tablica2.naziv_stupca = tablica3.naziv_stupca
```

Prilikom prirodnog spajanja u WHERE klauzuli je potrebno paziti da se navede naziv tablice prije naziva atributa kako bi se jednoznačno odredili uvjeti. Ne smije se napisati prirodni spoj kao u nastavku jer interpreter upita neće točno znati o atributu koje tablice se radi:

```
SELECT *
FROM mjesta, ustanove
WHERE postBr = postBr;
```

Upit će javiti grešku o korištenju dvosmislenog atributa u upitu:

```
Query: SELECT * FROM mjesta, ustanove WHERE postBr = postBr; LIMIT
0, 1000
```

Error Code: 1052

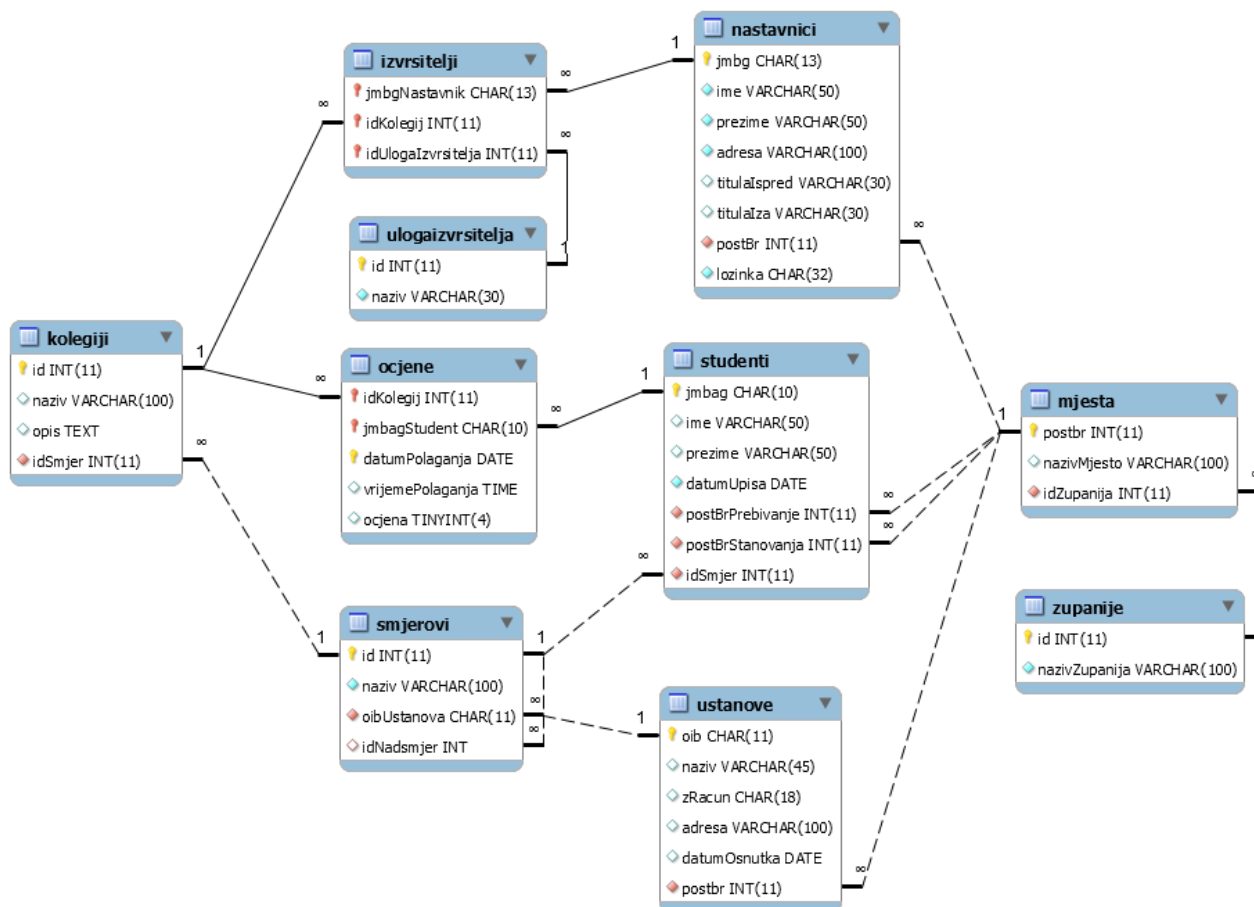
Column 'postBr' in where clause is ambiguous

Umjesto takvog upita, potrebno je napisati upit kao u nastavku gdje se interpreteru napominje točna tablica i atribut s kojim se radi:

```
SELECT *  
FROM mjesta, ustanove  
WHERE mjesta.postBr = ustanove.postBr;
```

11.2.4. Kako odrediti atribute preko kojih će se spajati tablice?

Atribute preko kojih se tablice međusobno spajaju najjednostavnije je odrediti pomoću sheme, odnosno dijagrama baza podataka kao na slici:



Slika 30. Shema baze podataka studenti s laboratorijskih vježbi

Iz dijagrama je vidljivo da se tablice spajaju na sljedeći način:

- kolegiji.id = ocjene.idKolegij
- kolegij.id = izvrsitelji.idKolegij
- kolegij.idSmjer = smjerovi.id
- izvrsitelji.idUlogaIzvrstelja = ulogaIzvrstelja.id
- izvrsitelji.jmbgNastavnik = nastavnici.jmbg
- ocjene.jmbagStudent = studenti.jmbag
- smjerovi.id = studenti.idSmjer
- smjerovi.oibUstanova = ustanove.oib
- smjerovi.id = smjerovi.idNadsmjer
- studenti.postBrPrebivanje = mjesta.postBr
- studenti.postBrStanovanja = mjesta.postBr
- ustanove.postBr = mjesta.postBr

- `nastavnici.postBr = mjesta.postBr`
- `mjesta.idZupanija = zupanije.id`

Naravno, moguće je i spajanje preko drugih atributa, ali ono je najčešće besmisleno i nepotrebno. Na primjer, moguće je spojiti studenta i nastavnika preko imena, ali taj rezultat bi bio apsolutno beskoristan:

```
SELECT *
FROM studenti, nastavnici
WHERE studenti.ime = nastavnici.ime;
```

Upit vraća rezultate, ali beskorisne:

idmj	ime	prezime	datumUpisa	postBrFrebivanje	postBrStanovanja	idSmjer	idmj	ime	prezime	adresa	titulaIspređ	titulalna	postBr	lozinka
001026584	Borna	Knežević	2010-09-29	10410	44000		90606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.		21000	e6999e93270eb1014b7118501a201a59
001026584	Borna	Knežević	2010-09-29	10410	44000		1710105981100707	Vesna	Ilić	Paška ulica 25		prof.	40000	d5fa903f8a4019ca80112e2402cc044ac
0016200033	Zvonimir	Čarković	2010-09-29	21000	20000		1910109626363604	Zvonimir	Ivančević	Šimunovića 39	dr.sc.	v. pred.	42000	92d3c1cf04d46c53e6323d9cbl7af60
0016201458	Ana	Radak	2011-09-27	42000	22000		161204979362905	Ana	Mihaljević	Palmotićeva ulica 6	prof. dr.sc.		10000	55ebe981813c9d023ef854da29ebf606
0016220011	Daniyel	Zečević	2010-09-26	21000	23000		190404964361006	Daniyel	Tkalčić	Modilova 96	dr.sc.		21000	806fad9a7e9e1a041b84440451a1b84
0016233522	Nikola	Habiyan	2011-09-26	42000	42000		161909987173977	Nikola	Bačić	Radiceva 3	prof. dr.sc.		10000	d5fe014b11c328fa07219e41970cbade
0035180940	Sonja	Šimpraga	2011-09-26	10000	21000		42306978362104	Sonja	Dumnyak	Ulica žrtava fašizma 23	prof. dr.sc.		21000	d44a0b0e4f6edec4f3d3e1bea1f511
0036450215	Josip	Pavković	2009-09-24	10000	20000		142011987166005	Josip	Ivanović	Šaharova ulica 5		prof.	10000	372ab6455cc324e4352454531023fde4
0036450215	Josip	Pavković	2009-09-24	10000	20000		142207987301807	Josip	Dukić	Jurišićeva 21	mr.sc.		10000	9f50a13b0c84f1aa366807698131a0a
0036499965	Lovro	Međlović	2008-09-26	10000	10410		152805952392713	Lovro	Horvatinđić	Škurinjska cesta 47	dr.sc.	v. pred.	10310	64255cc57954a1c231f08bfae3b7a782
0067402200	Borna	Pavić	2010-09-26	21000	10430		190606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.		21000	e6999e93270eb1014b7118501a201a59
0067402200	Nikola	Pavić	2010-09-26	21000	21000		191909987173977	Nikola	Bačić	Radiceva 3	prof. dr.sc.		10000	d5fe014b11c328fa07219e41970cbade
0108066366	Iva	Perković	2012-09-26	10000	51000		150907987330057	Iva	Klarin	Gojlanska ulica 18	dr.sc.	v. pred.	10000	9a92b020e954bdf39f776c428d18cccb
0112037622	Nikola	Gorišani	2011-09-28	51000	51000		211909987173977	Nikola	Bačić	Radiceva 3	prof. dr.sc.		10000	d5fe014b11c328fa07219e41970cbade
0112047238	Ana	Ujčić	2011-09-30	51000	51000		201204979362905	Ana	Mihaljević	Palmotićeva ulica 6	prof. dr.sc.		10000	55ebe981813c9d023ef854da29ebf606
0112053625	Boris	Miljuš	2011-09-30	51000	52210		210708980383302	Boris	Brčić	Zagrebačka 154	prof. dr.sc.		10000	ffdc0d63cb0b527cb40e1724ef34c13a
0112055201	Nikola	Šušec	2012-09-24	51000	52210		201909987173977	Nikola	Bačić	Radiceva 3	prof. dr.sc.		10000	d5fe014b11c328fa07219e41970cbade
0246025343	Ana	Miličević	2012-09-25	10000	52210		21204979362905	Ana	Mihaljević	Palmotićeva ulica 6	prof. dr.sc.		10000	55ebe981813c9d023ef854da29ebf606
0246025633	Zvonimir	Kolar	2010-09-29	10000	22300		41010962363604	Zvonimir	Ivančević	Šimunovića 39	dr.sc.	v. pred.	42000	92d3c1cf04d46c53e6323d9cbl7af60

11.2.5. Kako funkcionira prirodno spajanje?

Prirodno spajanje je po definiciji filtrirani kartezijev produkt. Izvršavanje prirodnog spajanja je opisano konceptualno:

- Prvo se određuje kartezijev produkt navedenih tablica
- Obavlja se filtriranje n-torki koje zadovoljavaju uvjete spajanja
- Ispisuju se vrijednosti svih atributa navedenih u dijelu za selekciju u 2 koraku ili se obavlja projekcija po atributima navedenim u listi za selekciju (ako je naveden DISTINCT kvalifikator)

U današnje vrijeme sve će baze podataka zapravo raditi prirodno spajanje pomoću JOIN naredbe koja koristi hash algoritme za spajanje umjesto zastarjelog filtriranja kartezijevog produkta. To je zato što sve baze podataka prije izvršavanja upita isti provjeravaju, optimiziraju i ubrzavaju kroz proces provjere unutar DBMS procesa (slika 3). Na taj način osigurava se brže i optimiziranje izvođenje upita što je jedna od primarnih zadaća svakog DBMS sustava.

11.2.6. Primjeri

Potrebno je ispisati sve studente i nazive županija njihovog stanovanja:

```
SELECT studenti.*, zupanije.nazivZupanija
FROM studenti, mjesta, zupanije
WHERE studenti.postBrStanovanja = mjesta.postBr
AND mjesta.idZupanija = zupanije.id;
```

Rezultati izvođenja upita (162 n-torke):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	nazivZupanija
0016216325	Toni	Tošeski	2011-09-28	42000	10000	17	Grad Zagreb
0016216326	Ivan	Horvat	2011-09-29	42000	10000	17	Grad Zagreb
0035177990	Irena	Dabić	2012-09-26	10000	10000	1	Grad Zagreb
0035179953	Tanja	Sunek	2012-09-26	10000	10000	2	Grad Zagreb
0035187555	Marko	Krišto	2011-09-27	10410	10000	9	Grad Zagreb
0036450023	Darko	Zidanić	2009-09-23	10000	10000	15	Grad Zagreb
0036452014	Alen	Martić	2009-09-22	10000	10000	14	Grad Zagreb
0036460022	Ivan	Matić	2008-09-24	10000	10000	15	Grad Zagreb
0036463254	Ivan	Zović	2010-09-27	10000	10000	14	Grad Zagreb
0036466211	Nada	Marulin	2010-09-27	10000	10000	15	Grad Zagreb
0067408996	Rajko	Posavi	2010-09-28	21000	10000	18	Grad Zagreb
0108065187	Tomislav	Obrovac	2012-09-26	40000	10000	7	Grad Zagreb
0108067363	Filip	Kljaić	2011-09-28	40000	10000	8	Grad Zagreb
0112047168	Marko	Erceg	2011-09-29	51000	10000	20	Grad Zagreb
0128056624	Stanislav	Oršulić	2009-09-21	10410	10000	10	Grad Zagreb

Potrebno je ispisati sve studente koji imaju mjesto stanovanja u Zagrebačkoj županiji:

```
SELECT studenti.*
FROM studenti, mjesta, zupanije
WHERE studenti.postBrStanovanja = mjesta.postBr
AND mjesta.idZupanija = zupanije.id
AND zupanije.nazivZupanija = "Zagrebačka županija";
```

Rezultati izvođenja upita (20 n-torki):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0035166854	Krešo	Šišić	2011-09-26	10410	10410	10
0035188827	Blaž	Đuranić	2009-09-21	10410	10410	9
0036463255	Ivan	Horvat	2010-09-28	10000	10410	14
0036499965	Lovro	Keglović	2008-09-26	10000	10410	15
0067025920	Ante	Cindrić	2011-09-29	21000	10410	18
0067028521	Helena	Džapo	2010-09-29	21000	10410	19
0067430739	Marijana	Adžija	2011-09-29	10410	10410	10
0067433447	Francika	Dragić	2011-09-30	21000	10410	9
0130421547	Dina	Panić	2010-09-28	10410	10410	10
0246007553	Nina	Katić	2012-09-26	10410	10410	9
0246027955	Dino	Rosenzweig	2012-09-26	10410	10410	9
1191200784	Dubravko	Antunović	2012-09-24	10410	10410	10
1191220888	Nikolina	Orlović	2012-09-24	10410	10410	9
1191230521	Anna	Sardalic Wojcikiewicz	2010-09-27	10000	10410	13

Potrebno je ispisati sve studente koji imaju adresu stanovanja u Karlovačkoj županiji:

```
SELECT studenti.*
FROM studenti, mjesta, zupanije
WHERE studenti.postBrStanovanja = mjesta.postBr
```

```
AND mjesta.idZupanija = zupanije.id
AND zupanije.nazivZupanija = "Karlovačka županija";
```

Rezultati izvođenja upita:

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 0016098017	Marina	Zadro	2011-09-28	42000	47000	17
☐ 0036468547	Sandra	Labak	2008-09-23	10000	47000	15
☐ 0313003860	Martina	Rak	2012-09-27	40000	47000	8
☐ 0313004207	Luka	Skupnjak	2012-09-27	40000	47000	8

Potrebno je ispisati sve studente koji studiraju na smjeru naziva „ekonomika poduzetništva“ i žive u županiji „Grad Zagreb“:

```
SELECT studenti.*
FROM studenti, mjesta, zupanije, smjerovi
WHERE studenti.postBrPrebivanje = mjesta.postBr
AND mjesta.idZupanija = zupanije.id
AND studenti.idSmjer = smjerovi.id
AND smjerovi.naziv = "ekonomika poduzetništva"
AND zupanije.nazivZupanija = "Grad Zagreb";
```

Rezultati izvođenja upita:

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 1191290785	Želimira	Ivček	2012-09-24	10000	31000	17

Potrebno je ispisati sve studente koji su dobili ocjenu 5 iz bilo kojeg predmeta:

```
SELECT DISTINCT studenti.*
FROM studenti, ocjene
WHERE studenti.jmbag = ocjene.jmbagStudent
AND ocjene.ocjena = 5;
```

Zato što postoji studenata koji su dobili po više ocjena izvrstan (5) javljaju se dupli rezultati. S obzirom na to da je u ovom slučaju bilo traženo ispisati samo studente koji su dobili ocjenu 5, a ne i predmete iz kojih su ocjene dobili, potrebno je koristiti DISTINCT izjavu.

Rezultati izvođenja upita (53 n-torke):

☐ jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐ 0128050002	Kristina	Žikić	2012-09-25	10000	34000	1
☐ 0035179953	Tanja	Sunek	2012-09-26	10000	10000	2
☐ 0128052105	Luka	Šimić	2012-09-25	10000	43000	2
☐ 0246029526	Dorotea	Stipanov	2010-09-30	10000	20000	3
☐ 0246033365	Marko	Noskov	2008-09-24	10000	10000	4
☐ 0246036655	Marija-Olgica	Kasumović	2008-09-23	10000	34000	4
☐ 0283016371	Ines	Pozaić	2012-09-26	32000	35000	5
☐ 0283018425	Anela	Pavelić	2012-09-26	32000	32000	5
☐ 0013025926	Antonio	Pavić	2012-09-25	32000	44000	5
☐ 0283016670	Dava	Baničević	2012-09-27	32000	31000	6

Potrebno je ispisati sve studente koji su imali predmet na smjeru „turizam“:

```
SELECT DISTINCT studenti.*
FROM studenti, ocjene, kolegiji, smjerovi
WHERE studenti.jmbag = ocjene.jmbagStudent
AND ocjene.idKolegij = kolegiji.id
AND kolegiji.idSmjer = smjerovi.id
AND smjerovi.naziv = 'turizam';
```

U ovom primjeru, slično kao i u prošlom potrebno koristiti DISTINCT izjavu kako ne bi dobivali duple rezultate. S obzirom na to da su neki studenti upisali smjer, ali ne i predmet s tog smjera, potrebno je povezati studenta preko ocjene i predmeta na smjer. Na ovaj način se u rezultatima dobivaju studenti koji su dobili ocjenu na određenom predmetu bez obzira bila ona prolazna ili ne, te također i oni koji su prešli na drugi smjer.

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0016200033	Zvonimir	Žarković	2010-09-29	21000	20000	19
☐	0067401609	Teo	Turković	2011-09-28	21000	32000	19
☐	0067402200	Borna	Pavić	2010-09-30	21000	10430	19
☐	0067408000	Nikola	Mucić	2010-09-28	21000	21000	19
☐	0067430279	Tajana	Vugrinec	2011-09-28	21000	23000	19
☐	0016220011	Danijel	Zečević	2010-09-30	21000	23000	19
☐	0067028521	Helena	Džapo	2010-09-29	21000	10410	19

Potrebno je ispisati sve studente koji su imali predmet na smjeru „održavanje zrakoplova“ i imaju mjesto stanovanja u županiji „Zagrebačka županija“:

```
SELECT DISTINCT studenti.*
FROM studenti, ocjene, kolegiji, smjerovi, mjesta, zupanije
WHERE studenti.jmbag = ocjene.jmbagStudent
AND ocjene.idKolegij = kolegiji.id
AND kolegiji.idSmjer = smjerovi.id
AND studenti.postBrStanovanja = mjesta.postBr
AND mjesta.idZupanija = zupanije.id
AND smjerovi.naziv = 'održavanje zrakoplova'
AND zupanije.nazivZupanija = 'Zagrebačka županija';
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0067430739	Marijana	Adžija	2011-09-29	10410	10410	10
☐	0130421547	Dina	Panić	2010-09-28	10410	10410	10
☐	1191200784	Dubravko	Antunović	2012-09-24	10410	10410	10
☐	0035166854	Krešo	Šišić	2011-09-26	10410	10410	10

Potrebno je ispisati sve studente koji su imali predmet na smjeru „održavanje zrakoplova“ ili imaju mjesto stanovanja u županiji „Grad Zagreb“:

```
SELECT DISTINCT studenti.*
FROM studenti, ocjene, kolegiji, smjerovi, mjesta, zupanije
WHERE studenti.jmbag = ocjene.jmbagStudent
  AND ocjene.idKolegij = kolegiji.id
  AND kolegiji.idSmjer = smjerovi.id
  AND studenti.postBrStanovanja = mjesta.postBr
  AND mjesta.idZupanija = zupanije.id
  AND smjerovi.naziv = 'održavanje zrakoplova'
  OR zupanije.nazivZupanija = 'Grad Zagreb';
```

Upit se ne izvršava ispravno zbog nedostatka ispravno postavljenih zagrada. Na zagrade je potrebno posebno obratiti pozornost kod prirodnog spajanja. Bez korištenja zagrada interpreter upita će u ovom slučaju pokušati napraviti prirodni spoj po zadanim uvjetima spajanja ili kartezijev produkt s uvjetom da je naziv županije „Grad zagreb“. Upit je ispravno napisan u nastavku:

```
SELECT DISTINCT studenti.*
FROM studenti, ocjene, kolegiji, smjerovi, mjesta, zupanije
WHERE (studenti.jmbag = ocjene.jmbagStudent
  AND ocjene.idKolegij = kolegiji.id
  AND kolegiji.idSmjer = smjerovi.id
  AND studenti.postBrStanovanja = mjesta.postBr
  AND mjesta.idZupanija = zupanije.id)
  AND (smjerovi.naziv = 'održavanje zrakoplova'
  OR zupanije.nazivZupanija = 'Grad Zagreb');
```

Rezultati izvođenja upita (40 n-torki):

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0035177990	Irena	Dabić	2012-09-26	10000	10000	1
☐	0246022345	Ivana	Babić	2012-09-25	10000	10000	1
☐	0246024883	Ivana	Jelenčić	2012-09-25	10000	10000	1
☐	0035179953	Tanja	Sunek	2012-09-26	10000	10000	2
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3
☐	0246033365	Marko	Noskov	2008-09-24	10000	10000	4
☐	0108065951	Jan	Dumančić	2009-09-21	32000	10290	5
☐	0108065150	Vanesa	Kovačević	2009-09-22	32000	10290	6
☐	0313006321	Nikola	Kupres	2009-09-25	40000	10290	7
☐	0108065187	Tomislav	Obrovac	2012-09-26	40000	10000	7
☐	0313005214	Josip	Marin	2008-09-22	40000	10290	8
☐	0035187555	Marko	Krišto	2011-09-27	10410	10000	9
☐	0067430739	Marijana	Adžija	2011-09-29	10410	10410	10
☐	0128052014	Tomislav	Paulić	2009-09-22	10410	21300	10

12. Unija

UNION izjava objedinjuje rezultate izvršavanja dva ili više upita u jednu projekciju. Projekcije izvršavanja svih upita povezanih unijom moraju sadržavati jednak broj stupaca i kompatibilne tipove podataka. kako bi se upiti uspješno spojili unijom. Postoje dva tipa unije:

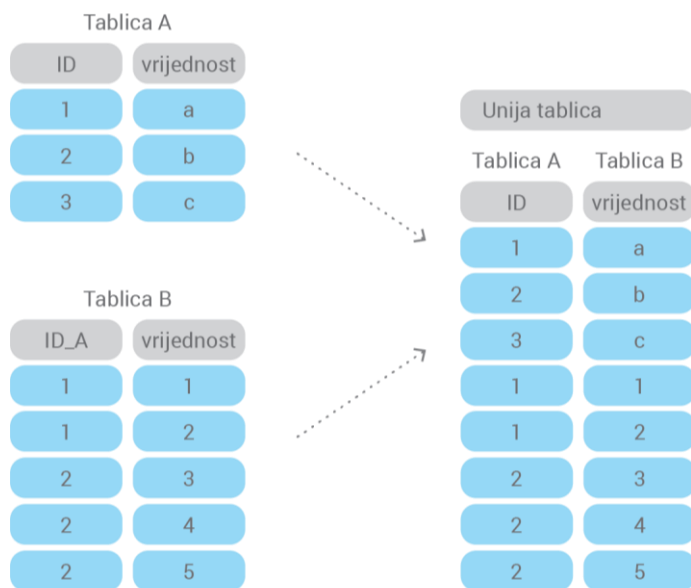
- UNION
- UNION ALL

Unija se često koristi kod skladišta podataka gdje podaci nisu savršeno normalizirani, a u slučaju MySQL baze podataka uniju je potrebno koristiti prilikom izrade FULL OUTER JOIN-a (poglavlje 12.6.).

Sintaksa:

```
SELECT naziv_stupca(s) FROM naziv_tablice1
UNION
SELECT naziv_stupca(s) FROM naziv_tablice2
```

12.1.1. Izgled unije tablica



Slika 31. Unija dvije tablice

12.1.2. Relacijska algebra unije

Ako postoje dvije tablice naziva A i B označava se s:

$$A \cup B$$

Za uniju vrijedi:

$$A \cup (B \cup C) = (A \cup B) \cup C = A \cup B \cup C$$

12.2. UNION

Union se koristi kada nije potrebno prikazati duplikate n-torki u projekciji. Na primjer ako postoje dvije tablice sa sljedećim podacima i upit kao u nastavku:

Student		Nastavnik	
Ime	postBr	Ime	postBr
Ivan	10000	Ivan	10001
Hrvoje	42000	Hrvoje	42000
Josip	20000	Stjepan	20000

```
SELECT * FROM Student
UNION
SELECT * FROM Nastavnik;
```

Upit će prikazati sljedeće podatke:

Ime	postBr
Ivan	10000
Hrvoje	42000
Josip	20000
Ivan	10001
Stjepan	20000

Kao što se može vidjeti u rezultatima upita s UNION izjavom nedostaju duplikati n-torki. Ako ih se mora uključiti u rezultat, potrebno je koristiti UNION ALL izjavu.

12.2.1. Primjeri:

Potrebno je ispisati imena i prezimena nastavnika i studenata u istoj projekciji.

```
SELECT studenti.ime, studenti.prezime FROM studenti
UNION
SELECT nastavnici.ime, nastavnici.prezime FROM nastavnici;
```

Rezultati izvođenja upita (211 n-torki):

☐ ime	prezime
☐ Tanja	Krog
☐ Matea	Omanović
☐ Antun	Mandić
☐ Antonio	Pavić
☐ Borna	Knežević
☐ Dora	Vidaković
☐ Vesna	Đukić
☐ Marina	Zadro
☐ Antonia	Čoić

Potrebno je ispisati imena i prezimena studenata i nastavnika u istoj projekciji i poredati ih po prezimenu uzlazno.

```
SELECT studenti.ime, studenti.prezime FROM studenti
UNION
SELECT nastavnici.ime, nastavnici.prezime FROM nastavnici
ORDER BY prezime;
```

Rezultati izvođenja upita (211 n-torki):

☐ ime	prezime
☐ Marijana	Adžija
☐ Hana	Amulić
☐ Ana	Andal
☐ Dubravko	Antunović
☐ Ivana	Babić
☐ Renato	Babojelić
☐ Nikola	Bačić
☐ Pavo	Baničević

U ovom primjeru prilikom rješenja moguće je primijetiti da se za sortiranje koristi naziv stupca gornjeg upita. To je iz razloga što se stupcu dodjeljuje naziv koji se koristi u prvom upitu. Ako je potrebno promijeniti naziv stupca, to je moguće napraviti s aliasom (14. poglavlje).

12.3. UNION ALL

UNION ALL se koristi kada je potrebno prikazati duplikate n-torki u projekciji. Na primjer ako postoje dvije tablice sa sljedećim podacima i upit kao u nastavku:

Student	
Ime	postBr
Ivan	10000
Hrvoje	42000
Josip	20000

Nastavnik	
Ime	postBr
Ivan	10001
Hrvoje	42000
Stjepan	20000

```
SELECT * FROM Student
UNION ALL
SELECT * FROM Nastavnik;
```

Upit će prikazati sljedeće podatke:

Ime	postBr
Ivan	10000
Hrvoje	42000
Josip	20000
Ivan	10001
Hrvoje	42000
Stjepan	20000

Za razliku od UNION izjave UNION ALL će prikazati i dupli redak (U ovom slučaju „Hrvoje“ s poštanskim brojem 42000) što je ujedno i jedina razlika između UNION i UNION ALL izjava.

13. JOIN klauzula

JOIN klauzula koristi se kada je kao rezultat SQL upita potrebno dobiti presjek podataka iz dvije ili više tablica. U novije vrijeme baze podataka imaju iznimno dobre optimizatore upita i gotovo je svejedno piše li se upit kao prirodno spajanje ili JOIN (ako je moguće napraviti upit pomoću prirodnog spajanja). Vanjske spojeve puno je jednostavnije napisati pomoću JOIN klauzule. Kod vanjskog spajanja relacije (odnosno tablice) se tretiraju nesimetrično. Jedna relacija je dominantna što ima za posljedicu da se sve njene n-torke pojavljuju u rezultatu bez obzira postoji li odgovarajuća n-torka u podređenoj relaciji.

Preporuka je umjesto prirodnog spajanja pisati JOIN klauzulu zbog mnogo veće preglednosti upita. JOIN klauzulu je moguće podijeliti na nekoliko osnovnih podvrsta spoja:

- INNER JOIN
- NATURAL JOIN
- CROSS JOIN
- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN

13.1. INNER JOIN

INNER JOIN je najčešće korišten način spajanja tablica. Inner join kreira tablicu s rezultatima spajajući n-torke po vrijednostima određenih stupaca (ON klauzula). Prilikom izvršavanja upita DBMS uspoređuje određeni stupac svake n-torke prve tablice s određenim stupcem svake n-torke druge tablice i ako se vrijednosti podudaraju, n-torke se spajaju i ulaze u projekciju. JOIN se također može objasniti na način da se prvo stvori Kartezijev produkt, a potom se redci filtriraju pomoću uvjeta napisanog u ON klauzuli (identično kao prirodno spajanje). Naravno, ovakav način je samo slikovit jer baze podataka koriste „hash join“ ili „sort-merge join“ algoritme koji rade mnogo brže od Kartezijevog produkta i filtriranja istog. Pomoću JOIN klauzule moguće je spojiti beskonačan broj tablica.

Sintaksa:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
INNER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
INNER JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

Ili skraćeno:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
JOIN tablica2 ON tablica1.stupac = tablica2.stupac
```

```
JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

13.1.1. Relacijska algebra JOIN klauzule

U osnovi izgleda identično kao i kod prirodnog spajanja. Za tablice R i S vrijedi:

$$R \bowtie S$$

Za tri ili više tablica koje su spojene JOIN-om bili prirodnima spojem vrijedi da je redoslijed spajanja nebitan:

$$R \bowtie (S \bowtie T) = (R \bowtie S) \bowtie T$$

Ako se nazivi stupaca razlikuju potrebno je koristiti *Theta* JOIN operator. U mnogo slučajeva se nazivi stupaca razlikuju prilikom spajanja tablica. Izgled relacijske algebre za *Theta* JOIN:

$$R \bowtie_{a\theta b} S$$

- R i S su relacije (tablice)
- a i b su atributi po kojima se spajaju relacije (tablice)
- θ -simbol *theta* join-a gdje operator može biti $>$, $<$, $>=$, $<=$, $=$

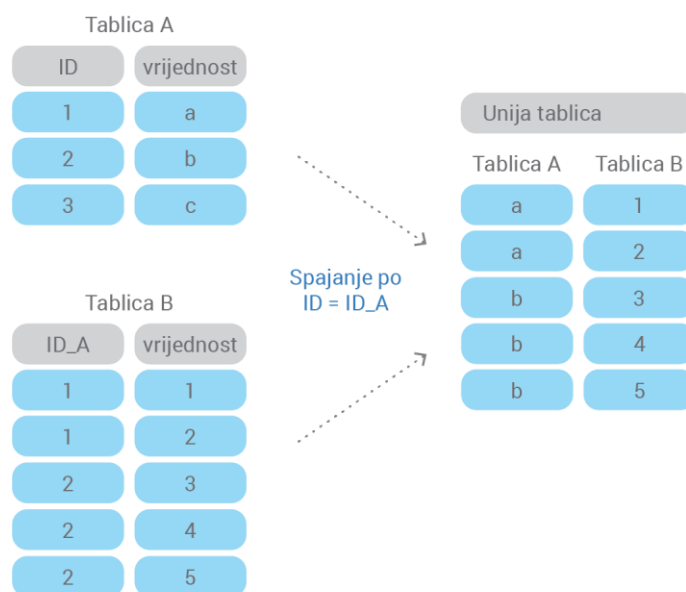
Simulacija selekcije JOIN-a u relacijskoj algebri izgleda kao u nastavku:

$$\sigma_{\theta}(R \times S)$$

- σ -simbol selekcije
- θ -simbol *theta* joina gdje operator može biti $>$, $<$, $>=$, $<=$, $=$

Prilikom spajanja dviju ili više tablica najčešće se koristi *Equijoin*. *Equijoin* je slučaj JOIN-a gdje se prilikom spajanja koristi znak jednakosti. To je najčešća vrsta spajanja dviju tablica.

13.1.2. Izgled INNER JOIN spoja



Slika 32. JOIN dvije tablice

13.1.3. Primjeri

Potrebno je ispisati sve nastavnike i nazive njihovih mjesta stanovanja.

```
SELECT nastavnici.*, mjesta.nazivMjesto FROM nastavnici
INNER JOIN mjesta ON nastavnici.postBr = mjesta.postBr;
```

ili

```
SELECT nastavnici.*, mjesta.nazivMjesto FROM nastavnici
JOIN mjesta ON nastavnici.postBr = mjesta.postBr;
```

Rezultati (51 n-torki):

idmjg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka	nazivMjesto
0110959390037	Petra	Jukić	Riječka 54	prof.vis.sk.		32000	378aaad427698bbcd8bd577b3914a91e	Vukovar
0205951330124	Vedran	Grubišić	Dobrilina 62	mr.sc.		10000	30f7799685334aed99267c75b5371a1fb	Zagreb
0305988391019	Lucija	Brčić	Naserov trg 1		prof.	10000	5d962f37c27a0dac27cfa4b5ff726f61	Zagreb
0404964361006	Danijel	Tkalčić	Nodilova 96	dr.sc.		21000	806fad8a67e9e1a041b8440451a1b84	Split
0501953210249	Anto	Bošnjak	Unska ulica 25		prof.	40000	db21dd893e3ddc0e1e4c64daba31995d	Čakovec
0509959360058	Leona	Vidović	Ulica svetog Duje 106	prof. dr.sc.		51000	67889942df943c1cf4a5f4df99887af4	Rijeka
0606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.		21000	e6993e93270eb1014b7118501a201a89	Split
0608955311207	Stjepan	Kovačević	Dubrovačka ulica 27	mr.sc.		42000	247d0bedf6f20caba466cad13ad720ee	Varaždin
0610975181955	Renato	Topić	Krešimirova 98	prof. dr.sc.		42000	9ad2279998eeddd857cdd2daecebaaea	Varaždin
0611973330036	Mila	Ilić	Ulica Alojzija Stepinca 2	dr.sc.	v. pred.	51000	4e099b86cb02c1431821a6b6c0f2f9b	Rijeka
07040897340304	Milica	Renata	Čandaka 95	dr.sc.		10000	61222f35e70a9a3f0a1f14ada370033	Zagreb

Pomoću INNER JOIN klauzule se dakle vrši presjek dva skupa brojeva, u ovom slučaju stupaca nastavnici.postBr i mjesta.postBr. Presjek je moguće prikazati matematički kao na sljedećoj slici:



Slika 33. INNER JOIN između tablica nastavnici i mjesto

Potrebno je ispisati sve nastavnike koji stanuju u Primorsko-goranskoj županiji.

```
SELECT nastavnici.* FROM nastavnici
INNER JOIN mjesto ON nastavnici.postBr = mjesto.postBr
INNER JOIN zupanije ON mjesto.idZupanija = zupanije.id
WHERE zupanije.nazivZupanija = 'Primorsko-goranska županija';
```

Rezultati izvođenja upita:

jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
0509959360058	Leona	Vidović	Ulica svetog Duje 106	prof. dr.sc.		51000	67889942df943c1cf4a5f4df99887af4
0611973330036	Mila	Ilić	Ulica Alojzija Stepinca 2	dr.sc.	v. pred.	51000	4e099b86cbd02c1431821a6b6c0f2f9b
1403968361918	Sanja	Ivanec	Mandićeva 65	prof. dr.sc.		51000	7b517185beee03aa53f9efc27e8d8067
2608948301518	Vladimir	Katić	Radićeva 32	dr.sc.		51000	a3d5f85f47657ab0390d74683fa0c0c9

Potrebno je ispisati unikatne nastavnike koji su izvršitelji na kolegijima na smjeru naziva „brodostrojarstvo“.

```
SELECT DISTINCT nastavnici.* FROM nastavnici
INNER JOIN izvršitelji ON nastavnici.jmbg =
    izvršitelji.jmbgNastavnik
INNER JOIN kolegiji ON izvršitelji.idKolegij = kolegiji.id
INNER JOIN smjerovi ON kolegiji.idSmjer = smjerovi.id
WHERE smjerovi.naziv = 'brodostrojarstvo';
```

Rezultati izvođenja upita:

jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
1403968361918	Sanja	Ivanec	Mandićeva 65	prof. dr.sc.		51000	7b517185beee03aa53f9efc27e8d8067
2608948301518	Vladimir	Katić	Radićeva 32	dr.sc.		51000	a3d5f85f47657ab0390d74683fa0c0c9

Potrebno je ispisati sve nastavnike koji rade na kolegiju „Baze podataka“, a žive u županiji „Grad Zagreb“:

```
SELECT DISTINCT nastavnici.*
FROM nastavnici INNER JOIN
    izvrsitelji ON nastavnici.jmbg =
        izvrsitelji.jmbgNastavnik INNER JOIN
    kolegiji ON izvrsitelji.idKolegij = kolegiji.id INNER JOIN
    mjesta ON nastavnici.postBr = mjesta.postBr INNER JOIN
    zupanije ON mjesta.idZupanija = zupanije.id
WHERE kolegiji.naziv = 'Baze podataka'
AND zupanije.nazivZupanija = 'Grad Zagreb';
```

Rezultati izvođenja upita:

☐ jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
☐ 2409954210041	TVRTKO	MILETIĆ	Gundulićeva 59		dipl.ing.rač.	10000	668f7f71056d78b87a801fbc853d917b
☐ 3103947210186	Tomica	Hrvat	Gospoka ulica 91	prof. dr.sc.		10000	67dala4ed66fcad0bbe3f8be21174d5a

Potrebno je ispisati sve nastavnike koji rade na kolegiju „Baze podataka“, a žive u županiji „Grad Zagreb“, imaju ulogu profesora i rade na smjeru „stručni studij informatike“:

```
SELECT DISTINCT nastavnici.*
FROM nastavnici INNER JOIN
    izvrsitelji ON nastavnici.jmbg =
        izvrsitelji.jmbgNastavnik INNER JOIN
    kolegiji ON izvrsitelji.idKolegij = kolegiji.id INNER JOIN
    mjesta ON nastavnici.postBr = mjesta.postBr INNER JOIN
    zupanije ON mjesta.idZupanija = zupanije.id INNER JOIN
    ulogaIzvrstjelja ON izvrsitelji.IdUlogaIzvrstjelja =
        ulogaIzvrstjelja.id INNER JOIN
    smjerovi ON smjerovi.id = kolegiji.idSmjer
WHERE kolegiji.naziv = 'Baze podataka' AND
    zupanije.nazivZupanija = 'Grad Zagreb' AND
    ulogaIzvrstjelja.naziv = 'profesor' AND
    smjerovi.naziv = 'Stručni studij informatike';
```

Rezultati izvođenja upita:

☐ jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
☐ 3103947210186	Tomica	Hrvat	Gospoka ulica 91	prof. dr.sc.		10000	67dala4ed66fcad0bbe3f8be21174d5a

Potrebno je ispisati sve nastavnike koji rade na kolegiju „Baze podataka“, a žive u županiji „Grad Zagreb“, imaju ulogu profesora i rade na smjeru „stručni studij informatike“ na ustanovi „Tehničko veleučilište u Zagrebu“:

```
SELECT DISTINCT nastavnici.*
FROM nastavnici INNER JOIN
    izvrsitelji ON nastavnici.jmbg =
        izvrsitelji.jmbgNastavnik INNER JOIN
    kolegiji ON izvrsitelji.idKolegij = kolegiji.id INNER JOIN
```

```

mjesti ON nastavnici.postBr = mjesti.postBr INNER JOIN
zupanije ON mjesti.idZupanija = zupanije.id INNER JOIN
ulogaIzvrstitelja ON izvrstitelji.IdUlogaIzvrstitelja =
ulogaIzvrstitelja.id INNER JOIN
smjerovi ON smjerovi.id = kolegiji.idSmjer INNER JOIN
ustanove ON ustanove.oib = smjerovi.oibUstanova
WHERE kolegiji.naziv = 'Baze podataka' AND
zupanije.nazivZupanija = 'Grad Zagreb' AND
ulogaIzvrstitelja.naziv = 'profesor' AND
smjerovi.naziv = 'Stručni studij informatike' AND
ustanove.naziv = 'Tehničko veleučilište u Zagrebu';

```

Rezultati izvođenja upita:

id	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
3103947210186	Tomica	Hrvat	Gospoška ulica 91	prof. dr.sc.		10000	67dala4ed66fcad0bbe3f8be21174d5a

13.2. NATURAL JOIN

NATURAL JOIN radi identično kao i INNER JOIN na način da automatski dodaje uvjet spajanja. Uvjet spajanja, tj. ON klauzulu u NATURAL JOIN-u nije potrebno pisati, ali je potrebno dobro obratiti pažnju da atributi preko kojih se spajaju dvije tablice budu identičnih naziva. Ako ne postoje atributi identičnih naziva u obje tablice, NATURAL JOIN će kreirati Kartezijev produkt što može biti jako opasno, te kreirati veliku i teško uočljivu grešku u programu.

Sintaksa:

```

SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
NATURAL JOIN tablica2
NATURAL JOIN tablica3

```

13.2.1. Primjeri:

Potrebno je ispisati sve nastavnike koji rade žive u Varaždinu.

```

SELECT nastavnici.*
FROM nastavnici NATURAL JOIN mjesti
WHERE mjesti.nazivMjesto = 'Varaždin';

```

Rezultati:

jmbag	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
0608955311207	Stjepan	Kovačević	Dubrovačka ulica 27	mr.sc.		42000	247d0bedf6f20caba466cad13ad720ee
0610975181955	Renato	Topić	Krešimirova 98	prof. dr.sc.		42000	9ad2279998eedd857cdd2daeecebaaa
1010962363604	Zvonimir	Ivanković	Šimunovićeve 39	dr.sc.	v. pred.	42000	92d2c1fc4d46c534e863229dcb17afd0
1102966363016	Žarko	Marković	Kupska ulica 101	prof. dr.sc.		42000	d9472c97406aled0f9a8580917503365
1707963340011	Ankica	Čeh	Ulica kralja Petra Krešimira IV 6	prof.		42000	0d94a5529152ca83d011daf25be975c9

Potrebno je ispisati sve studente i nazive njihovih mjesta stanovanja.

```
SELECT studenti.*, mjesta.nazivMjesto
FROM studenti NATURAL JOIN mjesta
```

Ovo rješenje je **netočno** zato što tablica mjesta i studenti nemaju identičan naziv stupca preko koje ih je potrebno spojiti, te nastaje Kartezijev produkt.

Točno rješenje potrebno je napisati pomoću INNER JOIN naredbe:

```
SELECT studenti.*, mjesta.nazivMjesto
FROM studenti INNER JOIN mjesta
ON studenti.postBrStanovanja = mjesta.postBr
```

Rezultati (162 n-torka):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	nazivMjesto
0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Zaprešić
0013020125	Matea	Omanović	2012-09-25	10410	44000	9	Sisak
0013022546	Antun	Mandić	2010-09-30	10410	31000	10	Osiijek
0013025926	Antonio	Pavić	2012-09-25	32000	44000	5	Sisak
0013026584	Borna	Knežević	2010-09-29	10410	44000	9	Sisak
0016096972	Dora	Vidaković	2011-09-26	42000	44000	16	Sisak
0016097990	Vesna	Brkić	2011-09-29	42000	43000	17	Bjelovar

13.3. CROSS JOIN

Cross join je zapravo Kartezijev produkt dvije tablice. CROSS JOIN dakle vraća umnožak dvije tablice.

Sintaksa:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
CROSS JOIN tablica2
CROSS JOIN tablica3
```

ili

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1, tablica2, tablica3
```

13.3.1. Primjer:

Potrebno je ispisati Kartezijev produkt tablice studenata i mjesta:


```
SELECT studenti.*, mjesta.nazivMjesto
FROM studenti CROSS JOIN mjesta
```

Rezultati (11016 n-torki):

id	imbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	nazivMjesto
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Zagreb
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Zaprešić
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Kupljenovo
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Ivanić-Grad
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Novoselec
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Vrbovec
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Gradec
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Velika Gorica
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Samobor
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Dubrovnik
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Split
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3	Marinka

13.4. LEFT OUTER JOIN

LEFT OUTER JOIN je vanjsko spajanje tablica kod koje je lijeva tablica dominantna što znači da će upit vratiti sve n-torke lijeve tablice, dok će podaci desne tablice imati NULL vrijednosti na mjestima gdje ne postoji veza na n-torku lijeve tablice.

Sintaksa:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
LEFT OUTER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
LEFT OUTER JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

Ili skraćeno:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
LEFT JOIN tablica2 ON tablica1.stupac = tablica2.stupac
LEFT JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

13.4.1. Relacijska algebra LEFT OUTER JOIN klauzule

Ako postoje dvije relacije (tablice) A i B, A tablica je lijeva, LEFT OUTER JOIN se označava kao u nastavku:

$$A \bowtie B$$

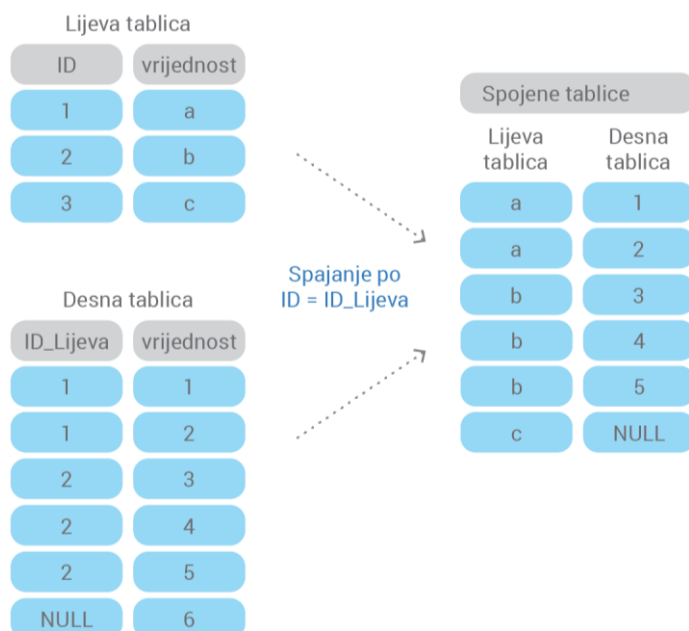
Formulu je moguće napisati pomoću baznih operatora:

$$(A \bowtie B) \cup (A - \pi_{a_1, a_2, \dots, a_n}(A \bowtie B)) \times \{(\omega, \dots, \omega)\}$$

- A i B su relacije
- π – projekcija za attribute a_1 do a_n iz tablice A

- $\omega, \dots \omega$ – redci unutar tablice koji se ne vežu za na tablicu B

13.4.2. Izgled LEFT OUTER JOIN spoja



Slika 34. LEFT OUTER JOIN spoj dvije tablice

13.4.3. Primjeri:

Potrebno je ispisati sve županije i sva mjesta koje one sadrže. Ako ne postoji ni jedno mjesto u određenoj županiji, potrebno je svejedno ispisati županiju, a za mjesto null vrijednosti:

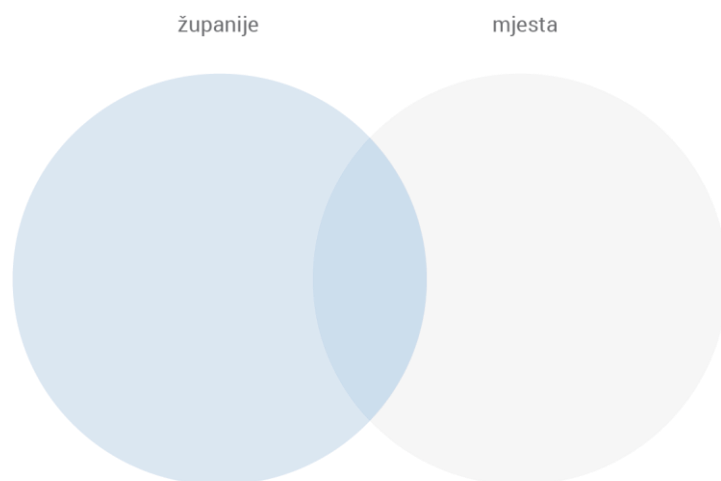
```
SELECT *
FROM zupanije LEFT OUTER JOIN
    mjesta ON zupanije.id = mjesta.idZupanija;
```

ili

```
SELECT *
FROM zupanije LEFT JOIN
    mjesta ON zupanije.id = mjesta.idZupanija;
```

Rezultati (70 n-torki):

id	nazivŽupanija	postbr	nazivMjesto	idŽupanija
1	Grad Zagreb	10000	Zagreb	1
1	Grad Zagreb	10290	Zaprešić	1
2	Zagrebačka županija	10295	Kupljenovo	2
2	Zagrebačka županija	10310	Ivanić-Grad	2
2	Zagrebačka županija	10315	Novoselec	2
2	Zagrebačka županija	10340	Vrbovec	2
2	Zagrebačka županija	10345	Gradec	2
2	Zagrebačka županija	10410	Velika Gorica	2
2	Zagrebačka županija	10430	Samobor	2
3	Krapinsko-zagorska županija	(NULL)	(NULL)	(NULL)
4	Sisačko-moslavačka županija	44000	Sisak	4
4	Sisačko-moslavačka županija	44010	Sisak-Caprag	4
5	Karlovačka županija	47000	Karlovac	5
5	Karlovačka županija	47285	Badelj	5



Slika 35. LEFT OUTER JOIN tablice zupanije i mjesta (zupanija je dominantna)

Ispisati sve smjerove i kolegije koji se održavaju na istima, a ako ne postoji ni jedan kolegij na smjeru potrebno je ispisati NULL vrijednosti:

```
SELECT *
FROM smjerovi LEFT OUTER JOIN
      kolegiji ON smjerovi.id = kolegiji.idSmjer;
```

Rezultati (52 n-torke):

id	naziv	oibUstanova	idNadsmjer	id	naziv	opis
1	Stručni studij informat	08814003451	22	1	Baze podataka	Student će steći znanja i vještine za diz...
1	Stručni studij informat	08814003451	22	2	Uvod u UNIX sustave	Student treba savladati osnovno znanje ra...
1	Stručni studij informat	08814003451	22	43	Tjelesna i zdravstvena	Odgojno-obrazovno područje podmi...
1	Stručni studij informat	08814003451	22	46	Napredne baze podataka	Student se treba upoznati s objektima i k...
2	Stručni studij računarst	08814003451	22	3	Uvod u UNIX sustave	Student treba savladati osnovno znanje ra...
2	Stručni studij računarst	08814003451	22	4	Fizika	Razumijevanje fizikalnih pojava i veličin...
3	Stručni studij elektrot	08814003451	(NULL)	5	Fizika	Razumijevanje fizikalnih pojava i veličin...
3	Stručni studij elektrot	08814003451	(NULL)	8	Osnove elektrotehnike	Stjecanje znanja iz teorije električnih k...
4	Stručni studij graditel	08814003451	(NULL)	6	Fizika	Razumijevanje fizikalnih pojava i veličin...
4	Stručni studij graditel	08814003451	(NULL)	7	Betonske konstrukcije	Stjecanje teoretskih i praktičkih znanja ...
5	studij trgovine	21720825730	(NULL)	9	Poslovna etika	Cilj je predmeta upoznati studente s etič...
5	studij trgovine	21720825730	(NULL)	10	Statistika	Opći ciljevi su ovladavanje teorijskim sp...
6	upravni studij	21720825730	(NULL)	11	Osnove građanskog prava	U općem djelu građanskog prava stječu se ...
6	upravni studij	21720825730	(NULL)	12	Javna nabava	Student će biti osposobljen za organizaci...
7	računarstvo	31444990605	(NULL)	15	Operacijski sustavi	Operacijski je sustav skup programa koji ...
7	računarstvo	31444990605	(NULL)	16	Baze podataka	Student će steći znanja i vještine za diz...
8	menadžment turizma i sr	31444990605	(NULL)	13	Ekonomika turizma	Stjecanje specifičnih znanja za prepoznav...
8	menadžment turizma i sr	31444990605	(NULL)	14	Sportski marketing	U okviru ovog predmeta studenti će se upo...
9	očna optika	09032023114	(NULL)	17	Anatomija i fiziologija	Stjecanje temeljitih znanja iz anatomije ...

Potrebno je ispisati sva mjesta koja se nalaze unutar Zagrebačke županije kao i nastavnike koji žive u istima. Ako ne postoji nastavnik koji živi u mjestu unutar županije, potrebno je svejedno ispisati mjesto, a unutar stupaca nastavnika NULL vrijednosti.

```
SELECT mjesta.nazivMjesto, nastavnici.*
FROM zupanije INNER JOIN
      mjesta ON zupanije.id = mjesta.idZupanija LEFT OUTER JOIN
      nastavnici ON mjesta.postBr = nastavnici.postBr
WHERE zupanije.nazivZupanija = 'Zagrebačka županija';
```

Rezultati:

☐ nazivMjesto	jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
☐ Kupljenovo	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Ivanić-Grad	1303966335145	Valentino	Posavec	Ulica dr. Ante Starčevića 9	prof. dr.sc.		10310	b02048802c7d982398fc192da5660338
☐ Ivanić-Grad	2805952392713	Lovro	Horvatinčić	Škurinjska cesta 47	dr.sc.	v. pred.	10310	6d255cc57954alc231fd8bfae3b7a782
☐ Novoselec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Vrbovec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Gradec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Velika Gorica	0804969306801	Stjepan	Lukić	Marko Polo 36	prof. dr.sc.		10410	120db3b28966d6678a63baf597811b30
☐ Velika Gorica	1409987320508	Zlatica	Vidaković	Zvonimirova 32		prof.	10410	e046cb08b64f423a16a079ecfbed4led
☐ Velika Gorica	1910956363012	Dražen	Debeljak	Trpimirova 15	dr.sc.	v. pred.	10410	9aa31e57ed4c9a4b6ce586f05b764fef
☐ Velika Gorica	2409952381906	Morana	Novak	Kolodvorska 37		prof.	10410	fc9e9539cald04da45df8c5d79ae8f18
☐ Velika Gorica	2808966330057	Mira	Mandić	Ulica Svetog Vlaho 95	prof. dr.sc.		10410	552bfc1303b329ede95722afcl7e2lf7
☐ Samobor	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Potrebno je ispisati ime i prezime nastavnika te naziv kolegija na kojemu rade. Potrebno je:

- ako nastavnik radi na više kolegija - ispisati sve kolegije,
- ako ne rade ni na jednom kolegiju - ispisati NULL vrijednosti.

```
SELECT nastavnici.ime, nastavnici.prezime, kolegiji.naziv
FROM nastavnici LEFT OUTER JOIN
      izvrsitelji ON nastavnici.jmbg = izvrsitelji.jmbgNastavnik
LEFT OUTER JOIN
      kolegiji ON izvrsitelji.idKolegij = kolegiji.id;
```

Rezultati:

☐ ime	prezime	naziv
☐ Petra	Jukić	Osnove građanskog prava
☐ Petra	Jukić	Osnove građanskog prava
☐ Vedran	Grubišić	Osnove elektrotehnike
☐ Lucija	Brčić	Energijske tehnologije
☐ Danijel	Tkalčić	Kvantitativne metode u managementu
☐ Anto	Bošnjak	Ekonomika turizma
☐ Leona	Vidović	Tereti u prometu
☐ Leona	Vidović	Planiranje prometne potražnje
☐ Borna	Blažević	Marketing destinacije
☐ Stjepan	Kovačević	Osnove poduzetništva
☐ Stjepan	Kovačević	Marketing
☐ Renato	Topić	Marketing
☐ Mila	Ilić	Planiranje prometne potražnje
☐ Miliro	Matić	Planiranje prometne potražnje

13.5. RIGHT OUTER JOIN

RIGHT OUTER JOIN je vanjsko spajanje tablica kod koje je desna tablica dominantna što znači da će upit vratiti sve n-torke desne tablice, dok će podaci lijeve tablice imati NULL vrijednosti na mjestima gdje ne postoji veza na n-torku desne tablice. RIGHT OUTER JOIN zapravo radi identično kao i LEFT OUTER JOIN, samo je desna tablica dominantna umjesto lijeve.

Sintaksa:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
RIGHT OUTER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
RIGHT OUTER JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

Ili skraćeno:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
RIGHT JOIN tablica2 ON tablica1.stupac = tablica2.stupac
RIGHT JOIN tablica3 ON tablica2.stupac = tablica3.stupac...
```

13.5.1. Relacijska algebra RIGHT OUTER JOIN klauzule

Ako postoje dvije relacije (tablice) A i B, A tablica je lijeva, RIGHT OUTER JOIN se označava kao u nastavku:

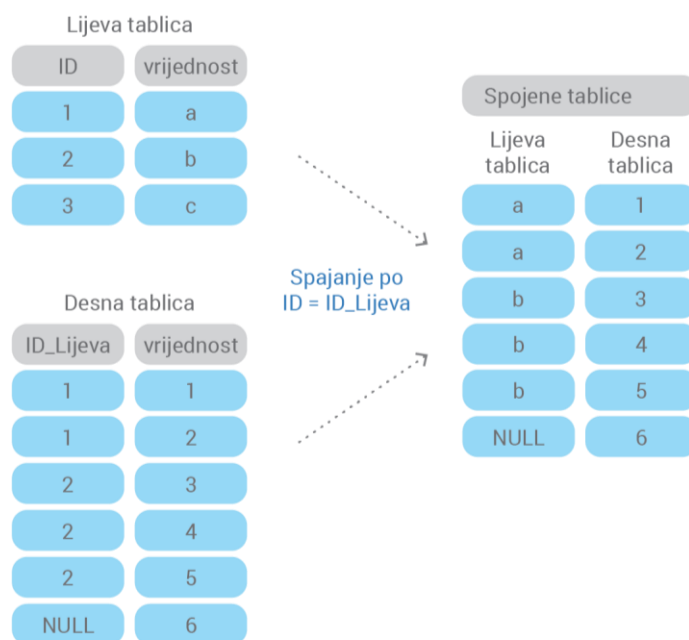
$$A \bowtie B$$

Formulu je moguće napisati pomoću baznih operatora:

$$(A \bowtie B) \cup (\{(\omega, \dots, \omega)\} \times (B - \pi_{b_1, b_2, \dots, b_n}(A \bowtie B)))$$

- A i B su relacije
- π – projekcija za atribute a_1 do a_n iz tablice A
- $\omega, \dots, \omega$ – redci unutar tablice koji se ne vežu za na tablicu A

13.5.2. Izgled RIGHT OUTER JOIN spoja



Slika 36. RIGHT OUTER JOIN spoj dvije tablice

13.5.3. Primjeri:

Potrebno je ispisati sve županije i sva mjesta koje one sadrže. Ako ne postoji ni jedno mjesto u određenoj županiji, potrebno je svejedno ispisati županiju, a za mjesto NULL vrijednosti:

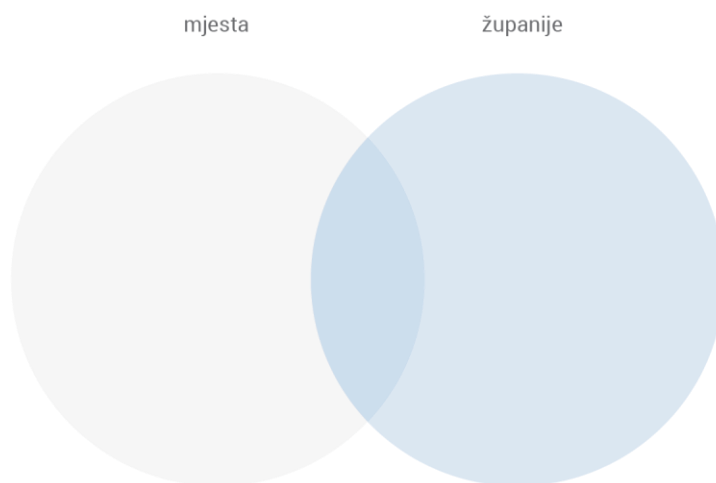
```
SELECT *
FROM mjesta RIGHT OUTER JOIN
      zupanije ON zupanije.id = mjesta.idZupanija;
```

ili

```
SELECT *
FROM mjesta RIGHT JOIN
      zupanije ON zupanije.id = mjesta.idZupanija;
```

Rezultati (70 n-torki):

id	nazivŽupanija	postbr	nazivMjesto	idŽupanija
1	Grad Zagreb	10000	Zagreb	1
1	Grad Zagreb	10290	Zaprešić	1
2	Zagrebačka županija	10295	Kupljenovo	2
2	Zagrebačka županija	10310	Ivanić-Grad	2
2	Zagrebačka županija	10315	Novoselec	2
2	Zagrebačka županija	10340	Vrbovec	2
2	Zagrebačka županija	10345	Gradec	2
2	Zagrebačka županija	10410	Velika Gorica	2
2	Zagrebačka županija	10430	Samobor	2
3	Krapinsko-zagorska županija	(NULL)	(NULL)	(NULL)
4	Sisačko-moslavačka županija	44000	Sisak	4
4	Sisačko-moslavačka županija	44010	Sisak-Caprag	4
5	Karlovačka županija	47000	Karlovac	5
5	Karlovačka županija	47285	Radatovići	5
6	Varaždinska županija	42000	Varaždin	6



Slika 37. RIGHT OUTER JOIN tablica mjesta i županije (županija je dominantna)

Ispisati sve smjerove i kolegije koji se održavaju na istima, a ako ne postoji ni jedan kolegij na smjeru potrebno je ispisati NULL vrijednosti:

```
SELECT *
FROM kolegiji RIGHT OUTER JOIN
smjerovi ON smjerovi.id = kolegiji.idSmjer;
```

Rezultati (52 n-torke):

id	naziv	opis	idSmjer	id	naziv	oibbStanova	idNadmjes
45	Pjelesna i zdravstvena kultura 3	Odgovorno-obrazovno područje podmirjenja biopsihosocijalnih ...	2275	16	16 informacijski i poslovni sustavi	02024002310	(NULL)
33	Osnove poduzetništva	Pojmovno određenje poduzetništva. Fovijesni i komparativn...	1558	17	17 ekonomika poduzetništva	02024002310	(NULL)
34	Marketing	Marketing kao filozofija poslovanja; Priroda marketinškog...	1468	17	17 ekonomika poduzetništva	02024002310	(NULL)
35	Mikroekonomija I	Razumijevanje principa ponude i potražnje te njihovo pove...	2108	18	18 poslovna ekonomija	84477684422	(NULL)
36	Kvantitativne metode u managementu	Upoznavanje sa osnovnim kvantitativnim metodama koje se m...	1368	18	18 poslovna ekonomija	84477684422	(NULL)
37	Matematika u ekonomiji	S ovim predmetom studenti bi trebali biti osposobljeni za...	1638	19	19 turizam	84477684422	(NULL)
38	Marketing destinacije	Specifičnosti upravljanja marketingom destinacije;Turisti...	1118	19	19 turizam	84477684422	(NULL)
39	Tehnička mehanika 2	Upoznati studente s teorijskim i iskustvenim spoznajama i...	2168	20	20 brodograđarstvo	76722145702	(NULL)
40	Sredstva pomorskog prometa	Cilj kolegija je upoznati studente sa stabilnošću broda u...	1418	20	20 brodograđarstvo	76722145702	(NULL)
41	Tereti u prometu	Osnovni cilj i zadatak kolegija je upoznavanja prirode ma...	2168	21	21 logistika i menadžment u pomorstvu	76722145702	(NULL)
42	Planiranje prometne potražnje	Oposobiti studenta za primjenu metoda kojima se utvrđuje...	918	21	21 logistika i menadžment u pomorstvu	76722145702	(NULL)
(NULL)	(NULL)	(NULL)	OK	(NULL)	22 INRO	08814003451	23
(NULL)	(NULL)	(NULL)	OK	(NULL)	23 Stručni studij	08814003451	(NULL)
(NULL)	(NULL)	(NULL)	OK	(NULL)	24 Specijalizirani studij	08814003451	(NULL)

Potrebno je ispisati sva mjesta koja se nalaze unutar Zagrebačke županije kao i nastavnike koji žive u istima. Ako ne postoji nastavnik koji živi u mjestu unutar županije, potrebno je svedjedno ispisati mjesto, a unutar stupaca nastavnika NULL vrijednosti.

```
SELECT mjesta.nazivMjesto, nastavnici.
FROM nastavnici RIGHT OUTER JOIN
    mjesta ON mjesta.postBr = nastavnici.postBr INNER JOIN
    zupanije ON zupanije.id = mjesta.idZupanija
WHERE zupanije.nazivZupanija = 'Zagrebačka županija';
```

Rezultati:

☐ nazivMjesto	jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
☐ Kupljenovo	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Ivanić-Grad	1303966335145	Valentino	Posavec	Ulica dr. Ante Starčevića 9	prof. dr.sc.		10310	b02048802c7d982398fc192da5660338
☐ Ivanić-Grad	2805952392713	Lovro	Horvatinčić	Škurinjska cesta 47	dr.sc.	v. pred.	10310	6d255cc57954alc231fd8bfae3b7a782
☐ Novoselec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Vrbovec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Gradec	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)
☐ Velika Gorica	0804969306801	Stjepan	Lukić	Marko Polo 36	prof. dr.sc.		10410	120db3b28966d6678a63baf597811b30
☐ Velika Gorica	1409987320508	Zlatica	Vidaković	Zvonimirova 32		prof.	10410	e046cb08b64f423a16a079ecfbed4led
☐ Velika Gorica	1910956363012	Dražen	Debeljak	Trpimirova 15	dr.sc.	v. pred.	10410	9aa31e57ed4c9a4b6ce586f05b764fef
☐ Velika Gorica	2409952381906	Morana	Novak	Kolodvorska 37		prof.	10410	fc9e9539cald04da45df8c5d79ae8f18
☐ Velika Gorica	2808966330057	Mira	Mandić	Ulica Svetog Vlaho 95	prof. dr.sc.		10410	552bfc1303b329ede95722afcl7e21f7
☐ Samobor	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Potrebno je ispisati ime i prezime nastavnika te naziv kolegija na kojemu rade. Ako rade na više kolegija je potrebno ispisati ih sve, a ako ne rade niti na jednom kolegiju je potrebno ispisati NULL vrijednosti.

```
SELECT nastavnici.ime, nastavnici.prezime, kolegiji.naziv
FROM kolegiji RIGHT OUTER JOIN
    izvrsitelji ON izvrsitelji.idKolegij = kolegiji.id
RIGHT OUTER JOIN
    nastavnici ON nastavnici.jmbg = izvrsitelji.jmbgNastavnik;
```

Rezultati (63 n-torke):

☐ ime	prezime	naziv
☐ Petra	Jukić	Osnove građanskog prava
☐ Petra	Jukić	Osnove građanskog prava
☐ Vedran	Grubišić	Osnove elektrotehnike
☐ Lucija	Brčić	Energijske tehnologije
☐ Danijel	Tkalčić	Kvantitativne metode u managementu
☐ Anto	Bošnjak	Ekonomika turizma
☐ Leona	Vidović	Tereti u prometu
☐ Leona	Vidović	Planiranje prometne potražnje
☐ Borna	Blažević	Marketing destinacije
☐ Stjepan	Kovačević	Osnove poduzetništva
☐ Stjepan	Kovačević	Marketing
☐ Renato	Topić	Marketing
☐ Mila	Ilić	Planiranje prometne potražnje
☐ Milivoj	Matić	Linearna algebra 1
☐ Milivoj	Matić	Mjera i integral

13.6. FULL OUTER JOIN

FULL OUTER JOIN je vanjsko spajanje tablica kod kojeg su obje tablice dominantne i podređene, što znači da će upit vratiti sve n-torke obje tablice, dok će se u slučaju nevezanih n-torki u tom trenutku podređena tablica imati NULL vrijednosti. FULL OUTER JOIN u MySQL-u ne postoji kao jedinstvena izjava (engl. statement), već ju je potrebno izvesti pomoću LEFT OUTER JOIN izjave u uniji s RIGHT OUTER JOIN izjavom.

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
FULL OUTER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
```

U MySQL-u:

```
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
LEFT OUTER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
UNION
SELECT tablica1.stupac, tablica2.stupac
FROM tablica1
RIGHT OUTER JOIN tablica2 ON tablica1.stupac = tablica2.stupac
```

13.6.1. Relacijska algebra FULL OUTER JOIN klauzule

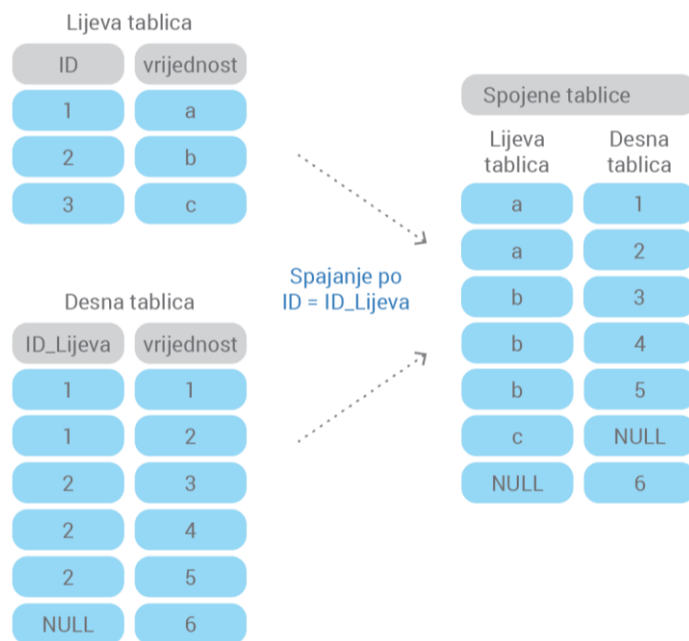
Ako postoje dvije relacije (tablice) A i B, A tablica je lijeva, RIGHT OUTER JOIN se označava kao u nastavku:

$$A \bowtie B$$

Formulu je moguće raspisati pomoću unije lijevog i desnog spoja:

$$(A \bowtie B) \cup (A \ltimes B)$$

13.6.2. Izgled FULL OUTER JOIN spoja



Slika 38. FULL OUTER JOIN spoj dvije tablice

13.6.3. Primjeri

Potrebno je prikazati nazive mjesta i nazive županija. Potrebno prikazati sve retke obje tablice.

U MS SQL Server DBMS-u upit izgleda ovako:

```
SELECT mjesta.nazivMjesto, zupanije.nazivZupanija
FROM mjesta FULL OUTER JOIN
      zupanije ON zupanije.sifZupanija = mjesta.sifZupanija
```

Rezultati (70 n-torki):

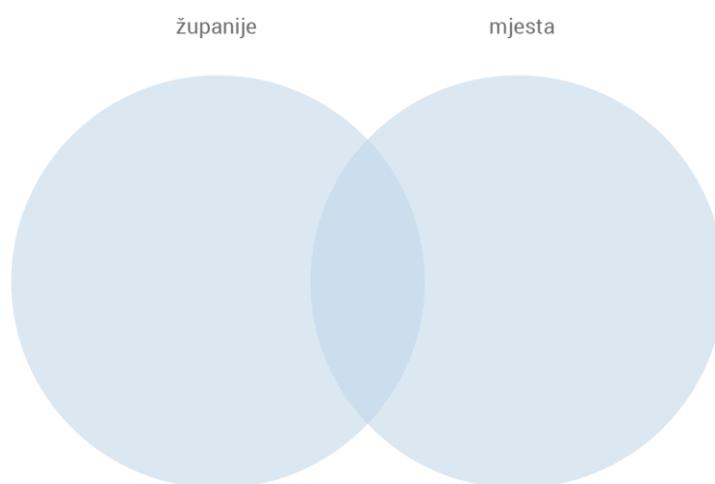
	nazivMjesto	nazivŽupanija
1	Zagreb	Grad Zagreb
2	Zagreb-Sloboština	Grad Zagreb
3	Zagreb-Noví Zagreb	Grad Zagreb
4	Zagreb-Dubrava	Grad Zagreb
5	Zagreb-Susedgrad	Grad Zagreb
6	Lučko	Zagrebačka
7	Gornji Stupnik	Zagrebačka
8	Zaprešić	Zagrebačka
9	Kupljenovo	Zagrebačka
10	Ivanić-Grad	Zagrebačka
11	Novoselec	Zagrebačka
12	Vrbovec	Zagrebačka
13	Gradec	Zagrebačka
14	Sesvete	Zagrebačka
15	Dugo Selo	Zagrebačka
16	Sveti Ivan Zelina	Grad Zagreb

Na MySQL DBMS-u upit je potrebno napisati na sljedeći način:

```
SELECT mjesta.nazivMjesto, zupanije.nazivZupanija
FROM mjesta LEFT OUTER JOIN
      zupanije ON zupanije.id = mjesta.idZupanija
UNION
SELECT mjesta.nazivMjesto, zupanije.nazivZupanija
FROM mjesta RIGHT OUTER JOIN
      zupanije ON zupanije.id = mjesta.idZupanija;
```

Rezultati (70 n-torki):

☐	nazivMjesto	nazivŽupanija
☐	Zagreb	Grad Zagreb
☐	Zaprešić	Grad Zagreb
☐	Kupljenovo	Zagrebačka županija
☐	Ivanić-Grad	Zagrebačka županija
☐	Novoselec	Zagrebačka županija
☐	Vrbovec	Zagrebačka županija
☐	Gradec	Zagrebačka županija
☐	Velika Gorica	Zagrebačka županija
☐	Samobor	Zagrebačka županija
☐	Dubrovnik	Dubrovačko-neretvanska županija
☐	Split	Splitsko-dalmatinska županija
☐	Makarska	Splitsko-dalmatinska županija



Slika 39. FULL OUTER JOIN tablica mjesta i županije

14. Aliasi

Alias je funkcionalnost podržana od svih relacijskih baza podataka koja služi promjeni naziva stupca ili tablica unutar upita. Najčešće se koristi kako bi se smanjila duljina koda upita, kako bi se povećala čitljivost programskog koda i kako bi se pojedina tablica mogla koristiti više puta (s različitim vezama) u jednom upitu. Postoje dvije vrste aliasa:

- Alias na naziv atributa (stupca)
- Alias na naziv tablica

14.1. ALIASI NA NAZIV ATRIBUTA

Alias na attribute služi kako bi se u projekciji izmijenio naziv stupca što je često jako korisno zato što se iz programskog jezika podacima pristupa pomoću naziva stupca. Iz razloga što baze podataka najčešće razvija jedna grupa programera, a aplikaciju druga grupa zbog što boljeg uzajamnog razumijevanja programskog koda dobra je praksa koristiti intuitivne nazive stupaca.

Sintaksa:

```
SELECT naziv_stupca AS zamjensko_ime  
FROM tablica
```

ili

```
SELECT naziv_stupca zamjensko_ime  
FROM tablica
```

Na primjer, moguće je koristiti neke funkcije u SELECT djelu naredbe. U slučaju da se polju ne dodjeli alias, ono će imati naziv cijele funkcije što je jako nepregledno:

```
SELECT DATE_FORMAT(STR_TO_DATE(jmbg, "%d%m"), "%d.%m."), prezime  
FROM nastavnici;
```

Izgled ispisa:

☐ DATE_FORMAT(STR_TO_DATE(jmbg, "%d%m"), "%d.%m.")	prezime
☐ 01.10.	Jukić
☐ 02.05.	Grubišić
☐ 03.05.	Brčić
☐ 04.04.	Tkalčić
☐ 05.01.	Bošnjak
☐ 05.09.	Vidović
☐ 06.06.	Blažević
☐ 06.08.	Kovačević
☐ 06.10.	Topić
☐ 06.11.	Ilić
☐ 07.04.	Matić
☐ 07.08.	Brkić

Umjesto da stupac ima naziv „DATE_FORMAT(STR_TO_DATE(jmbg, \"%d%m\"), \"%d.%m.\")“ koji je jako nepraktičan, bilo bi bolje stupcu dodijeliti alias npr. „danRodjenja“:

```
SELECT DATE_FORMAT(STR_TO_DATE(jmbg, \"%d%m\"), \"%d.%m.\") AS
danRodjenja, prezime
FROM nastavnici;
```

Izgled ispisa s aliasom:

☐	danRodjenja	prezime
☐	01.10.	Jukić
☐	02.05.	Grubišić
☐	03.05.	Brčić
☐	04.04.	Tkalčić
☐	05.01.	Bošnjak
☐	05.09.	Vidović
☐	06.06.	Blažević
☐	06.08.	Kovačević

14.1.1. Primjer:

Ispisati poštanske brojeve i nazive mjesta iz tablice mjesta. Neka naziv stupca za poštanski broj bude „postanski_broj“, a za naziv mjesta „naziv“.

```
SELECT postbr postanski_broj, nazivMjesto naziv FROM mjesta;
```

ili

```
SELECT postbr AS postanski_broj, nazivMjesto AS naziv FROM mjesta;
```

Rezultati izvođenja upita:

☐	postanski_broj	naziv
☐	10000	Zagreb
☐	10290	Zaprešić
☐	10295	Kupljenovo
☐	10310	Ivanić-Grad
☐	10315	Novoselec
☐	10340	Vrbovec
☐	10345	Gradec
☐	10410	Velika Gorica

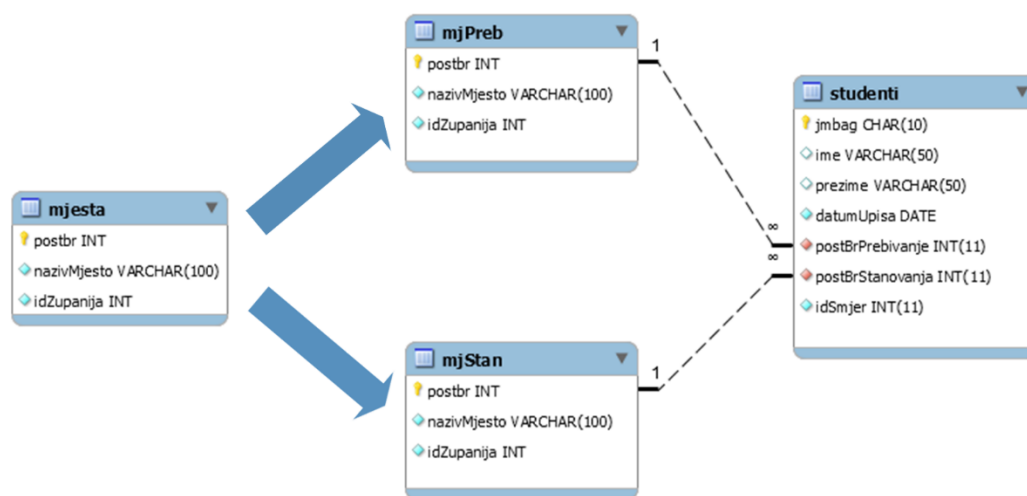
14.2. ALIASI NA NAZIV TABLICA

Alias na nazive tablica služe kako bi se promijenio naziv tablice zbog skraćivanja programskog koda, preglednosti istog ili višekratnog korištenja iste tablice unutar jednog upita.

Na primjer, potrebno je ispisati nazive mjesta stanovanja i nazive mjesta prebivanja svih studenata:

```
SELECT mjStan.nazivMjesto, mjPreb.nazivMjesto, studenti.*
FROM mjesta AS mjStan INNER JOIN
     studenti ON mjStan.postBr = studenti.postBrStanovanja
INNER JOIN
     mjesta AS mjPreb ON studenti.postBrPrebivanje =
     mjPreb.postBr;
```

U primjeru je prikazano kako se jedna tablica može koristiti za dobivanje više različitih informacija, odnosno 'više puta' unutar istog upita korištenjem aliasa. Čim se doda alias na naziv tablice, nju je nadalje u upitu potrebno referencirati isključivo pomoću njenog aliasa. Tablicu nije moguće više referencirati pomoću starog imena!



Slika 40. Dodavanje aliasa na tablicu mjesta

Obzirom da je bitno da tablica *mjesta* ima različite nazive, u gornjem primjeru moguće je za tablicu *mjesta* jednom koristiti njeno izvorno ime, a alias tek prilikom poziva tablice *mjesta* za dobivanje druge informacije. U nastavku je rješenje samo s jednim aliasom:

```
SELECT mjesta.nazivMjesto, mjPreb.nazivMjesto, studenti.*
FROM mjesta INNER JOIN
     studenti ON mjesta.postBr = studenti.postBrStanovanja
INNER JOIN
     mjesta AS mjPreb ON studenti.postBrPrebivanje =
     mjPreb.postBr;
```

Aliase je također moguće koristiti i s prirodnim spojem, a ne samo s JOIN izjavom:

```
SELECT mjesta.nazivMjesto, mjPreb.nazivMjesto, studenti.*
FROM mjesta, studenti, mjesta AS mjPreb
WHERE mjesta.postBr = studenti.postBrStanovanja AND
     studenti.postBrPrebivanje = mjPreb.postBr;
```

14.2.1. Primjeri:

Potrebno je ispisati sve unikatne studente koji su imali ocjenu veću od 3, mjesto prebivanja je Zagreb a mjesto stanovanja Bjelovar.

```
SELECT DISTINCT studenti.*
FROM studenti INNER JOIN
    ocjene ON studenti.jmbag = ocjene.jmbagStudent INNER JOIN
    mjesta ON studenti.postBrStanovanja= mjesta.postBr INNER JOIN
    mjesta AS mjPreb ON studenti.postBrPrebivanje = mjPreb.postBr
WHERE mjesta.nazivMjesto = 'Bjelovar' AND
    mjPreb.nazivMjesto = 'Zagreb' AND
    ocjene.ocjena > 3;
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0035182233	Ivan	Turk	2013-09-26	10000	43000	3
☐	0035192565	Kristina	Matleković	2015-09-27	10000	43000	4
☐	0036430456	Dragutina	Topić	2016-09-26	10000	43000	15
☐	0128052105	Luka	Šimić	2017-09-25	10000	43000	2

Potrebno je ispisati sve unikatne studente koji su imali ocjenu veću od 3, prebivaju u županiji „Grad zagreb“, a mjesto stanovanja je u županiji „Vukovarsko-srijemska županija“.

```
SELECT DISTINCT studenti.*
FROM studenti INNER JOIN
    ocjene ON studenti.jmbag = ocjene.jmbagStudent INNER JOIN
    mjesta ON studenti.postBrStanovanja= mjesta.postBr INNER JOIN
    zupanije ON mjesta.idZupanija = zupanije.id INNER JOIN
    mjesta AS mjPreb ON studenti.postBrPrebivanje =
        mjPreb.postBr INNER JOIN
    zupanije AS zupPreb ON mjPreb.idZupanija = zupPreb.id
WHERE zupanije.nazivZupanija = 'Vukovarsko-srijemska županija' AND
    zupPreb.nazivZupanija = 'Grad zagreb' AND
    ocjene.ocjena > 3;
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	1191220781	Marina	Krmpotić	2017-09-27	10000	32100	13

Potrebno je ispisati sve studente koji imaju mjesto stanovanja u županiji „Splitsko-dalmatinska županija“, a studiraju na ustanovi koja ima sjedište u županiji „Grad Zagreb“.

```
SELECT DISTINCT studenti.*
FROM studenti INNER JOIN
    mjesta AS stStan ON studenti.postBrStanovanja = stStan.postBr
    INNER JOIN
    zupanije AS stZup ON stStan.idZupanija = stZup.id INNER JOIN
    smjerovi ON studenti.idSmjer = smjerovi.id INNER JOIN
    ustanove ON smjerovi.oibUstanova = ustanove.oib INNER JOIN
```



```
mjesta AS usSjed ON ustanove.postBr = usSjed.postBr
INNER JOIN
zupanije AS zupSjed ON usSjed.idZupanija = zupSjed.id
WHERE stZup.nazivZupanija = 'Splitsko-dalmatinska županija'
AND zupSjed.nazivZupanija = 'Grad Zagreb';
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0035180940	Sonja	Šimpraga	2016-09-26	10000	21000	4
0246025333	Ksenija	Božić	2015-09-30	10000	21300	4
0036435111	Maja	Vuković	2016-09-27	10000	21300	14

Potrebno je ispisati sve studente koji imaju mjesto stanovanja u županiji „Splitsko-dalmatinska županija“, studiraju na ustanovi koja ima sjedište u županiji „Grad Zagreb“, imaju prebivanje u županiji „Grad Zagreb“ i predaju im nastavnici koji također žive u županiji „Grad Zagreb“:

```
SELECT DISTINCT studenti.*
FROM studenti INNER JOIN
    mjesta AS stStan ON studenti.postBrStanovanja = stStan.postBr
INNER JOIN
    zupanije AS stZup ON stStan.idZupanija = stZup.id INNER JOIN
    smjerovi ON studenti.idSmjer = smjerovi.id INNER JOIN
    ustanove ON smjerovi.oibUstanova = ustanove.oib INNER JOIN
    mjesta AS usSjed ON ustanove.postBr = usSjed.postBr INNER
JOIN
    zupanije AS zupSjed ON usSjed.idZupanija = zupSjed.id INNER
JOIN
    mjesta AS stPreb ON studenti.postBrPrebivanje = stPreb.postBr
INNER JOIN
    zupanije AS stZupPreb ON stPreb.idZupanija = stZupPreb.id
INNER JOIN
    kolegiji ON smjerovi.id = kolegiji.idSmjer INNER JOIN
    izvorsitelji ON kolegiji.id = izvorsitelji.idKolegij INNER JOIN
    nastavnici ON izvorsitelji.jmbgNastavnik = nastavnici.jmbg
INNER JOIN
    mjesta AS mjNastavnik ON mjNastavnik.postBr =
nastavnici.postBr INNER JOIN
    zupanije AS zupNastavnik ON zupNastavnik.id =
mjNastavnik.idZupanija
WHERE stZup.nazivZupanija = 'Splitsko-dalmatinska županija' AND
    zupSjed.nazivZupanija = 'Grad Zagreb' AND
    stZupPreb.nazivZupanija = 'Grad Zagreb' AND
    zupNastavnik.nazivZupanija = 'Grad Zagreb';
```

Rezultati izvođenja upita:

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0035180940	Sonja	Šimpraga	2016-09-26	10000	21000	4
0246025333	Ksenija	Božić	2015-09-30	10000	21300	4
0036435111	Maja	Vuković	2016-09-27	10000	21300	14

15. Limit

Nakon što SELECT naredba vrati set rezultata, koji puta ga je potrebno ograničiti na određene retke po broju redaka (npr. *paging* prilikom prikazivanja podataka korisniku). Limit dakle služi kako bi se ispisale n-torke po njihovom rednom broju. Svaka baza podataka ima drugačiju sintaksu za limit se navode neke od njih:

MySQL sintaksa:

```
SELECT * FROM tablica LIMIT brojRetkaKretanja, ukupanBrojRedaka
```

Oracle sintaksa:

```
SELECT * FROM tablica WHERE ROWNUM <= brojRetka
```

MS SQL Server sintaksa:

```
SELECT TOP brojRedakaOdPrvog * FROM tablica
```

Limit se na MySQL bazi podataka uvijek piše na kraju upita, a specifično je da se n-torke broje od nula kao prve što je ujedno i prvi argument limit kazujuće, a drugi argument je broj n-torki koji se želi prikazati.

15.1.1. Primjeri:

Pomoću limita je potrebno ispisati studenta s najduljim imenom i prezimenom.

```
SELECT *
FROM studenti
ORDER BY (LENGTH(ime) + LENGTH(prezime)) DESC
LIMIT 0,1;
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	1191230521	Anya	Sardelic Wojcikiewicz	2015-09-27	10000	10410	13

Potrebno je pomoću limita ispisati nastavnika prvog po abecedi.

```
SELECT * FROM nastavnici
ORDER BY prezime ASC, ime ASC
LIMIT 0,1;
```

Rezultati izvođenja upita:

☐	jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
☐	1908987173977	Nikola	Bačić	Radićeva 3	prof. dr.sc.		10000	df5e014b11c328fa07219a41970cbade

Potrebno je ispisati prvih deset nastavnika poredanih po abecedi.

```
SELECT * FROM nastavnici
ORDER BY prezime ASC, ime ASC
LIMIT 0,10;
```

Rezultati izvođenja upita:

jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
1908987173977	Nikola	Bačić	Radićeva 3	prof. dr.sc.		10000	df5e014b11c328fa07219a41970cbade
1804949363053	Daniela	Bačić	Branimirova 96	dr.sc.		10000	b362df18e5980dfdf981f99094748ecc
0606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.		21000	e6993e93270eb1014b7118501a201a89
0501953210249	Anto	Bošnjak	Unska ulica 25		prof.	40000	db21dd893e3ddc0e1e4c64daba31995d
0305988391019	Lucija	Brčić	Naserov trg 1		prof.	10000	5d962f37c27a0dac27cfa4b5ff726f61
0708980383302	Boris	Brkić	Zagrebačka 154	prof. dr.sc.		10000	ffdc0d63cb0b527cb40e17246f34c13a
1707963340011	Ankica	Čeh	Ulica kralja Petra Krešimira IV 6		prof.	42000	0d94a5529152ca83d011daf25be975c9
1910956363012	Dražen	Debeljak	Trpimirova 15	dr.sc.	v. pred.	10410	9aa31e57ed4c9a4b6ce586f05b764fef
2207987301807	Josip	Dukić	Jurišićeva 21	mr.sc.		10000	9f50a136bc84fala366807698131a0a
2306978362104	Sonja	Duvnjak	Ulica Žrtava fašizma 23	prof. dr.sc.		21000	dd4a0b6e48ded6ec4fd3e51bealf511

Potrebno je ispisati od desete do petnaeste n-torke nastavnika koji su poredani po abecedi.

```
SELECT * FROM nastavnici
ORDER BY prezime ASC, ime ASC
LIMIT 10,5;
```

Rezultati izvođenja upita:

jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka
0205951330124	Vedran	Grubišić	Dobrilina 62	mr.sc.		10000	30f7799685334aed99267c75b537lafb
2805952392713	Lovro	Horvatinčić	Škurinjska cesta 47	dr.sc.	v. pred.	10310	6d255cc57954alc231fd8bfae3b7a782
3103947210186	Tomica	Hrvat	Gospoka ulica 91	prof. dr.sc.		10000	67dala4ed66fcad0bbe3f8be21174d5a
0611973330036	Mila	Ilić	Ulica Alojzija Stepinca 2	dr.sc.	v. pred.	51000	4e099b86cbd02c1431821a6b6c0f2f9b
3105981300707	Vesna	Ilić	Paška ulica 25		prof.	40000	d85a9058fab489ca8812e2482ccd44ac

16. GROUP BY i HAVING klauzule

16.1. GROUP BY

Klauzula GROUP BY koristi se kako bi se grupirale n-torke i nad grupama računale statistike pomoću agregatnih funkcija. Grupiranje se obavlja na način da se n-torke koje imaju jednake vrijednosti atributa navedenih u listi za grupiranje kombiniraju u zajedničku grupu.

Sintaksa:

```
SELECT naziv_stupca, agregatna_funkcija(naziv_stupca)
FROM naziv_tablice
WHERE naziv_stupca operator vrijednost
GROUP BY naziv_stupca
```

GROUP BY klauzula uvijek dolazi nakon WHERE klauzule (ako postoji). Također je jako bitno da svi atributi ili izrazi koji se nalaze u listi za selekciju, a koji nisu unutar agregatnih funkcija **moraju** biti navedeni u GROUP BY listi.

Na primjer, potrebno je pobrojati broj mjesta po šifri županije:

```
SELECT idZupanija, COUNT(mjesta.postBr) AS brMjesta
FROM mjesta
GROUP BY idZupanija
```

Kako GROUP BY zapravo radi (teoretski):

postbr	nazivMjesto	idZup...	
10000	Zagreb	1	← Prva grupa od 2 retka
10290	Zaprešić	1	
10295	Kupljenovo	2	← Druga grupa od 7 redaka
10310	Ivanić-Grad	2	
10315	Novoselec	2	
10340	Vrbovec	2	
10345	Gradec	2	
10410	Velika Gorica	2	
10430	Samobor	2	
44010	Sisak-Caprag	4	← Treća grupa od 2 retka
44000	Sisak	4	

Prvo se napravi klasična projekcija, nakon čega se po šifri županije kreiraju grupe. Grupe se kreiraju na način da je jedna grupa zapravo identična vrijednost. Nakon što se kreiraju grupe, nad svakom grupom se izvrši agregatna funkcija.

Rezultati izvođenja upita (19 n-torka):

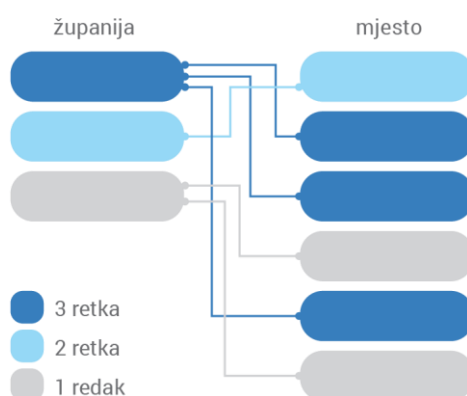
idŽupanija	brMjesta
1	2
2	7
4	2
5	2
6	1
8	1
9	1
10	7
11	3
12	1
13	1
14	6
15	7
16	9
17	9
18	5
19	2

GROUP BY se najčešće koristi na više od jedne tablice kako bi imao smisla. Na primjer, potrebno je izbrojati broj mjesta po nazivu županije:

```
SELECT nazivŽupanija, COUNT(mjesta.postBr) AS brMjesta
FROM mjesta
RIGHT OUTER JOIN zupanije ON mjesta.idŽupanija = zupanije.id
GROUP BY nazivŽupanija
```

Rezultati izvođenja upita (21 n-torka):

nazivŽupanija	brMjesta
Bjelovarsko-bilogorska županija	1
Brodsko-posavska županija	1
Dubrovačko-neretvanska županija	1
Grad Zagreb	2
Istarska županija	2
Karlovačka županija	2
Koprivničko-križevačka županija	0
Krapinsko-zagorska županija	0
Ličko-senjska županija	7
Međimurska županija	1
Osječko-baranjska županija	7
Požeško-slavonska županija	1
Primorsko-goranska županija	1
Sisačko-moslavačka županija	2



Slika 41. Brojanje mjesta unutar županije

U upitu se koristi vanjsko spajanje da se prikažu sve županije uključujući i one koje ne sadrže ni jedno mjesto (Koprivničko-križevačka županija i Krapinsko-zagorska županija). Kada bi se koristilo klasično unutarnje spajanje sve županije koje ne sadrže ni jedno mjesto ne bi bile ispisane.

16.2. HAVING

HAVING klauzula se koristi kada je potrebno filtrirati rezultate dobivene grupiranjem. Kao što je poznato od ranije (6. poglavlje) agregatne funkcije se ne smiju i ne mogu pisati unutar WHERE klauzule. Za rješenje te problematike koristi se HAVING klauzula. U HAVING klauzuli dopušteno je dodavati uvjete za attribute koji se nalaze u GROUP BY klauzuli ili uvjete s agregatnim funkcijama.

Sintaksa:

```
SELECT naziv_stupca, agregatna_funkcija(naziv_stupca)
FROM naziv_tablice
WHERE naziv_stupca operator vrijednost
GROUP BY naziv_stupca
HAVING naziv_stupca operator vrijednost [AND|OR
      agregatna_funkcija(naziv_stupca) operator vrijednost]
```

16.2.1. Primjeri:

Potrebno je ispisati prosjek ocjena svih studenata po njihovom JMBAG-u, imenu i prezimenu. Potrebno je ispisati samo one studente kojima je on veći od 2.

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime,
AVG(ocjene.ocjena)
FROM studenti INNER JOIN
      ocjene ON studenti.jmbag = ocjene.jmbagStudent
GROUP BY studenti.jmbag, studenti.prezime, studenti.ime
HAVING AVG(ocjene.ocjena) > 2;
```

Rezultati izvođenja upita (116 n-torki):

jmbag	ime	prezime	AVG(ocjene.ocjena)
0013022546	Antun	Mandić	4.0000
0013025926	Antonio	Pavić	5.0000
0013026584	Borna	Knežević	3.0000
0016096972	Dora	Vidaković	3.0000
0016097990	Vesna	Đukić	4.5000
0016098017	Marina	Zadro	5.0000
0016099969	Antonia	Čoić	4.5000
0016100313	Biljana	Vidović	3.0000
0016199452	Bernarda	Tomić	4.5000
0016200033	Zvonimir	Žarković	3.0000
0016201125	Denis	Šapina	5.0000
0016201458	Ana	Radak	4.6667
0016203355	Dino	Bošnjak	4.5000
0016216326	Ivan	Horvat	3.0000
0016220011	Danijel	Zečević	5.0000
0016233522	Nikola	Habijan	3.5000
0016235684	Tomislav	Vukosavljević	2.5000

Potrebno je ispisati broj ustanova po županiji. Potrebno je ispisati sve županije u kojima se nalazi barem jedna ustanova. Rezultate je potrebno sortirati po ustanova po županiji uzlazno.

```
SELECT zupanije.id, zupanije.nazivZupanija, COUNT(ustanove.oib)
FROM zupanije INNER JOIN
      mjesta ON zupanije.id = mjesta.idZupanija INNER JOIN
      ustanove ON mjesta.postBr = ustanove.postBr
GROUP BY zupanije.id, zupanije.NazivZupanija
HAVING COUNT(ustanove.oib) > 0
ORDER BY COUNT(ustanove.oib) ASC;
```

Rezultati izvođenja upita:

☐ nazivZupanija	COUNT(ustanove.oib)
☐ Varaždinska županija	1
☐ Vukovarsko-srijemska županija	1
☐ Zagrebačka županija	1
☐ Primorsko-goranska županija	1
☐ Međimurska županija	1
☐ Splitsko-dalmatinska županija	1
☐ Grad Zagreb	3

Potrebno je ispisati prvih 10 nastavnika koji rade na najviše kolegija, a kojima prezime počinja slovom „B“.

```
SELECT nastavnici.*, COUNT(kolegiji.id)
FROM nastavnici INNER JOIN
      izvrsitelji ON nastavnici.jmbg = izvrsitelji.jmbgNastavnik
INNER JOIN
      kolegiji ON izvrsitelji.idKolegij = kolegiji.id
GROUP BY nastavnici.jmbg
HAVING nastavnici.prezime LIKE 'B%'
ORDER BY COUNT(kolegiji.id) DESC
LIMIT 0,10;
```

Rezultati izvođenja upita:

☐ jmbg	ime	prezime	adresa	titulaIspred	titulaIza	postBr	lozinka	COUNT(kolegiji.id)
☐ 1908987173977	Nikola	Bačić	Radićeva 3	prof. dr.sc.		10000	df5e014b11c328fa07219a41970cbade	3
☐ 1804949363053	Daniela	Bačić	Branimirova 96	dr.sc.		10000	b362df18e5980dfdf981f99094748ecc	1
☐ 0708980383302	Boris	Brkić	Zagrebačka 154	prof. dr.sc.		10000	ffdc0d63cb0b527cb40e17246f34c13a	1
☐ 0501953210249	Anto	Bošnjak	Unska ulica 25		prof.	40000	db21dd893e3ddc0e1e4c64daba31995d	1
☐ 0305988391019	Lucija	Brčić	Naserov trg 1		prof.	10000	5d962f37c27a0dac27cfa4b5ff726f61	1
☐ 0606960383925	Borna	Blažević	Ulica Petra Jurčića 12	prof. dr.sc.		21000	e6993e93270eb1014b7118501a201a89	1

Potrebno je ispisati prosječnu ocjene po smjerovima.

```
SELECT smjerovi.naziv, AVG(ocjene.ocjena)
FROM smjerovi INNER JOIN
      kolegiji ON smjerovi.id = kolegiji.idSmjer INNER JOIN
      ocjene ON kolegiji.id = ocjene.idKolegij
GROUP BY smjerovi.id, smjerovi.naziv;
```

Rezultati izvođenja upita (21 n-torka):

☐ naziv	avg(ocjene.ocjena)
☐ Stručni studij informatike	3.3889
☐ Stručni studij računarstva	3.3750
☐ Stručni studij elektrotehnike	2.6364
☐ Stručni studij graditeljstva	3.0000
☐ studij trgovine	3.3000
☐ upravni studij	3.4000
☐ računarstvo	3.5000
☐ menadžment turizma i sporta	2.8182
☐ očna optika	3.5000
☐ održavanje zrakoplova	3.6250
☐ smjer matematika	3.0909
☐ smjer fizika	2.8750
☐ smjer kemije	3.2500

Potrebno je ispisati kolegij s najvećom prosječnom ocjenom i iznos prosječne ocjene.

```
SELECT kolegiji.naziv, AVG(ocjene.ocjena)
FROM kolegiji INNER JOIN
    ocjene ON kolegiji.id = ocjene.idKolegij
GROUP BY kolegiji.id, kolegiji.naziv
ORDER BY AVG(ocjene.ocjena) DESC
LIMIT 0,1;
```

Rezultat izvođenja upita:

☐ naziv	AVG(ocjene.ocjena)
☐ Tjelesna i zdravstvena kultura 2	5.0000

17. Podupiti

SELECT naredba ugniježdjena unutar WHERE ili SELECT klauzule neke druge SELECT, UPDATE ili DELETE naredbe naziva se podupit (engl. *subquery*). Upit u kojem je podupit sadržan naziva se vanjski upit (engl. *outer statement*).

Postoje tri tipa podupita:

- Usporedba izraza iz vanjskoga upita s rezultatom podupita
- Ispitivanje je li neki izraz iz vanjskog upita sadržan u rezultatu podupita
- Ispitivanjem da li se kao rezultat podupita pojavljuje barem jedna n-torka

17.1. USPOREDBA IZRAZA IZ VANJSKOG UPITA S PODUPITOM

Usporedba izraza iz vanjskog upita s podupitom se može izvršiti na dva načina. Ako podupit vraća isključivo jednu vrijednost (npr. agregatna funkcija bez grupiranja) tada je moguće podupit napisati bez ključne riječi. Ako upit vraća više od jedne vrijednosti potrebno je koristiti ključne riječi ALL ili ANY (SOME). Podupiti koji se koriste s ALL ili ANY ključnim riječima moraju vraćati jedan stupac za rezultat.

- ALL – sve n-torke dobivene podupitom moraju zadovoljiti uvjet usporedbe sa izrazom (engl. *Expression*). Ako podupit ne vrati ni jednu vrijednost, uvjet se evaluira kao istinit.
- SOME ili ANY (sinonimi) – uvjet će biti zadovoljen ako barem jedna n-torka iz podupita zadovoljava usporedbu sa izrazom. Ako podupit ne vrati niti jednu vrijednost, uvjet se evaluira kao lažan.

Na primjer, potrebno je ispisati sve studente koji su upisani na neku ustanovu prije nego što je upisana iti jedna ocjena.

```
SELECT * FROM studenti WHERE datumUpisa < ALL (SELECT
datumPolaganja FROM ocjene);
```

Rezultati izvođenja upita (13 n-torki):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0035182233	Ivan	Turk	2013-09-26	10000	43000	3
0036460022	Ivan	Matić	2013-09-24	10000	10000	15
0036468547	Sandra	Labak	2013-09-23	10000	47000	15
0036499965	Lovro	Keglović	2013-09-26	10000	10410	15
0246031254	Petra	Vujić	2013-09-22	10000	34000	3
0246033365	Marko	Noskov	2013-09-24	10000	10000	4
0246036655	Marija-Olgica	Kasumović	2013-09-23	10000	34000	4
0283021325	Robert	Stanić	2013-09-26	32000	32000	5
0283021548	Tomislav	Pokupić	2013-09-25	32000	32000	5
0313004222	Marko	Keser	2013-09-25	40000	40000	8
0313005214	Josip	Marin	2013-09-22	40000	10290	8
0313006332	Karla	Rolj	2013-09-24	40000	44000	8
0313008547	Sandra	Tinjić	2013-09-23	40000	44000	8

Ispisati sve studente za koje postoji barem jedan nastavnik da mu je prezime kraće:

```
SELECT * FROM studenti WHERE LENGTH(prezime) < ANY (SELECT
LENGTH(prezime) FROM nastavnici);
```

Rezultati izvođenja upita (162 n-torke):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0010081356	Tanja	Krog	2015-09-28	10000	10290	3
0013020125	Matea	Omanović	2017-09-25	10410	44000	9
0013022546	Antun	Mandić	2015-09-30	10410	31000	10
0013025926	Antonio	Pavić	2017-09-25	32000	44000	5
0013026584	Borna	Knežević	2015-09-29	10410	44000	9
0016096972	Dora	Vidaković	2016-09-26	42000	44000	16
0016097990	Vesna	Đukić	2016-09-29	42000	43000	17
0016098017	Marina	Zadro	2016-09-28	42000	47000	17
0016099969	Antonia	Čoić	2016-09-28	42000	35000	17
0016100313	Biljana	Vidović	2016-09-27	42000	35000	16
0016101784	David	Vučetić	2016-09-27	42000	42000	16

Ispisati sve studente za koje vrijedi da svi nastavnici imaju dulje prezime:

```
SELECT * FROM studenti WHERE LENGTH(prezime) > ALL (SELECT
LENGTH(prezime) FROM nastavnici);
```

Rezultati izvođenja upita (9 n-torki):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0128045526	Kristina	Košić-Kancir	2016-09-30	51000	22300	21
0128052335	Matej	Levanić Kutni	2015-09-27	51000	22000	20
1191230521	Any	Sardelic Wojcikiewicz	2015-09-27	10000	10410	13

Kao što je napomenuto na početku poglavlja, podupit koji se koristi kao usporedba s izrazom iz vanjskog upita mora vraćati jedan stupac rezultata. Ako upit vrati više od jednog stupca, dogodit će se pogreška prilikom izvođenja upita. Na primjer, ne smije se napisati upit kao u nastavku:

```
SELECT * FROM studenti WHERE LENGTH(prezime) > ALL (SELECT
LENGTH(prezime), ime FROM nastavnici);
```

Ako upit vraća jednu vrijednost (engl. *single-valued subquery*) ključne riječi ALL, ANY ili SOME se mogu ispustiti. U tom slučaju se vrijednost podupita može promatrati kao rezultat funkcije, a uvjet s podupitom kao usporedba s vrijednošću koja se dobije izvršavanjem funkcije. Ako podupit napisan na taj način vrati više od jedne vrijednosti, dogodit će se pogreška. Ako podupit ne vrati niti jednu vrijednost, uvjet će biti *false*.

Potrebno je ispisati JMBAG, ime i prezime studenata čiji je prosjek ocjena veći od ukupnog prosjeka ocjena.

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime
FROM studenti INNER JOIN
    ocjene ON ocjene.jmbagStudent = studenti.jmbag
GROUP BY studenti.jmbag, studenti.ime, studenti.prezime
HAVING AVG(ocjene.ocjena) >
    (SELECT AVG(ocjene.ocjena) FROM ocjene);
```

Rezultati izvođenja upita (75 n-torki):

jmbag	ime	prezime
0013022546	Antun	Mandić
0013025926	Antonio	Pavić
0016097990	Vesna	Đukić
0016098017	Marina	Zadro
0016099969	Antonia	Čoić
0016199452	Bernarda	Tomić
0016201125	Denis	Šapina
0016201458	Ana	Radak
0016203355	Dino	Bošnjak
0016220011	Danijel	Zečević
0016233522	Nikola	Habijan

Potrebno je ispisati JMBAG, ime i prezime studenata čiji je prosjek ocjena jednak maksimalnoj ocjeni.

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime
FROM studenti INNER JOIN
    ocjene ON ocjene.jmbagStudent = studenti.jmbag
GROUP BY studenti.jmbag, studenti.ime, studenti.prezime
HAVING AVG(ocjene.ocjena) =
    (SELECT MAX(ocjene.ocjena) FROM ocjene);
```

Rezultati izvođenja upita (24 n-torke):

jmbag	ime	prezime
0013025926	Antonio	Pavić
0016098017	Marina	Zadro
0016201125	Denis	Šapina
0016220011	Danijel	Zečević
0035179953	Tanja	Sunek
0036430456	Dragutin	Topić
0036452014	Alen	Martić
0067025919	Hana	Amulić
0067401609	Teo	Turković
0128050002	Kristina	Žikić
0128052105	Luka	Šimić

17.2. ISPITIVANJE JE LI NEKI IZRAZ IZ VANJSKOG UPITA SADRŽAN U REZULTATU PODUPITA

Ispitivanje nalazi li se neki izraz ili vrijednost iz vanjskog upita unutar rezultata podupita, izvršava se pomoću ključne riječi IN ili NOT IN (ako se ne želi da vrijednost bude sadržana unutar podupita). Operator IN je već korišten u ovom udžebniku kao operator nalazi li se neka vrijednost unutar skupine vrijednosti. Na identičan način radi s podupitima, samo što podupit vraća skupinu vrijednosti. Podupit smije vraćati samo jedan stupac, u protivnom će se dogoditi pogreška.

Na primjer, potrebno je ispisati sve studente koji nemaju niti jednu ocjenu.

```
SELECT * FROM studenti WHERE jmbag NOT IN
(SELECT jmbagStudent FROM ocjene);
```

Rezultati izvođenja upita:

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0108067363	Filip	Kljaić	2016-09-28	40000	10000	8
☐	0246026699	Sonja	Batelić	2017-09-25	10000	53285	1
☐	0313011025	Filip	Fadljević	2017-09-25	40000	44010	7
☐	1191220001	Tina	Kostelić	2017-09-28	10000	32245	13

17.3. ISPITIVANJE POJAVLJUJE LI SE KAO REZULTAT PODUPITA BAREM JEDNA N-TORKA

Upit u kojem se ispituje pojavljuje li se (ili ne pojavljuje) kao rezultat podupita barem jedna n-torka piše se pomoću EXISTS (ili NOT EXISTS) ključne riječi. Rezultat podupita može biti jedna ili više n-torki, a broj atributa (stupaca) koje vraća podupit nije ograničen na jednu, već je apsolutno svejedno koliko će ih se pojaviti.

Na primjer, potrebno je ispisati sve studente ako danas nije upisana niti jedna ocjena.

```
SELECT * FROM studenti WHERE NOT EXISTS (SELECT * FROM ocjene
WHERE datumPolaganja = CURDATE())
```

Ovaj primjer je poprilično besmislen, jer rijetko kome je potreban takav uvjet prikazivanja n-torki. Upiti s EXISTS ključnom riječi korisni su kada se koriste u koreliranim upitima.

17.4. KORELIRANI UPITI

Korelirani upiti (engl. *correlated subquery*) su oni kod kojih podupit pri evaluaciji koristi vrijednosti vanjskog upita. Svaka druga vrsta podupita naziva se nekorelirani podupit (engl. *uncorrelated*). Nekorelirani podupit se obavlja samo jednom, a zatim se njegov rezultat koristi za usporedbu s n-torkama dobivenim vanjskim upitom. Korelirani podupiti se moraju obaviti jednom za svaku n-torku iz vanjskog upita (Oprezno s velikim brojem n-torki!). Korelirani podupiti također mogu biti ugrađeni u IN podupite i ALL/ANY/SOME podupite.

Na primjer, potrebno je ispisati jmbag, prezime i ime studenata koji nemaju niti jednu ocjenu.

```
SELECT jmbag, prezime, ime
FROM studenti
WHERE NOT EXISTS
  (SELECT *
   FROM ocjene
   WHERE ocjene.jmbagStudent = studenti.jmbag)
```

Kao što se može vidjeti u podupitu se koristi korelacija s vanjskim upitom. Atribut „ocjene.jmbagStudent“ iz podupita mora biti jednak „studenti.jmbag“ atributu iz glavnog upita. Korelirani podupiti na neki način funkcioniraju kao spajanje tablica, samo što je proces puno zahtjevniji za računalo jer se podupit izvršava za svaku n-torku vanjskog upita.

Rezultati izvođenja upita:

☐ jmbag	prezime	ime
☐ 0108067363	Kljaić	Filip
☐ 0246026699	Batelić	Sonja
☐ 0313011025	Fadljević	Filip
☐ 1191220001	Kostelić	Tina

Gore navedeni primjer koreliranog upita se konceptualno obavlja na sljedeći način:

- dohvaća se prva n-torka iz relacije (tablice) studenti pa se koristeći vrijednost „studenti.jmbag“ određuje postoji li zapis u relaciji (tablici) ocjene s odgovarajućom vrijednosti „jmbagStudent“
- ako ne postoji, n-torka iz vanjskog upita se pojavljuje u rezultatu
- postupak se ponavlja za svaku n-torku relacije student

Potrebno je ispisati JMBAG, ime i prezime studenata koji imaju manji broj ocjena nego što je različitih kolegija na njihovom smjeru.

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime
FROM studenti LEFT OUTER JOIN
    ocjene ON ocjene.jmbagStudent = studenti.jmbag
GROUP BY jmbag, ime, prezime, studenti.idSmjer
HAVING COUNT(ocjene.idKolegij) <
    (SELECT COUNT(kolegiji.id)
     FROM kolegiji
     WHERE kolegiji.idSmjer = studenti.idSmjer);
```

Rezultati izvođenja upita (107 n-torki):

☐ jmbag	ime	prezime
☐ 0013020125	Matea	Omanović
☐ 0013022546	Antun	Mandić
☐ 0013025926	Antonio	Pavić
☐ 0013026584	Borna	Knežević
☐ 0016096972	Dora	Vidaković
☐ 0016098017	Marina	Zadro
☐ 0016100313	Biljana	Vidović

Potrebno je ispisati JMBAG, ime i prezime studenata za koje ne postoji neki drugi student s istim prosjekom.

```
SELECT studenti.jmbag, studenti.ime, studenti.prezime
FROM studenti INNER JOIN
    ocjene ON ocjene.jmbagStudent = studenti.jmbag
GROUP BY studenti.jmbag, studenti.ime, studenti.prezime
HAVING AVG(ocjene.ocjena) NOT IN
    (SELECT AVG(ocjene.ocjena)
     FROM studenti AS drugiStudenti INNER JOIN
         ocjene ON ocjene.jmbagStudent = drugiStudenti.jmbag
         WHERE drugiStudenti.jmbag <> studenti.jmbag
         GROUP BY drugiStudenti.jmbag);
```

Rezultati izvođenja upita:

☐ jmbag	ime	prezime
☐ 0016201458	Ana	Radak
☐ 0035177990	Irena	Dabić
☐ 0036460031	Dajana	Jerec
☐ 0067408000	Nikola	Mucić

17.5. PODUPITI U SELECT KLAUZULI

Podupiti se umjesto u WHERE klauzuli mogu koristiti i unutar SELECT klauzule. Unutar SELECT klauzule također mogu biti korelirani i nekorelirani. Za razliku od WHERE klauzule gdje je moguće koristiti ključne riječi za usporedbu podupita sa izrazom ili vrijednosti, podupit

unutar SELECT klauzule uvijek mora vraćati jednu vrijednost. (engl. *single-valued subquery*). Ako podupit unutar SELECT klauzule vrati više od jedne vrijednosti, dogodit će se pogreška. Ako podupit unutar SELECT klauzule ne vrati ni jednu vrijednost, kao vrijednost podupita uzima se NULL vrijednost.

Na primjer, uz svako mjesto potrebno je ispisati koliko klijenata u tom mjestu stanuje i koliko klijenata u tom mjestu ima registriran auto.

```
SELECT mjesta.postBr, mjesta.nazivMjesto,
(SELECT COUNT(*) FROM studenti WHERE studenti.postBrPrebivanje =
mjesta.postBr) AS prebiva,
(SELECT COUNT(*) FROM studenti WHERE studenti.postBrStanovanja =
mjesta.postBr) AS stanuje FROM mjesta;
```

Rezultati izvođenja upita (68 n-torki):

postBr	nazivMjesto	prebiva	stanuje
10000	Zagreb	64	27
10290	Zaprešić	0	7
10295	Kupljenovo	0	0
10310	Ivanić-Grad	0	0
10315	Novoselec	0	0
10340	Vrbovec	0	0
10345	Gradec	0	0
10410	Velika Gorica	20	14
10430	Samobor	0	6
20000	Dubrovnik	0	5

17.6. UGNJEŽĐIVANJE PODUPITA U PODUPITU

SQL naredba može sadržavati podupit u kojem je ugniježđen podupit unutar kojeg opet može biti ugniježđen podupit i tako dalje.

```
SELECT ... FROM ...
WHERE izraz1 >= ALL
(SELECT ... FROM ...
WHERE ...
AND izraz2 NOT IN
(SELECT ... FROM ...
WHERE ...
)
)
```

Pomoću podupita potrebno je ispisati podatke svih ustanova koje se nalaze u županiji „Grad Zagreb“:

```
SELECT ustanove.*
FROM ustanove
WHERE ustanove.postBr IN
(SELECT postBr
FROM mjesta
WHERE mjesta.idZupanija IN
```

```
(SELECT id
FROM zupanije
WHERE nazivZupanija = 'Grad Zagreb'));
```

Rezultati izvođenja upita:

☐	oib	naziv	zRacun	adresa	datumOsnutka	postbr
☐	08814003451	Tehničko Veleučilište u Zagrebu	2360000-1101350801	Vrbik 8	1998-05-27	10000
☐	28163265527	Prirodoslovno-matematički fakultet	2360000-1101522208	Horvatovac 102A	1946-06-08	10000
☐	57029260362	Fakultet elektrotehnike i računarstva	2360000-1101218080	Unska 3	1956-06-08	10000

17.7. UPDATE I DELETE NAREDBE

UPDATE naredba po ANSI SQL-92 standardu ne podržava JOIN naredbu, niti je moguće prirodno spajanje. MySQL dozvoljava spajanje više tablica pomoću JOIN naredbe unutar UPDATE naredbe, ali neke druge baze podataka ne podržavaju JOIN unutar UPDATE naredbe (npr. SQLite). U slučaju DELETE naredbe, trenutno na tržištu ne postoji baza podataka koja podržava DELETE naredbu s JOIN klauzulom. Ako je potrebno obrisati neki podatak s uvjetom iz neke druge tablice koja je vezana na onu čije se podatke želi obrisati, moraju se koristiti podupiti.

Na primjer, potrebno je svim nastavnicima iz županije "Grad Zagreb" promijeniti ime u "Petar", a prezime u „Hektorović“.

```
UPDATE nastavnici
SET ime = 'Petar',
    prezime = 'Hektorović'
WHERE postBr IN (
    SELECT postBr
    FROM mjesta JOIN
        zupanije ON zupanije.id= mjesta.idZupanija
    WHERE zupanije.nazivZupanija = 'Grad Zagreb');
```

Ili

```
UPDATE nastavnici INNER JOIN
    mjesta ON nastavnici.postBr = mjesta.postBr INNER JOIN
    zupanije ON zupanije.id = mjesta.postBr
SET ime = 'Petar',
    prezime = 'Hektorović'
WHERE zupanije.nazivZupanija = 'Grad Zagreb';
```

Potrebno je obrisati sve ocjene studenata koji studiraju na ustanovi „Fakultet elektrotehnike i računarstva“.

```
DELETE FROM ocjene
WHERE jmbagStudent IN (
    SELECT jmbag
    FROM studenti JOIN
        smjerovi ON studenti.idSmjer = smjerovi.id JOIN
        ustanove ON smjerovi.oibUstanova = ustanove.oib
    WHERE ustanove.naziv = "Fakultet elektrotehnike i računarstva");
```


17.8. SMJERNICE PRILIKOM KORIŠTENJA PODUPITA

17.8.1. NULL vrijednosti

Prilikom korištenja podupita potrebno je obratiti pažnju na NULL vrijednosti. Najveću pažnju prilikom korištenja podupita čiji rezultat može sadržavati NULL vrijednosti treba obratiti na uvjete za dohvat oblika:

- WHERE izraz NOT IN (podupit)

Ako se u rezultatu ovakvih podupita nalazi barem jedna NULL vrijednost, rezultat evaluacije uvjeta za dohvat će uvijek biti NULL vrijednost.

Na primjer, ako tablice mjesta i županija sadrže zapise kao u nastavku:

postBr	nazivMjesto	idZupanija	id	nazivZupanija
10000	Zagreb	21	21	Grad Zagreb
(NULL)	Garešnica	7	7	Bjelovarsko-bilogorska
43000	Bjelovar	(NULL)	0	Nepoznata županija

Upit

```
SELECT *
FROM zupanije
WHERE id IN (SELECT idZupanija FROM mjesta);
```

dati će rezultate:

id	nazivZupanija
21	Grad Zagreb
7	Bjelovarsko-bilogorska

a upit kao u nastavku:

```
SELECT *
FROM zupanije
WHERE id NOT IN (SELECT idZupanija FROM mjesta);
```

neće vratiti ni jedan rezultat:

id	nazivZupanija
----	---------------

17.8.2. Podupiti i učestalost korištenja

Podupite ne treba koristiti bez valjanog razloga zato što se jako često izvode sporije od klasičnih upita što je pogotovo izraženo kod koreliranih podupita. Osim što se sporije izvode, podupiti su mnogo nepregledniji od klasičnih upita.

Na primjer, potrebno je ispisati sve podatke o studentima koji imaju mjesto prebivanja u županiji „Grad Zagreb“.

Rješenje pomoću podupita:

```
SELECT * FROM studenti
WHERE postBrPrebivanje IN (
    SELECT postbr FROM mjesta
    WHERE idZupanija IN (
        SELECT id FROM zupanije
        WHERE nazivZupanija = 'Grad Zagreb'));
```

Rješenje pomoću spajanja tablica:

```
SELECT studenti.*
FROM studenti INNER JOIN
    mjesta ON studenti.postBrPrebivanje = mjesta.postBr INNER JOIN
    zupanije ON mjesta.idZupanija = zupanije.id
WHERE zupanije.nazivZupanija = 'Grad Zagreb';
```

Rezultati izvođenja upita (64 n-torke):

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0010081356	Tanja	Krog	2015-09-28	10000	10290	3
☐	0035177990	Irena	Dabić	2017-09-26	10000	10000	1
☐	0035179953	Tanja	Sunek	2017-09-26	10000	10000	2
☐	0035180940	Sonja	Šimpraga	2016-09-26	10000	21000	4
☐	0035182233	Ivan	Turk	2013-09-26	10000	43000	3
☐	0035192211	Karla	Brandis	2015-09-28	10000	40000	4
☐	0035192565	Kristina	Matleković	2015-09-27	10000	43000	4
☐	0036430456	Dragutina	Topić	2016-09-26	10000	43000	15
☐	0036430552	Armin	Čehajić	2016-09-30	10000	44000	15
☐	0036435111	Maja	Vuković	2016-09-27	10000	21300	14
☐	0036440706	Vicka	Zečević	2016-09-26	10000	31000	14

18. Primarni i strani ključevi

Primarni i strani ključevi služe za očuvanje entitetskog i referencijskog integriteta u bazi podataka. Svaka vrijednost stupca nad kojom je strani ključ mora biti uparena s vrijednošću stupca na koju se referencira, što je najčešće primarni ključ (ne mora nužno biti, iako je dobra praksa). To zapravo znači da se podaci ne smiju unositi nasumično bez ikakvih pravila.

Na primjer, svaka vrijednost atributa `student.postBrStanovanja` mora se nalaziti u skupu vrijednosti atributa `mjesta.postBr` zato što tablica *studenti* ima postavljen strani ključ na stupcu `studenti.postBrStanovanja` koji pokazuje na primarni ključ `mjesta.postBr`. Isto pravilo vrijedi i za stupac `studenti.postBrPrebivanje`.

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
0010081356	Tanja	Krog	2015-09-28	10000	10290	3
0013020125	Matea	Omanović	2017-09-25	10410	44000	9
0013022546	Antun	Mandić	2015-09-30	10410	31000	10
0013025926	Antonio	Pavić	2017-09-25	32000	44000	5
0013026584	Borna	Knežević	2015-09-29	10410	44000	9
0016096972	Dora	Vidaković	2016-09-26	42000	44000	16
0016097990	Vesna	Bukić	2016-09-29	42000	43000	17

postbr	nazivMjesto	idZupanija
10290	Zaprešić	1
31000	Osijek	15
43000	Bjelovar	8
44000	Sisak	4

Slika 42. Podaci unutar tablica mjesta i zupanije

18.1. PRIMARNI KLJUČ

Primarni ključ (engl. *primary key*) je skup od jednog ili više atributa (stupaca) koji jednoznačno određuje svaku n-torku u tablici. Postoje dvije vrste primarnog ključa:

- **Jednostavni primarni ključ** – ako se sastoji od samo jednog stupca
- **Složeni (kompozitni) primarni ključ** – ako se sastoji od dva ili više stupaca

Svaka tablica može imati definiran samo jedan primarni ključ. Svaki primarni ključ mora biti:

- **Jedinstven** – ne smiju postojati duplicirane vrijednosti primarnog ključa, odnosno svaka vrijednost primarnog ključa mora biti jedinstvena.
- **Postojan (definiran)** - Vrijednost primarnog ključa kao cjeline ne smije biti jednaka NULL vrijednosti. Ako je primarni ključ relacije složen, ni jedna njegova komponenta ne smije poprimiti NULL vrijednost.

Kod većine DBMS sustava (uključujući i MySQL) prilikom kreiranja primarnog ključa automatski se stvara indeks po kojem se sortiraju podaci (engl. *clustered indeks*). Ta kreacija indeksa pomaže bazi podataka da brže provjerava mogućnost duplih vrijednosti prilikom unosa svakog podatka. Implementiranjem pravila primarnog ključa osigurava se entitetski integritet.

18.1.1. Primjeri:

Potrebno je kreirati tablicu naziva "test" s dva atributa "sifTest" i "nazivTest" i primarnim ključem na "sifTest" atributu.

```
CREATE TABLE test(  
sifTest INT,  
nazivTesta VARCHAR(50),  
PRIMARY KEY (sifTest));
```

ili:

```
CREATE TABLE test(  
sifTest INT PRIMARY KEY,  
nazivTesta VARCHAR(50));
```

Potrebno je kreirati tablicu naziva "test" s tri atributa "sifTest", "sifTest2" i "nazivTest" i složenim primarnim ključem na "sifTest" i "sifTest2" atributima.

```
CREATE TABLE test(  
sifTest INT,  
sifTest2 INT,  
nazivTesta VARCHAR(50),  
PRIMARY KEY (sifTest, sifTest2));
```

Potrebno je kreirati tablicu naziva "test" s dva atributa "sifTest" i "nazivTest", primarnim ključem na "sifTest" atributu koji dodjeljuje automatske inkrementalne vrijednosti

```
CREATE TABLE test(  
sifTest INT NOT NULL AUTO_INCREMENT,  
nazivTesta VARCHAR(50),  
PRIMARY KEY (sifTest));
```

18.1.2. Primarni ključ i UNIQUE indeks:

Iako na naizgled slični, primarni ključ i UNIQUE indeks se vrlo razlikuju. Kako po svojoj upotrebi, tako i po načinu ponašanja:

- Primarni ključ ne smije sadržavati NULL vrijednosti
- UNIQUE indeks smije sadržavati (jednu) NULL vrijednost
- Tablice smiju sadržavati jedan primarni ključ
- Tablice smiju sadržavati više unique indeksa

18.2. STRANI KLJUČ

Strani ključ (engl. *Foreign key*) se koristi za povezivanje određenog stupca ili stupaca s drugim stupcem ili stupcima (kompozitni strani ključ). Najčešće se radi o vezi jednog stupca koja se referencira na primarni ključ. Strani ključ povezuje stupce na način da onemogućava

upis vrijednosti koju ne sadrži stupac na koji se referencira. Za razliku od primarnog ključa, strani ključ može omogućavati NULL vrijednosti (u tom slučaju nema spoja). Moguće je kreirati više stranih ključeva po tablici, a MySQL DBMS prilikom kreiranja stranog ključa automatski kreira indeks. Dakle vrijednost stranog ključa:

- Mora biti u skupu vrijednosti atributa na kojeg se referencira
- Smije poprimiti NULL vrijednost

Strani ključ na većini baza podataka omogućava kaskadne izmjene i kaskadno brisanje. To znači da će se prilikom brisanja n-torke iz tablice roditelja obrisati i sve n-torke iz tablice djeteta.

Sintaksa:

```
FOREIGN KEY [index_name] (index_col_name, ...)
REFERENCES tbl_name (index_col_name,...)
[ON DELETE reference_option]
[ON UPDATE reference_option]
reference_option:
RESTRICT | CASCADE | SET NULL | NO ACTION
```

18.2.1. Primjeri:

Potrebno je kreirati dvije tablice *roditelji* i *djeca* koje će biti spojene pomoću stranog ključa. Pomoću pravila stranog ključa implementirati da ako se obriše roditelj, potrebno je obrisati i svu njegovu djecu.

```
CREATE TABLE roditelji (
    sifra INT NOT NULL,
    imeRoditelj VARCHAR(50),
    PRIMARY KEY (sifra)
);

CREATE TABLE djeca (
    imeDijete VARCHAR(50),
    sifraRoditelj INT,
    FOREIGN KEY (sifraRoditelj) REFERENCES roditelji(sifra)
    ON DELETE CASCADE
);
```

Potrebno je kreirati dvije tablice *roditelji* i *djeca* koje će biti spojene pomoću stranog ključa, a ako se promjeni vrijednost primarnog ključa roditelja, potrebno je automatski spojiti svu njegovu djecu s roditeljem. Ako se obriše roditelj, potrebno je obrisati i svu njegovu djecu.

```
CREATE TABLE roditelji (  
    sifra INT NOT NULL,  
    imeRoditelj VARCHAR(50),  
    PRIMARY KEY (sifra)  
);  
  
CREATE TABLE djeca (  
    imeDijete VARCHAR(50),  
    sifraRoditelj INT,  
    FOREIGN KEY (sifraRoditelj) REFERENCES roditelji(sifra)  
    ON UPDATE CASCADE ON DELETE CASCADE  
);
```

19. Indeksi

Indeksi su dodatna podatkovna struktura koja ubrzava dohvat podataka iz baze. Indeksi se mogu koristiti na jednom ili više stupaca u tablici. Mogu se koristiti za osiguravanje unikatnih vrijednosti, doprinose očuvanju referencijskog integriteta i ubrzavaju pretragu po određenom atributu.

Osim što ubrzavaju pretragu podataka, indeksi usporavaju unos novog podatka u tablicu nad kojom se nalazi indeks. To se događa zbog toga što prilikom svakog unosa podatka osim izvršavanja promjene u tablici, potrebno je izvršiti i promjene na indeksu. Osim što usporavaju unos novih podataka, indeksi zauzimaju dodatni prostor na disku.

Kreiranjem indeksa formira se struktura B-stabla (ako nije drugačije naznačeno) koja omogućava izravan pristup do n-torki u relaciji tj. tablici. Operacije =, >, >=, <, <=, BETWEEN, LIKE 'X%...', MATCHES 'X*', sortiranje i grupiranje se zbog izravnog pristupa mnogo brže izvode. Optimizator DBMS-a (engl. *Database Management System*) nakon kreiranja indeksa odlučuje koji će se indeks koristiti prilikom pristupa, sortiranja, grupiranja i ostalih operacija nad podacima.

Ako ne postoji odgovarajući indeks na tablici, DBMS mora koristiti slijednu pretragu podataka što je mnogo sporije. Nakon prve slijedne pretrage (ovisno o DBMS-u) podaci se mogu predmemorirati (engl. *caching*) za slijedeću pretragu kako bi se brže izvršila. Naravno, predmemoriranje podataka oduzima dodatne resurse u obliku radne memorije i sporije radi od indeksa. Osim navedenog, predmemoriranje podataka nije stalno rješenje zato što se izvođenjem idućeg upita predmemoriraju neke druge n-torke.

U bazi podataka razlikuju se dva tipa indeksa. Prvi tip je indeks po kojem se sortiraju podaci (engl. *clustered index*), a drugi tip je sekundarni indeks (engl. *nonclustered index* ili *secondary index*). U MySQL DBMS-u indeks po kojem se sortiraju podaci se automatski kreira prilikom kreiranja primarnog ključa. Ako na tablici ne postoji primarni ključ uzima se prvi indeks koji ne dozvoljava duplikate. Sekundarni indeksi u MySQL bazi podataka funkcioniraju na način da u sebi imaju pokazivač na stupac koja je primarni ključ. Svaka tablica u bazi podataka može sadržavati jedan indeks po kojem se sortiraju podaci i više sekundarnih indeksa.

19.1. RAD S INDEKSIMA

19.1.1. Pregled i uklanjanje indeksa

Sintaksa pregleda indeksa:

```
SHOW INDEX FROM [ime_baze.]ime_tablice
```

Sintaksa uklanjanja indeksa:

```
DROP [ONLINE | OFFLINE] INDEX ime_indeksa ON ime_tablice
```

ONLINE – indeks se briše za vrijeme rada tablice

OFFLINE – tablica se zaključava prije brisanja indeksa (dok se indeks ne obriše, svi upiti koji se izvode nad tablicom su stavljeni u red čekanja)

Primjeri:

Potrebno je prikazati sve indekse koji se nalaze na tablici `kljent`.

```
SHOW INDEX FROM studenti;
```

Rezultati izvođenja upita:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
studenti	0	PRIMARY	1	id	A	162	(NULL)	(NULL)		BTREE		
studenti	1	fk_students_mjesto1	1	postBrPrebivanje	A	7	(NULL)	(NULL)		BTREE		
studenti	1	fk_students_mjesto2	1	postBrStanovanja	A	26	(NULL)	(NULL)		BTREE		
studenti	1	fk_students_smjerovil_idx	1	idSmjer	A	21	(NULL)	(NULL)		BTREE		

Potrebno je obrisati indeks „postBrPrebivanje“ iz tablice „studenti“.

```
DROP INDEX postBrPrebivanje ON studenti;
```

19.1.2. Kreiranje indeksa

Sintaksa stvaranja indeksa:

```
CREATE [ONLINE|OFFLINE] [UNIQUE|FULLTEXT|SPATIAL] INDEX
naziv_indeksa
[tip_indeksa] ON naziv_tablice(naziv_stupca_indeksa, ...)
[opcije_indeksa]
naziv_stupca_indeksa:
naziv_atributa [(duljina)] [ASC|DESC]
tip_indeksa:
USING {BTREE|HASH|RTREE}
opcije_indeksa:
KEY_BLOCK_SIZE [=] vrijednost | tip_indeksa | WITH PARSE
naziv_parser
```

UNIQUE – vrijednosti indeksa moraju biti jedinstvene

FULLTEXT – indeksiranje teksta kod myIsam storage enginea

SPATIAL – indeksiranje specifičnih stupaca, tipa POINT and G-GEOMETRY za dobivanje koordinata unutar poligona
(<http://dev.mysql.com/doc/refman/5.1/ja/creating-spatial-indexes.html> i <http://dev.mysql.com/doc/refman/5.1/ja/using-a-spatial-index.html>)

BTREE, **HASH**, **RTREE** – tip strukture indeksa

`KEY_BLOCK_SIZE` – sistemsko limitiranje veličine jednog bloka indeks polja

`PARSER` – način razlamanja riječi za fulltext indekse
(<http://dev.mysql.com/doc/refman/5.6/en/innodb-table-and-index.html#innodb-fulltext-index> i
<http://dev.mysql.com/doc/refman/5.6/en/full-text-plugins.html>)

Primjeri:

Potrebno je kreirati indeks nad stupcem „postBrPrebivanje“ na tablici „studenti“.

```
CREATE INDEX postBrPrebivanje ON studenti;
```

Potrebno je kreirati indeks nad stupcima „ime“ i „prezime“ u tablici „studenti“ na način da je ime sortirano uzlazno, a prezime silazno.

```
CREATE INDEX imePrezime_ix on studenti(ime ASC, prezime DESC);
```

Potrebno je kreirati indeks nad stupcima „ime“ i „prezime“ u tablici „studenti“ na način da je ime sortirano uzlazno, a prezime silazno uz izričito korištenje B-tree strukture.

```
CREATE INDEX imePrezime_ix on studenti(ime ASC, prezime DESC)  
USING BTREE;
```

Potrebno je stvoriti indeks koji dozvoljava samo unikatne vrijednosti nad stupcem „jmbag“ u tablici studenti.

```
CREATE UNIQUE INDEX jmbagstudent_ix ON studenti(jmbag);
```

19.2. TIPOVI INDEKSA

MySQL (ovisno o okruženju za pohranu podataka) podržava BTREE, HASH i RTREE tipove indeksa. Na InnoDB okruženju (na kojem se nalazi napravljena baza podataka za izvođenje laboratorijskih vježbi) podržan je samo BTREE tip indeksa.

Tablica 29. Tablica podržanih tipova indeksa

Storage Engine	Dozvoljeni tip indeksa
MyISAM	BTREE, RTREE
InnoDB	BTREE
MEMORY/HEAP	BTREE, HASH
NDB	BTREE, HASH

19.2.1. BTREE tip indeksa

BTREE je stablasta struktura podataka koja čuva podatke sortiranim, omogućava pretragu, sekvencijalni pristup, unos i brisanje podataka u logaritamskom vremenu. B-TREE je generalizacija binarnog stabla u kojem čvor može imati više od dvoje djece. Za razliku od samo-balansirajućih binarnih stabla, B-TREE je optimiziran za sustave s velikim blokovima podataka (npr. baze podataka).

Pravila koja vrijede za B-TREE strukturu indeksa:

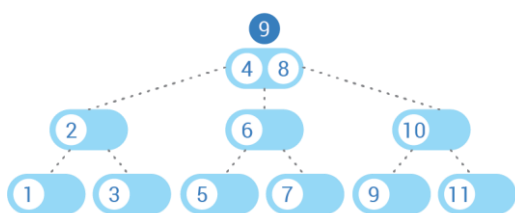
- Svaki čvor ima $\leq M$ djece
- Svaki čvor (osim korijena i lišća) ima $\geq m/2$ djece
- Korijen ima barem dvoje djece
- Sva lišća se nalaze na dnu najčešće u istom redu
- Čvor koji nije lišće sa k djece sadrži $k-1$ ključeva

Pretraga B-TREE stabla:

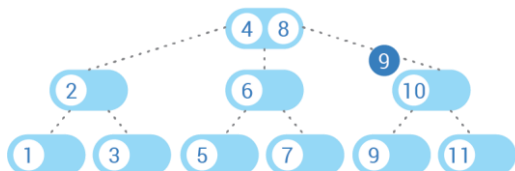
Pretraga uvijek počinje od korijena

- Za svaki horizontalni red čvorova
 - Ako je vrijednost koja se traži veća od desne vrijednosti sadržaja čvora prelazak na desno dijete
 - Ako je vrijednost koja se traži između obje vrijednosti sadržaja čvora prelazak na srednje dijete
 - Ako je vrijednost koja se traži manja od lijeve vrijednosti sadržaja čvora prelazak na lijeve dijete
- Gornji postupak se izvršava dok se ne dođe do željene vrijednosti

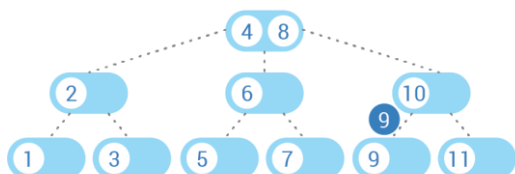
Objašnjenje postupka pretrage:



Na prvom čvoru se može vidjeti da postoje vrijednosti 4 i 8. Broj 9 je veći od broja 8, te se pretraga prebacuje na čvor desno od broja 8.



Na sljedećem čvoru se nalazi broj 10. Broj 9 je manji od broja 10, te se pretraga prebacuje na čvor lijevo od broja 10.



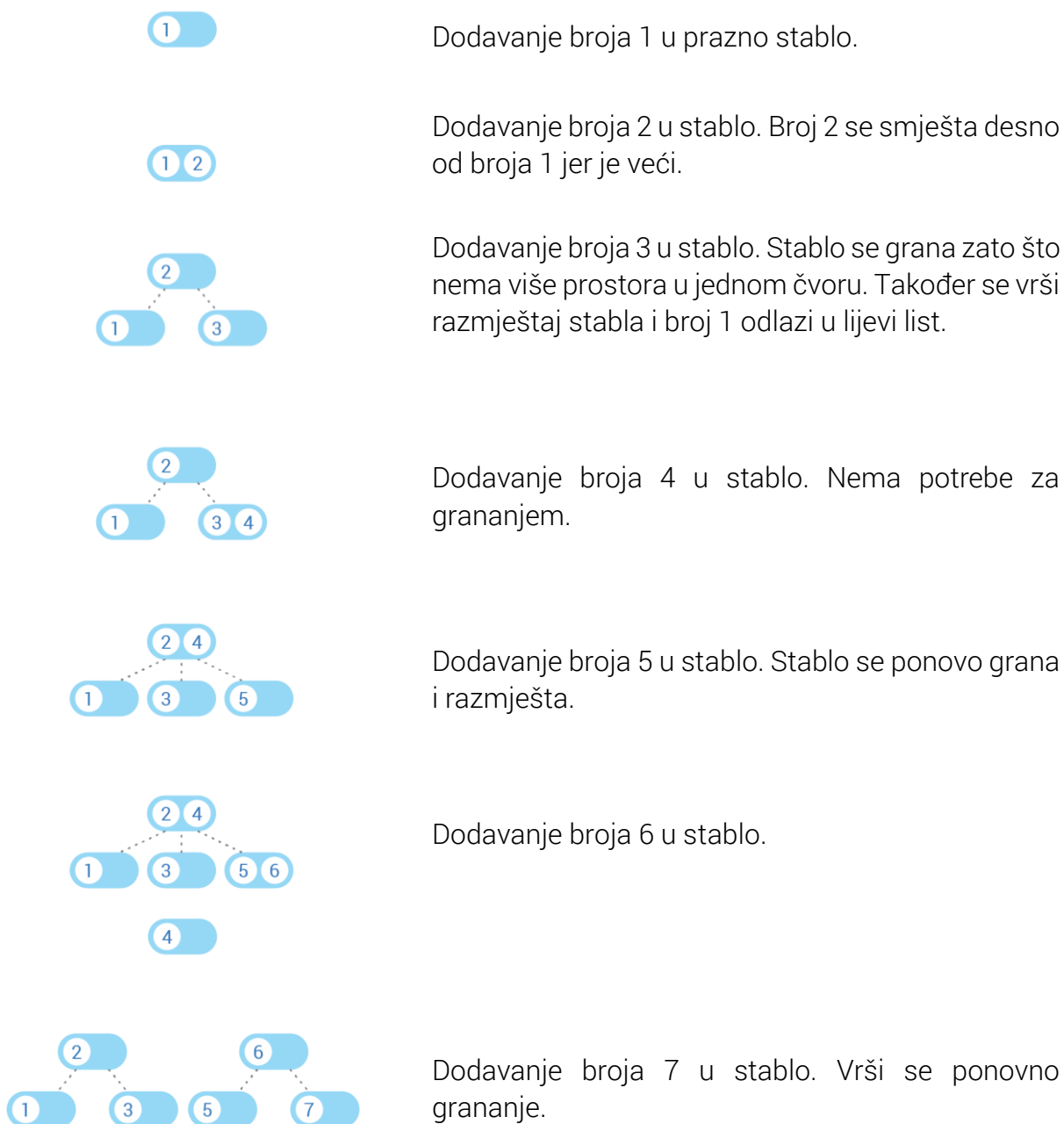
Vrijednost s brojem 9 je pronađena.

Slika 43. B-TREE - postupak pretrage

Postupak dodavanja elementa u B-TREE:

- Pretraživanjem po stablu pronađi list u koji treba dodati informaciju
 - Ako list sadrži manje elemenata od maksimalnog broja (2), ubaci element uz održanje redoslijeda
 - U protivnom, list je potrebno razdijeliti
 - Slaže se poredak od starih članova lista i novog ulaznog člana, uklanja se list
 - Vrijednost najmanjeg člana ide kao lijeva vrijednost novog lista
 - Vrijednost najvećeg člana ide kao desna vrijednost novog lista
 - Vrijednost srednjeg člana ide u roditelja novog lista, može uzrokovati dijeljenje roditelja kada se algoritam ponavlja ali za roditelja lista kao list.
 - Ako prelamanje dođe do korijena uzrokovati će dijeljenje korijena ako je korijen pun

Objašnjenje postupka dodavanja elementa:



Slika 44. B-TREE - postupak dodavanja elemenata

Brisanje elementa u B-TREE:

Postoje dvije strategije brisanja:

- Prvo se pronađe podatak koji je potrebno obrisati, nakon čega se obriše i rekonstruira stablo

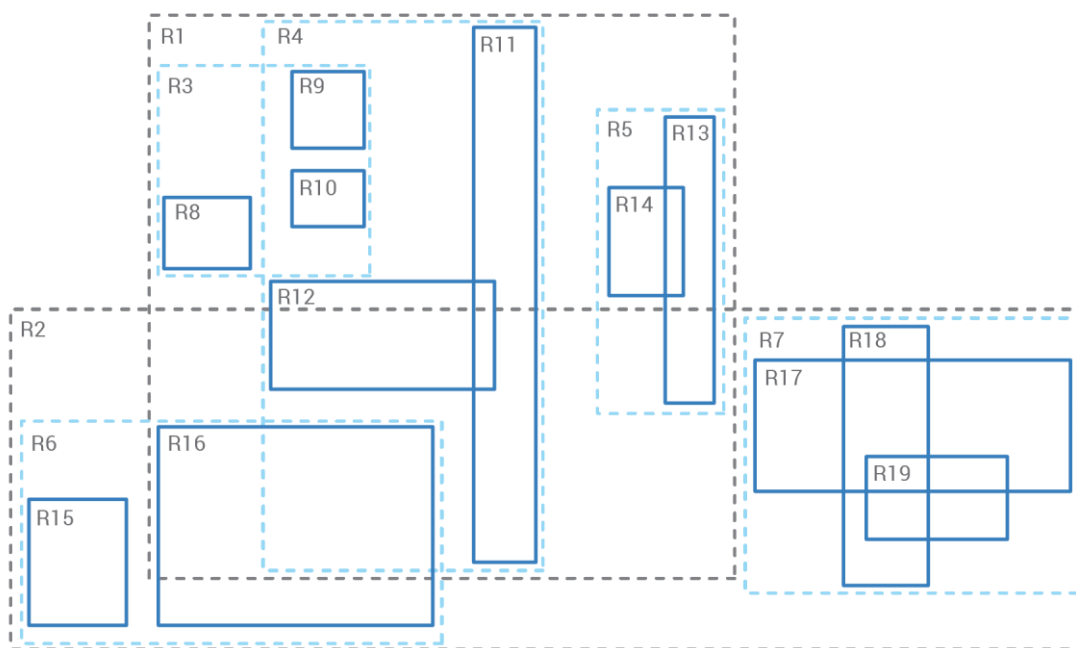
- Radi se prolaz prema podatku ali prije dolaska do podataka putem se rekonstruira stablo tako da po brisanju nije potrebno raditi daljnje rekonstrukcije

Pošto je prva strategija jednostavnija i već je objašnjena kroz pretragu i unos podataka, ona će se primjenjivati:

- Potrebno je doći do podatka kojeg je potrebno obrisati.
- Podatak se obriše
- S preostalim vrijednostima ponovo se izgradi stablo

19.2.2. R-TREE tip indeksa

R-TREE je struktura indeksa koja se specifično koristi za indeksiranje višedimenzionalnih informacija kao što su geografske koordinate i poligoni. Najčešće korištenje R-TREE stabla je primjerice traženje restorana u blizini trenutne lokacije. Osim vrlo različitih algoritama za sortiranje i pretragu podataka unutar poligona, R-TREE se od B-TREE indeksa razlikuje i po samoj strukturi stabla indeksa na način da R-TREE sadrži više vrijednosti po čvoru.

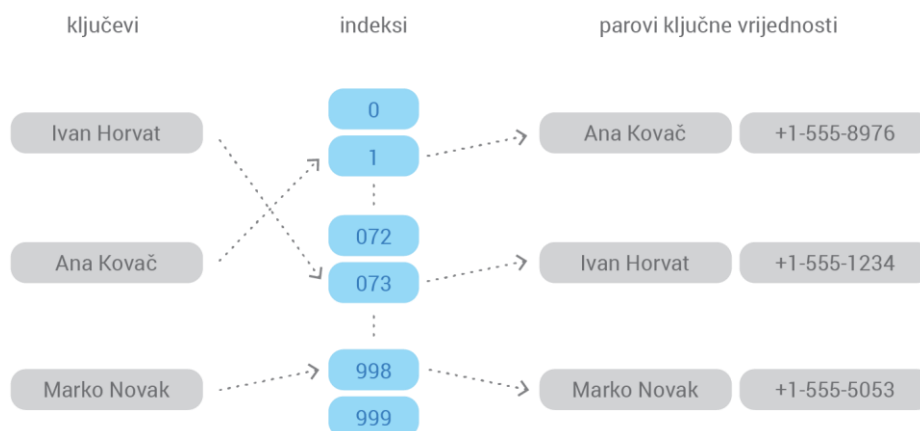


Slika 45. Prikaz rada R-TREE indeksa

19.2.3. HASH tip indeksa

Ovaj tip indeksa se svodi na generiranje kratkog hash zapisa nekog teksta. Za svaku indeksiranu vrijednost potrebno je napraviti HASH (npr. CRC32). Prilikom pretrage, uvjet u

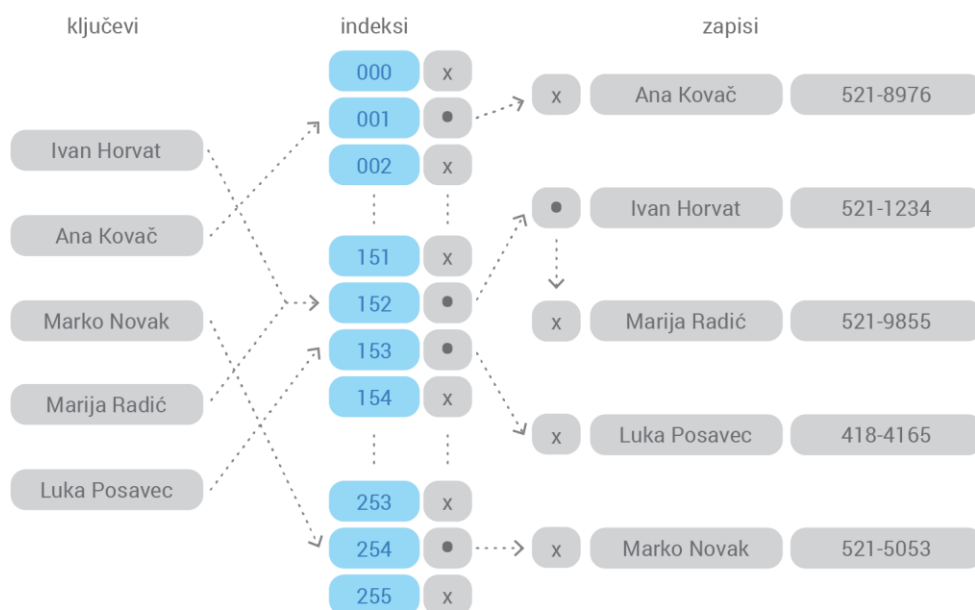
WHERE klauzuli se također pretvara u HASH nakon čega se vrši pretraga. Kao što se može zaključiti, po HASH-u se ne može sortirati, ali pretraga se vrši mnogo brže jer je potrebno usporediti mnogo manje vrijednosti od teksta. HASH indeks najjednostavnije je usporediti s indeksom pojmova na kraju knjige.



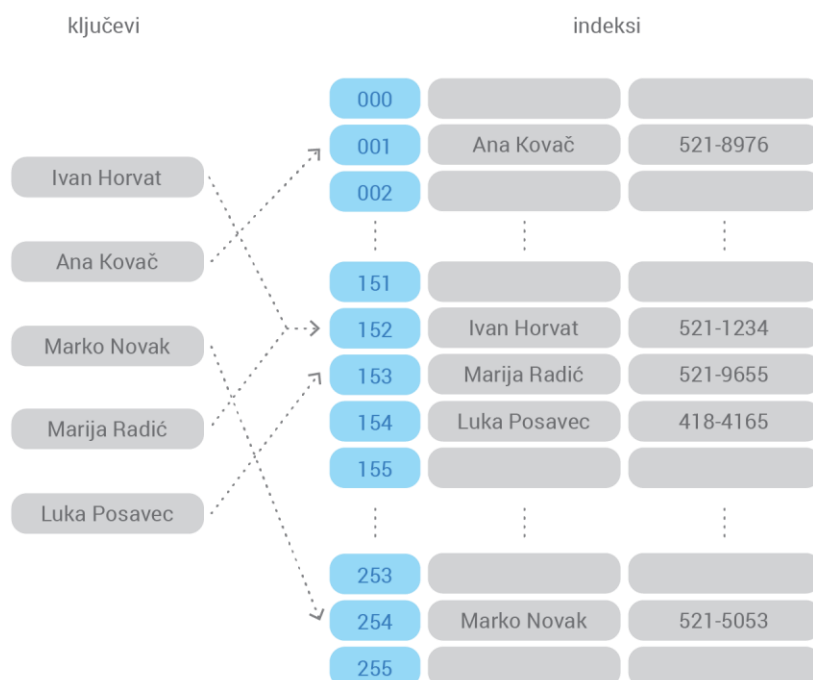
Slika 46. Prikaz rada HASH indeksa

Prilikom izrade HASH-a može se dogoditi kolizija, odnosno pojava da se od dva različita stringa dobije isti HASH. Postoji nekoliko metoda kojima je moguće riješiti problem kolizije HASH-a:

- Otvoreno adresiranje
- Ulančavanje
- Coalesced hashing
- Cuckoo hashing
- Robin Hood hashing
- 2-Choice Hashing
- Hopscotch hashing



Slika 47. Ulančavanje



Slika 48. Otvoreno adresiranje

19.3. OPĆENITO O INDEKSIMA

19.3.1. Automatsko kreiranje indeksa

DBMS će prilikom kreiranja pravila integriteta (primarni i strani ključevi) kreirati:

- PRIMARY KEY – nad atributima primarnog ključa kreira se DISTINCT INDEX (UNIQUE INDEX)
- UNIQUE KEY – nad atributima alternativnog ključa kreira se DISTINCT INDEX (UNIQUE INDEX)
- FOREIGN KEY – nad atributima stranog ključa kreira se INDEX (koji dopušta "duplikate")

19.3.2. Dodavanje indeksa unutar ALTER ili CREATE TABLE naredbe

Osnovna sintaksa:

```
ALTER [ONLINE | OFFLINE] [IGNORE] TABLE tbl_name
    [alter_specification [, alter_specification] ...]
```

alter_specification:

```
ADD [COLUMN] col_name column_definition
    [FIRST | AFTER col_name ]
| ADD [COLUMN] (col_name column_definition, ...)
| ADD {INDEX|KEY} [index_name]
    [index_type] (index_col_name, ...) [index_option] ...
| ADD [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (index_col_name, ...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    UNIQUE [INDEX|KEY] [index_name]
    [index_type] (index_col_name, ...) [index_option] ...
| ADD [CONSTRAINT [symbol]]
    FOREIGN KEY [index_name] (index_col_name, ...)
```

Identična sintaksa se koristi i u CREATE TABLE naredbi, tako da je nije potrebno dodatno objašnjavati.

Primjeri:

Potrebno je kreirati tablicu naziva "test" s dva atributa "sifTest" i "nazivTest", indeksom na "sifTest" atributu koji ne dozvoljava duplikate i koristi BTREE.

```
CREATE TABLE test(
sifTest INT,
nazivTest VARCHAR(50),
UNIQUE INDEX (sifTest) USING BTREE);
```


Potrebno je kreirati tablicu naziva "adrese" s tri atributa "grad", "ulica" i "kucniBr", indeksom nad svim atributima koji ne dozvoljava duplikate i koristi HASH.

```
CREATE TABLE adrese (
grad VARCHAR(50),
ulica VARCHAR(50),
kucniBr VARCHAR(5),
UNIQUE INDEX (grad, ulica, kucniBr) USING HASH);
```

19.3.3. Performanse upita s indeksom i bez indeksa

Kako bi bilo što jasnije koliko brzina izvođenja upita ovisi o indeksima u ovom je poglavlju obavljeno testiranje performansi upita sa indeksom i bez indeksa. Platforma na kojoj su mjerene performanse je MS SQL Server 2008 R2 iz razloga što daje detaljan uvid o načinu i brzini izvršavanja pojedinog upita, ali svaka baza podataka će se vrlo slično ponašati u ovim mjerenjima.

Tablica na kojoj su mjerene performanse upita sadrži 10,000,000 redaka kako bi rezultati bili što očigledniji i potpuniji. Tablica sadrži dva stupca „test“ i „test2“. Vršiti se pretraga po stupcu test. Između svih testiranja vrši se brisanje cache memorije.

testnaTablica			
	Column Name	Data Type	Allow Nulls
	test	varchar(50)	☐
	test2	varchar(50)	☐
			☐

Slika 49. Izgled tablice nad kojom se vrši pretraga

Upit koji se izvršava:

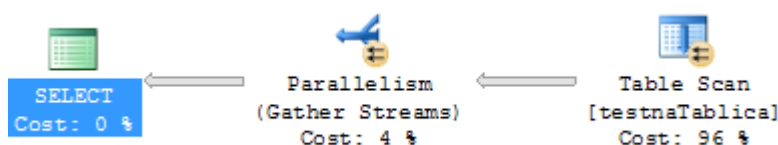
```
select * from testnaTablica where test = 'c'
```

Nakon izvršavanja upita bez indeksa dobiveni su sljedeći rezultati (u milisekundama):

Time Statistics	
Client processing time	7854
Total execution time	8588
Wait time on server replies	734

Slika 50. Statistika izvršavanja upita iz SQL Servera bez indeksa

Također, SQL Server posjeduje funkcionalnost grafičkog prikaza načina izvršavanja upita gdje se može vidjeti da prilikom pretrage pretražuje svaki redak unutar tablice (engl. *Table Scan*):



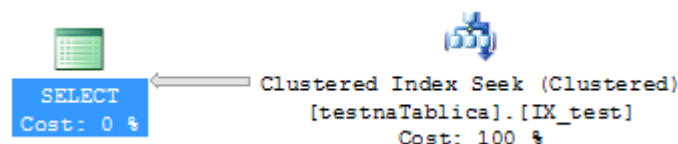
Slika 51. Plan izvođenja upita bez indeksa

Nakon izvršavanja upita bez indeksa i dobivanja rezultata, na tablici je kreiran indeks na stupcu „test“ (po kojem se vrši pretraga). Dobiveni rezultati nakon izvršavanja identičnog upita su prikazani na slici dolje.

Time Statistics	
Client processing time	21
Total execution time	86
Wait time on server replies	65

Slika 52. Statistika izvršavanja upita iz SQL Servera s indeksom

A grafički prikaz načina izvršavanja upita izgleda kao na slici dolje.



Slika 53. Plan izvođenja upita s indeksom

19.4. INDEKSI ZA PRETRAGU TEKSTA

Indeksi za pretragu teksta (engl. *Full-text index, FTI*) je tehnologija koja omogućava lingvističko pretraživanje teksta (engl. *Full-text search, FTS*) unutar baze podataka. S obzirom na brojne opcije koje omogućava (kao što je pretraživanje po frazama, indeksiranje teksta, rangiranje rezultata pretrage, primjena zabranjenih riječi te upotreba rječnika sinonima) pretraga po takvom indeksu služi i kao koristan alat. Najbolji primjer primjene ove tehnologije su internetske tražilice poput Google-a, Yahoo-a, Binga i slično.

19.4.1. Način rada indeksa za pretragu teksta

Moderna tehnologija za pretraživanje kao primjerice iFTS (*integrated full-text search*) se temelji na indeksiranju riječi koje su izvučene iz teksta namijenjenog pretraživanju. Indeksiranje funkcionira slično kao kazalo u knjigama. Za svaku riječ se bilježi iz kojeg je dokumenta, te na kojoj poziciji se nalazi unutar cijelog teksta. Većina modernih baza podataka koristi obrnutu strukturu indeksa za pohranjivanje FTS indeksa, koja razlama tekst po riječima i pohranjuje ih u indeks.

Ovakvo razlamanje teksta po riječima moguće je pokazati na rečenici: „Svi su jednaki, ali neki su jednakiji od drugih.“.

Tablica 30. Primjer obrnutog indeksiranja

Riječ	ID dokumenta	Pozicija
Svi	1	1
su	1	2
jednaki	1	3
ali	1	4

neki	1	5
su	1	6
jednakiji	1	7
od	1	8
drugih	1	9

Ključna polja kod obrnutog indeksa su riječ koja se indeksira, referenca na redak u tablici iz kojeg potječe dokument (ID dokumenta) i pozicija koja daje relativnu poziciju za svaku riječ u tekstu. Kako bi se podaci lakše i brže indeksirali bez viška podataka, koristi se lista zabranjenih riječi koji određene riječi ne indeksira. Za svaki jezik je ta lista drugačija, te se također može i dopuniti riječima za koje se ne želi da budu indeksirane. Za hrvatski jezik standardna lista zabranjenih riječi trebala bi biti: a, da, i, je, na, o, od, s, se, su, u, za. Do liste zabranjenih riječi moguće je doći pomoću jednostavnog upita:

```
SELECT * FROM INFORMATION_SCHEMA.INNODB_FT_DEFAULT_STOPWORD;
```

Uz korištenje navedene liste zabranjenih riječi za hrvatski jezik, primjer iz iduće tablice. izgledat će poprilično pojednostavljeno.

Tablica 31. Primjer obrnutog indeksiranja bez zabranjenih riječi

Riječ	ID dokumenta	Pozicija
Svi	1	1
jednaki	1	3
ali	1	4
neki	1	5
jednakiji	1	7
drugih	1	9

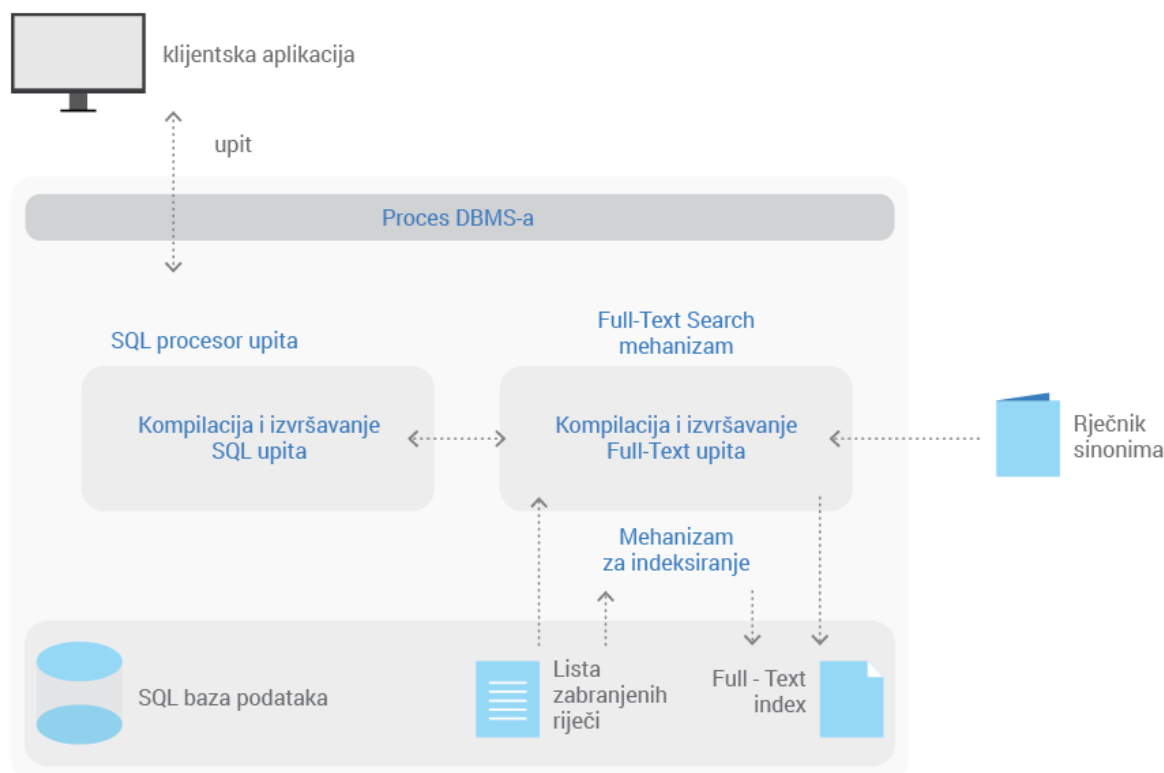
Listu zabranjenih riječi moguće je ručno podesiti na vlastitu tablicu. Najveći problem kod nekih DBMS sustava je taj što je lista zabranjenih riječi globalna (kao što je slučaj s MySQL-om). Problem dolazi do vidjela kada višejezične aplikacije moraju koristiti tekstualnu pretragu.

Kreiranje vlastite tablice za zabranjene riječi:

```
CREATE TABLE mojalista(value VARCHAR(30));
INSERT INTO mojalista(value) VALUES ('a'), ('da'), ('i'), ('je'),
('na'), ('o'), ('od'), ('s'), ('se'), ('su'), ('u'), ('za');
SET GLOBAL innodb_ft_server_stopword_table = 'studenti/mojalista';
```

19.4.2. Arhitektura lingvističkog pretraživanja

Arhitektura lingvističke pretrage se najčešće sastoji od nekoliko komponenata koje rade zajedno s DBMS-ovim mehanizmom za upite kako bi efikasno izvodili lingvistička pretraživanja. U nastavku je pojašnjena ilustracija arhitekture lingvističke pretrage.



Slika 54. Pojednostavljena arhitektura lingvističke pretrage

- Klijentska aplikacija – kreira FTS upite i šalje ih SQL procesoru upita koji se potom brine o tome da upit odgovara sintaksi.
- Proces DBMS-a – sadrži SQL procesor upita i mehanizam za tekstualnu pretragu.
- SQL procesor upita – sastoji se od nekoliko komponenti koje su zadužene za ovjeravanje (engl. *validation*) SQL upita, kompiliranje upita, izradu planova upita, te izvršavanje upita na bazi podataka.
- FTS procesor upita – nakon što SQL procesor upita primi FTS upit, on ga prosljeđuje FTS procesoru upita. FTS procesor upita je zadužen za raščlambu i validaciju upita, konzultiranje s tekstualnim indeksom radi ispunjavanja zahtjeva, te rad sa SQL procesorom upita kako bi se vratili svi potrebni rezultati.
- Mehanizam za indeksiranje – radi s drugim komponentama kako bi dobio tekstualni sadržaj iz dokumenata, označio sadržaj i popunio tekstualni indeks. Neke od

komponentata s kojima ovaj mehanizam radi, a koji nisu prikazani u dijagramu su: sakupljač, rukovatelj protokolom, filtri i razlagači riječi.

- Full-Text indeks – to je struktura obrnutog indeksa koji je pridružen određenoj tablici. Mehanizam za indeksiranje popunjava FTS indeks, a FTS procesor upita se konzultira s indeksom kako bi ispunio zahtjeve pretraživanja.
- Lista zabranjenih riječi – kao što je već bilo rečeno, to je lista riječi koje će se zanemariti prilikom indeksiranja. Mehanizam za indeksiranje i FTS procesor upita se konzultiraju s listom zabranjenih riječi prilikom indeksiranja i izvršavanja upita kako bi eliminirali zabranjene riječi iz indeksa i pojma koji se traži.
- Rječnik sinonima – Neki DBMS sustavi ga posjeduju, a neki ne. On omogućuje proširivanje pretraživanja korištenjem sinonima riječi u pojmu koji se pretražuje. Na primjer, ako se pretražuje tekst za pojam „auto“, uz pomoć rječnika sinonima, upit će također tražiti i pojmove „automobil“, „vozilo“.

19.4.3. Kreiranje tekstualnog indeksa u MySQL-u

Na MySQL DBMS-u kreiranje tekstualnih indeksa je veoma jednostavno, ali sama pretraga ne obiluje velikim mogućnostima. Potrebno je obratiti pažnju na to da će tekstualni indksi raditi samo pod „MyIsam“ i „InnoDB“ okruženjima.

Na postojećoj tablici tekstualni indeks je moguće dodati pomoću naredbe:

```
ALTER TABLE tablica
ADD FULLTEXT INDEX NazivIndeksa (stupac1 ASC, stupac2 DESC);
```

Na primjer, na postojećoj tablici „kolegiji“ potrebno je dodati tekstualni indeks nad stupcima naziv i opis:

```
ALTER TABLE kolegiji
ADD FULLTEXT INDEX IndexKolegijiFullText (naziv ASC, opis ASC);
```

Prilikom kreiranja nove tablice u bazi podataka, tekstualni indeks je potrebno posebno naznačiti slično kao i klasičan indeks.

Na primjer, potrebno je kreirati tablicu članaka s poljima naslov i tekst koji su indeksirani pomoću tekstualnog indeksa:

```
CREATE TABLE clanci (
id INT PRIMARY KEY,
naslov VARCHAR(200),
tekstClanka TEXT,
FULLTEXT (naslov,tekstClanka)
```

)

19.4.4. Pretraga po tekstualnom indeksu u MySQL-u

Najvažnije funkcije koje se upotrebljavaju za tekstualnu pretragu na MySQL DBMS-u su MATCH i AGAINST čije korištenje može biti ponešto drugačije od standardnih SQL funkcija. Unutar MATCH funkcije potrebno je odrediti nad kojim stupcima će se pretraga obavljati, a unutar AGAINST funkcije potrebno je navesti pojam koji se pretražuje i način pretrage.

Primjer:

Unutar tablice kolegiji potrebno je pretražiti opis i naslov kolegija za riječ „baze“:

```
SELECT * FROM kolegiji
  WHERE MATCH (naziv, opis)
  AGAINST ('baze' IN NATURAL LANGUAGE MODE);
```

ili

```
SELECT * FROM kolegiji
  WHERE MATCH(naziv, opis)
  AGAINST ('baze' IN NATURAL LANGUAGE MODE);
```

Rezultati:

☐	id	naziv	opis		idSmjer
☐	1	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	1
☐	16	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	7
☐	46	Napredne baze podataka	Student se treba upoznati s objektima i kontrolama za pri...	167B	1
☐	21	Linearna algebra 1	Preko jednostavnih primjera (sustavi linearnih jednadžbi)...	182B	11

Na MySQL DBMS-u, stupci nad kojima se vrši pretraga moraju biti identične onima nad kojima se nalazi tekstualni indeks. Nemoguće je podesiti tekstualni indeks nad dva stupca i tada pretraživati samo po jednom stupcu. Ako postoji takav slučaj, indeks treba odjeliti za svaki stupac zasebno.

Ako je potrebno prikazati relevantnost rezultata, upit je potrebno napisati u potpunosti drugačije i njegova sintaksa se kosi s ostalim pravilima pisanja dobrog SQL upita.

Primjeri:

Unutar tablice kolegiji potrebno je pretražiti opis i naslov kolegija za riječ „baze“ i sortirati rezultate po rang pretrage:

```
SELECT kolegiji.*, MATCH(naziv, opis)
AGAINST('baze podataka' IN NATURAL LANGUAGE MODE) AS rang
FROM kolegiji
ORDER BY rang DESC;
```

Rezultati:

id	naziv	opis	idSmjer	rang
1	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	1
16	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	7
46	Napredne baze podataka	Student se treba upoznati s objektima i kontrolama za pri...	167B	1
21	Linearna algebra 1	Preko jednostavnih primjera (sustavi linearnih jednadžbi)...	182B	11
2	Uvod u UNIX sustave	Student treba savladati osnovno znanje rada na otvorenim ...	80B	1
3	Uvod u UNIX sustave	Student treba savladati osnovno znanje rada na otvorenim ...	80B	2
4	Fizika	Razumijevanje fizikalnih pojava i veličina, opisanih u ši...	97B	2
5	Fizika	Razumijevanje fizikalnih pojava i veličina, opisanih u ši...	97B	3

Kako bi se izbjegli ne relevantni rezultati moguće je napisati jedan od dva moguća upita:

```
SELECT kolegiji.*, MATCH(naziv, opis)
AGAINST('baze podataka' IN NATURAL LANGUAGE MODE) AS rang
FROM kolegiji
WHERE MATCH(naziv, opis)
AGAINST('baze podataka' IN NATURAL LANGUAGE MODE)
ORDER BY rang DESC;
```

ili

```
SELECT kolegiji.*, MATCH(naziv, opis)
AGAINST('baze podataka' IN NATURAL LANGUAGE MODE) AS rang
FROM kolegiji
HAVING rang>0
ORDER BY rang DESC
```

MySQL DBMS ima mogućnost naprednije pretrage pomoću „IN BOOLEAN MODE” modifikatora. Pomoću tog modifikatora moguće je koristiti posebne znakove „+ i -”, koji označavaju mora li se neka riječ pojaviti unutar teksta ili ne.

Primjeri:

Potrebno je prikazati samo podatke tablice *kolegiji* u kojima postoji riječ „baze” i ne postoji riječ „podataka”.

```
SELECT * FROM kolegiji
WHERE MATCH(naziv, opis)
AGAINST('+baze -podataka' IN BOOLEAN MODE);
```

Rezultati:

id	naziv	opis	idSmjer
21	Linearna algebra 1	Preko jednostavnih primjera (sustavi linearnih jednadžbi)...	182B

Potrebno je prikazati samo podatke tablice *kolegiji* u kojima postoji riječ „baze” i obvezno postoji riječ „podataka”.

```
SELECT * FROM kolegiji
```

```
WHERE MATCH(naziv, opis)
AGAINST('+baze +podataka' IN BOOLEAN MODE);
```

Rezultati:

☐	id	naziv	opis		idSmjer
☐	1	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	1
☐	16	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	7
☐	46	Napredne baze podataka	Student se treba upoznati s objektima i kontrolama za pri...	167B	1

20. Transakcije

20.1. DEFINICIJA I OSNOVNO NAČELO TRANSAKCIJA

Transakcija (engl. *transaction*) je skup od jedne ili više SQL naredbi koje čine logičku jedinicu rada nad bazom podataka. Sastoji se od niza logički povezanih radnji, a korisnik ili aplikacija ih trebaju provesti kao nedjeljivu cjelinu. U slučajevima kada transakcija sadrži nekoliko operacija koje se odnose na bilo kakvu izmjenu podataka u bazi, ona mora biti u potpunosti izvedena i potvrđena (engl. *committed*) ili opovrgnuta (engl. *rolled back*). To je osnovno načelo transakcija koje se temelji na principu „Svi za jednog, jedan za sve“ odnosno načelu „Sve ili ništa“. Transakcija se dakle promatra kao jedinica posla nad bazom podataka koja mora

- a) biti provedena u cijelosti ili
- b) ne smije biti provedena uopće.

Ako je transakcija iz bilo kojeg razloga prekinuta i nije obavljena do kraja, ona mora biti neutralizirana kako bi se baza vratila u konzistentno stanje sa ciljem očuvanja integriteta podataka u bazi. Svi podatci koje je transakcija do trenutka prekida promijenila morali bi biti vraćeni na svoje polazne vrijednosti. Jedan od najčešćih slučajeva pogreške u bazi je neizvršavanje transakcije u potpunosti i neizvršavanje neutralizacije. Transakcija nije objekt koji postoji u bazi, nema svoje ime i ne može se *backupirati* sa ostalim podacima ili objektima, osim ako nije sastavni dio nekog drugog pohranjenog objekta.

Koncept transakcije objašnjen je na dijagramu operacija unutar jedne transakcije, na primjeru baze podataka banke. U slučaju kada klijent A uplaćuje klijentu B na bankovni račun 400kn, ne smije se dogoditi da se novac izgubi ili da se pojavi višak novca. Ako u bilo kojem trenutku dođe do pogreške, bazu podataka ja potrebno vratiti u početno konzistentno stanje.



Slika 55. Dijagram izvođenja transakcije

20.2. GRANICE TRANSAKCIJE

```

[SET AUTOCOMMIT = 0;]
START TRANSACTION ili BEGIN [WORK];
...
COMMIT [WORK]; ili ROLLBACK [WORK];
[SET AUTOCOMMIT = 1;]
    
```

Naredba `START TRANSACTION` ili `BEGIN WORK` označiti će početak transakcije. Nakon toga se navode sve SQL naredbe koje je potrebno izvesti u sklopu započete transakcije. Na kraju se navodi:

```

COMMIT [WORK]
ili
ROLLBACK [WORK]
    
```

- a) COMMIT – označava uspješan završetak transakcije i trajno pohranjuje sve promjene izvedene tijekom transakcije koje tada postaju vidljive i ostalim sesijama, korisnicima ili transakcijama
- b) ROLLBACK – označava neuspješan završetak i poništava sve do sada napravljeno od naredbe START ili BEGIN, koristi se za prekid transakcije unutar koje je uočena pogreška ili se obavlja automatski kada je transakcija prekinuta izvana

Početak i kraj se za svaku transakciju mogu definirati:

- **eksplicitno** naredbama BEGIN / COMMIT / ROLLBACK
- **implicitno** pri čemu svaka operacija izmjene u bazi predstavlja transakciju, njezin uspješan završetak predstavlja COMMIT a neuspješan ROLLBACK.

U bazi prilikom svake operacije izmjene izvršava se zapravo jedna transakcija, omeđena sa START i COMMIT. Primjerice:

```
UPDATE izvrsitelji
    SET idUlogaIzvrstelja = 2
    WHERE jmbgNastavnik = '0809970383313' AND idKolegij = 14;
```

Navedena naredba ažuriranja u tablici izvrsitelji ekvivalent je idućoj transakciji.

```
START TRANSACTION;
    UPDATE izvrsitelji
        SET idUlogaIzvrstelja = 2
        WHERE jmbgNastavnik = '0809970383313'
AND idKolegij = 14;
COMMIT;
```

20.3. PREDUVJETI ZA KORIŠTENJE TRANSAKCIJA

20.3.1. Način rada automatskog potvrđivanja

Kako bi skup naredbi predstavljao transakciju, i ponašao se kao jedinstvena cjelina, nužno je raditi u okruženju isključenog automatskog potvrđivanja izmjena (engl. *autocommit*) koje svaka pojedina naredba izvršava. Pretpostavljen način rada upravo jest automatsko potvrđivanje svake izvedene naredbe, te se svaka sesija pokreće s postavljenim *autocommit* načinom rada. To znači da će svaka promjena napravljena primjerice naredbom DELETE biti zapisana u bazi njezinim izvršavanjem. Ako bi se transakcija koja sadrži više naredbi pokretala u ovom načinu rada, ne bi bilo moguće držati se načela „sve ili ništa“. Zato je potrebno prije početka transakcije isključiti način rada automatskog potvrđivanja postavljanjem sesijske varijable AUTOCOMMIT na vrijednost nula. Ovisno o verziji DBMS-a,

to će se odraditi automatski pozivanjem naredbe BEGIN WORK, odnosno preporuča se prije BEGIN WORK izvesti:

```
SET AUTOCOMMIT = 0;
```

Po završetku transakcije potrebno je ponovno uključiti način rada automatskog potvrđivanja, što će ovisno o verziji, DBMS odraditi sam pokretanjem COMMIT ili ROLLBACK naredbe, odnosno preporuča se izvesti:

```
SET AUTOCOMMIT = 1;
```

Podatak o načinu rada pohranjen je u meta bazi podataka information_schema. Ovdje se nalaze „podatci o podacima“ kao što su nazivi baza, tablica, stupaca, privilegija, zaključavanja i slično. To je baza podataka sa informacijama o bazama podataka na MySQL poslužitelju. Stanje automatskog potvrđivanja za sesiju nalazi se u tablici SESSION_VARIABLES, redak AUTOCOMMIT.

```
SELECT * FROM information_schema.SESSION_VARIABLES
      WHERE variable_name = "autocommit";
```

Od verzije 5.7 umjesto korištenjem gornje naredbe, do istih je podataka moguće doći pomoću naredbe SHOW:

```
SHOW VARIABLES WHERE variable_name = "autocommit";
```

☐ Variable name	Value
☐ autocommit	ON

```
SET AUTOCOMMIT = 0;
```

```
SHOW VARIABLES WHERE variable_name = "autocommit";
```

☐ Variable name	Value
☐ autocommit	OFF

20.3.2. Mehanizmi pohrane podataka

Drugi i ne manje važan preduvjet uspješnog korištenja transakcija je uporaba transakcija nad tablicama koje koriste mehanizam pohrane podataka (engl. *storage engine*) koji podržava transakcije. Potrebno je koristiti InnoDB mehanizam pohrane za tablice koje se koriste unutar transakcije. Od MySQL verzije 5.7 upravo je ovo pretpostavljeni mehanizam pohrane za DBMS. Alternativa koju je moguće izvesti koristeći mehanizam pohrane koji ne podržava transakcije, a potrebno ga je koristiti radi nekih drugih njegovih boljih svojstava, su pseudotransakcije. U njima se imitira transakcija u tablicama koje ne podržavaju ACID transakcije korištenjem zaključavanja podataka koji će se koristiti u transakciji.

20.4. PRIMJERI

Izraditi tablicu menze koja će sadržavati osnovne podatke o menzi, te unijeti jednu n-torku u tablicu. U tablici je potrebno koristiti mehanizam pohrane podataka koji podržava transakcije.

```
CREATE TABLE menze (
  id INT(11) NOT NULL AUTO_INCREMENT,
  naziv VARCHAR(50) DEFAULT NULL,
  adresa VARCHAR(250) DEFAULT NULL,
  PRIMARY KEY (idMenze)
) ENGINE=INNODB DEFAULT CHARSET=utf8;

INSERT INTO menze (naziv, adresa)
VALUES ('Menza FSB', 'Ivana Lučića 5');

SELECT * FROM menze;
```

☐	▼ id	naziv	adresa
☐	1	Menza FSB	Ivana Lučića 5

Unijeti dvije nove n-torke u tablicu menze te napraviti opoziv unosa podataka.

```
SET AUTOCOMMIT = 0;

SHOW VARIABLES WHERE variable_name = "autocommit"; /* u rezultatu
je vidljivo da je autocommit isključen */

START TRANSACTION;

  INSERT INTO menze (naziv, adresa)
    VALUES ('Menza Voltino', 'Konavoska 2');

  INSERT INTO menze (naziv, adresa)
    VALUES ('Menza Vrbik', 'Vrbik 8a');

  SELECT * FROM menze;

ROLLBACK;

SET AUTOCOMMIT = 1;
```

Provjera podataka u tablici menze nakon završetka transakcije.

```
SELECT * FROM menze;
```

☐	▼ id	naziv	adresa
☐	1	Menza FSB	Ivana Lučića 5

Iz rezultata je vidljivo da je transakcija uspješno opozvana, u tablici menze i dalje postoji samo jedna n-torka.

Unijeti dvije nove menze u tablicu 'menze' te napraviti potvrditi naredbe.

```
SET AUTOCOMMIT = 0;

START TRANSACTION;

  INSERT INTO menze (naziv, adresa)
```

```
VALUES ('Menza Voltino', 'Konavoska 2');
INSERT INTO menze (naziv, adresa)
VALUES ('Menza Vrbik', 'Vrbik 8a');
SELECT * FROM menze;
COMMIT;
SET AUTOCOMMIT = 1;
```

Provjera podataka u tablici menze nakon završetka transakcije.

```
SELECT * FROM menze;
```

☐ id	naziv	adresa
☐ 1	Menza FSB	Ivana Lučića 5
☐ 4	Menza Vrbik	Vrbik 8a
☐ 5	Menza Voltino	Konavoska 2

Transakcija je uspješno potvrđena, u tablici su sada tri n-torke. Id sa vrijednostima 2 i 3 je iskorišten u transakciji koja je opozvana, transakcija koja je potvrđena je unijela n-torke s id vrijednostima 4 i 5.

Potrebno je izraditi još jednu tablicu za podatke o menzama i postaviti mehanizam pohrane koji ne podržava transakcije.

```
CREATE TABLE menze2 (
  id INT(11) NOT NULL AUTO_INCREMENT,
  naziv VARCHAR(50) DEFAULT NULL,
  adresa VARCHAR(250) DEFAULT NULL,
  PRIMARY KEY (id)
) ENGINE=MYISAM DEFAULT CHARSET=utf8;
```

Unijeti dvije nove menze u tablicu 'menze2' te napraviti opoziv unosa podataka.

```
SET AUTOCOMMIT = 0;
START TRANSACTION;
INSERT INTO menze2 (naziv, adresa)
VALUES ('Menza Voltino', 'Konavoska 2');
INSERT INTO menze2 (naziv, adresa)
VALUES ('Menza Vrbik', 'Vrbik 8a');
ROLLBACK;
SET AUTOCOMMIT = 1;
SELECT * FROM menze2;
```

☐ id	naziv	adresa
☐ 1	Menza Voltino	Konavoska 2
☐ 2	Menza Vrbik	Vrbik 8a

Unatoč naredbi ROLLBACK koja je izvršena na kraju transakcije, n-torke su ipak trajno zapisane u bazi obzirom da je korišten mehanizam pohrane podataka koji ne podržava

transakcije. Pokrenuti transakciju u prvoj sesiji za unos menze Cassandra u tablicu 'menze'. Prije nego transakcija završi, pokrenuti novu sesiju te provjeriti jesu li u njoj vidljivi podatci uneseni u prvoj sesiji a nisu potvrđeni (COMMIT). Završiti transakciju te napraviti istu provjeru u drugoj sesiji ponovno.

Prva sesija:

```
SET AUTOCOMMIT=0;
START TRANSACTION;
INSERT INTO menze (naziv,
adresa) VALUES ('Menza
Cassandra', 'Ul. Grada Vukovara
39');
SELECT * FROM menze;
```

☐ id	naziv	adresa
☐ 1	Menza FSB	Ivana Lučića 5
☐ 4	Menza Vrbik	Vrbik 8a
☐ 5	Menza Volti	Konavoska 2
☐ 6	Menza Cassa	Ul. Grada Vukovara 39

```
COMMIT;
SET AUTOCOMMIT =1;
SELECT * FROM menze;
```

☐ id	naziv	adresa
☐ 1	Menza FSB	Ivana Lučića 5
☐ 4	Menza Vrbik	Vrbik 8a
☐ 5	Menza Volti	Konavoska 2
☐ 6	Menza Cassa	Ul. Grada Vukovara 39

Druga sesija:

```
SELECT * FROM menze;
```

☐ id	naziv	adresa
☐ 1	Menza FSB	Ivana Lučića 5
☐ 4	Menza Vrbik	Vrbik 8a
☐ 5	Menza Volti	Konavoska 2

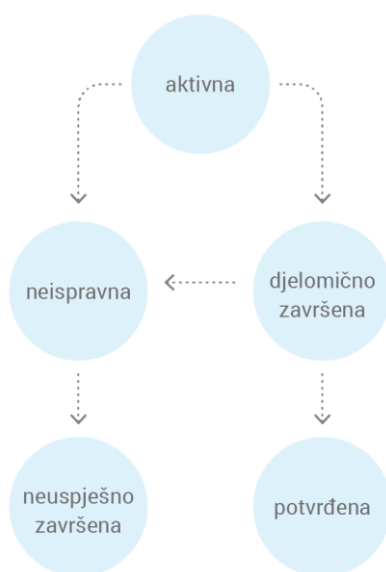
```
SELECT * FROM menze;
```

☐ id	naziv	adresa
☐ 1	Menza FSB	Ivana Lučića 5
☐ 4	Menza Vrbik	Vrbik 8a
☐ 5	Menza Volti	Konavoska 2
☐ 6	Menza Cassa	Ul. Grada Vukovara 39

20.5. STANJA TRANSAKCIJA

Transakcija svojim izvršavanjem provodi bazu iz jednog konzistentnog stanja u drugo. Za vrijeme izvođenja transakcije, ona se može nalaziti u jednom od navedenih stanja:

- **aktivna** (engl. *active*) – tijekom izvođenja transakcije
- **djelomično završena** (engl. *partially committed*) – nakon što je obavljena posljednja operacija transakcije
- **neispravna** (engl. *failed*) – nakon što se ustanovi da nije moguće nastaviti normalno izvođenje transakcije
- **neuspješno završena** (engl. *aborted*) – nakon što su poništeni svi efekti transakcije i baza podataka je vraćena u stanje kakvo je bilo prije nego što je transakcija počela
- **potvrđena** (engl. *committed*) – transakcija je uspješno završena



Slika 56. Stanja transakcija

20.6. ACID

Transakcije u MySQL-u temeljene su na ACID modelu. ACID model je skup pravila modela baze podataka koji naglašavaju aspekte pouzdanosti koji su važni za poslovne podatke. InnoDB omogućava strogo podržavanje ACID modela, tako da u nepredviđenim situacijama podaci neće biti oštećeni niti će rezultati biti iskrivljeni iznimnim događajima poput rušenja softvera i hardverskih kvarova. ACID svojstva su svojstva ACID modela, a odnose se na pravila:

- održavanja integriteta (točnosti podataka) i
- održavanja konzistentnosti (točnosti informacija) baze podataka.

O ACID svojstvima se brine sam DBMS, odnosno sustav za upravljanje transakcijama (engl. *transaction manager*). ACID svojstva odnosno transakcije dodatno opterećuju memoriju i brzinu, te ako sustavu u određenoj specifičnoj namjeni nisu potrebne, nije ih potrebno slijediti.

Akronim ACID označava slijedeća svojstva:

- **A** – engl. *Atomicity* - **atomarnost**
- **C** – engl. *Consistency* - **konzistentnost**
- **I** – engl. *Isolation* - **izolacija**
- **D** – engl. *Durability* - **izdržljivost**

20.6.1. Atomarnost

Transakcija se mora obaviti u cijelosti ili se uopće ne smije obaviti, svojstvo je atomarnosti.

Transakcija može biti neizvršena u cijelosti u tri slučaja:

1. Prekinuta ili neuspješno završena
 - zbog pogrešaka pri izvođenju
 - ako je prekinuta od strane DBMS-a, bit će resetirana i pokrenuta ispočetka
2. Prilikom pada sustava
 - primjerice prilikom nestanka struje u vrijeme izvođenja transakcije
3. Nepredviđene situacije
 - neočekivane vrijednosti podataka koji se unose
 - nemogućnost pristupa memoriji (disku)

MySQL svojstvo atomarnosti osigurava zapisivanjem rezultata naredbi transakcije samo u međuspremnik memorije (engl. *buffer*). Ove rezultate iz memorije zapisuje na disk i binarni dnevnik (engl. *binary log*) tek prilikom potvrđivanja transakcije (eksplicitni ili implicitni *commit*).

Primjer:

Ako je klijentu A s računa oduzet određeni iznos, transakcija može biti završena tek nakon što je klijentu B dodan taj iznos, ili se obje radnje moraju opovrgnuti.

20.6.2. Konzistentnost

Transakcijom, baza podataka prelazi iz jednog konzistentnog stanja u drugo konzistentno stanje. Korisnik ili aplikacija koji potvrđuju transakciju, moraju osigurati da će transakcija po završetku ostaviti bazu podataka u ispravnom stanju.

MySQL svojstvo konzistentnosti osigurava održavanjem dnevnika transakcija u kojemu zapisuje sve promjene u bazi i ostavlja revizijski trag za oporavak transakcije. Osim toga MySQL osigurava mehanizme zaključavanja koji omogućuju da su svi podatci korišteni u transakciji zaključani prilikom pokretanja dovoljno dugo da se ili izvrši transakcija ili sve vrati natrag.

Primjer:

Ako definiramo prebacivanje određenog iznosa s računa klijenta A klijentu B, nakon što oduzmemo iznos s računa A, moramo ISTI iznos dodati računu B (DBMS se neće sam za to pobrinuti). Suma stanja oba računa ista je na početku i na kraju.

20.6.3. Izolacija

Kada se paralelno odvijaju dvije ili više transakcija, njihov učinak mora biti kao da su se odvijale jedna nakon druge.

Kako bi to osigurali, DBMS-ovi koriste posebne algoritme za organizaciju, primjerice algoritam serijskog zakazivanja (engl. *Serial Scheduling*). Varijable semafora na strani poslužitelja i mehanizmi za zaključavanje djeluju kao upravitelji kako bi pomogli programima da upravljaju vlastitim mehanizmima izolacije. MySQL-ov InnoDB mehanizam za osiguravanje ovog svojstva koristi finu granulaciju zaključavanja na razini retka.

Primjer:

Za vrijeme izvođenja transakcije prebacivanja novca između računa klijenta A i klijenta B, ostale transakcije ne mogu čitati stanje računa A i B sve dok prva transakcija ne završi.

Svojstvo izolacije usko je povezano sa svojstvom serijabilnosti koje nije dio ACID-a. Neka se u višekorisničkoj bazi podataka izvodi nekoliko transakcija paralelno tako da se pojedini dijelovi tih transakcija izvode vremenski izmiješano. Ako je konačan učinak njihovog izvođenja isti kao da su se one izvršavale serijski, kažemo da se radi o serijabilnom izvršavanju transakcija.

20.6.4. Izdržljivost

Ako je transakcija završila svoj posao, njezini efekti ne smiju biti izgubljeni ako se dogodi kvar sustava, čak ni u situaciji kada se kvar dogodi neposredno nakon završetka transakcije. Ako je pak do kvara došlo za vrijeme izvođenja transakcije, DBMS se mora pobrinuti za

vraćanje u konzistentno stanje pomoću dnevnika u kojem su zapisane sve promjene na disku.

MySQL osigurava svojstvo izdržljivosti upravo održavanjem binarnog dnevnika transakcija. U slučaju kvara hardvera ili naglog zaustavljanja sustava, oporavak izgubljenih podataka relativno je jednostavan zadatak – korištenjem posljednje sigurnosne kopije u kombinaciji s dnevnikom kada se sustav ponovo pokrene. Sve InnoDB tablice u potpunosti su izdržljive, dok su MyISAM tablice (i ostale koje ne podržavaju transakcije) izdržljive samo djelomično.

Primjer:

Kada je jednom obavljen transfer novca između računa, nema povratka u slučaju pogreške već novac ostaje na računu klijenta B.

20.7. OGRANIČENJA TRANSAKCIJA

Neke naredbe unutar transakcije ne mogu biti opozvane jer se njihovim izvođenjem izvodi implicitno potvrđivanje promjena (COMMIT). One će u tom trenutku ujedno i završiti transakciju kao da je napravljen COMMIT prije izvršavanja naredbe. Radi se većinom o DDL naredbama poput CREATE DATABASE, CREATE TABLE, DROP DATABASE, DROP TABLE, ALTER TABLE, CREATE FUNCTION, ALTER FUNCTION i slično. Ovo je pravilo koje nije univerzalno ako se radi o ključnoj riječi TEMPORARY. Popis tih naredbi za MySQL verziju 5.7 nalazi se na poveznici <http://dev.mysql.com/doc/refman/5.7/en/implicit-commit.html>.

Transakcije se ne mogu ugnježdživati jer početak transakcije izvršava eksplicitni COMMIT čime potvrđuje i zatvara transakciju unutar koje bi se eventualno ugnijezdila. Jedna aplikacija u određenom trenutku može imati aktivnu samo jednu transakciju. Transakcije ne podržavaju opoziv (ROLLBACK) DDL naredbi

Primjer:

Unutar transakcije potrebno je izraditi tablicu menze3. Opovrgnuti promjene te provjeriti postoji li tablica.

```
SELECT * FROM menze3; /* provjera nepostojanja tablice menze3 */
```

```
1 queries executed, 0 success, 1 ERRORS, 0 WARNINGS
QUERY: SELECT * FROM menze3 LIMIT 0, 1000
Error CODE: 1146
TABLE 'studenti.menze3' doesn't exist
```

```
SET autocommit=0;
BEGIN WORK;
CREATE TABLE menze3 (
```

```

        id INT(11) NOT NULL AUTO_INCREMENT,
        naziv VARCHAR(50) DEFAULT NULL,
        адреса VARCHAR(250) DEFAULT NULL,
        PRIMARY KEY (id)
    ) DEFAULT CHARSET=utf8;
ROLLBACK;
SET autocommit=1;

```

```
SELECT * FROM menze3;
```

Ispis naredbom SELECT iz tablice menze3 vraća nula n-torki odnosno više ne vraća poruku o pogreški, dakle tablica je stvorena a naredba ROLLBACK nije opozvala CREATE TABLE naredbu.

id	naziv	adresa
----	-------	--------

20.8. TOČKE POHRANJIVANJA

InnoDB omogućava dodatnu korisnu značajku, a to je mogućnost opovrgavanja dijela transakcije umjesto opovrgavanja transakcije u cijelosti. Navedeno se postiže korištenjem točaka pohranjivanja (engl. *savepoint*). Točka pohranjivanja je korisnički definirana točka unutar transakcije koja se koristi za označavanje specifičnih podstanja unutar transakcije. U slučaju kvara transakcije, ove pohranjene točke omogućuju vraćanje samo određenih dijelova transakcije, a ne cjelokupne transakcije. Pohranjene točke unutar transakcije postavljaju su pomoću naredbe SAVEPOINT koja prihvaća identifikator koji je definiran od strane korisnika. ROLLBACK TO SAVEPOINT naredba tada se može koristiti za opovrgavanje transakcije u tijeku na tu točku, opovrgavajući sve promjene izvršene **nakon** pohranjene točke.

Postavljanje točke pohranjivanja kako bi se kasnije mogao napraviti opoziv do te točke:

```
SAVEPOINT imeTockePohranjivanja;
```

Opoziv svih naredbi nakon imenovane točke pohranjivanja:

```
ROLLBACK TO SAVEPOINT imeTockePohranjivanja;
```

Brisanje postavljene točke pohranjivanja:

```
RELEASE SAVEPOINT;
```

Primjer:

Nastavnicima Morani Novak i Tvrtku Miletiću potrebno je u atribut titulalspred unijeti vrijednost 'dr.sc.'. Dodavanje titule Morani potrebno je potvrditi, a dodavanje titule Tvrtku opovrgnuti.

```
SELECT * FROM nastavnici
WHERE (ime = 'Morana' AND prezime = 'Novak')
OR (ime = 'Tvrtko' AND prezime = 'Miletić');
```

☐ jmbg	ime	prezime	adresa	titu...	titulaIza	postBr	lozinka
☐ 2409952381906	Morana	Novak	Kolodvorska 37		prof.	10410	fc9e9539ca1d04d
☐ 2409954210041	Tvrtko	Miletić	Gundulićeva 59		dipl.ing.r	10000	668f7f71056d78b

```
SET AUTOCOMMIT = 0;
```

```
BEGIN;
```

```
UPDATE nastavnici
SET titulaIspred = 'dr.sc.'
WHERE ime = 'Morana' AND prezime = 'Novak';
SAVEPOINT pohrana;
UPDATE nastavnici
SET titulaIspred = 'dr.sc.'
WHERE ime = 'Tvrtko' AND prezime = 'Miletić';
/* provjera: */
SELECT * FROM nastavnici
WHERE (ime = 'Morana' AND prezime = 'Novak')
OR (ime = 'Tvrtko' AND prezime = 'Miletić');
```

☐ jmbg	ime	prezime	adresa	titu...	titulaIza	postBr	lozinka
☐ 2409952381906	Morana	Novak	Kolodvorska 37	dr.sc.	prof.	10410	fc9e9539ca1d04d
☐ 2409954210041	Tvrtko	Miletić	Gundulićeva 59	dr.sc.	dipl.ing.r	10000	668f7f71056d78b

```
ROLLBACK TO SAVEPOINT pohrana;
/* provjera: */
SELECT * FROM nastavnici
WHERE (ime = 'Morana' AND prezime = 'Novak')
OR (ime = 'Tvrtko' AND prezime = 'Miletić');
```

☐ jmbg	ime	prezime	adresa	titu...	titulaIza	postBr	lozinka
☐ 2409952381906	Morana	Novak	Kolodvorska 37	dr.sc.	prof.	10410	fc9e9539ca1d04d
☐ 2409954210041	Tvrtko	Miletić	Gundulićeva 59		dipl.ing.r	10000	668f7f71056d78b

```
COMMIT;
```

```
SET AUTOCOMMIT = 1;
```

21. Pohranjeni zadatci

Pohranjeni zadaci (engl. *routines*) neovisni su dijelovi koda namijenjeni višekratnoj uporabi koja enkapsulira određenu funkcionalnost, a mogu se pozivati unutar aplikacija ili drugih SQL upita. Koncept pohranjenih zadataka blizak je programerskom svijetu kroz koji višekratnu uporabljivost koda postaje obilježje SQL upita. U pohranjenim se zadacima prožimaju mogućnosti SQL upita s lokalnim i globalnim varijablama, naredbama za kontrolu toka programa te naredbama za rukovanje iznimkama.

Prilikom porasta složenosti poslovne logike aplikacija koje se vežu na bazu podataka, česta je pojava repetitivnog pisanja niza istih SQL naredbi. Primjerice, više puta piše se ista INSERT naredba za unos n-torke u tablicu ili se rade isti izračuni nad određenim setom podataka. Tada je poželjno pohraniti niz ovakvih naredbi u pohranjeni zadatak koji će se nalaziti na poslužitelju baze podataka i izvršavati na način da aplikacija na klijentskoj strani ne mora prenositi niz naredbi do poslužitelja baze, već samo ime zadatka koji će obaviti radnje nad bazom.

Pohranjeni zadatci ujedno su objekti baze podataka koji se mogu izvoziti zajedno s ostalim objektima i podacima iz baze u obliku sigurnosne kopije. Obično imaju svoj naziv i definiran način izvođenja. Ovisno o vrsti pohranjenog zadatka, raditi će se o pozivu pomoću ključnih riječi CALL odnosno SELECT ili o automatskom pozivanju pohranjenog zadatka o čemu će biti riječi kasnije u tekstu.

MySQL baza podataka uvodi pohranjene zadatke od verzije 5.0 krajem 2005.-te godine. U toj verziji bila je razvijena podrška za procedure, funkcije, okidače te unaprijedive pogleda. Sintaksa korištena u MySQL pohranjenim zadacima slična je onoj koja se u Oracle bazi koristi u sklopu njenog PL/SQL-a odnosno SQL Server Transact-SQL-a.

21.1. PREDNOSTI POHRANJENIH ZADATAKA

Pohranjivanje SQL upita na poslužitelju u obliku pohranjenih zadataka ima sljedeće prednosti:

- Izbjegava se **repetitivno pisanje istih naredbi** čime se smanjuje mogućnost pogreške. Time se također dobiva na preglednosti i efikasnosti samoga koda aplikacije.
- Pohranjeni zadaci nalaze se na poslužitelju baze podataka umjesto u samoj aplikaciji. U kontekstu klijent-poslužitelj arhitekture, brže je i **manje se opterećuje prijenosni kanal** ako se prenosi samo naziv pohranjenog zadatka, u odnosu na prijenos niza naredbi koje se trebaju izvršiti.
- Olakšano korištenje iste poslovne logike **različitim aplikacijama** te je povećana **prenosivost aplikacije na drugu platformu**. Razna sučelja i aplikacije napisane primjerice u jeziku PHP, Java, VB, Excel, pozivaju se primjerice pomoću JDBC,

JDBC/ODBC ili ODBC programskih sučelja istog pohranjenog zadatka. Skup pohranjenih procedura može djelovati kao svojevrsan API za rad s podacima.

- Programer može jednostavno pozvati pohranjeni zadatak **bez znanja o modelu** odnosno dizajnu baze podataka.
- Povećana je **sigurnost i izolacija podataka** jer postoji mogućnost dodjeljivanja dozvola (GRANT) za pristup podacima na razini pohranjenih zadataka. SPL (*Stored Procedure Language*) omogućava zaštitu podataka od neovlaštene uporabe na razini pohranjenih zadataka. Korisniku se dodijeli dozvola za obavljanje definiranog pohranjenog zadatka, umjesto dozvole za pristup skupu podataka. Time je precizno određen način na koji korisnik smije obaviti operacije nad podacima (brisanje, izmjena, unos, dohvat).
- Logika baze podataka ostaje u samoj bazi, čime se **povećava robusnost** aplikacija na razini **konzistentnosti podataka** i izbjegava nedosljednost podataka.
- **Nadograđene su osnovne funkcionalnosti** baze podataka aritmetikom, varijablama, kontrolom toka i petljama.

21.2. NEDOSTATCI POHRANJENIH ZADATAKA

- Umjesto dohvata podataka za što su baze optimizirane, od pohranjenog zadatka očekuje se rad s logičkim operacijama i poslovnom logikom. Posljedica je opterećenje poslužitelja odnosno opterećenje CPU-a i memorije.
- Postoje ograničenja u pohranjenim zadacima u obliku ograničenosti naredbi koje se mogu koristiti. SPL nije kompleksan kao PHP, C++, VB, Java i ostali.
- Upravljanje pogreškama (engl. *error handling*) je loše što uzrokuje složenost razvoja pohranjenog zadatka jer je potrebno dobro znanje SQL jezika.
- Nedostatkom se može smatrati što je poslovna logika u bazi umjesto u aplikaciji. Time se lako dovodi da je dio logike u bazi a dio u aplikaciji. Ovo je prednost u slučajevima operacija nad kompleksnim setom rezultata, kada je bolje koristiti pohranjeni zadatak kako se složenost entiteta ne bi prenosila u aplikacijski sloj.
- Otežan je prijenos pohranjenih zadataka između različitih DBMS-ova.

21.3. PODJELA POHRANJENIH ZADATAKA

Pohranjeni zadaci dijele se na:

- pohranjene rutine (engl. *stored routines*):
 - pohranjene procedure (engl. *stored procedures*),
 - pohranjene funkcije (engl. *stored functions*),
- okidače (engl. *triggers*),
- događaje (engl. *events*).

Ovisno o vrsti pohranjenog zadatka, on će imati različitu sintaksu, način pozivanja ali i inicijalnu funkcionalnu namjenu. Iako se iste funkcionalnosti baze podataka mogu izvesti na više načina, kroz tekst će biti objašnjene karakteristike i specifičnosti za pojedine vrste pohranjenih zadataka.

Pohranjene rutine odnosno procedure i funkcije imaju više sličnosti nego razlika, no upravo iz razlike čine specifičnima za pojedine namjene. Važno je imati na umu da govoreći o funkcijama u ovom poglavlju, govori se o korisnički definiranim funkcijama, njihovoj sintaksi i primjeni. Podsjetimo se, funkcije općenito podijelili smo na string funkcije, matematičke, datumske, agregatne, ostale i korisničke funkcije.

Deklaracija procedura i deklaracija funkcija međusobno su vrlo slične te je sličan i način njihovog izvođenja. Glavne razlike između pravila procedura i funkcija mogu se svesti na sljedeće:

- Funkcije uvijek vraća jedan rezultat, dok je za proceduru karakteristično da često vraća više ili pak niti jedan rezultat.
- Funkcije ne mogu koristiti SQL naredbe koje podrazumijevaju transakcijsku obradu (COMMIT, ROLLBACK).
- Funkcija ne može sama sebe rekurzivno pozivati.
- Funkcije moraju vratiti rezultat.

22. Varijable

22.1. VRSTE VARIJABLI

Varijable je općenito dijele na dvije skupine, a to su **korisničke varijable** i **sistemske varijable**. Razlikujemo dvije vrste **korisnički varijabli**:

- **Globalne varijable**
 - varijable u MySQL-u koje počinju znakom @,
 - vrijede u jednoj sesiji (spoju na poslužitelj) – prilikom zatvaranja sesije otpuštaju se zauzeti resursi te se vrijednosti varijabli poništavaju,
 - vidljive su unutar bloka pohranjenog zadatka ali i u sesiji,
 - nije ih potrebno deklarirati.
- **Lokalne varijable**
 - varijable deklarirane unutar bloka BEGIN ... END pohranjenog zadatka,
 - ne počinju sa znakom @,
 - vidljive su isključivo unutar bloka pohranjenog zadatka,
 - deklaracija se mora nalaziti uvijek na vrhu procedure, poslije ključne riječi BEGIN i obavezno prije početka nekog drugog koda.

Osnovna sintaksa je:

```
SET @var_name = expr [, @var_name = expr] ...
```

Osim korisnički definiranih varijabli u sklopu poslužitelja baze podataka postoje i **sistemske varijable** u kojima su pohranjeni podatci primjerice o sustavu, o trenutnim postavkama i atributima sustava ili sesije, zapisanim u meta bazi naziva *information_schema*.

Vrijednosti sistemskih globalnih ili sesijskih varijabli moguće je dohvatiti naredbom SHOW. Osnovna sintaksa je prikazana u nastavku, pri čemu je moguće dohvatiti vrijednost određene varijable na razini sustava ili na razini sesije:

```
SHOW [GLOBAL | SESSION] VARIABLES  
[LIKE 'pattern' | WHERE expr]
```

Rezultat izvođenja naredbe:

```
SHOW VARIABLES;
```

Variable name	Value
auto_increment_increment	1
auto_increment_offset	1
autocommit	ON
automatic_sp_privileges	ON
avoid_temporal_upgrade	OFF
back_log	80
basedir	C:\Program Files\MySQL\MySQL Server 5.7\
big_tables	OFF
bind_address	*
binlog_cache_size	32768
binlog_checksum	CRC32
binlog_direct_non_transactional_updates	OFF
binlog_error_action	ABORT_SERVER
binlog_format	ROW
binlog_group_commit_sync_delay	0
binlog_group_commit_sync_no_delay_count	0
binlog_gtid_simple_recovery	ON
binlog_max_flush_queue_time	0

...

22.2. DODJELJIVANJE VRIJEDNOSTI VARIJABLAMA

Varijabli je moguće dodijeliti vrijednost korištenjem ključne riječi **SET**:

- direktnim dodjeljivanjem vrijednosti određenog tipa,
- dodjeljivanjem vrijednosti nekoliko varijabli,
- kao rezultat matematičke operacije,
- pohrana rezultata ugrađene funkcije i slično.

Primjeri za navedeno su u nastavku:

```
SET var1 = 1234;
SET @var2 = 5678;
SET @var3 = @var1 + @var2;
SET @var4 = 4, @var5 = 5;
SET var5 = "vrijednost_varijable";
SET @var6 = (SELECT AVG(ocjena) FROM ocjene);
SET @var7 =
    (SELECT MAX(id) FROM kolegiji);
SET var8 =
    (SELECT MAX(id) FROM kolegiji);
```

U varijablu je moguće pohraniti (jednu) vrijednost izvođenja funkcije koristeći ključnu riječ **INTO**:

```
SELECT
    MAX(id) INTO najveći1
```

```
        FROM kolegiji;  
SELECT  
        MAX(id) INTO @najveci2  
        FROM kolegiji;
```

23. Pohranjene rutine

23.1. KORACI IZRADE RUTINE

Izrada svake procedure ili funkcije sastojati će se od nekoliko ključnih dijelova koji će biti objašnjeni u ovom poglavlju. U poglavljima o procedurama detaljnije se navode specifičnosti i ulazi se dublje u sintaksu.

1. Izradi proceduru ili funkciju (ključna riječ **CREATE PROCEDURE** odnosno **CREATE FUNCTION**).
2. Definiraj ime procedure odnosno funkcije.
3. Definiraj parametre.
4. Definiraj tijelo procedure ili funkcije (ako se procedura sastoji od više naredbi, potrebno ih je definirati unutar **BEGIN ... END** bloka).

Dodatno se uvodi nulti i peti korak ako se tijelo procedure sastoji od više naredbi što obično jest slučaj. U tim koracima definira se promjena graničnika kako bi se rutina smatrala jedinstvenom cjelinom.

Pohranjenom zadatku može se dodijeliti ime od najviše 64 znaka, pri čemu se ime ne smije sastojati samo od brojki. Ime može sadržavati i posebne znakove, ali u tom slučaju moraju se označiti s unazadnim navodnikom (`). Preporuča se izbjegavati nazivanje pohranjenih podataka imenima baze, tablice ili atributa.

23.2. ODABIR GRANIČNIKA

Proceduru ili funkciju kao vrstu pohranjenog zadatka, te također i okidač, želimo izvesti na način da se izvedu sve naredbe od **CREATE** do **END**. U pretpostavljenom načinu rada DBMS prekida izvođenje upita kada dođe do kraja trenutnog upita. Kraj upita označava **graničnik** (engl. **DELIMITER**) čija je pretpostavljena vrijednost „;“ (točka zarez). Graničnik je znak ili niz znakova koji govori MySQL-u da se na tom mjestu nalazi kraj SQL naredbe.

Da bi se izbjeglo zaustavljanje izvođenja pohranjenog zadatka prilikom nailaska na prvi „;“, potrebno je privremeno izmijeniti vrijednost graničnika na neku drugu vrijednost – u ovom primjeru mijenja se na „//“.

Graničnik može poprimiti bilo koju vrijednost,

- //
- #
- nekolmeGranicnika

ali se ne preporuča korištenje oznaka koje bi se mogle pronaći u tijelu procedure:

- *
- (,)
- =, <, >

Po završetku procedure graničnik je potrebno vratiti na njegovu inicijalnu vrijednost naredbom „DELIMITER ;“.

Prilikom rada s MySQL bazom podataka te alatima kao što su SQLyog ili MySQL Workbench, valja voditi računa da je korak promjene vrijednosti graničnika sintaksno vrlo osjetljiv. Tako će promjena vrijednosti graničnika „DELIMITER //“ rezultirati pogreškom ako se prije ključne riječi DELIMITER nalazi bilo što u tom retku. Preporuka je dakle da se ključna riječ DELIMITER nalazi na početku retka bez vodećih razmaka niti tabova.

Također kod definiranja vrijednosti graničnika, pogrešno je izostaviti razmak nakon ključne riječi a prije nove vrijednosti jer se ovo ne smatra navođenjem vrijednosti graničnika već njegovom definicijom. Potrebno je dakle u trenutku kada se graničnik postavlja primjerice na vrijednost „//“ koristiti sintaksu „DELIMITER //“, a nikako „DELIMITER//“.

24. Pohranjene procedure

24.1. IZRADA PROCEDURE

Za pohranjene procedure karakteristično je da obavljaju niz SQL naredbi te da mogu vratiti skup rezultata. Rezultat ne mogu vratiti direktno (kao što to može funkcija) već rezultat vraćaju preko varijabli ili koristeći SELECT naredbu. Procedura može primiti varijable koje mogu biti modificirane unutar tijela procedure.

Procedura se definira prema sljedećim koracima:

1. Izradi proceduru (ključna riječ **CREATE PROCEDURE**).
2. Definiraj ime procedure.
3. Definiraj ulazne i izlazne parametre.
4. Definiraj tijelo procedure (ako se procedura sastoji od više naredbi, potrebno ih je definirati unutar **BEGIN ... END** bloka).

Osnovna sintaksa:

```
CREATE
    [DEFINER = { korisnik | CURRENT_USER }]
    PROCEDURE ime_procedure ([parameteri[,...]])
    [karakteristike ...]
    tijelo_procedure

parameteri:
    [ IN | OUT | INOUT ] ime_parametra tip

tip:
    bilo koji MySQL tip podatka

karakteristike:
    COMMENT 'string'
    | LANGUAGE SQL
    | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
    | SQL SECURITY { DEFINER | INVOKER }
```

Naredbom CREATE je procedura kreirana i pohranjena u sklopu baze. Posebno je potrebno pozvati proceduru i predati joj odgovarajuće parametre. Navođenje karakteristika procedure ne utječe direktno na njezino izvođenje već se odnosi na definiranje dodatnih opisnih atributa same procedure u obliku uputa onome koji će ju koristiti.

24.2. POZIVANJE PROCEDURE

Procedura se poziva ključnom riječi CALL nakon čega slijedi ime procedure te navođenje vrijednosti parametara koje se proceduri predaje i varijabli u koje se rezultat pohranjuje.

Osnovna sintaksa:

```
CALL ime_procedure ([parametri[,...]])
```

24.3. IZMJENA PROCEDURE

MySQL omogućava naredbom ALTER PROCEDURE izmijeniti samo određene karakteristike procedure. Ako postoji potreba za izmjenom tijela procedure, s obzirom da DBMS ne dozvoljava više procedura istog imena, potrebno je proceduru obrisati (DROP) te je nakon toga ponovno izraditi sa izmijenjenim tijelom (CREATE).

```
ALTER PROCEDURE ime_procedure [karakteristike ...]
```

karakteristike:

```
    COMMENT 'string'  
| LANGUAGE SQL  
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }  
| SQL SECURITY { DEFINER | INVOKER }
```

Definiranje karakteristika procedure je opcionalno i služi isključivo kao komentar (MySQL ne provjerava istinitost navedenih karakteristika). Moguće je slijedeće:

- COMMENT – radi se o MySQL ekstenziji koja se koristi samo za opis – ova je informacija vidljiva prilikom poziva SHOW CREATE FUNCTION
- LANGUAGE – karakteristika koja govori u kojem je jeziku procedura pisana – server ignorira ovu karakteristiku i podržava samo SQL jezik
- CONTAINS SQL – procedura sadrži naredbe za čitanje ili pisanje u bazu (primjerice može sadržavati CURDATE() ugrađenu funkciju)
- NO SQL – procedura ne sadrži SQL upite
- READS SQL DATA – procedura sadrži isključivo naredbe čitanja podataka (SELECT)
- MODIFIES SQL DATA – procedura mijenja podatke (naredbama INSERT, UPDATE i DELETE)
- SQL SECURITY – može poprimiti vrijednost DEFINER ili INVOKER što daje kontekst sigurnosti – poziva li se procedura sa privilegijama (GRANT) korisnika koji ju je izradio (DEFINER) ili korisnika koji ju poziva (INVOKER) – korisnik koji ju poziva mora imati dodijeljenu EXECUTE privilegiju nad tom procedura

Klauzula DEFINER specificira koji MySQL korisnik se koristi kada se provjeravaju dozvole za pristup prilikom izvršavanja procedura koja ima postavljenu SECURITY DEFINER karakteristiku.

24.4. BRISANJE PROCEDURE

Osnovna sintaksa:

```
DROP PROCEDURE [IF EXISTS] ime_procedure;
```

Klauzula IF EXISTS sprečava pojavu pogreške prilikom izvođenja naredbe brisanja ako procedura s danim imenom ne postoji. U tom se slučaju neće dogoditi pogreška već će se samo ispisati upozorenje o nepostojanju procedure koja se pokušava obrisati. Navođenje ove klauzule nije obavezno ali se preporuča radi ugodnijeg rada prilikom izrade procedure kao i ostalih pohranjenih zadataka.

24.5. PRIMJER

Napisati proceduru koja ispisuje sve podatke o svim studentima:

```
DELIMITER //                                → Privremena promjena graničnika
CREATE PROCEDURE ispisiStudente ()          → Deklaracija procedure
BEGIN
    SELECT * FROM studenti;                 → Tijelo procedure
END; //
DELIMITER ;                                → Vraćanje vrijednosti graničnika na ;
```

Gornjim je kodom procedura kreirana i pohranjena, ali ne i pozvana. Nakon deklaracije procedure slijedi tijelo procedure. Ako se tijelo sastoji od više naredbi, obavezno je definirati ih unutar BEGIN ... END bloka.

Primjer poziva gornje procedure:

```
CALL ispisiStudente ();
```

Pozivanje gornje procedure rezultirat će sljedećim skupom podataka (162 n-torke):

☐	jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer
☐	0010081356	Tanja	Krog	2010-09-28	10000	10290	3
☐	0013020125	Matea	Omanović	2012-09-25	10410	44000	9
☐	0013022546	Antun	Mandić	2010-09-30	10410	31000	10
☐	0013025926	Antonio	Pavić	2012-09-25	32000	44000	5
☐	0013026584	Borna	Knežević	2010-09-29	10410	44000	9
☐	0016096972	Dora	Vidaković	2011-09-26	42000	44000	16
☐	0016097990	Vesna	Đukić	2011-09-29	42000	43000	17
☐	0016098017	Marina	Zadro	2011-09-28	42000	47000	17
☐	0016099969	Antonia	Čoić	2011-09-28	42000	35000	17
☐	0016100313	Biljana	Vidović	2011-09-27	42000	35000	16
☐	0016101784	David	Vučetić	2011-09-27	42000	42000	16
☐	0016199452	Bernarda	Tomić	2011-09-30	42000	42000	17
☐	0016200033	Zvonimir	Žarković	2010-09-29	21000	20000	19
☐	0016201125	Denis	Šapina	2011-09-27	42000	42000	17
☐	0016201458	Ana	Radak	2011-09-27	42000	22000	16
☐	0016203355	Dino	Bošnjak	2011-09-30	42000	22300	16
☐	0016209876	Kristina	Pešerović	2011-09-29	42000	22300	17
☐	0016216325	Toni	Tošeski	2011-09-28	42000	10000	17
☐	0016216326	Ivan	Horvat	2011-09-29	42000	10000	17
☐	0016220011	Danijel	Zečević	2010-09-30	21000	23000	19

Primjer brisanja procedure ispisiStudente:

```
DROP PROCEDURE IF EXISTS ispisiStudente;
```

Primjer brisanja nepostojeće procedure:

```
DROP PROCEDURE IF EXISTS proceduraKojaNePostoji;
```

Ako *proceduraKojaNePostoji* ne postoji u bazi podataka, MySQL će vratiti samo upozorenje kao u nastavku.

```
0 row(s) affected, 1 warning(s)
```

```
Execution Time : 0.068 sec
```

```
Transfer Time : 0 sec
```

```
Total Time : 0.069 sec
```

```
Note Code : 1305
```

```
PROCEDURE mydb.proceduraKojaNePostoji does not exist
```

24.6. DEFINIRANJE PARAMETARA

Procedura ako ne prima ulazne parametre, svojim pozivanjem će uvijek vraćati isti rezultat. Iznimka je ako ovisi o nekim globalno definiranim varijablama na koje se poziva unutar tijela procedure. Procedura *ispisiStudente* tip je pohranjenog zadatka koji nema ulazne parametre te će prilikom poziva procedure uvijek vratiti isti skup rezultata (svi podaci o svim studentima). Potpuniji smisao procedura dobiva ako joj se definira ulazni parametar u odnosu na kojeg je u mogućnosti vratiti skup rezultata koji će ovisiti o tom parametru. Parametar nije ništa drugo već varijabla pomoću koje procedura komunicira

Osnovna sintaksa:

```
CREATE PROCEDURE imeProcedure ([IN] x tipPodatka, INOUT y  
tipPodatka, OUT z tipPodatka)  
...tijelo procedure...
```

Postoje sljedeće vrste parametara:

- Ulazni parametri – IN
 - Koriste se kao ulaz u proceduru
 - Ako procedura unutar koda procedure mijenja vrijednost ove varijable, to neće utjecati na njen poziv

- Ako se ne navede tip ulaza, podrazumijeva se IN
- Izlazni parametri – OUT
 - Koristi se samo kao izlaz iz procedure
 - Procedura može mijenjati vrijednost ove varijable
- Ulazno-izlazni parametri – INOUT
 - Koristi se kao ulaz u i izlaz iz procedure
 - Preko istog parametra se proceduri daje podatak te procedura u njega zapisuje i vraća rezultat
 - Poželjno je koristiti ovakvu vrstu parametra kada se radi o istom tip ulaznog i izlaznog podatka, odnosno potrebno je biti oprezan što će se dogoditi prilikom pretvorbe ako tip ulaznog i izlaznog podatka nisu isti

24.7. PRIMJERI

Primjer (ulazni parametar):

Napisati proceduru koja ispisuje sve podatke klijentima sa zadanim poštanskim brojem prebivanja.

```
DELIMITER //
```

```
DROP PROCEDURE IF EXISTS ispisiStudenteIzMjesta //
```

```
CREATE PROCEDURE ispisiStudenteIzMjesta(IN zadanoMjesto  
VARCHAR(50))  
BEGIN  
    SELECT *  
        FROM studenti JOIN mjesta ON postBrPrebivanje=postbr  
        WHERE nazivMjesto=zadanoMjesto;  
END; //
```

```
DELIMITER ;
```

Već u drugom redu prethodnog isječka koda, a prije CREATE, navodi se DROP PROCEDURE IF EXISTS što je dobra praksa prilikom razvijanja procedure. Često će biti potrebno promijeniti funkcionalnost ili samo ispraviti tipfeler ranije izrađene procedure odnosno ponovno izvršiti CREATE. Obzirom da u MySQL bazi nije moguće samo ponoviti CREATE naredbu (vratiti će se pogreška da procedura s tim imenom već postoji), dobra praksa je prvo odraditi DROP pa nakon toga CREATE.

Ako se prilikom deklaracije funkcije navodi da ona prima ulazne parametre koji se označavaju ključnom riječi IN, odnosno ako se ne navede oznaka parametra pretpostavlja se da se radi od IN parametru, primijetiti da je linija koda:

```
CREATE PROCEDURE ispisiStudenteIzMjesta(IN zadanoMjesto  
VARCHAR(50))
```

identična idućoj liniji:

```
CREATE PROCEDURE ispisiStudenteIzMjesta (zadanoMjesto VARCHAR(50))
```

Potrebno je voditi računa o navođenju parametara prilikom poziva procedure. Proceduri treba predati točno onoliko parametara koliko je navedeno prilikom deklaracije procedure

Primjer poziva procedure:

```
CALL ispisiStudenteIzMjesta ('Zagreb');
```

Rezultat pozivanja procedure sa vrijednošću ulaznog parametra 'Zagreb' (64 n-torke):

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	postbr	nazivMjesto	idZupanija
0010081356	Tanja	Krog	2010-09-28	10000	10290	3	10000	Zagreb	1
0035177990	Irena	Dabić	2012-09-26	10000	10000	1	10000	Zagreb	1
0035179953	Tanja	Sunek	2012-09-26	10000	10000	2	10000	Zagreb	1
0035180940	Sonja	Šimpraga	2011-09-26	10000	21000	4	10000	Zagreb	1
0035182233	Ivan	Turk	2008-09-26	10000	43000	3	10000	Zagreb	1
0035192211	Karla	Brandis	2010-09-28	10000	40000	4	10000	Zagreb	1
0035192565	Kristina	Matleković	2010-09-27	10000	43000	4	10000	Zagreb	1
0036430456	Dragutina	Topić	2011-09-26	10000	43000	15	10000	Zagreb	1
0036430552	Armin	Čehajić	2011-09-30	10000	44000	15	10000	Zagreb	1
0036435111	Maja	Vuković	2011-09-27	10000	21300	14	10000	Zagreb	1
0036440706	Vicka	Zečević	2011-09-26	10000	31000	14	10000	Zagreb	1
0036450023	Darko	Zidanić	2009-09-23	10000	10000	15	10000	Zagreb	1
0036450215	Josip	Pavković	2009-09-24	10000	20000	14	10000	Zagreb	1
0036452014	Alen	Martić	2009-09-22	10000	10000	14	10000	Zagreb	1
0036459870	Klaudija	Buljević	2009-09-25	10000	20000	15	10000	Zagreb	1
0036460022	Ivan	Matić	2008-09-24	10000	10000	15	10000	Zagreb	1
0036460031	Dajana	Jerec	2010-09-28	10000	10430	14	10000	Zagreb	1
0036460111	Ozren	Savić	2011-09-26	10000	10430	14	10000	Zagreb	1
0036463254	Ivan	Zović	2010-09-27	10000	10000	14	10000	Zagreb	1

Primjer (izlazni parametar):

Potrebno je napisati proceduru koja će prebrojati studente iz zadanog mjesta te preko parametra vratiti broj studenata.

```
DELIMITER //
DROP PROCEDURE IF EXISTS prebrojiStudenteIzMjesta //
CREATE PROCEDURE prebrojiStudenteIzMjesta
    (IN zadanoMjesto VARCHAR(50), OUT brojStudenata INT)
BEGIN
    SELECT
        COUNT(*) INTO brojStudenata
    FROM studenti JOIN mjesta ON postBrPrebivanje=postbr
    WHERE nazivMjesto=zadanoMjesto;
END; //
DELIMITER ;
```

U WHERE klauzuli selektirani su samo oni studenti kojima je mjesto prebivanja definirano putem ulaznog parametra (*nazivMjesto=zadanoMjesto*). Rezultat ugrađene funkcije COUNT, pomoću ključne riječi INTO pohranjuje se u izlaznu varijablu *brojStudenata*, koju procedura automatski vraća kao izlaz iz procedure.

Primjer poziva procedure i ispisa rezultata:

```
CALL prebrojiStudenteIzMjesta ('Zagreb',@br);
SELECT @br AS broj_studenata;
```

Prilikom poziva procedure potrebno je osim definiranja vrijednosti za ulazni parametar (primjerice 'Zagreb'), definirati i ime (globalne) varijable u koju će se pohraniti vrijednost izlazne varijable odnosno rezultat izvođenja procedure. Varijabla može imati proizvoljan naziv.

Pozivom procedure neće se (u ovom slučaju) ispisati rezultat već će se on nalaziti u globalnoj varijabli *@br*. Sadržaj te varijable ispisuje se narednom SELECT. Rezultat pozivanja procedure s vrijednošću ulaznog parametra 'Zagreb' je sljedeći:

broj_studenata
64

Ovaj primjer moguće je riješiti i bez izlaznog parametra na način da procedura vrati rezultat kroz SELECT klauzulu definiranu unutar tijela procedure:

```
DELIMITER //
DROP PROCEDURE IF EXISTS prebrojiStudenteIzMjesta //
CREATE PROCEDURE prebrojiStudenteIzMjesta(IN zadanoMjesto
VARCHAR(50))
BEGIN
    SELECT
        COUNT(*)
        FROM studenti JOIN mjesta ON postBrPrebivanje=postbr
        WHERE nazivMjesto=zadanoMjesto;
END; //
DELIMITER ;
```

Primjer poziva procedure:

```
CALL prebrojiStudenteIzMjesta ('Zagreb');
```

Pozivanjem ove procedure direktno se ispisuje rezultat. Primjer poziva sa vrijednošću ulaznog parametra 'Zagreb' je sljedeći:

COUNT (*)
64

Treći je način vraćanje rezultata preko globalne varijable:

```
DELIMITER //
DROP PROCEDURE IF EXISTS prebrojiStudenteIzMjesta //
CREATE PROCEDURE prebrojiStudenteIzMjesta(IN zadanoMjesto
VARCHAR(50))
BEGIN
    SELECT
        COUNT(*) INTO @n
        FROM studenti JOIN mjesta ON postBrPrebivanje=postbr
        WHERE nazivMjesto=zadanoMjesto;
```

```
END; //
```

```
DELIMITER ;
```

Primjer poziva procedure i ispisa rezultata iz globalne varijable @n:

```
CALL prebrojiStudenteIzMjesta ('Zagreb');
```

```
SELECT @n;
```

@n
64

Primjer (ulazno-izlazni parametar):

Potrebno je napisati proceduru koja će za zadanog studenta vratiti poštanski broj mjesta prebivanja.

```
DELIMITER //
```

```
DROP PROCEDURE IF EXISTS nadiPbrStudenta //
```

```
CREATE PROCEDURE nadiPbrStudenta (INOUT studentMjesto INT)
```

```
BEGIN
```

```
    SELECT postBrPrebivanje INTO studentMjesto
```

```
        FROM studenti WHERE jmbag=studentMjesto;
```

```
END; //
```

```
DELIMITER ;
```

Varijabla *studentMjesto* ima ulogu prenošenja ulaznog podatka proceduri (WHERE *jmbag=studentMjesto*) te primanja rezultata procedure (*SELECT postBrPrebivanje INTO studentMjesto*).

Primjer poziva procedure i ispisa rezultata:

```
SET @var = 0010081356;
```

```
CALL nadiPbrStudenta (@var);
```

```
SELECT @var AS pbr_prebivanja;
```

Prilikom poziva procedure, ulazno-izlazna varijabla komunicirat će sa sesijom preko globalne varijable *@var*. Varijabli *@var* ključnom riječju **SET** pridodana je vrijednost 0010081356 prije poziva procedure. Procedura traži poštanski broj prebivanja studenta čiji je JMBAG jednak 0010081356, te ga pohranjuje u *@var*. Naredbom CALL rezultat neće biti ispisan, već će biti upisan u *@var*. Sadržaj globalne varijable *@var* ispisuje se naredbom SELECT. Rezultat je u nastavku.

pbr_prebivanja
10000

25. Pohranjene funkcije

25.1. IZRADA FUNKCIJE

Pohranjena funkcija vrsta je pohranjene rutine koja direktno vraća rezultat. Ne može vratiti skup rezultata niti skup podataka, već vraća skalarnu vrijednost. Za nju je karakteristično da s bazom podataka komunicira isključivo putem ulaznih parametara, dok izlazne parametre ne podržava. Podatke prima preko ulaznih varijabli koje može modificirati. S obzirom da rezultat ne može biti vraćen preko parametra, prilikom deklaracije funkcije nužno je navesti tip podatka kojeg će funkcija (obavezno) vratiti.

Za razliku od procedura, funkcija mora vratiti rezultat i ne može ispisivati podatke unutar BEGIN ... END bloka.

Funkcija se definira prema sljedećim koracima:

1. Izradi funkciju (ključna riječ **CREATE FUNCTION**)
2. Definiraj ime funkcije
3. Definiraj ulazne i izlazne parametre -> svi su parametri ulaznog tipa (drugi tipovi nisu dozvoljeni) te se iz tog razloga **ne navodi** ključna riječ IN prije imena parametra
4. Ključna riječ RETURNS i tip parametra koji funkcija vraća
5. Definirati je li funkcija **DETERMINISTIC** ili **NOT DETERMINISTIC**
6. Definirati tijelo procedure (ako se procedura sastoji od više naredbi, potrebno ih je definirati unutar **BEGIN ... END** bloka)
7. Unutar tijela funkcije naredbom **RETURN** definirati izlaz iz funkcije

Pravila koja su vrijedila za procedure, a tiču se definiranja graničnika, izmjene i brisanja procedura vrijede i za pohranjene funkcije.

Osnovna sintaksa:

```
CREATE
    [DEFINER = { user | CURRENT_USER }]
    FUNCTION ime_funkcije [parametri[,...]]
    RETURNS tip
    [karakteristike...]
    tijelo_funkcije

parametri:
    ime_parametra tip

tip:
    bilo koji validan MySQL tip podatka
```

karakteristike:

```
COMMENT 'string'
| LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY { DEFINER | INVOKER }
```

DETERMINISTIC i NOT DETERMINISTIC

Prilikom deklaracije funkcije potrebno je definirati je li funkcija deterministička ili je nedeterministička. Ako funkcija ima determinističko obilježje, tada se kaže da je strogo određena. Za iste će ulazne parametre uvijek vratiti isti rezultat. Navedeno se podrazumijeva u slučaju replikacije i u situacijama kada se unutar funkcije pozivaju ugrađene funkcije tipa RAND() i NOW(). Pretpostavljena vrijednost je NOT DETERMINISTIC.

Gornjim kodom funkcija je kreirana i pohranjena, ali ne i pozvana. Obavezno je nakon navođenja imena i parametara definirati tip podatka kojeg funkcija vraća.

25.2. POZIVANJE FUNKCIJE

Funkcija se poziva ključnom riječi SELECT nakon čega slijedi ime funkcije te navođenje vrijednosti parametara koji se funkciji predaju. Obzirom da funkcija vraća rezultat direktno, nije potrebno navoditi varijable niti parametre u koje će ona pohraniti rezultat. Prilikom poziva funkcije važno je predati joj broj parametara koji očekuje u skladu s deklaracijom same funkcije.

Osnovna sintaksa:

```
SELECT ime_funkcije ([parametri[,...]])
```

25.3. IZMJENA FUNKCIJE

MySQL omogućava naredbom ALTER FUNCTION izmijeniti samo određene karakteristike funkcije. Ako postoji potreba za izmjenom tijela funkcije, s obzirom da DBMS ne dozvoljava više funkcija istog imena, potrebno je funkciju obrisati te je nakon toga ponovno izraditi s izmijenjenim tijelom.

```
ALTER FUNCTION ime_funkcije [karakteristike ...]
```

karakteristike:

```
COMMENT 'string'
| LANGUAGE SQL
```

```
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY { DEFINER | INVOKER }
```

25.4. BRISANJE FUNKCIJE

Osnovna sintaksa:

```
DROP FUNCTION [IF EXISTS] ime_funkcije;
```

Navođenje klauzule IF EXISTS je proizvoljno te funkcionira na isti način kako je opisano u poglavlju o brisanju procedura.

25.5. DEFINIRANJE PARAMETARA

Funkcija podržava isključivo ulazne parametre. Pri tome se ne smije navoditi ključna riječ IN kao oznaka tipa parametra jer druga opcija ne postoji. Navođenje ključne riječi IN rezultiralo bi sintaksnom pogreškom.

Osnovna sintaksa:

```
CREATE FUNCTION imeFunkcije (x tipPodatka)
...tijelo funkcije...
```

25.6. PRIMJERI

Napisati funkciju koja ispisuje prosječnu ocjenu svih studenata:

```
DELIMITER $$
DROP FUNCTION IF EXISTS vratiProsjecnuOcjenu $$
CREATE FUNCTION vratiProsjecnuOcjenu() RETURNS DECIMAL(4,3)
DETERMINISTIC
BEGIN
    DECLARE vrati DECIMAL(4,3) DEFAULT NULL;
    SELECT AVG(ocjena) INTO vrati FROM ocjene;
    RETURN vrati;
END; $$
DELIMITER ;
```

Ako je unutar tijela funkcije potrebno odraditi složeniji rad s podacima, praktično je koristiti lokalne varijable unutar tijela funkcije kako je vidljivo na gornjem primjeru. Ako se u RETURN klauzuli vraća vrijednost određene lokalne varijable, tu je varijablu potrebno deklarirati na početku tijela funkcije, upravo kao i sve ostale lokalne varijable.

Obzirom da je ovo primjer u kojem se ne odrađuje složeniji rad s podacima niti logika izračuna unutar samog tijela funkcije, moguće je izbjeći uporabu lokalne varijable te rezultat SELECT-a vratiti direktno u RETURN klauzuli.

```
DELIMITER $$
DROP FUNCTION IF EXISTS vratiProsjecnuOcjenu $$
CREATE FUNCTION vratiProsjecnuOcjenu() RETURNS DECIMAL(4,3)
DETERMINISTIC
BEGIN
    RETURN (SELECT AVG(ocjena) FROM ocjene);
END; $$
DELIMITER ;
```

Primjer poziva gornje funkcije:

```
SELECT vratiProsjecnuOcjenu ();
```

Pozivanje gornje funkcije dati će sljedeći rezultat:

☐ vratiProsjecnuOcjenu()
☐ 3.250

Napisati funkciju koja za zadanog studenta vraća njegovu prosječnu ocjenu.

```
DELIMITER $$
DROP FUNCTION IF EXISTS vratiProsjecnuOcjenuStudenta $$
CREATE FUNCTION vratiProsjecnuOcjenuStudenta(zadaniStudent INT)
RETURNS DECIMAL(4,3)
DETERMINISTIC
BEGIN
    DECLARE prosjekStudenta DECIMAL(4,3) DEFAULT NULL;
    SELECT AVG(ocjena) INTO prosjekStudenta
        FROM ocjene
        JOIN studenti ON ocjene.jmbagStudent = studenti.jmbag
        WHERE studenti.jmbag = zadaniStudent;
    RETURN prosjekStudenta;
END; $$
DELIMITER ;
```

Primjer poziva funkcije:

```
SELECT vratiProsjecnuOcjenuStudenta(0016201458);
```

☐ vratiProsjecnuOcjenuStudenta(0016201458)
☐ 4.667

Izrađenu je funkciju moguće iskoristiti za ispis prosječne ocjene svakog studenta, je ju iskoristiti unutar SELECT-a koji za rezultat vraća skup n-torki. Potrebno je voditi računa o tome da će se funkcija pozvati za svakog studenta odnosno 162 puta.

```
SELECT
    studenti.jmbag AS 'JMBAG',
    studenti.ime AS 'IME',
    studenti.prezime AS 'PREZIME',
    vratiProsjecnuOcjenuStudenta(jmbag) AS 'PROSJEK'
FROM studenti;
```

Poziv gornjeg upita dati će rezultat kao u nastavku (162 n-torke):

☐ JMBAG	IME	PREZIME	PROSJEK
☐ 0010081356	Tanja	Krog	2.000
☐ 0013020125	Matea	Omanović	2.000
☐ 0013022546	Antun	Mandić	4.000
☐ 0013025926	Antonio	Pavić	5.000
☐ 0013026584	Borna	Knežević	3.000
☐ 0016096972	Dora	Vidaković	3.000
☐ 0016097990	Vesna	Đukić	4.500
☐ 0016098017	Marina	Zadro	5.000
☐ 0016099969	Antonia	Čoić	4.500
☐ 0016100313	Biljana	Vidović	3.000
☐ 0016101784	David	Vučetić	2.000
☐ 0016199452	Bernarda	Tomić	4.500
☐ 0016200033	Zvonimir	Žarković	3.000
☐ 0016201125	Denis	Šapina	5.000
☐ 0016201458	Ana	Radak	4.667
☐ 0016203355	Dino	Bošnjak	4.500
☐ 0016209876	Kristina	Pešerović	2.000
☐ 0016216325	Toni	Tošeski	2.000
☐ 0016216326	Ivan	Horvat	3.000

...

26. Okidači

Okidač (engl. *trigger*) je objekt baze podataka koji se asocira s tablicom i aktivira u trenutku određenog događaja nad tablicom. Čini ga niz definiranih SQL naredbi koje se automatski izvode prilikom unosa, izmjene ili brisanja podataka. Okidači implementiraju funkcionalnost baze podataka koja se mora održati kada dođe do promjene nad podacima u tablici. Obzirom da se aktivira automatski, ako je izrađen i vezan za određenu tablicu, nije ga moguće zaobići putem aplikacije koja koristi dotičnu bazu podataka.

Okidači su korisni kod automatiziranja operacija nad bazom na način da se smanjuje teret zadataka koje bi aplikacija morala izvršiti, kao i kod očuvanja integriteta baze podataka. Neki od primjera primjene okidača su:

- Izvođenje automatskih izračuna
- Održavanje poslovne logike
- Izmjena podataka u jednoj tablici kao odgovor na promjene u drugoj
- Evidentiranje promjena u podacima (korisnički izrađeni logovi)
- Stvaranje snimke podataka prije promjene (za eventualno poništavanje funkcionalnosti)
- Očuvanje integriteta podataka kao nadogradnja na već ugrađene mehanizme DBMS-a

Okidač je vezan isključivo uz pravu tablicu, i ne može biti vezan uz privremenu tablicu niti uz pogled. Svaki okidač je objekt u jednoj bazi podataka i vrijedi samo za tu bazu. Prilikom brisanja baze podataka, bit će obrisani i svi njeni okidači.

26.1. VRSTE OKIDAČA

Okidač se može smatrati pohranjenom procedurom koja se ne poziva naredbom CALL već se automatski aktivira prilikom izvršavanja odrađenih akcija od strane korisnika. Omogućava manipuliranje podacima prije nego što stvarno budu uneseni, izmijenjeni ili obrisani, odnosno aktivira se prilikom naredbe INSERT, UPDATE ili DELETE. Događaj koji aktivira okidač zove se **aktivirajuća operacija** (engl. *triggering event*). Potrebno je eksplicitno specificirati listu naredbi koje se izvršavaju kad se dogodi aktivirajuća operacija. Skup tih naredbi naziva se **aktivirana operacija** (engl. *triggered action list*). Aktivirane operacije izvršavaju se kod eksplicitno specificiranih **uvjeta** (engl. *condition*).

Prema aktivirajućoj operaciji okidači se dijele na tri kategorije:

- INSERT
- UPDATE

- DELETE

Prema vremenu izvođenja dijele se na dvije podkategorije:

- BEFORE

- AFTER

Na jednoj tablici nije moguće izraditi dva okidača koji će biti isti i prema aktivirajućoj operaciji i prema vremenu izvođenja. Primjerice ne mogu postojati dva before insert okidača na istoj tablici, ali može jedan before insert i drugi koji je after insert. Ako već postoji before insert okidač a potrebno je dodati još jedan isto po aktivirajućoj operaciji i vremenu okidanja, potrebno je već postojeći okidač nadograditi s logikom novog. Time se proširenjem postojećeg okidača uspješno ugrađuje dodatna logika a izbjegava potreba za dva istovjetna okidača na jednoj tablici.

Okidač se pokreće prilikom svakog rada s tablicom nad kojom je definiran bez obzira zadovoljava li uvjete i hoće li vidljivo utjecati na podatke.

26.2. IZRADA OKIDAČA

Okidači kada se izrade, mogu se izvoziti s bazom podataka isto kao i ostali pohranjeni zadatci. Ime svakog okidača treba biti jedinstveno u sklopu baze. Da bi korisnik u bazi mogao izraditi okidač, potrebno je da su mu dodijeljene odgovarajuće privilegije (TRIGGER privilege u MySQL 5.1.6+ ili SUPER privilege u MySQL 5.0.x). U nastavku je osnovna sintaksa za izradu okidača.

```
CREATE

    [DEFINER = { user | CURRENT_USER }]
    TRIGGER ime_okidaca
    vrijeme događaj
    ON tablica FOR EACH ROW
    tijelo_okidaca

vrijeme: { BEFORE | AFTER }
događaj: { INSERT | UPDATE | DELETE }
```

Danom sintaksom izrađuje se okidač *ime_okidaca* koji je asociran s tablicom *tablica*, u skladu s definiranim vremenom i okidajućoj operaciji. Klauzula FOR EACH ROW je obavezna po MySQL sintaksi. Označava da će okidač biti izvršen za svaki redak na kojeg se odnosi DML naredba koja je aktivirala okidač. Prema ANSI standardu moguća je FOR EACH STATEMENT klauzula koje će biti podržana u jednoj od budućih MySQL verzija. Obzirom da okidač u većini slučajeva izvršava više SQL naredbi, one njih je potrebno nakon FOR EACH ROW izjave definirati unutar BEGIN ... END bloka.

Na ovaj način je okidač izrađen i pohranjen u sklopu baze. Obzirom na prirodu okidača, nije ga potrebno zasebno pozivati kao što je to slučaj s funkcijom ili procedurom, već će se okidač automatski pokretati prilikom rada s tablicom sa kojom je asociran.

26.3. BRISANJE OKIDAČA

Osnovna sintaksa:

```
DROP TRIGGER [IF EXISTS] [ime_baze.]ime_okidaca
```

26.4. IZMJENA OKIDAČA

Okidač nakon što je izrađen, nije moguće modificirati sa DDL naredbom ALTER, već ga je potrebno prvo obrisati (DROP TRIGGER), te nakon toga ponovno izraditi s izmijenjenom logikom.

26.5. REFERENCIRANJE NA STARE / NOVE VRIJEDNOSTI

U tijelu okidača moguće je referencirati se na stare vrijednosti atributa u tablici koje će biti promijenjene ili obrisane odnosno na nove koje će biti unesene u tablicu. Navedeno je moguće aliasima NEW odnosno OLD nakon kojih se navodi naziv stupca na koji se podatak odnosi. *NEW.imeStupca* odnosi se na podatak koji će tek biti unesen u bazu, odnosno *OLD.imeStupca* odnosi se na postojeći atribut u tablici koji će biti izbrisan ili promijenjen. Navedene oznake postoje i kod BEFORE i kod AFTER okidača. Alias OLD i NEW nisu osjetljive na mala i velika slova, odnosno sintaksno je identično navesti *new.imeStupca* i *NEW.imeStupca*.

Primjerice, ako na određenoj tablici postoji BEFORE UPDATE okidač, i ta tablica sadrži atribut *stupac1*, vrijednost u stupcu koja će tek biti izmijenjena je *OLD.stupac1*, a vrijednost nakon izmjene je *NEW.stupac1*.

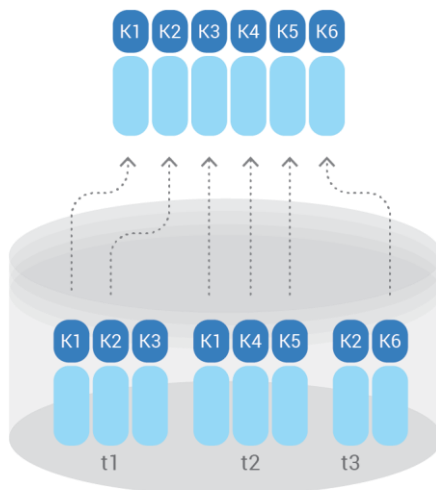
Popis tipova okidača i postojećih pripadajućih aliasa:

- ✓ INSERT -> NEW
- ✓ UPDATE -> NEW, OLD
- ✓ DELETE -> OLD

NEW.imeStupca

- Sadrži podatke koji će biti uneseni u tablicu prilikom izvođenja naredbe INSERT ili UPDATE
- Moguće je korisnički manipulirati s ovim podacima prije nego se unesu u originalnu tablicu uz koju je okidač vezan (primjerice SET NEW.imeStupca=novaVrijednost)

- Navedena mogućnost promjene vrijednosti u retku NEW moguća je isključivo kod BEFORE okidača
- Po završetku rada okidača, podaci koji imaju alias NEW, automatski će biti uneseni u originalnu tablicu



Slika 57. Princip rada okidača

Primjer

Potrebno je napraviti okidač koji će osigurati da se prilikom unosa ocjene, datum polaganja uvijek postavlja na današnji datum.

```
DELIMITER $$
DROP TRIGGER IF EXISTS provjeriDatumOcjene $$
CREATE TRIGGER provjeriDatumOcjene
  BEFORE INSERT ON ocjene
  FOR EACH ROW
  BEGIN
    SET NEW.datumPolaganja = CURDATE();
  END
$$
DELIMITER ;
```

Okidajuća operacija okidača je INSERT jer se provjera treba odraditi prilikom unosa nove ocjene, te se radi o BEFORE tipu okidača jer se vrijednost datuma provjerava prije nego je ocjena unesena u tablicu. Datum polaganja koji se unosi, postavlja se pomoću aliasa NEW.datumPolaganja.

Primjer naredbe koja će aktivirati okidač:

```
INSERT INTO ocjene
VALUES (1, '0016099969', '2017-03-10', '12:00:00', 3);
```

Izvođenje gornje INSERT naredbe rezultirati će drugačijom vrijednosti za datum polaganja ispita. U nastavku je ispis ocjena iz tablice *ocjena* za kolegij s ID-om 1.

```
SELECT *
FROM ocjene
WHERE idKolegij = 1 AND jmbagStudent = '0016099969';
```

☐	idKolegij	jmbagStudent	datumPolaganja	vrijemePolaganja	ocjena
☐	1	0016099969	2017-07-04	12:00:00	3

OLD.imeStupca

- Sadrži podatke koji će biti izbrisani u tablici izvođenjem naredbe DELETE ili promijenjeni izvođenjem naredbe UPDATE
- Ove je podatke moguće isključivo čitati, ali ne i mijenjati

Primjer:

Potrebno je napisati okidač koji će onemogućiti izmjenu ocjene koja je već unesena. Ako se pokušava izmijeniti dotična n-torka, potrebno je umjesto nove ocjene zadržati staru.

```
DELIMITER $$
DROP TRIGGER IF EXISTS neMijenjaJOcjenu $$
CREATE TRIGGER neMijenjaJOcjenu
BEFORE UPDATE ON ocjene
FOR EACH ROW
BEGIN
    SET NEW.ocjena = OLD.ocjena;
END
$$
DELIMITER ;
```

Okidajuća operacija ovog okidača je UPDATE jer se radi o provjeri prilikom izmjene ocjene, vrijeme okidanja je BEFORE jer je potrebno provjeriti obaviti provjeru prije nego se podatak unese u dotičnu tablicu.

Primjer naredbe koja će aktivirati okidač:

```
UPDATE ocjene
SET ocjena = 5
WHERE jmbagStudent = '0016099969' AND idKolegij = 1;
```

Dohvat podataka radi provjere rezultata za ocjenu koja je u prethodnom primejru unesena u tablicu *ocjene*. U vraćenom skupu podataka vidljivo je da ocjena nije postavljena na vrijednost 5 već da je ostala 3.

```
SELECT *
FROM ocjene
WHERE idKolegij = 1 AND jmbagStudent = '0016099969';
```

☐	idKolegij	jmbagStudent	datumPolaganja	vrijemePolaganja	ocjena
☐	1	0016099969	2017-07-04	12:00:00	3

26.6. UPOTREBA BEFORE I AFTER OKIDAČA

Definiranjem vremena pokretanja okidača koje može biti BEFORE i AFTER, definirano je kada se naredbe navedene u tijelu okidača izvršavaju, da li prije ili poslije DML izjava (INSERT, UPDATE ili DELETE) koje su pokrenule okidač. Glavna razlika između ove dvije vrste okidača je ta da kod AFTER okidača nije moguće mijenjati vrijednosti koje će biti unesene ili ažurirane u tablici (NEW.imeStupca) na koju se okidač odnosi. U tom je slučaju kasno pokušavati mijenjati radnju aktivirajuće operacije. Podaci NEW.imeStupca koji su uneseni u bazu za ovaj okidač postaju nepromijenjeni, a pokušaj njihovog mijenjanja rezultirat će pogreškom. Primjer ovog slučaja dan je u nastavku.

Primjer:

Potrebno je napraviti okidač koji će osigurati da se u tablicu *mjesto* ne može unijeti novi zapis takav da mu vrijednost u stupcu nazivMjesto sadrži manje od 2 slova. Ako se unosi takvo mjesto, potrebno mu je u naziv dodati sufiks „_test“.

```
DELIMITER $$
CREATE TRIGGER duljinaNazivaKolegija
AFTER INSERT ON kolegiji FOR EACH ROW
BEGIN
    IF LENGTH(NEW.naziv)<2 THEN
        SET NEW.naziv=CONCAT(NEW.naziv, "_test");
    END IF;
END
$$
DELIMITER ;
```

Izvođenje gornjeg koda rezultirat će pogreškom:

Error Code: 1362

Updating of NEW row is not allowed in after trigger

Ispravno je:

```
DELIMITER $$
CREATE TRIGGER duljinaNazivaKolegija
```



```

BEFORE INSERT ON kolegiji FOR EACH ROW
BEGIN
    IF LENGTH(NEW.naziv)<2 THEN
        SET NEW.naziv=CONCAT(NEW.naziv, "_test");
    END IF;
END
$$
DELIMITER ;

```

Primjer INSERT naredbe koja će aktivirati okidač:

```

INSERT INTO kolegiji (naziv, opis, idSmjer)
VALUES ('N', 'Novi testni kolegij', 17);

```

U nastavku je ispis svih kolegija iz tablice kolegij, posljednja n-torka je unesena sa sufiskom "_test".

id	naziv	opis	idSmjer
31	Poslovno komuniciranje	Usvajanje općih znanja potrebnih za uspješniju interperso...	166B 16
32	Obrada teksta i slike	usvajanje znanja i razumijevanja: postupka kreiranja doku...	151B 16
33	Osnove poduzetništva	Pojmovno određenje poduzetništva. Povijesni i komparativn...	155B 17
34	Marketing	Marketing kao filozofija poslovanja; Priroda marketinškog...	146B 17
35	Mikroekonomija I	Razumijevanje principa ponude i potražnje te njihovo pove...	210B 18
36	Kvantitativne metode u managementu	Upoznavanje sa osnovnim kvantitativnim metodama koje se m...	136B 18
37	Matematika u ekonomiji	S ovim predmetom studenti bi trebali biti osposobljeni za...	163B 19
38	Marketing destinacije	Specifičnosti upravljanja marketingom destinacije;Turisti...	111B 19
39	Tehnička mehanika 2	Upoznati studente s teorijskim i iskustvenim spoznajama i...	216B 20
40	Sredstva pomorskog prometa	Cilj kolegija je upoznati studente sa stabilnošću broda u...	141B 20
41	Tereti u prometu	Osnovni cilj i zadatak kolegija je upoznavanja prirode ma...	216B 21
42	Planiranje prometne potražnje	Osposobiti studenta za primjenu metoda kojima se utvrđuje...	91B 21
43	Tjelesna i zdravstvena kultura 2	Odgojno-obrazovno područje podmirenja biopsihosocijalnih ...	227B 1
44	Tjelesna i zdravstvena kultura 2	Odgojno-obrazovno područje podmirenja biopsihosocijalnih ...	227B 10
45	Tjelesna i zdravstvena kultura 3	Odgojno-obrazovno područje podmirenja biopsihosocijalnih ...	227B 16
46	Napredne baze podataka	Student se treba upoznati s objektima i kontrolama za pri...	167B 1
47	N_test	Novi testni kolegij	19B 17
*	(Auto) (NULL)	(NULL)	OK (NULL)

Iako se u većini slučajeva potrebne funkcionalnosti mogu riješiti sa BEFORE okidačima, AFTER okidače će biti korisno upotrijebiti za radnje koje se logički trebaju izvršiti tek po uspješnom završetku aktivirajuće operacije. Primjerice, revizijske aktivnosti najbolje je izvršiti tek po uspješnom završetku DML naredbe (aktivirajuće operacije). Prilikom izrade sigurnosne kopije (engl. *backup*) također se najčešće koriste AFTER okidači. Često će se koristiti za obavljanje određenih operacija u drugim tablicama nakon uspješne DML operacije na vezanoj tablici.

Primjer:

Potrebno je izraditi okidač koji će prilikom unosa podataka u tablicu ustanove ujedno napuniti i tablicu ustanove_backup sa tim istim podacima.

```

DELIMITER $$
DROP TRIGGER IF EXISTS backupUstanove $$
CREATE TRIGGER backupUstanove
    AFTER INSERT ON ustanove FOR EACH ROW
    BEGIN
        INSERT INTO ustanove_backup
            VALUES (NEW.oib, NEW.naziv, NEW.zRacun,
NEW.adresa, NEW.datumOsnutka, NEW.postbr);
    END
$$
DELIMITER ;

```

Primjer naredbe koja će aktivirati okidač:

```

INSERT INTO ustanove
    VALUES ('08491264236', 'Novi fakultet', '1360000-
4561846257', 'Nova cesta 52', '2007-01-01', 51000);

```

Rezultat izvođenja gornje INSERT naredbe:

☐ oib	naziv	zRacun	adresa	datumOsnutka	postbr
☐ 08491264236	Novi fakultet	1360000-4561846257	Nova cesta 52	2007-01-01	51000
* (NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

26.7. OGRANIČENJA OKIDAČA

Okidač ne smije koristiti naredbe koje eksplicitno označavaju početak i kraj transakcije. Potrebno je izbjeći primjerice izazivanje neželjenog završetka transakcije u trenutku kada je transakcija izvršila određenu DML naredbu koja je aktivirala okidač.

Ako postoji pogreška u okidaču, aktivirajuća operacija neće se izvesti. U slučaju da na tablici postoje i BEFORE i AFTER okidač te ako dođe do pogreške prilikom izvođenja BEFORE okidača, tada se niti AFTER okidač neće izvesti.

MySQL do sada nije razvio mogućnost izazivanja pogreške (raise exception) kako bi obavijestio aplikaciju pod kojom radi da nešto nije u redu. Ako je potrebno direktno prekinuti izvođenje određene radnje, rješenje je prouzrokovati pogrešku što će rezultirati prekidom rada okidača te prekidom aktivirajuće operacije. To se može napraviti na nekoliko načina:

- Unosom podatka u nepostojeći stupac
- Unosom NULL vrijednosti u stupac za koji je definirano da ne smije poprimiti NULL vrijednost
- Pozivom nepostojeće pohranjene procedure

```

DELIMITER $$
CREATE TRIGGER odj BEFORE DELETE ON odjel
FOR EACH ROW

```

```

BEGIN
    CALL nepostojeca_procedura();
END;
$$
DELIMITER ;

```

Primjer naredbe koja će aktivirati okidač:

```
DELETE FROM smjerovi WHERE id = 20000;
```

Izvođenje gornje naredbe prouzrokovat će (namjerno izazvanu) pogrešku:

Error Code: 1305

```
PROCEDURE studenti.nepostojeca_procedura does not exist
```

26.8. OKIDAČI I EVIDENTIRANJA PROMJENA U BAZI PODATAKA

Kroz prethodna poglavlja već su dani primjeri primjene okidača. Jedna od primjena je snimanje informacija o izmjenama u bazi na korisnički definiran način.

Primjer:

Prilikom upisa ili izmjene ocjene iz nekog kolegija, potrebno je podatke o izmjeni pohraniti u tablicu ocjene_log. Potrebno je pohraniti i podatke o korisniku koji je unio ili izmijenio ocjenu te vremenski žig. Tablicu ocjene_log potrebno je kreirati u istoj bazi podataka.

U ovom primjeru potrebno je obavljati gotovo istu radnju za unos i izmjenu podataka, no obzirom da su aktivirajuće operacije različite, biti će potrebno izraditi dva okidača sa sličnim funkcionalnostima.

```

CREATE TABLE ocjene_log (
    idKolegij INT(11) NOT NULL,
    jmbagStudent CHAR(10) COLLATE cp1250_croatian_ci NOT NULL,
    datumPolaganja DATE NOT NULL,
    ocjenaStara TINYINT(4) DEFAULT NULL,
    ocjenaNova TINYINT(4) DEFAULT NULL,
    korisnik VARCHAR(255) DEFAULT NULL,
    vremenskiZig DATETIME DEFAULT NULL,
    PRIMARY KEY (idKolegij, jmbagStudent, datumPolaganja)
) ENGINE=INNODB DEFAULT CHARSET=cp1250 COLLATE=cp1250_croatian_ci

```

Okidač za evidentiranje unesenih ocjena:

```

DELIMITER %%
DROP TRIGGER IF EXISTS insertOcjeneLog %%
CREATE TRIGGER insertOcjeneLog

```

```
AFTER INSERT ON ocjene FOR EACH ROW
BEGIN
    INSERT INTO ocjene_log
        (idKolegij,jmbagStudent,datumPolaganja,ocjenaStara,ocjenaNova,korisnik,vremenskiZig)
        VALUES (NEW.idKolegij,
                NEW.jmbagStudent,
                NEW.datumPolaganja,
                NULL,
                NEW.ocjena,
                CURRENT_USER(),
                NOW());
END;
%%
DELIMITER ;
```

Primjer naredbe koja će aktivirati okidač:

```
INSERT INTO ocjene
VALUES (2, '0016097990', '2017-01-30', '15:30:00', 1);
```

Okidač za evidentiranje promijenjenih ocjena:

```
DELIMITER %%
DROP TRIGGER IF EXISTS updateOcjeneLog %%
CREATE TRIGGER updateOcjeneLog
AFTER UPDATE ON ocjene FOR EACH ROW
BEGIN
    INSERT INTO ocjene_log
        (idKolegij,jmbagStudent,datumPolaganja,ocjenaStara,ocjenaNova,korisnik,vremenskiZig)
        VALUES (NEW.idKolegij,
                NEW.jmbagStudent,
                NEW.datumPolaganja,
                OLD.ocjena,
                NEW.ocjena,
                CURRENT_USER(),
                NOW());
END;
%%
DELIMITER ;
```

Primjer naredbe koja će aktivirati okidač:

```
UPDATE ocjene
  SET ocjena = 3
  WHERE (idKolegij = 2 AND jmbagStudent = '0016097990' AND
datumPolaganja = '2017-01-30');
```

Tablica ocjene_log:

☐	idKolegij	jmbagStudent	datumPolaganja	ocjenaStara	ocjenaNova	korisnik	vremenskiZig
☐	1	0013025926	2016-09-11	(NULL)	2	root@localhost	2016-09-12 10:53:10
☐	7	0016097990	2016-09-19	(NULL)	3	root@localhost	2016-09-22 10:59:14
☐	2	0016097990	2017-01-30	(NULL)	1	stuslu@localhost	2017-01-31 13:37:42
☐	2	0016097990	2017-01-30	1	2	stuslu@localhost	2017-01-31 13:38:45
☐	30	0016199452	2017-02-05	(NULL)	3	root@localhost	2017-02-07 11:15:53
☐	1	0016201458	2017-02-10	1	5	root@localhost	2017-02-12 13:31:13
☐	46	0016201458	2017-02-16	(NULL)	2	root@localhost	2017-02-17 11:19:02
☐	30	0016199452	2017-02-05	3	5	root@localhost	2017-07-07 13:32:34
*	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

27. Događaji

MySQL događaj (engl. *event*) je vrsta pohranjenog zadatka a služi za obavljanje radnji koje su uvjetovane vremenom. Za razliku od pohranjenih rutina koje se pokreću eksplicitno naredbama CALL i SELECT, te za razliku od okidača koji se pokreću automatski prilikom obavljanja INSERT, UPDATE ili DELETE naredbi nad tablicama u bazi, događaj je objekt koji se pokreće u ovisnosti o vremenu, a ponekad se naziva i vremenski okidač. Događaj je moguće definirati da se pokreće jednom u određenom trenutku ili da se ponavlja u skladu sa zadanim vremenskim intervalom. Događaji postoje od verzije MySQL 5.1 i nude svojevrsnu alternativu zakazanim zadacima (engl. *cron jobs*).

27.1. IZRADA DOGAĐAJA

Osnovna sintaksa:

```
CREATE
```

```
[DEFINER = { korisnik | CURRENT_USER }]
EVENT [IF NOT EXISTS] ime_događaja
ON SCHEDULE raspored
[ON COMPLETION [NOT] PRESERVE]
[ENABLE | DISABLE | DISABLE ON SLAVE]
[COMMENT 'string']
DO tijelo_događaja;
```

raspored:

```
AT timestamp [+ INTERVAL interval] ...
| EVERY interval
[STARTS timestamp [+ INTERVAL interval] ...]
[ENDS timestamp [+ INTERVAL interval] ...]
```

interval:

```
quantity {YEAR | QUARTER | MONTH | DAY | HOUR | MINUTE |
          WEEK | SECOND | YEAR_MONTH | DAY_HOUR | DAY_MINUTE |
          DAY_SECOND | HOUR_MINUTE | HOUR_SECOND |
MINUTE_SECOND}
```

Kao i kod ostalih pohranjenih zadataka, ako se tijelo događaja *tijelo_događaja* sastoji od više naredbi, potrebno ih je omeđiti unutar BEGIN i END bloka. Također je potrebno izvršiti i privremenu promjenu graničnika na neku drugu vrijednost.

Dinamiku pokretanja događaja moguće je definirati na nekoliko načina:

1. Pokretanje jednom u točno definiranom trenutku

```
AT 'YYYY-MM-DD HH:MM.SS'
```

Primjer:

```
AT '2017-01-01 02:00.00'
```

2. Pokretanje jednom nakon isteka točno određenog intervala

```
AT CURRENT_TIMESTAMP + INTERVAL n [HOUR|MONTH|WEEK|DAY|MINUTE]
```

Primjer:

```
AT CURRENT_TIMESTAMP + INTERVAL 1 DAY
```

3. Pokretanje u točno određenim intervalima (zauvijek)

```
EVERY n [HOUR|MONTH|WEEK|DAY|MINUTE]
```

Primjer:

```
EVERY 1 DAY
```

4. Pokretanje u točno određenim intervalima za vrijeme određenog vremenskog perioda

```
EVERY n [HOUR|MONTH|WEEK|DAY|MINUTE] STARTS date ENDS date
```

Primjer:

```
EVERY 1 DAY
```

```
STARTS CURRENT_TIMESTAMP + INTERVAL 1 WEEK
```

```
ENDS '2025-01-01 00:00.00'
```

27.2. OMOGUĆAVANJE DOGAĐAJA ZA DBMS

Na početku rada s događajima potrebno je provjeriti jesu li omogućeni na DBMS-u. Pretpostavljeno stanje je da događaji nisu uključeni, što je moguće provjeriti sa slijedećom naredbom:

```
SHOW VARIABLES
```

```
WHERE VARIABLE_NAME = 'event_scheduler';
```

☐ Variable name	Value
☐ event_scheduler	OFF

Globalno uključivanje postavljanja događaja, moguće je na četiri načina:

```
SET GLOBAL event_scheduler = ON;
```

```
SET @@global.event_scheduler = ON;
SET GLOBAL event_scheduler = 1;
SET @@global.event_scheduler = 1;
```

Ponovna provjera:

```
SHOW VARIABLES
    WHERE VARIABLE_NAME = 'event_scheduler';
```

☐ Variable name	Value
☒ event_scheduler	ON

Ako je potrebno isključiti događaje, koriste se naredbe (jedna od ponuđenih):

```
SET GLOBAL event_scheduler = OFF;
SET @@global.event_scheduler = OFF;
SET GLOBAL event_scheduler = 0;
SET @@global.event_scheduler = 0;
```

27.3. IZMJENA DOGAĐAJA

Osnovna sintaksa:

```
ALTER
    [DEFINER = { user | CURRENT_USER }]
    EVENT naziv_događaja
    [ON SCHEDULE raspored]
    [ON COMPLETION [NOT] PRESERVE]
    [RENAME TO novi_naziv_događaja]
    [ENABLE | DISABLE | DISABLE ON SLAVE]
    [COMMENT 'string']
    [DO tijelo_događaja]
```

Većina gornjih klauzula objašnjena je u ranijim poglavljima te se neće nadalje detaljnije komentirati.

27.4. BRISANJE DOGAĐAJA

Osnovna sintaksa:

```
DROP EVENT [IF EXISTS] event_name
```

27.5. PRIMJER

Neka je na tablici *izvrsitelji* implementiran *soft delete* princip brisanja podataka. Tablica izvrsitelji neka sadrži još jedan atribut *deletedTS*, tipa podatka datetime i pretpostavljene

vrijednosti NULL. Pretpostavimo da će se brisanje n-torki u tablici *izvršitelji* odvijati tako da se ažurira atribut *deletedTS* dotične n-torke i postavi na vrijednost trenutnog vremena i datuma. Time bi se (neobrisane) n-torke iz tablice izvršitelji dohvaćale pomoću:

```
SELECT * FROM izvršitelji WHERE deletedTS IS NOT NULL;
```

Potrebno je izraditi tablicu *arhivaizvršitelji* koja će biti iste strukture kao izvršitelji ali predviđena za arhiviranje podataka.

Nakon toga izraditi događaj koji će jednom dnevno prebacivati sve obrisane podatke iz tablice *izvršitelji* u *arhivaizvršitelji* radi optimizacije podataka u tablici *izvršitelji*.

```
ALTER TABLE izvršitelji
    ADD COLUMN deletedTS DATETIME DEFAULT NULL;
CREATE TABLE arhivaIzvršitelji LIKE izvršitelji;
ALTER TABLE arhivaIzvršitelji ENGINE=ARCHIVE;
1 queries executed, 0 success, 1 ERRORS, 0 WARNINGS
QUERY: ALTER TABLE arhivaIzvršitelji ENGINE=ARCHIVE

Error CODE: 1069
Too many KEYS specified; MAX 1 KEYS allowed
/* mehanizam pohrane arhive ne podržava više od jednog indexa na
tablici pa će u nastavku biti obrisani */
ALTER TABLE arhivaIzvršitelji
    DROP INDEX fk_izvršitelji_nastavnic1;
ALTER TABLE arhivaIzvršitelji
    DROP INDEX fk_izvršitelji_kolegij1;
ALTER TABLE arhivaIzvršitelji
    DROP INDEX fk_izvršitelji_ulogaIzvršitelja1;
ALTER TABLE arhivaizv arhivaIzvršitelji rsitelji
    DROP PRIMARY KEY;
/* promjena mehanizma pohrane na arhive nakon obrisanih indeksa */
ALTER TABLE arhivaIzvršitelji ENGINE=ARCHIVE;
1 queries executed, 1 success, 0 ERRORS, 0 WARNINGS
QUERY: ALTER TABLE arhivaIzvršitelji ENGINE=ARCHIVE
```

Dohvat podataka iz tablice izvršitelji:

```
SELECT * FROM izvršitelji WHERE deletedTS IS NULL;
```

jmbgNastavnik	idKolegij	idUlogaIzvrstjelja	deletedTS
0110959390037	11	1	(NULL)
0110959390037	11	2	(NULL)
0205951330124	8	1	(NULL)
0305988391019	28	1	(NULL)
0404964361006	36	2	(NULL)
0501953210249	13	1	(NULL)
0509959360058	41	1	(NULL)
0509959360058	42	2	(NULL)
0606960383925	38	1	(NULL)
0608955311207	33	1	(NULL)
0608955311207	34	1	(NULL)
0610975181955	34	2	(NULL)

Operacija brisanja sada se izvršava naredbom update. U nastavku su obrisane dvije n-torke:

```
UPDATE izvrstjelji SET deletedTS = NOW()
WHERE jmbgNastavnik = '0110959390037'
AND idKolegij = 11 AND idUlogaIzvrstjelja = 1;
UPDATE izvrstjelji SET deletedTS = NOW()
WHERE jmbgNastavnik = '0110959390037'
AND idKolegij = 11 AND idUlogaIzvrstjelja = 2;
SELECT * FROM izvrstjelji;
```

jmbgNast...	idKolegij	idUlogaIzvrstjelja	deletedTS
0110959390037	11	1	2018-02-10 18:25:54
0110959390037	11	2	2018-02-10 18:25:56
0205951330124	8	1	(NULL)
0305988391019	28	1	(NULL)
0404964361006	36	2	(NULL)
0501953210249	13	1	(NULL)

Izrada događaja.

```
DELIMITER //
CREATE EVENT arhivirajIzvrstjeljeDnevno
ON SCHEDULE EVERY 1 DAY
STARTS '2017-01-01 02:00:00' ENABLE DO
BEGIN
    INSERT INTO arhivaIzvrstjelji
        SELECT * FROM izvrstjelji WHERE deletedTS IS NOT NULL;
    DELETE FROM izvrstjelji WHERE deletedTS IS NOT NULL;
END//
DELIMITER ;
```

Nakon izvršavanja događaja tablica izvrstjelji arhiva sadržati će (za sada) dvije n-torke:

```
SELECT * FROM arhivaIzvrstjelji;
```

☐ jmbgNastavnik	idKolegij	idUlogaIzvršitelja	deletedTS
☐ 0110959390037	11	1	2018-02-10 18:25:54
☐ 0110959390037	11	2	2018-02-10 18:25:56
* (NULL)	(NULL)	(NULL)	(NULL)

Postojeće događaje moguće je dohvatiti naredbom SHOW EVENTS:

SHOW EVENTS;

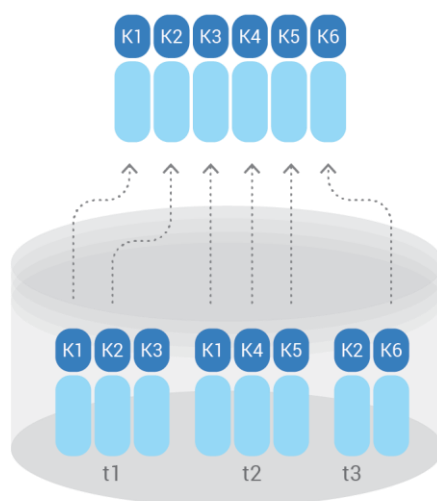
☐ Db	Name	Definer	Time zone	Type
☐ studenti	arhivirajIzvršiteljeDnevno	root@localhost	SYSTEM	RECURRING
☐ studenti	arhivirajIzvršiteljeDvijeSekunde	root@localhost	SYSTEM	RECURRING

Događaji podržavaju i kursore tako da se funkcionalnost ovog tipa može implementirati i detaljnijim provjerama korištenjem kursora unutar tijela događaja.

28. Pogledi

Pogled (engl. *view*) je vrsta privremene tablice koja nastaje kao rezultat SQL upita nad jednom ili više tablica. To je dinamički objekt u bazi podataka, koji enkapsulira najčešće složeni SQL upit, pohranjen je u sklopu baze te se kao i ostali objekti, kreira korištenjem naredbe CREATE i dodjeljivanjem naziva tom pogledu. Za razliku od privremene tablice koja postoji isključivo u sesiji, pogled je pohranjen u memoriji a SELECT upit od kojeg se sastoji izvršava se svaki puta kod ispisa podataka iz pogleda. Podatci koje pogled sadrži upravo iz tog razloga nisu duplicirani jer fizički ne postoje. Pogled predstavlja jednostavnu metodu pohrane SQL naredbe u bazu podataka.

Pogledi se najčešće primjenjuju za analizu ili pripremu podataka sa ciljem nenarušavanja normalizacije baze podataka. Podatci se time pripremaju u obliku pogodnim za analizu a bez potrebe za promjenom relacijskog modela baze i bez dupliciranja podataka. Nad pogledom se dodatno mogu postavljati različiti upiti.



Slika 58. Izrada pogleda iz projekcije i selekcije tri tablice

Za pogled nije nužno da sadrži podatke iz tablica, a može sadržavati:

- SELECT upite
- JOIN-ove (unutarnje, vanjske)
- unije
- podupite
- agregirane i izračunate vrijednosti

Poglede je moguće ugnježdavati odnosno izraditi pogled ne samo iz SELECTA već i iz postojećeg pogleda.

28.1. PREDNOSTI KORIŠTENJA POGLEDA

1. Omogućava pojednostavljenje složenih upita

Izradom pogleda definiran je SQL izraz koji povezuje mnogo fizičkih tablica. Koristeći pogled moguće je enkapsulirati složenost strukture baze podataka prema krajnjem korisniku ili vanjskoj aplikaciji. Time oni ne moraju biti upoznati sa potrebnim relacijama u bazi već koriste jednostavne SELECT upite za dohvat podataka umjesto složenih.

2. Omogućava jednostavno ograničavanje pristupa podacima

Nije potrebno dodjeljivati dozvole korisnicima nad cijelim skupom osjetljivih podataka, već je moguće dodjeljivati dozvole na razini pogleda (samo nad onim podacima koji su potrebni).

3. Postojanje dodatne zaštite podataka

Ako se izradi pogled koji je namijenjen samo čitanju, korisnici koji mu pristupaju neće imati mogućnosti mijenjati dotične podatke već ih samo čitati.

4. Pogled može sadržavati atribut s izračunatim / agregiranim vrijednostima

Nije uobičajeno u sklopu tablica u bazi pohranjivati izračunate vrijednosti radi izbjegavanja redundancije i narušavanja normalizacije, no pogledi mogu sadržavati takve attribute. Potrebno je voditi računa o tome da se vrijednosti takvih atributa izračunavaju prilikom pokretanja upita.

5. Kompatibilnost prilikom promjene strukture baze

U stvarnim informacijskim sustavima prilikom dodavanja novih informacija ili softverskih funkcionalnosti, ponekad je potrebno mijenjati strukturu baze podataka. Uvedene promjene mogu nerijetko utjecati na SELECT upit za dobivanje podataka prema krajnjem korisniku ili aplikaciji. Umjesto da se mijenja struktura baze, moguće je izmijeniti samo pogled čime krajnji korisnik ne vidi da je došlo do većih izmjena strukture već i dalje dohvaća podatke iz „istog” pogleda.

28.2. NEDOSTATCI KORIŠTENJA POGLEDA

1. Sporost dobivanja rezultata

Brzina čitanja podataka iz pogleda može biti manja u odnosu ako se čita iz tablice čime se narušavaju performanse. Ovo je posebno izraženo ako se pogled dobiva SELECT upitom iz drugog pogleda.

2. Ovisnost o tablicama

Obzirom da se pogled temelji na tablicama, izmjena strukture tablice može rezultirati potrebom izmjene definicije pogleda.

28.3. IZRADA POGLEDA

```
CREATE [ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]  
VIEW [ime_baze].[ime_pogleda]
```

```
AS [SELECT izraz]
```

Definiranje željenog algoritma je opcionalno, pri čemu se automatski postavlja na vrijednost UNDEFINED u slučaju da nije eksplicitno naveden.

Mogući algoritmi su:

1.MERGE

MERGE algoritam kombinira projekcije i selekcije iz SELECT upita prije izvođenja pogleda u jedan upit, nakon čega se izvodi samo jedan upit (nad definiranim podupitom – upit iz definicije pogleda postaje podupit) kojim se dobiva result set. On nije moguć ako se pogled odnosi na agregatne funkcije (MAX, MIN, COUNT, AVG, SUM, ...), DISTINCT, GROUP BY, HAVING, LIMIT, UNION, UNION ALL kao i na podupite. Ako bi pogled kojemu je definiran algoritam MERGE sadržavao navedeno, automatski će se promijeniti algoritam na UNDEFINED.

2. TEMPTABLE

Prilikom dohvata podataka iz pogleda, dohvaćaju se podatci iz fizičkih tablica s kojima se puni privremena tablica na temelju definiranog pogleda. Nakon toga se izvršava SELECT nad privremenom tablicom. Ovaj algoritam sporije radi zbog vremena potrebnog za punjenje privremene tablice i zbog nepostojanja indeksa u privremenoj tablici nad kojom se postavljaju daljnji upiti.

3.UNDEFINED

U slučaju nenavođenja preferiranog algoritma, pretpostavljena vrijednost je UNDEFINED. U tom će slučaju DBMS sam procijeniti koji algoritam upotrijebiti iako će se najčešće preferirati MERGE radi boljih performansi. S druge strane, ako su određeni podatci zaključani, prilikom izrade privremene tablice ključevi će biti otpušteni, pa je u tom slučaju bolja brzina dobivanja rezultata jer DBMS ne mora čekati na otpuštanje ključeva.

Primjer:

Izraditi pogled koji će vraćati današnji datum u obliku dd.mm.gggg. Napisati primjer uporabe.

```
CREATE VIEW datum AS  
SELECT DATE_FORMAT(CURDATE(), '%m.%d.%Y.');
```

Primjer ispisa iz pogleda:

```
SELECT * FROM datum;
```

DBMS će tretirati pogled kao i tablicu, što je vidljivo iz rezultata poziva naredbi SHOW TABLES i SHOW FULL TABLES:

☐ Tables in studenti
☐ datum
☐ izvrsitelji
☐ kolegiji
☐ mjesta
☐ nastavnici
☐ ocjene
☐ smjerovi
☐ studenti
☐ ulogaizvrsitelja
☐ ustanove
☐ zupanije

☐ Tables in studenti	Table type
☐ datum	VIEW
☐ izvrsitelji	BASE TABLE
☐ kolegiji	BASE TABLE
☐ mjesta	BASE TABLE
☐ nastavnici	BASE TABLE
☐ ocjene	BASE TABLE
☐ smjerovi	BASE TABLE
☐ studenti	BASE TABLE
☐ ulogaizvrsitelja	BASE TABLE
☐ ustanove	BASE TABLE
☐ zupanije	BASE TABLE

28.4. PREGLED, BRISANJE I IZMJENA POGLEDA

```
SHOW CREATE VIEW [ime_baze].[ime_pogleda];
```

```
DROP VIEW [ime_baze].[ime_pogleda];
```

```
ALTER [ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]
VIEW [ime_baze].[ime_pogleda]
AS [SELECT izraz]
```

Umjesto naredbe ALTER, prilikom razvoja moguće je koristiti CREATE OR REPLACE VIEW izraz.

```
CREATE OR REPLACE
[ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]
VIEW [ime_baze].[ime_pogleda]
AS [SELECT izraz]
```

28.5. PRIMJERI

Primjer - pogled iz podupita

Izraditi pogled koji će ispisivati samo one nastavnike s ulogom profesora koji nisu evidentirali niti jednu neprolaznu ocjenu.

```
CREATE VIEW prof_pozit AS
SELECT *
FROM nastavnici JOIN izvrsitelji ON jmbg = jmbgNastavnik
JOIN ulogaizvrsitelja ON idUlogaIzvrsitelja =
ulogaizvrsitelja.id
WHERE ulogaizvrsitelja.naziv = 'profesor' AND idKolegij NOT
IN
(SELECT DISTINCT (kolegiji.id)
FROM kolegiji JOIN ocjene ON kolegiji.id = idKolegij
```

```
WHERE ocjena = 1);
```

Primjer ispisa iz pogleda:

```
SELECT * FROM prof_pozit;
```

Primjer – pogled kao unija

Izraditi pogled *recent_activity* koji će prikazivati aktivnosti upisa studenata na studij te polaganja kolegija. Za svaki zapis je potrebno ispisati o kojoj aktivnosti je riječ. Koristeći pogled potrebno je ispisati ispravno sortirane aktivnosti počevši od najnovije.

```
CREATE OR REPLACE VIEW recent_activity AS
SELECT datumUpisa AS datum, 'upisan student', CONCAT(ime, " ",
prezime, " ", jmbag) AS student, smjerovi.naziv AS opis
FROM studenti JOIN smjerovi ON studenti.idSmjer =
smjerovi.id
UNION ALL
SELECT datumPolaganja AS datum, 'unesena ocjena', CONCAT(ime, " ",
prezime, " ", jmbag) AS student, kolegiji.naziv AS opis
FROM studenti JOIN ocjene ON studenti.jmbag =
ocjene.jmbagStudent
JOIN kolegiji ON ocjene.idKolegij = kolegiji.id;
```

Ispis iz pogleda sa sortiranjem po zajedničkom stupcu datum:

```
SELECT * FROM recent_activity ORDER BY datum DESC;
```

datum	upisan student	student	opis
2017-08-08	unesena ocjena	Antonio Pavić 0013025926	Baze podataka
2017-08-08	unesena ocjena	Vesna Đukić 0016097990	Betonske konstrukcije
2017-07-04	unesena ocjena	Antonia Čoić 0016099969	Baze podataka
2017-02-17	unesena ocjena	Bernarda Tomić 0016199452	Tjelesna i zdravstvena kultura 3
2017-02-17	unesena ocjena	Ana Radak 0016201458	Tjelesna i zdravstvena kultura 3
2017-02-16	unesena ocjena	Ana Radak 0016201458	Tjelesna i zdravstvena kultura 3
2017-02-16	unesena ocjena	Ana Radak 0016201458	Napredne baze podataka
2017-02-10	unesena ocjena	Ana Radak 0016201458	Baze podataka
2017-02-05	unesena ocjena	Bernarda Tomić 0016199452	Izrada web-projekta
2017-01-30	unesena ocjena	Vesna Đukić 0016097990	Uvod u UNIX sustave

...

Ovime je vidljivo da će ORDER BY nad pogledom nadjačati eventualno sortiranje unutar pogleda.

Dohvat današnjih aktivnosti (kada bi baza bila *live*):

```
SELECT * FROM recent_activity WHERE datum = CURDATE() ORDER BY
datum DESC
```

Potrebno je izraditi pogled koji će za svakog studenta osim njegovih osobnih podataka (jmbag, ime, prezime) vraćati i izvještaj o:

- prosječnoj ocjeni
- ukupnom broju ocjena
- ukupnom broju petica
- ukupnom broju četvorki
- ukupnom broju trojki
- ukupnom broju dvojki
- ukupnom broju jedinica

```
CREATE OR REPLACE VIEW ocjeneStudenata AS
SELECT jmbag, ime, prezime,
AVG(ocjena) AS prosjek,
COUNT(ocjena) AS brojOcjena,
(SELECT COUNT(*) FROM ocjene
WHERE ocjene.jmbagStudent = jmbag
AND ocjene.ocjena = 5) AS Petice,
(SELECT COUNT(*) FROM ocjene
WHERE ocjene.jmbagStudent = jmbag
AND ocjene.ocjena = 4) AS Cetvorke,
(SELECT COUNT(*) FROM ocjene
WHERE ocjene.jmbagStudent = jmbag
AND ocjene.ocjena = 3) AS Trojke,
(SELECT COUNT(*) FROM ocjene
WHERE ocjene.jmbagStudent = jmbag
AND ocjene.ocjena = 2) AS Dvojke,
(SELECT COUNT(*) FROM ocjene
WHERE ocjene.jmbagStudent = jmbag
AND ocjene.ocjena = 1) AS Jedinice
FROM studenti
JOIN ocjene ON studenti.jmbag = ocjene.jmbagStudent
GROUP BY jmbag;
```

Ispis iz pogleda:

```
SELECT * FROM ocjeneStudenata;
```

jmbag	ime	prezime	prosjeak	brojOcjena	Petice	Cetvorke	Trojke	Dvojke	Jedinice
0010081356	Tanja	Krog	2.0000	2	0	0	0	2	0
0013020125	Matea	Omanović	2.0000	1	0	0	0	1	0
0013022546	Antun	Mandić	4.0000	1	0	1	0	0	0
0013025926	Antonio	Pavić	3.5000	2	1	0	0	1	0
0013026584	Borna	Knežević	3.0000	1	0	0	1	0	0
0016096972	Dora	Vidaković	3.0000	1	0	0	1	0	0
0016097990	Vesna	Đukić	4.2500	4	2	1	1	0	0
0016098017	Marina	Zadro	5.0000	1	1	0	0	0	0
0016099969	Antonia	Čoić	4.0000	3	1	1	1	0	0
0016100313	Biljana	Vidović	3.0000	1	0	0	1	0	0
0016101784	David	Vučetić	2.0000	1	0	0	0	1	0
0016199452	Bernarda	Tomić	3.7500	4	2	1	0	0	1
0016200033	Zvonimir	Žarković	3.0000	1	0	0	1	0	0
0016201125	Denis	Šapina	5.0000	1	1	0	0	0	0
0016201458	Ana	Radak	4.0000	7	4	1	0	2	0
0016203355	Dino	Bošnjak	4.5000	2	1	1	0	0	0
0016209876	Kristina	Pešerović	2.0000	1	0	0	0	1	0
0016216325	Toni	Tošeski	2.0000	1	0	0	0	1	0

...

Potrebno je izraditi isti pogled bez korištenja podupita već sa uvjetovanjem u SELECT-u.

```
CREATE OR REPLACE VIEW ocjeneStudenata AS
SELECT jmbag, ime, prezime, datumUpisa,
AVG(ocjena) AS prosjek, COUNT(ocjena) AS brojOcjena,
COUNT(IF (ocjene.ocjena=5, ocjene.ocjena, NULL)) if_Petice,
COUNT(IF (ocjene.ocjena=4, ocjene.ocjena, NULL)) if_Cetvorke,
COUNT(IF (ocjene.ocjena=3, ocjene.ocjena, NULL)) if_Trojke,
COUNT(IF (ocjene.ocjena=2, ocjene.ocjena, NULL)) if_Dvojke,
COUNT(IF (ocjene.ocjena=1, ocjene.ocjena, NULL)) if_Jedinice
FROM studenti
JOIN ocjene ON studenti.jmbag = ocjene.jmbagStudent
GROUP BY jmbag;
```

Ispis iz pogleda:

```
SELECT * FROM ocjeneStudenata;
```

28.6. UNAPRIJEDIVI POGLEDI

U određenim slučajevima moguće je izraditi unaprijedivi pogled (engl. updatable view). Za takav pogled je karakteristično da podržava INSERT, UPDATE i DELETE naredbe. Navedene naredbe izvode se nad pogledom a time će mijenjati sadržaj u tablicama na koje se pogled odnosi. Pogled ne može biti unaprijediv ako sadrži:

- agregatne funkcije
- DISTINCT

- GROUP BY
- HAVING
- UNION, UNION ALL
- podupit
- neunaprijedivi pogled u WHERE klauzuli
- aliase
- TEMPTABLE algoritam

Primjer:

```
CREATE OR REPLACE VIEW kolegiji_view AS
SELECT * FROM kolegiji;
```

Ispis podataka iz tablice kolegiji i pogleda kolegiji_view dati će isti set rezultata:

```
SELECT * FROM kolegiji;
SELECT * FROM kolegiji_view;
```

id	naziv	opis		idSmjer
1	Baze podataka	Student će steći znanja i vještine za dizajniranje i kre...	143B	1
2	Uvod u UNIX sustave	Student treba savladati osnovno znanje rada na otvorenim...	80B	1
3	Uvod u UNIX sustave	Student treba savladati osnovno znanje rada na otvorenim...	80B	2
4	Fizika	Razumijevanje fizikalnih pojava i veličina, opisanih u š...	97B	2
5	Fizika	Razumijevanje fizikalnih pojava i veličina, opisanih u š...	97B	3
6	Fizika	Razumijevanje fizikalnih pojava i veličina, opisanih u š...	97B	4
7	Betonske konstrukcije	Stjecanje teoretskih i praktičkih znanja i vještina, pom...	147B	4
8	Osnove elektrotehnike	Stjecanje znanja iz teorije električnih krugova i metoda...	93B	3
9	Poslovna etika	Cilj je predmeta upoznati studente s etičkim pitanjima s...	130B	5
10	Statistika	Opći ciljevi su ovladavanje teorijskim spoznajama kao i ...	109B	5
11	Osnove građanskog prava	U općem djelu građanskog prava stječu se znanja ne samo ...	211B	6
12	Javna nabava	Student će biti osposobljen za organizaciju i provedbu p...	78B	6
13	Ekonomika turizma	Stjecanje specifičnih znanja za prepoznavanje utjecaja t...	141B	8
14	Sportski marketing	U okviru ovog predmeta studenti će se upoznati s osnovam...	155B	8

...

Izvođenje UPDATE naredbe nad kolegiji_view rezultira promjenom u fizičkoj tablici:

```
UPDATE kolegiji_view SET opis = CONCAT('2017-10-08', ': ', opis);
SELECT * FROM kolegiji;
```

id	naziv	opis		idSmjer
1	Baze podataka	2017-10-08: Student će steći znan...	155B	1
2	Uvod u UNIX sustave	2017-10-08: Student treba savlada...	92B	1
3	Uvod u UNIX sustave	2017-10-08: Student treba savlada...	92B	2
4	Fizika	2017-10-08: Razumijevanje fizikal...	109B	2
5	Fizika	2017-10-08: Razumijevanje fizikal...	109B	3
6	Fizika	2017-10-08: Razumijevanje fizikal...	109B	4
7	Betonske konstrukcije	2017-10-08: Stjecanje teoretskih ...	159B	4
8	Osnove elektrotehnike	2017-10-08: Stjecanje znanja iz t...	105B	3
9	Poslovna etika	2017-10-08: Cilj je predmeta upoz...	142B	5
10	Statistika	2017-10-08: Opći ciljevi su ovlad...	121B	5
11	Osnove građanskog prava	2017-10-08: U općem djelu građans...	223B	6
12	Javna nabava	2017-10-08: Student će biti ospos...	90B	6

...

Izvođenje INSERT naredbe nad kolegiji_view rezultira unosom n-torke u fizičkoj tablici:

```
INSERT INTO kolegiji_view
```

```
(id, naziv, opis, idSmjer)
VALUES (47, 'Informacijska sigurnost', 'Prvo izvođenje
kolegija', 2);
SELECT * FROM kolegiji ORDER BY id DESC;
```

id	naziv	opis		idSmjer
47	Informacijska sigurnost	Prvo izvođenje kolegija	24B	2
46	Napredne baze podataka	2017-10-08: Student se treba upoz...	179B	1
45	Tjelesna i zdravstvena kultura 3	2017-10-08: Odgojno-obrazovno pod...	239B	16
44	Tjelesna i zdravstvena kultura 2	2017-10-08: Odgojno-obrazovno pod...	239B	10
43	Tjelesna i zdravstvena kultura 2	2017-10-08: Odgojno-obrazovno pod...	239B	1
42	Planiranje prometne potražnje	2017-10-08: Osposobiti studenta z...	103B	21
41	Tereti u prometu	2017-10-08: Osnovni cilj i zadata...	228B	21
40	Sredstva pomorskog prometa	2017-10-08: Cilj kolegija je upoz...	153B	20
39	Tehnička mehanika 2	2017-10-08: Upoznati studente s t...	228B	20

Brisanje prethodno unesene n-torke nad kolegiji_view rezultira brisanjem n-torke u fizičkoj tablici:

```
DELETE FROM kolegiji_view WHERE id = 47;
SELECT * FROM kolegiji WHERE id = 47;
```

id	naziv	opis	idSmjer
----	-------	------	---------

(rezultat je nula n-torki)

29. Upravljanje tokom pohranjenih zadataka

29.1. KONTROLA TOKA

Kontrolom toka moguće je preusjeravati tok izvođenja pohranjenog zadatka u ovisnosti je li zadani uvjet ispunjen. MySQL unutar pohranjenih zadataka omogućava kontrolu toka kroz IF i CASE uvjetovanje. Kontrola toka i petlje u MySQL-u slične su drugim programskim jezicima

29.1.1. IF uvjetovanje

Osnovni oblik kontrole toka pohranjenog zadatka je pomoću IF uvjetovanja. U slučaju da je uvjet ispunjen izvršava se jedan skup naredbi, a u slučaju da nije, izvršava se drugi skup definiran pomoću ELSE izjave ili se ulazi u novo grananje pomoću ELSEIF izjave.

Osnovna sintaksa:

```
IF uvjet
    THEN naredbe
    [ELSEIF uvjet1 THEN naredbe]
    ...
    [ELSE naredbe]
END IF;
```

Primjer (IF uvjetovanje s jednim grananjem):

Potrebno je napisati funkciju koja će dohvatiti trenutno vrijeme s poslužitelja, te ako je dan u tjednu (ponedjeljak do petak), ispisati odgovarajuću poruku.

```
DELIMITER //
DROP FUNCTION IF EXISTS danUTjednu //
CREATE FUNCTION danUTjednu()
RETURNS VARCHAR(255) DETERMINISTIC
BEGIN
    DECLARE poruka VARCHAR(255);
    IF DAYOFWEEK(NOW()) BETWEEN 2 AND 6 THEN
        SET poruka = 'Danas je radni dan';
    END IF;
    RETURN poruka;
END //
DELIMITER ;
```

Primjer poziva funkcije:

```
SELECT danUTjednu();
```

Rezultat poziva funkcije:

☐	danUTjednu()
☐	Danas je radni dan

Primjer (IF uvjetovanje s više grananja):

Potrebno je napisati funkciju koja će dohvatiti trenutno vrijeme s poslužitelja, te ispisati odgovarajuću poruku o tome koji je trenutno dan u tjednu.

```
DELIMITER //
DROP FUNCTION IF EXISTS danUTjednu //
CREATE FUNCTION danUTjednu()
RETURNS VARCHAR(255) DETERMINISTIC
BEGIN
    DECLARE poruka VARCHAR(255);
    IF DAYOFWEEK(NOW()) = 2 THEN
        SET poruka = 'Danas je ponedjeljak';
    ELSEIF DAYOFWEEK(NOW()) = 3 THEN
        SET poruka = 'Danas je utorak';
    ELSEIF DAYOFWEEK(NOW()) = 4 THEN
        SET poruka = 'Danas je srijeda';
    ELSEIF DAYOFWEEK(NOW()) = 5 THEN
        SET poruka = 'Danas je četvrtak';
    ELSEIF DAYOFWEEK(NOW()) = 6 THEN
        SET poruka = 'Danas je petak';
    ELSEIF DAYOFWEEK(NOW()) = 7 THEN
        SET poruka = 'Danas je dubota';
    ELSE
        SET poruka = 'Danas je nedjelja';
    END IF;
    RETURN poruka;
END //
DELIMITER ;
```

Primjer poziva funkcije:

```
SELECT danUTjednu();
```

Rezultat poziva funkcije:

☐	danUTjednu()
☐	Danas je ponedjeljak

29.1.2. CASE uvjetovanje

U slučajevima kada postoji potreba prilikom uvjetovanja kositi više grananja, umjesto IF uvjetovanja praktičnije je koristiti CASE uvjetovanje.

Osnovna sintaksa (1):

```
CASE
    WHEN izraz_1 THEN naredbe_1
    ...
    WHEN izraz_n THEN naredbe_n
    ELSE naredbe
END CASE
```

Osnovna sintaksa (2):

```
CASE izraz
    WHEN vrijednost_1 THEN naredbe_1
    ...
    WHEN vrijednost_n-1 THEN naredbe_n-1
    ELSE vrijednost_n
END CASE
```

Primjer:

Potrebno je napisati funkciju koja će dohvatiti trenutno vrijeme s poslužitelja, te ispisati odgovarajuću poruku o tome koji je trenutno dan u tjednu.

```
DELIMITER //
CREATE FUNCTION danUTjednu() RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE vrati VARCHAR(50);
    CASE DAYOFWEEK(CURDATE())
        WHEN 2 THEN SET vrati='Danas je ponedjeljak';
        WHEN 3 THEN SET vrati='Danas je utorak';
```

```

        WHEN 4 THEN SET vrati='Danas je srijeda';
        WHEN 5 THEN SET vrati='Danas je četvrtak';
        WHEN 6 THEN SET vrati='Danas je petak';
        ELSE SET vrati='Danas je vikend';
    END CASE;
    RETURN vrati;
END; //
DELIMITER ;

```

Primjer poziva funkcije:

```
SELECT danUTjednu();
```

Rezultat poziva funkcije:

☐	danUTjednu()
☐	Danas je ponedjeljak

29.2. PETLJE

Petlje u pohranjenim zadacima omogućavaju višekratno ponavljanje skupa istih naredbi, ovisno o tome je li određen uvjet ispunjen ili nije. Moguće su tri vrste petlji kao što je opisano u nastavku.

29.2.1. WHILE petlja

WHILE petlja iterira definirane naredbe sve dok je zadovoljen određeni uvjet, pri čemu se taj uvjet provjerava na početku iteracije petlje.

Osnovna sintaksa:

```

WHILE uvjet DO
    naredbe
END WHILE

```

Primjer:

U primjeru je napisana procedura u kojoj se WHILE petlja iterira n puta. U svakoj se iteraciji povećava vrijednost varijable i za 1, te se ispisuje sa SELECT naredbom.

```

DELIMITER $$
DROP PROCEDURE IF EXISTS proc $$
CREATE PROCEDURE proc_while(n INT)
BEGIN
    DECLARE i INT;

```



```
        SET i = 0;
        WHILE i<n DO
            SET i = i + 1;
            SELECT i;
        END WHILE;
    END;
$$
DELIMITER ;
```

Primjer poziva procedure:

```
CALL proc_while(5);
```

Poziv procedure rezultirat će sa 5 skupova rezultata odnosno sa ispisom vrijednosti: 1, 2, 3, 4, 5.

29.2.2. REPEAT ... UNTIL petlja

REPEAT petlja iterira definirane naredbe sve dok se ne zadovolji određeni uvjet, pri čemu se taj uvjet provjerava na kraju iteracije petlje.

Osnovna sintaksa:

```
REPEAT
    naredbe;
    UNTIL uvjet
END REPEAT
```

Primjer:

U primjeru je napisana procedura u kojoj se REPEAT petlja iterira n puta. U svakoj se iteraciji povećava vrijednost varijable i za 1, te se ispisuje sa SELECT naredbom.

```
DROP PROCEDURE IF EXISTS proc_repeat;
DELIMITER $$
CREATE PROCEDURE proc_repeat(n INT)
    BEGIN
        DECLARE i INT;
        SET i = 0;
        REPEAT
            SET i = i + 1;
            SELECT i;
        UNTIL i >= 5
```

```
        END REPEAT;  
    END;  
$$  
DELIMITER ;
```

Primjer poziva procedure:

```
CALL proc_repeat(5);
```

Poziv procedure rezultirat će sa 5 skupova rezultata odnosno sa ispisom vrijednosti: 1, 2, 3, 4, 5.

29.2.3. LOOP petlja

LOOP petlja izvršava niz naredbi sve dok ne dođe do prekida izazvanog LEAVE izjavom. Ako se LEAVE koristi u kombinaciji sa IF uvjetovanjem, moguće je definirati petlju koja se izvodi sve dok se ne ispuni određeni uvjet. Navođenjem LEAVE ključne riječi izlazi se iz izvođenja LOOP petlje. ITERATE izjavom prelazi se u sljedeću iteraciju petlje od trenutka navođenja izjave ITERATE.

Osnovna sintaksa:

```
Ime_petlje: LOOP  
    Naredbe;  
    [IF uvjet THEN  
        LEAVE ime_petlje;  
    END IF;]  
    [IF uvjet THEN  
        ITERATE ime_petlje;  
    END IF;]  
END LOOP;
```

Primjer:

U primjeru je napisana procedura u kojoj se LOOP petlja iterira n puta. U svakoj se iteraciji povećava vrijednost varijable i za 1, te se ispisuje sa SELECT naredbom.

```
DROP PROCEDURE proc_loop;  
DELIMITER $$  
CREATE PROCEDURE proc_loop(n INT)  
BEGIN  
    DECLARE i INT;  
    SET i = 0;  
    petlja: LOOP
```

```

        SET i = i + 1;
        SELECT i;
        IF i >= n THEN
            LEAVE petlja;
        END IF;
    END LOOP;
END; $$
DELIMITER ;

```

Primjer poziva procedure:

```
CALL proc_loop(5);
```

Poziv procedure rezultirat će sa 5 skupova rezultata odnosno sa ispisom vrijednosti: 1, 2, 3, 4, 5.

29.3. REKURZIJE U POHRANJENIM PROCEDURAMA

Rekurzivne procedure su one koje pozivaju same sebe i koriste se kao elegantna opcija rješavanja složenijih programskih struktura. Glavni nedostatak uporabe rekurzije je veliko opterećenje radne memorije čime pohranjena procedure koja sadrži rekurziju postaje manje efikasna i skalabilna u odnosu na procedure koje se ne temelje na rekurziji. Većinu funkcionalnosti koje su izrađene rekurzivnim pozivom procedure moguće je implementirati iterativnim izvršavanjem seta naredbi ili procedure. MySQL podržava rekurzivne procedure ali ne i rekurzivne funkcije.

Navedeno će biti prikazano na primjeru izračuna n-tog elementa Fibonaccijevog niza. Fibonaccijev niz je niz u kojem je svaki element niza izračunava kao zbroj prethodna dva elementa.

```

DELIMITER &&
DROP PROCEDURE IF EXISTS rekurzivni_fibonacci &&
CREATE PROCEDURE rekurzivni_fibonacci(n INT, OUT fib_element INT)
BEGIN
    DECLARE n_1 INT;
    DECLARE n_2 INT;

    IF (n = 0) THEN
        SET fib_element = 0;
    ELSEIF (n = 1) THEN
        SET fib_element = 1;
    
```

```

ELSE
    CALL rekurzivni_fibonacci(n-1, n_1);
    CALL rekurzivni_fibonacci(n-2, n_2);
    SET fib_element = (n_1 + n_2);
END IF;
END; &&
DELIMITER ;

```

Poziv procedure za prvi član niza:

```

CALL rekurzivni_fibonacci(1, @ fib_1);
SELECT @fib_1;

```

☐	@fib_1
☐	1

Poziv procedure za drugi član niza:

```

CALL rekurzivni_fibonacci(1, @ fib_2);
SELECT @fib_2;

```

Poziv ne izvršava uspješnim izvođenjem već se radi pretpostavljenog ograničenja MySQL servera na rekurzije javlja pogreška kao u nastavku.

Error Code: 1456

Recursive limit 0 (as set by the max_sp_recursion_depth variable) was exceeded for routine rekurzivni_fibonacci

Pretpostavljeno ograničenje za izvođenje rekurzije je nula što se vidi kao odgovor na naredbu SHOW VARIABLES kao u nastavku – vrijednost sistemske varijable je nula. Ovo ograničenje je postavljeno radi zaštite od preopterećenja resursa.

```
SHOW VARIABLES LIKE 'max_sp_recursion_depth';
```

☐	Variable_name	Value
☐	max_sp_recursion_depth	0

Vrijednost ove sistemske varijable moguće je promijeniti sa SET naredbom, kao i vrijednost bilo koje druge varijable.

```

SET max_sp_recursion_depth = 10;
SHOW VARIABLES LIKE 'max_sp_recursion_depth';

```

☐	Variable_name	Value
☐	max_sp_recursion_depth	10

U nastavku su dani primjeri poziva rekurzije s većom dubinom.

```
CALL rekurzivni_fibonacci(2, @fib_2);  
SELECT @fib_2;
```

@fib_2
1

```
CALL rekurzivni_fibonacci(3, @fib_3);  
SELECT @fib_3;
```

@fib_3
2

```
CALL rekurzivni_fibonacci(4, @fib_4);  
SELECT @fib_4;
```

@fib_4
3

```
CALL rekurzivni_fibonacci(9, @fib_9);  
SELECT @fib_9;
```

@fib_9
34

Primjer nerekurzivne procedure za izračun n-tog Fibonaccijevog elementa je u nastavku.

```
DELIMITER &&  
DROP PROCEDURE IF EXISTS fibonacci &&  
CREATE PROCEDURE fibonacci(n INT, OUT fib_element INT)  
BEGIN  
    DECLARE m INT DEFAULT 0;  
    DECLARE k INT DEFAULT 1;  
    DECLARE i INT;  
    DECLARE tmp INT;  
  
    SET m=0;  
    SET k=1;  
    SET i=1;  
  
    WHILE (i<=n) DO  
        SET tmp=m+k;  
        SET m=k;  
        SET k=tmp;  
        SET i=i+1;  
    END WHILE;  
    SET fib_element=tmp;  
END
```

```

        END WHILE;
        SET fib_element=m;
END; &&
DELIMITER ;

```

```

CALL fibonacci(1, @fib_1);
SELECT @fib_1;

```

☐	@fib_1
☐	1

```

CALL fibonacci(2, @fib_2);
SELECT @fib_2;

```

☐	@fib_2
☐	1

```

CALL fibonacci(3, @fib_3);
SELECT @fib_3;

```

☐	@fib_2
☐	1

```

CALL fibonacci(4, @fib_4);
SELECT @fib_4;

```

☐	@fib_4
☐	3

```

CALL fibonacci(5, @fib_9);
SELECT @fib_9;

```

☐	@fib_9
☐	34

30. Kursori

Prilikom dohvata podataka sa SELECT naredbom, ako je potrebno dohvatiti i obraditi skup od više n-torki pri čemu se sa svakom upravlja zasebno, potrebno je koristiti kursor. Kursor je objekt koji omogućava dohvat pojedinog zapisa iz rezultata SELECT upita. Služi isključivo za dohvat podataka, a ne za ažuriranje, unos niti brisanje podataka. U slučajevima kada se pomoću naredbe SELECT dohvaća više od jednog zapisa, koristi se kursor za iterativnu obradu svakog zapisa zasebno. Kursori su podržani u okviru pohranjenih procedura i funkcija. MySQL omogućava slijedni dohvat od prvog do zadnjeg zapisa, pri čemu se kursor ne može koristiti za čitanje prethodnih zapisa niti skočiti na točno određeni zapis.

30.1. KADA KORISTITI KURSOR?

Kursori se u pravilu koriste kada u svakoj iteraciji dohvata n-torke postoji potreba za obradom zapisa u više tablica ili složenijom obradom dohvaćenog zapisa iz jedne tablice. Također se koriste kada se određene operacije ne moraju izvršiti nad svim dohvaćenim zapisima već samo nad onima koji zadovoljavaju određeni uvjet.

U toj situaciji moguće je prilikom dohvata podatka s kursorom te dodatnim uvjetovanjima ili kontrolom toka pohranjene rutine, efikasno izvršiti potrebnu logiku. Oni se u pravilu koriste u sklopu pohranjenih rutina.

30.2. KADA NE KORISTITI KURSOR?

Svakako bi se trebalo izbjegavati korištenje kursora u slučajevima kada se može napisati SQL upit koji je u mogućnosti obraditi sve potrebne zapise odjednom. Rutina se optimizira ako se umjesto kursora koristi složeniji SQL upit, naravno u slučajevima kada je to izvedivo.

Nuspojava korištenja kursora je usporavanje izvođenja rutine jer se vrti petlja unutar koje se prilikom svake iteracije izvršava niz određenih naredbi. Unutar svake iteracije petlje, podatak iz n-torke se zapisuje u lokalnu varijablu te se daljnji izračuni u glavnom izvršavaju pomoću njih. Druga je nuspojava složenost koda i otežano debugiranje za rutine koje koriste kursor.

30.3. RAD S KURSOROM

Rad s kursorom izvodi se u četiri glavna koraka čija su obilježja prikazana u idućoj tablici.

Tablica 32. Rad s kursorom

Naredba	Akcija
DECLARE	Deklaracija skupa podataka koje dohvaća kursor
OPEN	Dohvat skupa podataka prema deklaraciji
FETCH	Dohvat pojedine n-torke iz skupa
CLOSE	Zatvaranje kursora i oslobađanje resursa

30.3.1. Deklaracija kursora

Deklaracija kursora služi za definiranje skupa podataka kroz koji će se kursorom iterativno prolaziti. Kursor se deklarira na način da se nakon dodjeljivanja imena definira SELECT naredba koja vraća skup n-torki.

Osnovna sintaksa:

```
DECLARE ime_kursora CURSOR FOR  
SELECT _izraz;
```

Za kursor nije nužno da dohvaća n-torke iz jedne tablice, već one koje su rezultat SELECT naredbe koliko god ona složena bila. Važno je da se kursor deklarira na početku procedure odnosno funkcije i to prije prve naredbe, ali nakon što su deklarirane sve lokalne varijable koje će se koristiti u samoj proceduri ili funkciji. Iznimno je bitno poštivati ovaj redoslijed jer će u suprotnom doći do sintaksne pogreške.

Primjerice potrebno je obaviti određene operacije nad zapisima u tablici *studenti*. Kursor će biti deklariran na način da dohvaća samo identifikator studenta (jmbag) i poštanski broj prebivanja. Ako bi prilikom daljnjeg rada bilo potrebno raditi isključivo s podacima o prebivanju, svakako je preporučljivo dohvaćati s kursorom i stupac ili skup stupaca koje će **jednoznačno definirati** svaku dohvaćenu n-torku u dohvaćenom skupu.

```
DECLARE mojKursor CURSOR FOR  
SELECT jmbag, postBrPrebivanje FROM studenti;
```

30.3.2. Otvaranje kursora

Nakon deklaracije kursor je potrebno otvoriti naredbom OPEN. Time se izvršava SELECT naredba iz deklaracije te se podaci dohvaćaju u spremnik kroz koji će se iterativno prolaziti i čitati podatke. To je zapravo privremena tablica u kojoj se nalaze podatci s kojima kursor dalje radi.

Osnovna sintaksa:

```
OPEN ime_kursora;
```

30.3.3. Dohvat podataka iz kursora

Podaci se u ovom koraku dohvaćaju iz pojedine n-torke koja je dio skupa rezultata dobivenih otvaranjem kursora. Za dohvat se koristi naredba FETCH.

Osnovna sintaksa:

```
FETCH ime_kursora INTO var_1 [, var_2, ..., var_n];
```

Kursor funkcionira kao pokazivač koji pokazuje na trenutnu n-torku u skupu rezultata, međutim on ne zna sam u kojem trenutku treba dohvatiti sljedeću n-torku. Mehanizam

kursora funkcionira na način da nakon što se kursor otvori, pokreće se petlja te se prilikom svake iteracije petlje naredbom FETCH dohvaća podatke iz trenutne n-torke. Podaci će se tom prilikom pohraniti u lokalne varijable nad kojima će se obavljati daljnje operacije. Važno je deklarirati točno onoliko lokalnih varijabli koliko se atributa čita SELECT naredbom definiranom u deklaraciji kursora.

Primjer:

```
FETCH mojKursor  
    INTO jmbag_tmp, postBrPrebivanje_tmp;
```

U gornjem će primjeru kursor prilikom svake iteracije petlje dohvaćati jmbag te poštanski broj prebivanja studenta na kojeg se odnosi trenutna n-torka u tablici *studenti* te ih pohraniti u lokalne varijable *jmbag_tmp* te *postBrPrebivanje_tmp*.

Primjer:

```
DECLARE mojKursor CURSOR FOR  
    SELECT * FROM studenti;
```

Deklaracija kursora na ovakav način jest moguća ali nije preporučljiva jer zvjezdicom nismo točno definirali broj atributa koje dohvaćamo te ne možemo biti sigurni koliko je lokalnih varijabli potrebno koristiti prilikom FETCH naredbe. Ako kasnije prilikom rada s bazom podataka dođe do promjene broja atributa u tablici *studenti*, broj dohvaćenih atributa sa zvjezdicom i broj predviđenih varijabli unutar rutine više neće odgovarati te će svaki poziv rutine rezultirati pogreškom.

30.3.4. Zatvaranje kursora

Po završetku rada s kursorom potrebno ga je zatvoriti čime će se osloboditi resursi zauzeti kursorom. Naredba za zatvaranje kursora je CLOSE. Iako DBMS garantira da će prilikom završetka procedure odnosno funkcije resursi automatski biti oslobođeni, preporuka je ipak eksplicitno zatvoriti kursor naredbom CLOSE.

Osnovna sintaksa:

```
CLOSE ime_kursora;
```

30.3.5. Primjer

Primjer 1. dio

Napisati proceduru koja će koristeći kursor dohvatiti i ispisati za svaku *ustanovu* njen oib, naziv te opis radi li se o starijoj ili novijoj ustanovi. Ustanova će se u ovom primjeru proglasiti starom ako je osnovana prije 20 ili više godina u odnosu na trenutnu godinu i godinu osnutka.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS etiketiraj$$
CREATE PROCEDURE etiketiraj()
BEGIN
    DECLARE t_oib CHAR(11);
    DECLARE t_naziv VARCHAR(45);
    DECLARE t_osnutak DATE;
    DECLARE k_ustanove CURSOR FOR
        SELECT oib, naziv, datumOsnutka FROM ustanove;
    OPEN k_ustanove;
    petlja:LOOP
        FETCH k_ustanove INTO t_oib, t_naziv, t_osnutak;
        IF YEAR(CURDATE()) - YEAR(t_osnutak) >= 20
            THEN SELECT t_oib AS oib, t_naziv AS naziv,
                'starija ustanova' AS opis;
        ELSE
            SELECT t_oib AS oib, t_naziv AS naziv,
                'novija ustanova' AS opis;
        END IF;
    END LOOP petlja;
    CLOSE k_ustanove;
    SELECT 'kraj'; /* naredba dodana za provjeru izvođenja
procedure */
END; $$
DELIMITER ;
```

Primjer poziva procedure:

```
CALL etiketiraj();
```

Deklaracijom kursora *k_ustanove* odnosno naredbom OPEN u privremeni spremnik bit će povučeni podaci kako je prikazano u nastavku (devet n-torki). Sadržaj tog spremnika nigdje

se ne ispisuje – ovdje je ispisan njegov sadržaj pomoću naredbe iz deklaracije kursora (SELECT oib, naziv, datumOsnutka FROM ustanove) radi lakšeg razumijevanja mehanizma rada kursora.

☐ oib	naziv	datumOsnutka
☐ 02024882310	Fakultet organizacije i informatike	1974-03-29
☐ 08814003451	Tehničko Veleučilište u Zagrebu	1998-05-27
☐ 09032023114	Veleučilište Velika Gorica	2002-06-12
☐ 21720825730	Veleučilište "Ladislava Ružička"	2005-07-15
☐ 28163265527	Prirodoslovno-matematički fakultet	1946-06-08
☐ 31444990605	Međimursko veleučilište u Čakovcu	2005-09-24
☐ 57029260362	Fakultet elektrotehnike i računarstva	1956-06-08
☐ 76722145702	Pomorski fakultet	1914-11-26
☐ 84477684422	Ekonomski fakultet	1974-06-15



Prilikom prve iteracije, kursor pokazuje na prvu n-torku

Sadržaji atributa oib, naziv i datumOsnutka u svakoj se iteraciji upisuju u varijable t_oib, t_naziv i t_osnutak

Petlja LOOP izvodila bi se beskonačno puta jer nije definiran broj iteracija niti su korištene naredbe LEAVE ili ITERATE. U ovom slučaju izvoditi će se dok ne dođe do posljednjeg zapisa u dohvaćenom skupu podataka (u tablici *ustanove*). Iteriranjem će se devet puta ispisati podatci o ustanovi te opis 'starija ustanova' odnosno 'novija ustanova'. Primijetiti da s obzirom da je SELECT naredba pozvana devet puta, dobiva se i devet rezultata. Rezultat izvođenja poziva procedure je u nastavku:

☐ oib	naziv	opis
☐ 02024882310	Fakultet organizacije i informatike	starija ustanova

☐ oib	naziv	opis
☐ 08814003451	Tehničko Veleučilište u Zagrebu	novija ustanova

☐ oib	naziv	opis
☐ 09032023114	Veleučilište Velika Gorica	novija ustanova

☐ oib	naziv	opis
☐ 21720825730	Veleučilište "Ladislava Ružička"	novija ustanova

...

Kada započne iteracija s rednim brojem 10, naredba **FETCH** k_ustanove INTO t_oib, t_naziv, t_osnutak neće uspjeti dohvatiti podatak te će izvođenje rezultirati ispisom sljedeće poruke:

```
Query: call etiketiraj()
```

Error Code: 1329

No data - zero rows fetched, selected, or processed

```
Execution Time : 0.001 sec
Transfer Time  : 0.014 sec
Total Time     : 0.015 sec
```

Obzirom da procedura nije uspjela dohvatiti sljedeći podatak, nasilno se prekinulo njezino izvođenje. Dokaz za ovo je neizvršavanje naredbe SELECT 'kraj' – posljednje što je procedura vratila su podatci za Ekonomski fakultet, odnosno naredba SELECT 'kraj' nije se izvršila.

☐ oib	naziv	opis
☐ 84477684422	Ekonomski fakultet	starija ustanova

Naravno da ovakav ispis rezultata nije pregledan niti je pogodan kao forma za izvještaj, te će se u narednim primjerima dati unapređenje koda dok se postepeno objašnjava način rada kursora. Jedna od predloženih solucija za elegantniji ispis je unos ovih podataka u privremenu tablicu te ispis podataka iz privremene tablice. Navedeno će biti pokazano u poglavlju

Privremene tablice u pohranjenim zadatcima.

Primjer 2. dio

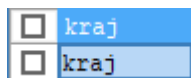
Unapređenje prethodnog primjera odnosno izbjegavanje pogreške uzrokovane prevelikim brojem iteracija petlje, moguće je izvesti tako da se prvo prebroji koliko će n-torki biti dohvaćeno prilikom otvaranja kursora te nakon toga zavrtiti petlju točno toliko puta (primjerice REPEAT...UNTIL, ili WHILE...DO).

```
DELIMITER $$
DROP PROCEDURE IF EXISTS etiketiraj$$
CREATE PROCEDURE etiketiraj()
BEGIN
    DECLARE t_oib CHAR(11);
    DECLARE t_naziv VARCHAR(45);
    DECLARE t_osnutak DATE;
    DECLARE i, n int;
    DECLARE k_ustanove CURSOR FOR
    SELECT oib, naziv, datumOsnutka FROM ustanove;
    set i = 0;
    SELECT COUNT(*) INTO n FROM ustanove;
    OPEN k_ustanove;
    WHILE i<n DO
        FETCH k_ustanove INTO t_oib, t_naziv, t_osnutak;
        IF YEAR(CURDATE()) - YEAR(t_osnutak) >= 20
            THEN SELECT t_oib AS oib, t_naziv AS naziv,
                'starija ustanova' AS opis;
        ELSE
            SELECT t_oib AS oib, t_naziv AS naziv,
                'novija ustanova' AS opis;
        END IF;
        SET i=i+1;
    END WHILE;
    CLOSE k_ustanove;
    SELECT 'kraj'; /*naredba dodana za provjeru izvođenja
procedure*/
END; $$
DELIMITER ;
```

Primjer poziva procedure:

```
CALL etiketiraj();
```

U ovom će se primjeru petlja iterirati točno devet puta te nakon izvršavanja petlje neće biti nikakve pogreške niti prekida, nego će procedura nastaviti s izvođenjem te ispisati 'kraj'.



Primjer 3. dio

Umjesto prebrojavanja n-torki agregatnom funkcijom COUNT(*) moguće je upotrijebiti ugrađenu funkciju FOUND_ROWS(). Ona vraća broj n-torki koje su rezultat prethodnog SQL upita. U ovom će slučaju dohvatiti broj n-torki dohvaćenih naredbom OPEN *k_ustanove* te ga pohraniti u varijablu *n*. Točno će se toliko iteracija petlje izvršiti koliko je zapisano u varijabli *n*. Ako je uspješno izvršen SELECT upit iz deklaracije kursora, taj će taj broj odgovarati ukupnom broju ustanova.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS etiketiraj$$
CREATE PROCEDURE etiketiraj()
BEGIN
    DECLARE t_oib CHAR(11);
    DECLARE t_naziv VARCHAR(45);
    DECLARE t_osnutak DATE;
    DECLARE i, n int;
    DECLARE k_ustanove CURSOR FOR
        SELECT oib, naziv, datumOsnutka FROM ustanove;
    SET i = 0;
    SELECT found_rows() INTO n;
    OPEN k_ustanove;
    WHILE i < n do
        FETCH k_ustanove INTO t_oib, t_naziv,
t_osnutak;

        IF YEAR(CURDATE()) - YEAR(t_osnutak) >= 20
            THEN SELECT t_oib AS oib, t_naziv AS
naziv,
                'starija ustanova' AS opis;
        ELSE
```

```
        SELECT t_oib AS oib, t_naziv AS naziv,  
              'novija ustanova' AS opis;  
    END IF;  
    SET i=i+1;  
END WHILE;  
CLOSE k_ustanove;  
SELECT 'kraj'; /*naredba dodana za provjeru izvođenja  
procedure*/  
END; $$  
DELIMITER ;
```

Primjer poziva procedure:

```
CALL etiketiraj();
```

Drugi način izbjegavanja nasilnog prekida procedure i poruke o pogreški za nemogućnost dohvata iduće n-torke je hvatanjem oznake o pogreški te reagiranjem na odgovarajući način uz pomoć *handler-a*. Implementacija ovog mehanizma opisana je u u idućem poglavlju.

31. Conditioni i handleri

31.1. RAD S CONDITIONIMA

Prilikom rada s pohranjenim zadatcima biti će potrebe definirati ponašanje pohranjenog zadatka u slučaju da dođe do određenih unaprijed predviđenih događaja odnosno pogrešaka. U tom slučaju moguće je definirati izuzetke (engl. *condition*), za određene situacije, dodijeliti im naziv te koristiti te izuzetke u daljnjim provjerama toka izvođenja pohranjenog zadatka. Hrvatska inačica *conditiona* kao objekta u svijetu baza podataka može biti *izuzetak*, no obzirom da je to izraz koji nije ustaljen u kontekstu baza podataka, u ovom tekstu koristiti će se engleska inačica riječi odnosno *condition*.

Osnovna sintaksa:

```
DECLARE ime CONDITION FOR uvjet
uvjet:
    mysql_error_code
| SQLSTATE [VALUE] sqlstate_value
```

Condition je potrebno deklarirati prije deklaracije kursora i *handlera*. Deklarira se za odrađeni uvjet koji može biti:

- SQLSTATE vrijednost ili
- mysql_error_code.

SQLSTATE vrijednost je string od pet znamenaka koji je ANSI SQL standardiziran. SQLSTATE vrijednosti mogu se grupirati na sljedeći način:

- Kod za uspješno izvršavanje – '00000' odnosno '0' (ne preporuča se koristiti)
- SQLWARNING = SQLSTATE koji počinje sa '01'
- NOT FOUND = SQLSTATE koji počinje sa '02' (relevantno u kontekstu kursora, definira se ponašanje za trenutak kada kursor dođe do posljednje n-toke u skupu podataka)
- SQLEXCEPTION = sve što ne počinje sa '00', '01' ili '02'

Kodovi svih MySQL pogrešaka odnosno **mysql_error_code** mogu se pronaći na službenim stranicama MySQL-a odnosno na poveznici <http://dev.mysql.com/doc/refman/5.7/en/error-messages-server.html> (ili drugoj ovisno o verziji MySQL-a koja je vidljiva u URL-u) a pregled češćih dan je u nastavku.

Tablica 33. Popis MySQL poruka po pogreški i SQLSTATE kodova

MySQL error code	SQLSTATE code	Poruka po pogreški
1011	HY000	Error on delete of '%s' (errno: %d)
1021	HY000	Disk full (%s); waiting for someone to free some space ...
1022	23000	Can't write; duplicate key in table '%s'
1027	HY000	'%s' is locked against change
1036	HY000	Table '%s' is read only
1048	23000	Column '%s' cannot be null
1062	23000	Duplicate entry '%s' for key %d
1099	HY000	Table '%s' was locked with a READ lock and can't be updated
1100	HY000	Table '%s' was not locked with LOCK TABLES
1104	42000	The SELECT would examine more than MAX_JOIN_SIZE rows; check your WHERE and use SET SQL_BIG_SELECTS=1 or SET SQL_MAX_JOIN_SIZE=# if the SELECT is okay
1106	42000	Incorrect parameters to procedure '%s'
1114	HY000	The table '%s' is full
1150	HY000	Delayed insert thread couldn't get requested lock for table %s
1165	HY000	INSERT DELAYED can't be used with table '%s' because it is locked with LOCK TABLES
1242	21000	Subquery returns more than 1 row
1263	22004	Column set to default value; NULL supplied to NOT NULL column '%s' at row %ld
1264	22003	Out of range value adjusted for column '%s' at row %ld
1265	1000	Data truncated for column '%s' at row %ld
1312	0A000	SELECT in a stored program must have INTO
1317	70100	Query execution was interrupted
1319	42000	Undefined CONDITION: %s
1325	24000	Cursor is already open
1326	24000	Cursor is not open
1328	HY000	Incorrect number of FETCH variables
1329	2000	No data to FETCH

1336	42000	USE is not allowed in a stored program
1337	42000	Variable or condition declaration after cursor or handler declaration
1338	42000	Cursor declaration after handler declaration
1339	20000	Case not found for CASE statement
1348	HY000	Column '%s' is not updatable
1357	HY000	Can't drop a %s from within another stored routine
1358	HY000	GOTO is not allowed in a stored program handler
1362		
1363		

Većina pogrešaka može se definirati na oba navedena načina. U teoriji, korištenje SQLSTATE vrijednosti koja je univerzalna za SQL jezik, rezultirat će da je sam kod lakše prenijeti na druge platforme i samim time bit će bolji izbor. S druge je strane u praksi mala vjerojatnost da će se pohranjeni zadaci prenositi na drugi RDBMS. Jezik pohranjenih procedura kod Oracle i SQL Servera u potpunosti je nekompatibilan s MySQL-om, dok je DB2 djelomično kompatibilan. Također je činjenica da svi MySQL kodovi pogreške nemaju odgovarajuću SQLSTATE vrijednost pa je preporuka da se *conditioni* radije deklariraju za određeni MySQL kod za pogrešku.

Primjeri izrađenih *conditiona*:

```
DECLARE err_nepostojeca_tablica CONDITION FOR 1050;
```

```
DECLARE err_strani_kljuc CONDITION FOR 1216;
```

```
DECLARE err_nepostojeca_tablica CONDITION FOR SQLSTATE '42S01';
```

31.2. RAD S HANDLERIMA

Hvatač (engl. *handler*) je objekt u bazi podataka koji služi za hvatanje poruke o određenoj pogreški nakon čega reagira na definirani način. Hrvatska inačica *handlera* kao objekta u svijetu baza podataka može biti *hvatač*, no obzirom da je to izraz koji također nije ustaljen u kontekstu baza podataka, u ovom tekstu koristiti će se engleska inačica izraza odnosno *handler*.

Prilikom deklaracije *handlera* obavezno je definirati na koju će se pogrešku *handler* odnositi, što se može učiniti na dva načina.

[1] Prvi način je definiranje uvjeta pomoću *CONDITION* klauzule.

[2] Drugi način je umetanje oznake o pogreški u sam *handler*.

Uvjet odnosno *condition* u kojem se *handler* aktivira preporuča se koristiti zbog dodjeljivanja imena uvjetu što će rezultirati boljom preglednošću procedure, pogotovo u slučajevima kada je potrebno definirati više *handlers* za više različitih uvjeta.

Nakon što je definiran uvjet u kojem se *handler* pokreće, potrebno je deklarirati i sam *handler* za zadani tip pogreške. *Handler* je u tijelu pohranjenog zadatka obavezno potrebno deklarirati nakon deklaracije varijabli i *conditiona*, a nikako prije.

Osnovna sintaksa:

```
DECLARE akcija HANDLER
    FOR uvjet [, uvjet_2, uvjet_3] ...
    izraz

akcija:
    CONTINUE
| EXIT
| UNDO

uvjet:
    mysql_error_code
| SQLSTATE [VALUE] sqlstate_value
| condition_name
| SQLWARNING
| NOT FOUND
| SQLEXCEPTION
```

U *akcija* dijelu definirana je akcija koja se obavlja prilikom pokretanja *handlera*:

- CONTINUE – nastavlja se izvođenje trenutnog pohranjenog zadatka od dijela nakon naredbe koja je aktivirala *handler*
- EXIT – prekida se izvođenje svih naredbi definiranih unutar BEGIN ... END bloka, završava se s izvođenjem pohranjenog zadatka
- UNDO – u planu je da bude implementiran u nekim od idućih verzija MySQL-a

U nastavku je definiran *handler* koji reagira na pogrešku definiranu *conditionom* *err_nepostojeca_tablica* uvjetom iz prethodnog poglavlja.

```
DECLARE err_nepostojeca_tablica CONDITION FOR 1050;
DECLARE EXIT HANDLER FOR err_nepostojeca_tablica
BEGIN
```

```
SET @table=-1;  
END;
```

U slučaju događaja pogreške *err_nepostojeca_tablica*, globalna se varijabla *@table* postavlja na vrijednost -1, odnosno čitanjem vrijednosti te varijable u samoj proceduri moguće je utvrditi je li došlo do pogreške 1050. Operacije koje je potrebno izvršiti prilikom aktiviranja *handlera* definirane su unutar BEGIN ... END bloka. Ako će se izvršiti samo jedna operacija tada je moguće izostaviti BEGIN ... END oznake.

Osim što je moguće za svaki uvjet definirati *condition* i dodijeliti mu odgovarajuće ime, deklaraciju *conditiona* i *handle-a* moguće je objediniti u samo jednu deklaraciju. Gornji primjeri deklaracije *condition-a* i *handler-a* tada će izgledati kao u nastavku:

```
DECLARE EXIT HANDLER FOR 1050  
BEGIN  
SET @table=-1;  
END;
```

31.3. PRIMJERI DEKLARACIJE I UPOTREBE HANDLERA

1. Ako se pojavi bilo koja pogreška (osim NOT FOUND), nastavlja se s izvođenjem pohranjenog zadatka uz postavljanje vrijednosti lokalne varijable *_error* na vrijednost 1:

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION  
SET _error=1;
```

2. Ako se pojavi bilo koja pogreška (osim NOT FOUND), prekida se izvršavanje trenutnog bloka pohranjenog zadatka izvođenjem ROLLBACK naredbe te ispisivanjem poruke o pogreški:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION  
BEGIN  
ROLLBACK;  
SELECT 'Dogodila se pogreška - završavam s radom';  
END;
```

3. Ako se pojavila pogreška 1062 (duplicirana vrijednost ključa), nastavlja se s izvođenjem pohranjenog zadatka uz ispis odgovarajuće pogreške:

```
DECLARE CONTINUE HANDLER FOR 1062
    SELECT 'Pogreška dupliciranog unosa';
```

4. Ako se pojavila SQLSTATE pogreška 23000 (duplicirana vrijednost ključa), nastavlja se s izvođenjem pohranjenog zadatka uz ispis odgovarajuće pogreške:

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '23000'
    SELECT 'Pogreška dupliciranog unosa';
```

5. Ako je došlo do SQLEXCEPTION odnosno bilo kojeg SQLSTATE čija vrijednost ne počinje sa '00', '01', or '02', postavi globalnu varijablu @exc.

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
    SET @exc=1;
END;
```

6. Ako kursor naredbom FETCH nije uspio dohvatiti podatak, nastavlja se s izvođenjem ostatka pohranjenog zadatka uz postavljanje varijable l_error na vrijednost 1:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET l_done=1;
```

Ili

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '02000'
    SET l_done=1;
```

Ili

```
DECLARE CONTINUE HANDLER FOR 1329
    SET l_done=1;
```

7. Ako dođe do unosa zapisa s dupliciranim ključem, nastavlja se izvođenje i inkrementira se globalna varijabla @x.

```
CREATE TABLE test.t (s1 INT, PRIMARY KEY (s1));

DELIMITER //
CREATE PROCEDURE duplicate()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLSTATE "23000"
        SET @x=1;
    SET @x=1;
```

```

INSERT INTO test.t VALUES (1);
SET @x=2;
INSERT INTO test.t VALUES (1);
SET @x=3;
END;
//
DELIMITER ;

```

Poziv procedure:

```

CALL duplicate ();
SELECT @x;

```

Izvođenjem procedure varijabla @x dobiva vrijednost 3 što znači da INSERT naredba koja je rezultirala greškom radi dupliciranog unosa po ključu nije prekinula nastavak izvršavanja procedure, već se procedura nastavila izvoditi zahvaljujući *continue handleru*.

☐	@x
☐	3

31.4. PRIMJENA CONDITIONA I HANDLERA U KURSORIMA

31.4.1. Primjer iz poglavlja o kursorima

Primjer iz poglavlja u kojemu se objašnjavaju kursori, dorađen sa *conditionima* i *handlerima* izgledati će kao u nastavku. Prisjetimo se, u primjeru se ispisuju ustanove s oznakom radi li se o novijoj ili starijoj godini prema kriteriju osnutka u posljednjih 20 godina.

```

DELIMITER $$
DROP PROCEDURE IF EXISTS etiketiraj$$
CREATE PROCEDURE etiketiraj()
BEGIN
    DECLARE t_oib CHAR(11);
    DECLARE t_naziv VARCHAR(45);
    DECLARE t_osnutak DATE;
    DECLARE greska BOOL DEFAULT FALSE;
    DECLARE k_ustanove CURSOR FOR
        SELECT oib, naziv, datumOsnutka FROM ustanove;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET greska=TRUE;
    OPEN k_ustanove;

```

```

        petlja:LOOP
        FETCH k_ustanove INTO t_oib, t_naziv, t_osnutak;
        IF greska = TRUE
            THEN LEAVE petlja;
            /*provjerava pojavu pogreške NOT FOUND - važno
napraviti odmah nakon FETCH naredbe*/
        END IF;
        IF YEAR(CURDATE()) - YEAR (t_osnutak) >= 20
            THEN SELECT t_oib AS oib, t_naziv AS naziv,
                'starija ustanova' AS opis;
        ELSE
            SELECT t_oib AS oib, t_naziv AS naziv,
                'novija ustanova' AS opis;
        END IF;
        END LOOP petlja;
    CLOSE k_ustanove;
    SELECT 'kraj'; /*naredba dodana za provjeru izvođenja
procedure*/
END; $$
DELIMITER ;

```

Poziv procedure:

```
CALL etiketiraj();
```

U slučaju pogreške NOT FOUND koja će se pojaviti nakon što je kursor obradio posljednju n-torku iz skupa podataka, odnosno prilikom dohвата iduće nepostojeće n-toke, aktivirat će se *continue handler* postavljajući zastavicu *greska* na vrijednost TRUE. Ako je s procedurom sve u redu, *greska* ima pretpostavljenu vrijednost FALSE. Unutar same petlje preporuča se provjeravanje zastavice *greska* odmah nakon dohвата n-torke pomoću naredbe FETCH kako bi *handler* reagirao u odgovarajućem trenutku. U primjeru je to učinjeno pomoću IF uvjeta, kojim se definira da ako je došlo do pogreške NOT FOUND (*greska* =TRUE), tada se izlazi iz petlje i nastavlja s ostalim naredbama u samoj proceduri.

Obzirom da se radi o samo dva moguća stanja (je li došlo do pogreške ili nije), ovdje je korišten BOOL tip podatka za varijablu *greska*. Dotičnoj je varijabli moguće postaviti željeni tip podatka ovisno o potrebnoj funkcionalnosti, boolean kao što je ovdje, tekstualni tip sa porukom i slično.

31.4.2. Primjeri

Tablici studenti dodati novi atribut *polozeno* inicijalno postavljen na -1. Napisati proceduru koja će u novostvoreni atribut upisati koliko je kolegija student položio sa prolaznom ocjenom.

```
ALTER TABLE studenti ADD COLUMN polozeno INT DEFAULT -1;
```

Nakon dodavanja stupca položeno, pregled tablice studenti je u nastavku.

jmbag	ime	prezime	datumUpisa	postBrPrebivanje	postBrStanovanja	idSmjer	polozeno
0010081356	Tanja	Krog	2010-09-28	10000	10290	3	-1
0013020125	Matea	Omanović	2012-09-25	10410	44000	9	-1
0013022546	Antun	Mandić	2010-09-30	10410	31000	10	-1
0013025926	Antonio	Pavić	2012-09-25	32000	44000	5	-1
0013026584	Borna	Knežević	2010-09-29	10410	44000	9	-1
0016096972	Dora	Vidaković	2011-09-26	42000	44000	16	-1
0016097990	Vesna	Đukić	2011-09-29	42000	43000	17	-1
0016098017	Marina	Zadro	2011-09-28	42000	47000	17	-1
0016099969	Antonia	Čoić	2011-09-28	42000	35000	17	-1
0016100313	Biljana	Vidović	2011-09-27	42000	35000	16	-1
0016101784	David	Vučetić	2011-09-27	42000	42000	16	-1
0016199452	Bernarda	Tomić	2011-09-30	42000	42000	17	-1

... (ukupno 162 n-torke)

```
DELIMITER $$
DROP PROCEDURE IF EXISTS evidentirajProlaze$$
CREATE PROCEDURE evidentirajProlaze()
BEGIN
    DECLARE t_jmbag CHAR(10);
    DECLARE n_stud, t_n_ocj INT;
    DECLARE not_found BOOL DEFAULT FALSE;
    DECLARE k_stud CURSOR FOR
        SELECT studenti.jmbag FROM studenti;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET not_found = TRUE;
    SELECT FOUND_ROWS() INTO n_stud;
    OPEN k_stud;
    petlja:LOOP
        FETCH k_stud INTO t_jmbag;
        IF not_found=TRUE THEN
            LEAVE petlja;
        END IF;
        SELECT COUNT(*) INTO t_n_ocj
        FROM studenti JOIN ocjene
            ON studenti.jmbag = ocjene.jmbagStudent
```



```

        WHERE studenti.jmbag = t_jmbag and ocjena > 1;
    UPDATE studenti SET polozeno = t_n_ocj
        WHERE jmbag = t_jmbag;
    END LOOP petlja;
    CLOSE k_stud;
END; $$
DELIMITER ;

```

Poziv procedure:

```
CALL evidentirajProlaze();
```

Nakon izvođenja procedure, stanje u tablici *studenti* prikazano je u nastavku – popunjen je atribut *polozeno*.

jmbag	ime	prezime	datumUpisa	postBrPrebivanja	postBrStanovanja	idSmjer	polozeno
0010081356	Tanja	Krog	2010-09-28	10000	10290	3	2
0013020125	Matea	Omanović	2012-09-25	10410	44000	9	1
0013022546	Antun	Mandić	2010-09-30	10410	31000	10	1
0013025926	Antonio	Pavić	2012-09-25	32000	44000	5	1
0013026584	Borna	Knežević	2010-09-29	10410	44000	9	1
0016096972	Dora	Vidaković	2011-09-26	42000	44000	16	1
0016097990	Vesna	Đukić	2011-09-29	42000	43000	17	2
0016098017	Marina	Zadro	2011-09-28	42000	47000	17	1
0016099969	Antonia	Čoić	2011-09-28	42000	35000	17	2
0016100313	Biljana	Vidović	2011-09-27	42000	35000	16	1
0016101784	David	Vučetić	2011-09-27	42000	42000	16	1
0016199452	Bernarda	Tomić	2011-09-30	42000	42000	17	2
0016200033	Zvonimir	Žarković	2010-09-29	21000	20000	19	1
0016201125	Denis	Šapina	2011-09-27	42000	42000	17	1

Primjer (nastavak):

Modificirati proceduru iz prethodnog zadatka da obradi samo one studente koji studiraju na zadanom smjeru Tehničkog veleučilišta u Zagrebu. Procedura neka vrati broj zapisa koji su dohvaćeni prilikom otvaranja kursora te broj zapisa koje je uistinu obradila (broj studenata za koje je prebrojala pozitivne ocjene).

Ovakav ispis poslužit će kao kontrola za optimizaciju koda – želimo u kursor dohvatiti isključivo one n-torke koje će se obrađivati u tijelu procedure. Iz tog razloga sve WHERE klauzule te uvjete oćenito postavljamo u SELECT upit koji se navodi već prilikom deklaracije kursora. Zaključno, već u početku želimo dohvatiti samo one studente koji studiraju na zadanom smjeru Tehničkog veleučilišta u Zagrebu.

Alternativno rješenje koje je lošije, bilo bi izraditi kursor koji dohvaća apsolutno sve studente pa tek unutar petlje za svakog studenta provjerava pripadnost zadanom smjeru Tehničkog veleučilišta u Zagrebu. Time narušavamo brzinu izvođenja procedure jer nepotrebno povećavamo broj iteracija u petlji za istu funkcionalnost.

```
DELIMITER $$
```

```

DROP PROCEDURE IF EXISTS evidentirajProlaze$$
CREATE PROCEDURE evidentirajProlaze(IN z_smjer INT)
BEGIN
    DECLARE t_jmbag CHAR(10);
    DECLARE t_n_ocj, dohvaceno, obradeno INT DEFAULT 0;
    /*obavezno je varijablu obradeno postaviti na 0 jer ju
    inkrementiramo u petlji pa mora imati početnu vrijednost -
    primijetiti da postavljanje na NULL nebi bilo ispravno jer NULL +
    1 nije moguća operacija */
    DECLARE not_found BOOL DEFAULT FALSE;
    DECLARE k_stud CURSOR FOR
        SELECT studenti.jmbag
        FROM studenti
        JOIN smjerovi ON studenti.idSmjer = smjerovi.id
        JOIN ustanove ON smjerovi.oibUstanova = ustanove.oib
        WHERE ustanove.naziv = "Tehničko veleučilište u
Zagrebu"
        AND smjerovi.id = z_smjer;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET not_found = TRUE;
    OPEN k_stud;
    SELECT FOUND_ROWS() INTO dohvaceno;
    petlja:LOOP
        FETCH k_stud INTO t_jmbag;
        IF not_found=TRUE THEN
            LEAVE petlja;
        END IF;
        SELECT COUNT(*) INTO t_n_ocj
        FROM studenti JOIN ocjene
            ON studenti.jmbag = ocjene.jmbagStudent
            WHERE studenti.jmbag = t_jmbag AND ocjena > 1;
        UPDATE studenti SET polozeno = t_n_ocj
            WHERE jmbag = t_jmbag;
        SET obradeno = obradeno + 1;
    END LOOP petlja;
    CLOSE k_stud;
    SELECT dohvaceno AS br_dohvacenih_zapisa,
        obradeno AS br_obradenih_zapisa;

```

```
END; $$
```

```
DELIMITER ;
```

Poziv procedure za smjer s id-om vrijednosti 1:

```
CALL evidentirajProlaze(1);
```

Procedura će obraditi samo sedam zapisa što je broj studenata na zadanom smjeru željene ustanove. Ispis iz procedure vidljiv je u nastavku, te potvrđuje ispravno izveden broj iteracija petlje LOOP.

☐ br_dohvacenih_zapisa	br_obradenih_zapisa
☐ 7	7

Primjer:

Izraditi funkciju koja će proći kroz sve zapise u tablici *kolegiji* koji se odnose na kolegije iz zadane *ustanove*. Ustanovu je potrebno zadati preko naziva (pri čemu ćemo procijeniti da ne postoje dvije ustanove identičnog naziva, odnosno da se ustanova može zadavati primjerice kroz padajući izbornik u aplikaciji koja bi koristila ovu funkciju). Funkcija treba u postojeću vrijednost atributa *opis* dodati opis '*negativan kolegij u godini zadana_godina – N negativnih*'. Funkcija kao ulazni parametar također prima godinu za koju broji negativne ocjene. Funkcija vraća 1 ako je uspjela obraditi sve potrebne (dohvaćene) zapise (kolegije), odnosno u suprotnom vraća 0.

```
DROP FUNCTION IF EXISTS negativni;
DELIMITER //
CREATE FUNCTION negativni(nazivUstanove VARCHAR(45), godina INT)
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE t_id, t_brOcjena, dohvat, obrada INT default 0;
    DECLARE kraj BOOL DEFAULT FALSE;
    DECLARE kursor CURSOR FOR
        SELECT k.id, COUNT(ocjena) FROM kolegiji k
        JOIN ocjene o ON k.id = idKolegij
        JOIN smjerovi s ON k.id = s.id
        JOIN ustanove u ON oibUstanova = oib
        WHERE u.naziv = nazivUstanove
        AND ocjena > 1 AND YEAR(datumPolaganja) = godina
        GROUP BY k.id
        HAVING COUNT(ocjena) > 0;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET kraj = TRUE;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION RETURN 0;
```

```

OPEN kursor;
SELECT FOUND_ROWS() INTO dohvat;
petlja:LOOP
    FETCH kursor INTO t_id, t_brOcjena;
    IF kraj = TRUE THEN LEAVE petlja; END IF;
    UPDATE kolegiji SET opis =
        CONCAT("negativan kolegij u godini ", godina, " - ",
            t_brOcjena, " negativnih; ", opis)
    WHERE kolegiji.id = t_id;
    SET obrada = obrada + 1;
END LOOP;
CLOSE kursor;
IF (dohvat = obrada) THEN RETURN 1;
ELSE RETURN 0;
END IF;
END ; //
DELIMITER ;

```

Poziv funkcije sa dva ulazna podatka:

```
SELECT negativni ('Tehničko Veleučilište u Zagrebu', 2013);
```

id	naziv	↑ opis		idSmjer
9	Poslovna etika	Cilj je predmeta upoznati studente s etičkim pitanjima suvremenog druš...	130B	5
40	Sredstva pomorskog prometa	Cilj kolegija je upoznati studente sa stabilnošću broda u neoštećenom ...	141B	20
25	Opća kemija	Čiste tvari. Homogene i heterogene smjese. Razdvajanje smjesa. Kemijsk...	164B	13
27	Digitalna logika	Digitalni sustavi veličine iz stvarnog svijeta, prethodno pretvorene u...	113B	14
34	Marketing	Marketing kao filozofija poslovanja; Priroda marketinškog poslovanja; ...	146B	17
3	Uvod u UNIX sustave	negativan kolegij u godini 2013 - 2 negativnih; Student treba savladat...	128B	2
4	Fizika	negativan kolegij u godini 2013 - 3 negativnih; Razumijevanje fizikaln...	145B	2
1	Baze podataka	negativan kolegij u godini 2013 - 3 negativnih; Student će steći znanj...	191B	1
2	Uvod u UNIX sustave	negativan kolegij u godini 2013 - 3 negativnih; Student treba savladat...	128B	1
43	Ijelesna i zdravstvena kultura 2	Odgojno-obrazovno područje podmićenja biopsihosocijalnih motiva za kre...	227B	1
44	Ijelesna i zdravstvena kultura 2	Odgojno-obrazovno područje podmićenja biopsihosocijalnih motiva za kre...	227B	10
45	Ijelesna i zdravstvena kultura 3	Odgojno-obrazovno područje podmićenja biopsihosocijalnih motiva za kre...	227B	16
10	Statistika	Opći ciljevi su ovladavanje teorijskim spoznajama kao i vještinama izr...	109B	5
15	Operacijski sustavi	Operacijski je sustav skup programa koji djeluju kao posrednici između...	119B	7

...

32. Privremene tablice u pohranjenim zadatcima

Privremena tablica je vrsta tablice koja se sastoji od definiranog broja atributa te može sadržavati određeni skup n-torki. Za razliku od *prave* tablice, privremena tablica nije trajno pohranjena u bazi podataka već je vezana uz sesiju u kojoj je definirana. Rad s privremenim tablicama (čitanje, pisanje, ažuriranje, brisanje) identičan je radu s tablicama pohranjenim u bazi.

Obzirom da je privremena tablica vezana uz sesiju odnosno korisnika koji je sesiju inicirao, moguće je da više korisnika istovremeno koristi različite privremene tablice istog imena. Korisnik iz druge sesije ne može pristupiti privremenoj tablici iz prve sesije. Zatvaranjem sesije u kojoj je privremena tablica napravljena, tablica će biti obrisana.

Spomenuto je kako su okidači mehanizmi koji se vežu za tablice u bazi podataka, no nije ih moguće vezati za privremene tablice.

Privremena tablica vezana je isključivo uz sesiju, ne postoji u sklopu baze te ju nije moguće izvesti (*backupirati*) zajedno s bazom. Namijenjena je privremenom radu s privremenim skupom podataka gdje se najčešće podatci unutar jedne logičke cjeline koja je vezana uz sesiju, iz privremene tablice ispisuju i obliku različitih izvještaja. Sličnoj namjeni mogu poslužiti i pogledi (*view*), koji mogu postojati u sklopu baze podataka ali su skromniji po pitanju logike koja se može odraditi kako bi se pogled izradio. U privremene tablice jednostavno je puniti podatke ili rezultate različitih izračuna unutar pohranjenog zadatka, dok je pogled definiran samo pomoću SELECT upita.

32.1. IZRADA PRIVREMENE TABLICE

DDL naredbe za stvaranje odnosno brisanje privremene tablice iste su kao naredbe za rad 'običnim' tablicama, osim što u sebi sadrže ključnu riječ TEMPORARY.

Osnovna sintaksa:

```
CREATE TEMPORARY TABLE
    ime_privremene_tablice (ime_atributa tip_podatka, ...)
```

Moguće je izraditi privremenu tablicu koja ima iste attribute kao neka tablica iz baze podataka na slijedeći način:

```
CREATE TEMPORARY TABLE
    ime_privremene_tablice
    LIKE ime_originalne_tablice
```

Osim preuzimanja strukture postojeće tablice prilikom stvaranja nove privremene tablice, moguće je preuzeti i podatke iz originalne tablice te njima napuniti privremenu. Navedeno se odnosi na sve podatke ili pak na projekciju pa i selekciju podataka.

```
CREATE TEMPORARY TABLE
    ime_privremene_tablice
    AS SELECT atribut_1 [, atribut_2, ..., atribut_n]
```

```
FROM ime_originalne_tablice  
WHERE uvjet;
```

32.2. BRISANJE TABLICE

Osnovna sintaksa za brisanje privremene tablice:

```
DROP TEMPORARY TABLE [IF EXISTS]  
ime_privremene_tablice
```

32.3. PRIMJERI

Napisati proceduru koja će za zadanu *ustanovu* ispisati sve kolegije te oznaku radi li se o teškom ili laganom kolegiju. Kriterij je prosječna ocjena svih studenata na kolegijima sa zadane ustanove. Ispisati abecedno po nazivu kolegija.

```
DROP PROCEDURE IF EXISTS tezinaKolegija;  
DELIMITER //  
CREATE PROCEDURE tezinaKolegija(nazivUstanove VARCHAR(45))  
BEGIN  
    DECLARE t_id, i, n_kursor INT DEFAULT 0;  
    DECLARE t_naziv VARCHAR(100) DEFAULT NULL;  
    DECLARE prosjek, t_prosjek DECIMAL (4,3);  
    DECLARE kursor CURSOR FOR  
        SELECT k.id, k.naziv, AVG(ocjena) FROM ocjene o  
        JOIN kolegiji k ON o.idKolegij = k.id  
        JOIN smjerovi s ON k.id = s.id  
        JOIN ustanove u ON s.oibUstanova = u.oib  
        WHERE u.naziv = 'Tehničko Veleučilište u Zagrebu'  
        GROUP BY k.id, k.naziv  
        ORDER BY k.naziv;  
    SELECT AVG(ocjena) INTO prosjek  
    FROM ocjene o  
    JOIN kolegiji k ON o.idKolegij = k.id  
    JOIN smjerovi s ON k.id = s.id  
    JOIN ustanove u ON s.oibUstanova = u.oib  
    WHERE u.naziv = nazivUstanove;  
    OPEN kursor;  
    SELECT FOUND_ROWS() INTO n_kursor;  
    WHILE (i < n_kursor) DO  
        FETCH kursor INTO t_id, t_naziv, t_prosjek;
```

```

        IF (t_prosjek > prosjek) THEN
            SELECT t_id AS IdKolegij, t_naziv AS NazivKolegij,
                   'težak kolegij' AS Opis;
        ELSE SELECT t_id AS IdKolegij, t_naziv AS
NazivKolegij,
                   'lagan kolegij' AS Opis;
        END IF;
        SET i = i + 1;
    END WHILE;
    CLOSE kursor;
END ; //
DELIMITER ;

```

Poziv procedure:

```
CALL tezinaKolegija('Tehničko Veleučilište u Zagrebu');
```

Ispis rezultata daje onoliko setova ispisa koliko postoji n-torki u spremnik podataka koji se stvorio prilikom izvođenja naredbe OPEN kursor, jer se prilikom svake iteracije petlje izvršava jedan SELECT. U nastavku je ispis.

☐ IdKolegij	NazivKolegij	Opis
☐ 1	Baze podataka	lagan kolegij

☐ IdKolegij	NazivKolegij	Opis
☐ 4	Fizika	težak kolegij

☐ IdKolegij	NazivKolegij	Opis
☐ 3	Uvod u UNIX sustave	lagan kolegij

☐ IdKolegij	NazivKolegij	Opis
☐ 2	Uvod u UNIX sustave	lagan kolegij

Poziv procedure rezultirat će sa četiri ispisa jer je toliko kolegija ocjenjivano na ustanovi. Uzork tomu je pozivanje SELECT naredbe unutar svake iteracije petlje, te svaka daje po jedan ispis (SELECT t_id AS IdKolegij, t_naziv AS NazivKolegij, 'težak kolegij' AS Opis i SELECT t_id AS IdKolegij, t_naziv AS NazivKolegij, 'lagan kolegij' AS Opis).

Ovakav je rezultat nepregledan i nepraktičan te se javlja potreba za optimizacijom strukture ispisa. Ideja je izbjeći ispisivanje podataka naredbom SELECT unutar svake iteracije petlje, privremeno ih pamtit, te ih ispisati tek na kraju procedure ili po završetku petlje. Ulogu privremenog spremnika podataka preuzet će privremena tablica.

Unutar tijela pohranjenog zadatka potrebno je izraditi privremenu tablicu sa tri potrebna atributa, te prilikom svake iteracije petlje unositi n-torku u privremenu tablicu umjesto da se

ta n-torka ispisuje naredbom SELECT. Po završetku rada s kursorom, privremena tablica će biti napunjena te se njen sadržaj može ispisati pomoću (jedne) naredbe SELECT.

```
DROP PROCEDURE IF EXISTS tezinaKolegija;
DELIMITER //
CREATE PROCEDURE tezinaKolegija(nazivUstanove VARCHAR(45))
BEGIN
    DECLARE t_id, i, n_kursor INT DEFAULT 0;
    DECLARE t_naziv VARCHAR(100) DEFAULT NULL;
    DECLARE prosjek, t_prosjek DECIMAL (4,3);
    DECLARE kursor CURSOR FOR
        SELECT k.id, k.naziv, AVG(ocjena) FROM ocjene o
        JOIN kolegiji k ON o.idKolegij = k.id
        JOIN smjerovi s ON k.id = s.id
        JOIN ustanove u ON s.oibUstanova = u.oib
        WHERE u.naziv = 'Tehničko Veleučilište u Zagrebu'
        GROUP BY k.id, k.naziv
        ORDER BY k.naziv;
    SELECT AVG(ocjena) INTO prosjek
    FROM ocjene o
    JOIN kolegiji k ON o.idKolegij = k.id
    JOIN smjerovi s ON k.id = s.id
    JOIN ustanove u ON s.oibUstanova = u.oib
    WHERE u.naziv = nazivUstanove;
    DROP TEMPORARY TABLE IF EXISTS kategoriziranjeKolegija;
    CREATE TEMPORARY TABLE kategoriziranjeKolegija
        (id INT, naziv VARCHAR(100), tezina VARCHAR(20));
    OPEN kursor;
    SELECT FOUND_ROWS() INTO n_kursor;
    WHILE (i < n_kursor) DO
        FETCH kursor INTO t_id, t_naziv, t_prosjek;
        IF (t_prosjek > prosjek) THEN
            /* SELECT t_id AS IdKolegij, t_naziv AS NazivKolegij,
            'težak kolegij' AS Opis; */
            INSERT INTO kategoriziranjeKolegija VALUES
                (t_id, t_naziv, 'težak kolegij');
```



```

        /* ELSE SELECT t_id AS IdKolegij, t_naziv AS
NazivKolegij, 'lagan kolegij' AS Opis; */

        ELSE INSERT INTO kategoriziranjeKolegija VALUES

            (t_id, t_naziv, 'lagan kolegij');

    END IF;

    SET i = i + 1;

END WHILE;

CLOSE kursor;

SELECT * FROM kategoriziranjeKolegija;

END ; //

DELIMITER ;

```

Poziv procedure:

```
CALL tezinaKolegija('Tehničko Veleučilište u Zagrebu');
```

Ispis iz procedure koji je zapravo ispis sadržaja privremene tablice:

id	naziv	tezina
1	Baze podataka	lagan kolegij
4	Fizika	težak kolegij
3	Uvod u UNIX sustave	lagan kolegij
2	Uvod u UNIX sustave	lagan kolegij

Pokazano rješenje ujedno je i metoda kako iz funkcije koja inače vraća jedan rezultat možemo (prividno) vratiti više rezultata. Funkciji će se svakako definirati njen standardni izlaz, ali osim toga pohranjivat ćemo podatke u privremenoj tablici koje ćemo naknadno ispisati. Važno je da se sadržaj pohranjene tablice poziva izvan funkcije jer u suprotnom se dobiva poruka po pogreški da funkcija ne može vratiti više rezultata.

Error Code: 1415

Not allowed to return a result set from a function

Primjer:

Potrebno je napisati funkciju koja će ispisati koliko *studenata* prebiva i stanuje u zadanom *mjestu*. Također je potrebno ispisati i popis tih *studenata*. Mjesto je potrebno zadati preko poštanskog broja.

```

DELIMITER %%

DROP FUNCTION IF EXISTS pronadiStudente %%

```

```

CREATE FUNCTION pronadiStudente (ulaz INT) RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE kraj INT DEFAULT 0;
    DECLARE t_jmbag CHAR(10) DEFAULT NULL;
    DECLARE t_ime, t_prez VARCHAR(50) DEFAULT NULL;
    DECLARE k CURSOR FOR
        SELECT jmbag, ime, prezime
        FROM studenti
        WHERE postBrPrebivanje=ulaz AND postBrStanovanja =
ulaz;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET kraj=1;
    DROP TEMPORARY TABLE IF EXISTS studentiIzMjesta;
    CREATE TEMPORARY TABLE studentiIzMjesta
        (jmbag CHAR(10), ime VARCHAR(50), prezime
VARCHAR(50));
    OPEN k;
    vrsti:LOOP
        FETCH k INTO t_jmbag, t_ime, t_prez;
        IF kraj=1 THEN LEAVE vrsti;
        END IF;
        INSERT INTO studentiIzMjesta
            VALUES (t_jmbag, t_ime, t_prez);
    END LOOP;
    CLOSE k;
    RETURN (SELECT COUNT(*) FROM studentiIzMjesta);
END; %%
DELIMITER ;

```

Poziv funkcije i rezultat prebrojavanja:

```
SELECT pronadiStudente(10000);
```

Poziv funkcije će rezultirati ispisom broja studenata koji prebivaju i stanuju u zadanom mjestu. Popis svih studenata nalazi se u privremenoj tablici *studentiIzMjesta*.

☐	pronadiStudente(10000)
☐	17

Ispis sadržaja privremene tablice *studentiIzMjesta* koji se poziva izvan tijela funkcije.

```
SELECT * FROM studentiIzMjesta;
```

☐	jmbag	ime	prezime
☐	0035177990	Irena	Dabić
☐	0035179953	Tanja	Sunek
☐	0036450023	Darko	Zidanić
☐	0036452014	Alen	Martić
☐	0036460022	Ivan	Matić
☐	0036463254	Ivan	Zović
☐	0036466211	Nada	Marulin
☐	0246022345	Ivana	Babić
☐	0246024883	Ivana	Jelenčić
☐	0246033365	Marko	Noskov
☐	1191219570	Renato	Babojelić
☐	1191220782	Mare	Mistrić
☐	1191221406	Ana	Anđal
☐	1191221499	Ivan	Horvat
☐	1191230621	Valentina	Benšek
☐	1191231669	Nikola	Kirasić
☐	1191303785	Miro	Pavlović

33. Zaštita od neovlaštenog pristupa podacima

33.1. ZAŠTITA BAZE PODATAKA

Svaka baza podataka mora biti zaštićena od nedopuštenih radnji u svrhu očuvanja sigurnosti i integriteta podataka u bazi. Osiguravanje sigurnosti baze podataka odnosi se na osiguravanje ovlaštenosti korisnika za akcije koje pokušavaju izvesti. S druge strane, osiguravanje integriteta odnosi se na osiguravanje da su akcije koje korisnici pokušavaju izvesti ispravne. O oba slučaja moraju biti definirana pravila koja korisnici ne smiju narušiti.

Glavni vid zaštite je postavljanje ograničenja fizičkog pristupa do računala na kojem se podaci nalaze. Osim fizičke postoji i softverski vid zaštite koji se ugrađuje u DBMS. Jedan od načina zaštite je dodjeljivanje ovlaštenja definiranim korisnicima za određene radnje. Oni će podacima moći pristupiti isključivo koristeći odgovarajuću identifikaciju, odnosno kombinaciju korisničkog imena i lozinke.

Gotovo svi DBMS-ovi imaju mogućnost rada na Internetu. Bazama se može pristupiti s drugih računala koja se nalaze u drugim gradovima ili državama. Navedeno nije samo prednost već je ujedno i mana jer povlači pitanje dodatne zaštite od neovlaštenog pristupa podacima. Komunikacija na Internetu bazirana je na IP protokolu gdje računala međusobno komuniciraju preko paketa čija je struktura točno određena. Svaki IP paket dodjeljuje se jednoj vrsti Internet usluge odnosno komunikacije, a identificira ga se preko broja odnosno porta. Svaka usluga na Internetu ima svoj broj porta (FTP 20/21, SSH 22, Telnet 23, Mail 25, WWW 80, ...). Baze podataka također imaju svoje portove za komunikaciju. Tako je port MySQL baze port 3306.

Kako zaštititi bazu podataka na Internetu ?

- Isključiti mrežne mogućnosti DBMS-a
- Dopustiti pristup DBMS-u samo lokalnim programima (localhost)
- Dopustiti pristup DBMS-u samo računalima unutar lokalne mreže
- Dopustiti nekima, ali uz identifikaciju (korisnik/lozinka)
- Koristiti šifriranu komunikaciju (ssl/ssh, dvostruki ključevi, . . .)

U nastavku će biti opisani postupci stvaranja korisnika, dodjeljivanja lozinke te dodjeljivanja ovlaštenja korisnicima za strogo određene radnje u bazi podataka.

33.2. STVARANJE KORISNIKA

Svaki je korisnik u bazi podataka definiran kombinacijom korisničkog imena te oznake lokacije (adrese klijenta) s koje može pristupati podacima.

Oblik identifikacijske oznake klijenta:

```
'korisnickoIme'@'adresaKlijenta'
```

Oznaka adrese klijenta može biti:

- IP adresa - primjerice: 31.127.106.14

- Ime poslužitelja – primjerice: localhost
- Puni naziv domene - primjerice: somehost.example.com
- Oznaka % koja označava bilo koju adresu ili skup adresa – primjerice: '%', '34.190.68.%', '%.primjer.com'

Kako bi se dva korisnika razlikovala, nužno je da imaju različite obje oznake. To povlači da će korisnik 'admin'@'192.168.5.10' biti različit od korisnika 'admin'@'192.168.5.%'. Ako je korisnik definiran na način da mu nije navedena oznaka lokacije s koje se smije spajati (primjerice 'admin'), tada se pretpostavlja da bazi može pristupiti s bilo koje lokacije (primjerice 'admin'@'%').

Primjer:

Provjeriti podatke o trenutno spojenom korisniku.

```
SELECT CURRENT_USER();
/* ili */
SELECT USER();
```

☐ USER()
☐ root@localhost

Osnovna sintaksa:

```
CREATE USER imeKorisnika [IDENTIFIED BY [PASSWORD] 'lozinka']
[,imeKorisnika2 [IDENTIFIED BY [PASSWORD] 'lozinka2']] ...
```

Svaka promjena stvaranja i brisanja korisnika te dodjeljivanja i brisanja dozvola određenom korisniku zapisana je u DBMS-u u posebnoj bazi podataka naziva *mysql*. Stvaranje novog korisnika rezultirat će dodavanjem nove n-torke u tablici *mysql.user*. Dohvatom podataka iz te tablice moguće je provjeriti specifikacije određenog korisnika.

Primjer:

Potrebno je stvoriti novog korisnika *ihorvat* sa lokalnog poslužitelja te mu dodijeliti lozinku 1234.

```
CREATE USER 'ihorvat'@'localhost' IDENTIFIED BY '1234';
```

Izvršavanje gornje CREATE USER naredbe rezultirat će sljedećom n-torkom u tablici *mysql.user*:

☐ Host	User	Select_priv	Insert_priv	Update_priv
☐ localhost	ihorvat	N	N	N

Prema gornjoj sintaksi moguće je stvoriti korisnika te mu opcionalno odmah dodijeliti i lozinku. Ako mu se ne dodijeli lozinka, u sigurnosnom pogledu to je nedostatak u sustavu, te odgovornost ostaje na administratoru da prilikom kreiranja korisnika odnosno najkasnije prilikom dodjeljivanja dozvola, dodijeli korisniku i odgovarajuću lozinku. Ako se lozinka i ne dodijeli u početku, moguće ju je naknadno dodijeliti naredbom SET PASSWORD.

Osnovna sintaksa:

```
SET PASSWORD [FOR imeKorisnika] = PASSWORD('lozinka')
```

U osnovnoj verziji uporabe naredbe SET PASSWORD, lozinka će biti postavljena trenutno spojenom korisniku.

Primjer:

Potrebno je promijeniti lozinku trenutno logiranom korisniku na vrijednost *root*.

```
SET PASSWORD = PASSWORD('root');
```

Lozinke nikada nisu u navedenoj tablici u bazi *mysql* pohranjene u izvornom obliku već su kriptirane. MySQL kriptira lozinke prema vlastitom algoritmu, pri čemu je za svaku predviđen 41 byte (od verzije MySQL 4.1). U gornjem kodu korištena je funkcija PASSWORD() koja se brine za kriptiranje lozinke. Kada se lozinka ne bi pohranila u kriptiranom obliku, došlo bi do pogreške prilikom pokušaja spajanja dotičnog korisnika na DBMS jer MySQL očekuje da je lozinka zapisana kriptirano te čitajući iz tablice *user.authentication_string* uvijek prvo provodi dekripciju.

Osim promjene lozinke trenutno spojenom korisniku, moguće je promijeniti lozinku nekom zadanom korisniku.

Primjer:

Promijeniti lozinku prethodno stvorenom korisniku *ihorvat* (koji se spaja s lokalnog poslužitelja) na vrijednost *novi1234*.

```
SET PASSWORD FOR 'ihorvat'@'localhost' = PASSWORD('novi1234');
```

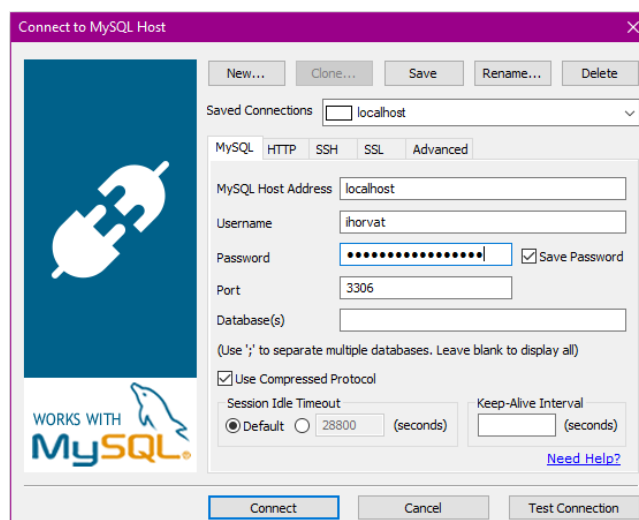
Treći je način postavljanja lozinke (osim ...IDENTIFIED BY klauzule i SET PASSWORD naredbe) direktnim ažuriranjem retka u tablici *mysql.user*.

Primjer:

Promijeniti lozinku prethodno stvorenom korisniku *ihorvat* (koji se spaja s lokalnog poslužitelja) na vrijednost *novaLozinka*. Promjenu lozinke potrebno je izvršiti ažuriranjem odgovarajuće tablice u bazi *mysql*.

```
UPDATE mysql.user
    SET user.password=PASSWORD('novaLozinka')
    WHERE user.authentication_string = 'ihorvat'
    AND user.host='localhost';
FLUSH PRIVILEGES;
```

Određene promjene vezane uz upravljanje korisnicima i njihovim dozvolama bit će vidljive tek po ponovnom pokretanju baze podataka odnosno nakon izvođenja naredbe FLUSH PRIVILEGES. Nakon toga moguće je spojiti se na server kao korisnik *ihorvat* korištenjem definirane lozinke.



Slika 59. Spajanje na poslužitelj kao ihorvat

33.3. DODJELJIVANJE DOZVOLA

Dozvole korisnicima za rad s podacima u MySQL bazi moguće je dodijeliti na dva načina. Prvi i preporučljiv način je koristeći naredbu GRANT. Sintaksa je čista i jednostavna, te se na brz način može dodijeliti odgovarajuća dozvola nekom korisniku. Kao što je moguće stvoriti novog korisnika izmjenom odnosno unosom podataka u tablici *mysql.user*, moguće je i dodijeliti odgovarajuće dozvole manipulacijom podataka u odgovarajućim tablicama u bazi *mysql*. To je nešto složeniji način jer je potrebno poznavati strukturu tablica u kojima su dozvole pohranjene. Ta se metoda preporuča samo u delikatnijim zahtjevima i situacijama.

Osnovna sintaksa:

GRANT

```
priv_type [(column_list)]
    [, priv_type [(column_list)]] ...
ON [object_type] priv_level
TO user_specification [, user_specification] ...
[REQUIRE {NONE | ssl_option [[AND] ssl_option] ...}]
[WITH with_option ...]
```

U nastavku su navedene opcije za dane dijelove naredbe:

object_type:

```
TABLE
| FUNCTION
| PROCEDURE
```

priv_level:

<code>*</code>		cijeli sustav
<code> *.*</code>		sve baze u sustavu
<code> db_name.*</code>		sve tablice u bazi db_name
<code> db_name.tbl_name</code>	→	tablica tbl_name u bazi db_name
<code> tbl_name</code>		tablica tbl_name
<code> db_name.routine_name</code>		pohranjeni zadatak routine_name u bazi db_name

user_specification:

```

user
[
    IDENTIFIED BY [PASSWORD] 'password'
    | IDENTIFIED WITH auth_plugin [AS 'auth_string']
]

```

ssl_option:

```

SSL
| X509
| CIPHER 'cipher'
| ISSUER 'issuer'
| SUBJECT 'subject'

```

with_option:

```

GRANT OPTION
| MAX_QUERIES_PER_HOUR count
| MAX_UPDATES_PER_HOUR count
| MAX_CONNECTIONS_PER_HOUR count
| MAX_USER_CONNECTIONS count

```

Primjer:

Dodijeliti sve dozvole nad svim podacima u sustavu korisniku *admin* sa svih adresa.

```
GRANT ALL ON *.* TO 'admin'@'%';
```


Primjer:

Dodijeliti sve dozvole nad svim podacima u sustavu korisniku *admin* sa svih adresa. Neka navedeni korisnik ima mogućnost dodjeljivanja dozvola drugim korisnicima

```
GRANT ALL ON *.* TO 'admin'@'%' WITH GRANT OPTION;
```

Primjer:

Dodijeliti dozvole čitanja, dodavanja i ažuriranja podataka u tablici *ocjene* korisniku *korisnik25* sa bilo kojeg poslužitelja u domeni *servis.com*. Korisnik *korisnik25* se spaja koristeći lozinku *mojaLozinka*. Neka *korisnik25* može ostvariti najviše 25 konekcija u satu.

```
GRANT SELECT, INSERT, UPDATE
ON studenti.ocjene
TO 'korisnik25'@'%.servis.com'
IDENTIFIED BY 'mojaLozinka'
WITH MAX_CONNECTIONS_PER_HOUR 25;
```

Stanje u tablici *mysql.tables_priv* nakon izvođenja gornjeg upita:

Host	Db	User	Tab...	Grantor	Table_priv
%servis.com	autoradionica	korisnik25	nalog	root@localhost	:21 Select, Insert, Update, Delete, Create, Drop, References, Ind

Ako se naredbom *grant* pokušavaju dodijeliti dozvole korisniku koji ne postoji, korisnik će automatski biti kreiran.

33.4. RAZINE DODJELJIVANJA DOZVOLA

33.4.1. Dozvole pristupa cijelom sustavu

- ✓ Evidentirane su u tablici *mysql.user*

Primjeri:

```
GRANT ALL
ON *.*
TO 'imeKorisnika'@'nazivDomene';

GRANT SELECT, INSERT, DELETE
ON *.*
TO 'imeKorisnika'@'nazivDomene';
```

33.4.2. Dozvole pristupa bazi podataka

- ✓ Evidentirane su u tablici *mysql.db*

Primjeri:

```
GRANT ALL ON imeBaze.*
TO 'imeKorisnika'@'nazivDomene';
```

```
GRANT SELECT, INSERT
      ON imeBaze.*
      TO 'imeKorisnika'@'nazivDomene';
```

33.4.3. Dozvole pristupa tablici

- ✓ Evidentirane su u tablici *mysql.tables_priv*

Primjeri:

```
GRANT ALL
      ON imeBaze.imeTablice
      TO 'imeKorisnika'@'nazivDomene';
GRANT SELECT, UPDATE, DELETE
      ON imeBaze.imeTablice
      TO 'imeKorisnika'@'nazivDomene';
```

33.4.4. Dozvole pristupa atributu

- ✓ Evidentirane su u tablici *mysql.columns_priv*

Primjer:

```
GRANT SELECT (stupac1), UPDATE (stupac1, stupac2)
      ON imeBaze.imeTablice
      TO 'imeKorisnika'@'nazivDomene';
```

33.4.5. Dozvole pristupa pohranjenom zadatku

- ✓ Evidentirane su u tablici *mysql.procs_priv*

Primjeri:

```
GRANT CREATE ROUTINE
      ON imeBaze.*
      TO 'imeKorisnika'@'nazivDomene';
GRANT EXECUTE
      ON PROCEDURE imeBaze.imeProcedure
      TO 'imeKorisnika'@'nazivDomene';
```

33.5. UKIDANJE DODIJELJENIH DOZVOLA

Kao i kod dodjeljivanja, dozvole se mogu ukinuti na dva načina. Prvi je način naredbom REVOKE, a drugi ažuriranjem odgovarajućih tablica u bazi MySQL. Sve što je oko izbora načina navedeno za dodjeljivanje dozvola, vrijedi i za ukidanje dozvola.

Osnovna sintaksa (1):

```
REVOKE
```

```
priv_type [(column_list)]
[, priv_type [(column_list)]] ...
ON [object_type] priv_level
FROM user [, user] ...
```

Osnovna sintaksa (2):

```
REVOKE ALL PRIVILEGES, GRANT OPTION
```

```
FROM user [, user] ...
```

Primjer:

Ukinuti dozvole ažuriranja podataka u tablici *ocjene* korisniku *korisnik25* sa bilo kojeg poslužitelja u domeni *servis.com*.

```
REVOKE UPDATE ON studenti.ocjene FROM 'korisnik25'@'%.servis.com';
```

Kao i kod dodjeljivanja dozvola, određene promjene bit će vidljive tek po ponovnom pokretanju baze podataka odnosno nakon izvođenja naredbe `FLUSH PRIVILEGES`.

33.6. PREGLED POSTOJEĆIH DOZVOLA

Ako je određenom korisniku potrebno ukinuti određene dozvole, a nismo sigurni koje trenutno ima dodijeljene, potrebno je prvo s naredbom `SHOW GRANTS` provjeriti trenutno stanje korisnikovih dozvola.

Osnovna sintaksa:

```
SHOW GRANTS FOR imeKorisnika;
```

Primjer:

Provjeriti dozvole za ranije stvorenog korisnika *ihorvat*.

```
SHOW GRANTS FOR 'ihorvat'@'localhost';
```

Obzirom da je korisnik samo bio stvoren naredbom `CREATE USER` a nisu mu kasnije dodjeljivane dozvole, navedeni korisnik ima samo dozvolu `USAGE` (korištenje) nad sustavom.

```
Grants for ihorvat@localhost
GRANT USAGE ON *.* TO 'ihorvat'@'localhost' IDENTIFIED BY PASSWORD '*'
```

33.7. PROMJENA KORISNIČKOG IMENA

Osnovna sintaksa:

```
RENAME USER stariKorisnik TO noviKorisnik
```

```
[, stariKorisnik2 TO noviKorisnik2] ...
```

Kod radnje promjene korisničkog imena treba biti oprezan jer će starije verzije MySQL-a ažurirati samo podatke u tablici *mysql.user*, odnosno neće promijeniti ostale zapise koji se odnose na korisnikove dozvole. Posljedica toga je da stare dozvole ostaju aktivne bez obzira

što stari korisnik više ne postoji. Postoji opasnost da kada bi se u budućnosti stvorio novi korisnik istog imena kao što je bio stari, odjednom bi imao dodijeljene (a potencijalno i opasne) dozvole.

Iz tog se razloga preporuča prvo koristiti naredbu REVOKE pa nakon toga GRANT, odnosno trajno obrisati starog korisnika ako više nije potreban.

33.7.1. Brisanje korisnika

Osnovna sintaksa:

```
DROP USER imeKorisnika;
```

34. Zaključavanje podataka

34.1. ISTOVREMENI PRISTUP PODACIMA

Rad korisnika s bazom podataka najčešće se svodi na izvršavanje unaprijed definiranih transakcija. Pojam transakcije odnosi se na skup od jedne ili više radnji koje čuvaju konzistenciju i korektnost podataka, a moraju se sve izvršiti u potpunosti ili se ne smije izvršiti niti jedna. Iako transakcija sa korisničkog stajališta predstavlja jednu nedjeljivu cjelinu, ona se obično realizira kao niz od nekoliko elementarnih zahvata u bazi.

Velike baze podataka moraju riješiti pitanje konkurentnosti baze i omogućiti istovremeni pristup podacima većem broju korisnika. U višekorisničkim bazama nekoliko transakcija izvodit će se paralelno, pri čemu će se osnovne operacije u transakciji vremenski ispreplitati. Da bi integritet baze bio očuvan, DBMS mora osigurati da učinak transakcija koje se izvršavaju paralelno mora biti kao da su se izvršavale slijedno, odnosno mora biti zadovoljeno svojstvo serijabilnosti (engl. *serializability*). Navedeno svojstvo ne može se a priori osigurati, a nekontrolirano izvršavanje transakcija može lako dovesti do neželjenih efekata.

Neka je izrađena tablica *racuni* te su u nju unesene tri n-torke.

```
CREATE TABLE racuni (  
  id INT(11) NOT NULL AUTO_INCREMENT,  
  stanje DECIMAL(8,2) DEFAULT NULL,  
  PRIMARY KEY (id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8  
INSERT INTO racuni VALUES (5000.00);  
INSERT INTO racuni VALUES (9000.00);  
INSERT INTO racuni VALUES (17000.00);
```

Pregled podataka u tablici:

```
SELECT * FROM racuni;
```

☐	id	stanje
☐	1	5000.00
☐	2	9000.00
☐	3	17000.00

U velikim sustavima ovakve tablice će imati na tisuće n-torki, te će rad u višekorisničkom okruženju sa većim količinama podataka iziskivati preciznost i vremensku usklađenost korisnika koji koriste ili mijenjaju podatke. Primjerice, pohranjeni zadatak koji pomoću kursora čita podatke te ih obrađuje, biti će zahjtevan po pitanju vremena i resursa za DBMS. Jednostavno ažuriranje stanja računa prema proceduri niže može trajati od nekoliko milisekundi do nekoliko sekundi.

```
DELIMITER //
```

```

DROP PROCEDURE IF EXISTS povecajStanje//
CREATE PROCEDURE povecajStanje ()
BEGIN
    DECLARE pomBr INT;
    DECLARE kraj BOOL;
    DECLARE kur CURSOR FOR
        SELECT id FROM racuni WHERE stanje>1000;
    DECLARE CONTINUE HANDLER
        FOR NOT FOUND SET kraj=FALSE;
    OPEN kur;
    SET kraj=TRUE;
    petlja: LOOP
        FETCH kur INTO pomBr;
        IF NOT kraj THEN
            LEAVE petlja;
        END IF;
        UPDATE racuni SET stanje = stanje * 1.1
            WHERE id=pomBr;
    END LOOP;
    CLOSE kur;
END;
//
DELIMITER ;

```

U slučaju kada nekoliko korisnika istovremeno obavlja operacije nad bazom, bitno je da se one odvijaju serijabilno te da se učinci međusobno ne preklapaju.

```

CALL povecajStanje ();
SELECT * FROM racuni;

```

☐	id	stanje
☐	1	5500.00
☐	2	9900.00
☐	3	18700.00

34.2. POTENCIJALNI PROBLEMI KOD PARALELNOG PRISTUPANJA PODACIMA

Postoje četiri općenita slučaja nastanka problema kod paralelnog pristupanja podacima.

- izgubljene izmjene (engl. *lost update*)
- prljavo čitanje (engl. *dirty read*)

- neponovljivo čitanje (engl. *nonrepeatable read*)
- sablasti (engl. *phantoms*)

Ta problematika može se riješiti zaključavanjem odgovarajućih podataka. Transakcija koja želi pristupiti nekom podatku najprije mora „uzeti“ odgovarajući ključ i njime zaključati dotični dio baze. Čim je obavila svoju operaciju, transakcija treba „vratiti“ lokot i time otključati podatke. Kad transakcija naiđe na podatke koji su već zaključani, ona mora čekati dok ih prethodna transakcija ne otključa. Time se izbjegava (sasvim) istovremeni pristup istom podatku.

34.2.1. Izgubljene izmjene

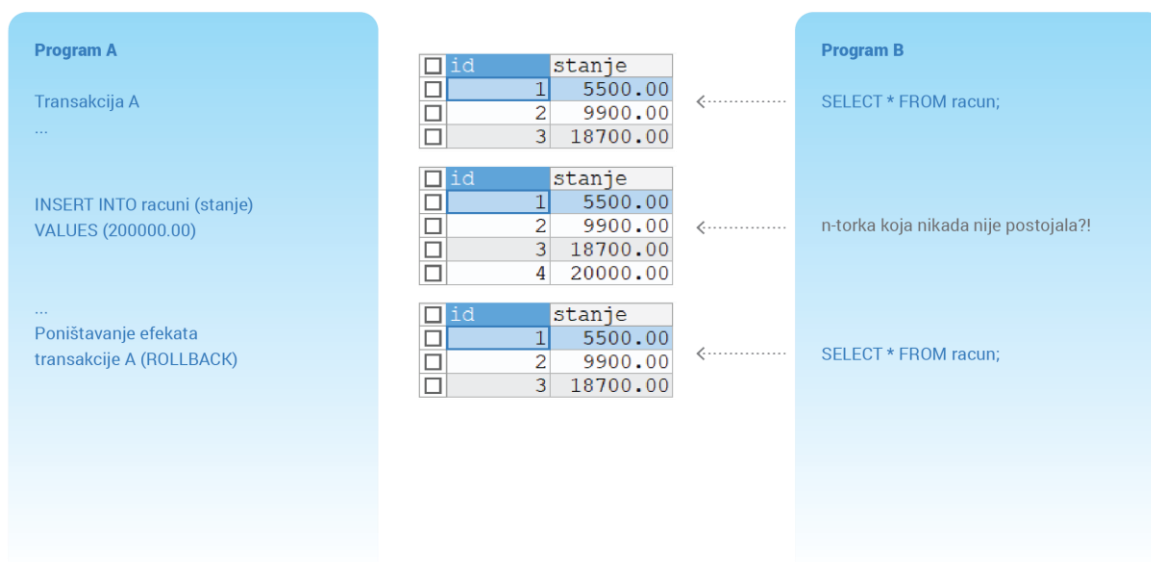
Do izgubljenih izmjena dolazi kada dvije transakcije paralelno pišu u bazu, a sačuvan je učinak samo one koja je zadnja završila. U primjeru prodaje avionskih karata na dvije različite lokacije (web aplikacija, šalter u agenciji, web servis i slično), ako se rezervacija događa istovremeno, moguć je scenarij kao u nastavku.



Slika 60. Problem kod paralelnog pristupanja podacima – izgubljene izmjene

34.2.2. Prljavo čitanje

Slučaj prljavnog čitanja nastaje kada prva transakcija piše, dok druga čita. Ovisno u kojem trenutku druga transakcija čita podatak, može vidjeti n-torke koje nikad nisu postojale u bazi jer je transakcija završila s opozivom (ROLLBACK).



Slika 61. Problem kod paralelnog pristupanja podacima – prljavo čitanje

34.2.3. Neponovljivo čitanje

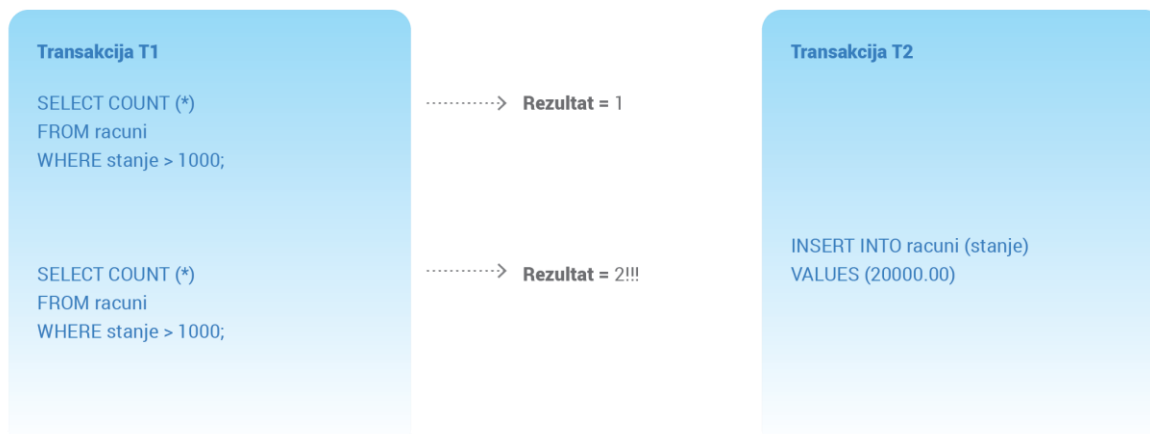
Neponovljivo čitanje nastaje kada prva transakcija čita određeni podatak, dok u međuvremenu druga napravi promjenu u bazi, nakon čega će prva transakcija za isti onaj upit čitanja podataka dobiti različiti rezultat.



Slika 62. Problem kod paralelnog pristupanja podacima – neponovljivo čitanje

34.2.4. Sablasne n-torke

Prva transakcija dobiva različit rezultat prilikom primjerice prebrojavanja n-torki jer je druga transakcija u međuvremenu unijela promjenu u tablicu. Time vidi n-torke za koje nema informaciju kako su nastale. One se zovu sablasne n-torke.



Slika 63. Problem kod paralelnog pristupanja podacima – sablasne n-torke

34.3. ZAKLJUČAVANJE PODATAKA U BAZI PODATAKA

Problematika paralelnog pristupanja podacima može se riješiti zaključavanjem odgovarajućih podataka. Transakcija koja želi pristupiti nekom podatku najprije mora „uzeti“ odgovarajući ključ i njime zaključati dotični dio baze. Čim je obavila svoju operaciju, transakcija treba „vratiti“ lokot i time otključati podatke. Kad transakcija naiđe na podatke koji su već zaključani, ona mora čekati dok ih prethodna transakcija ne otključa. Time se izbjegava (sasvim) istovremeni pristup istom podatku.

DBMS treba omogućiti paralelno izvođenje u što je moguće većoj mjeri, a efekt transakcija koje se obavljaju paralelno mora biti isti kao da su se obavljale jedna iza druge. Time se zadržava svojstvo **serijabilnosti** (engl. *Serializability*).



Slika 64. Paralelno izvršavanje zaključavanja podataka u dvije sesije

Dio DBMS-a je upravo **sustav za upravljanjem zaključavanja** (engl. *locking manager*) koji zaključava zapise i upravlja slučajevima kada postoji više zahtjeva za zaključavanjem istog podatka. Korisnik koji pokušava zaključati podatke mora imati odgovarajuće dozvole za zaključavanje (LOCK TABLES) i čitanje (SELECT) nad objektom kojeg zaključava.

34.4. PROTOKOL DVOFAZNOG ZAKLJUČAVANJA (2PL)

Svojstvo serijabilnosti bit će zadovoljeno ako sve transakcije prilikom upotrebe ključeva poštuju protokol dvofaznog zaključavanja (engl. *Two-phase locking protocol*). 2PL protokol sastoji se od dvije faze:

1. **Faza pribavljanja ključeva – faza rasta** (engl. *growing phase*)
2. **Faza otpuštanja ključeva – faza sužavanja** (engl. *shrinking phase*)

Prema protokolu transakcija mora zatražiti ključ prije obavljanja operacija nad objektom (tablicom, n-torkom...) te ga mora držati zaključanog sve dok ne završi potrebne operacije. Nakon otpuštanja ključa, transakcija više ne smije zatražiti nikakav ključ. Faza otpuštanja ključeva najčešće je izvedena kroz jednu operaciju na kraju transakcije (COMMIT ili ROLLBACK).

34.5. VRSTE KLJUČEVA

U ovisnosti o operacijama koje će transakcija obavljati nad podacima, potrebno ih je zaključati s odgovarajućim ključem. Ovisno s kojom je vrstom ključa podatak zaključan, drugi proces će moći na ograničen način baratati s tim podacima. Postoje tri vrste ključa.

34.5.1. Ključ za pisanje/izmjenu – WRITE LOCK

Transakcija T1 će s ovim ključem zaključati podatak da bi mogla pisati. Riječ je o **ekskluzivnom zaključavanju** (engl. *exclusive lock*), odnosno za to vrijeme druga transakcija T2 ne može ni pisati niti čitati navedeni podatak, sve dok ga prva ne otključa. Isto tako ga ne može niti zaključati. Transakcija T1 za to vrijeme može i čitati i pisati podatke koje je zaključala. Svaka operacija izmjene podataka implicitno postavlja ključ za pisanje.

34.5.2. Ključ za čitanje - READ LOCK

Transakcija T1 će s ovim ključem zaključati podatak da bi mogla čitati podatke. Riječ je o **zajedničkom zaključavanju** pri čemu transakcija T2 može zaključati podatak za čitanje (ne i za pisanje). Transakcija T1 također neće moći pisati po zaključanim podacima jer ih je zaključala (samo) sa read lockom.

34.5.3. Unaprijedivi ključ – PROMOTABLE LOCK

Ovo je ključ koji omogućava DBMS-u da predviđa buduće akcije. Na početku će T1 postaviti ključ za čitanje, a prije izmjene podataka ključ se unaprjeđuje u ključ za pisanje. Unaprijedivi ključ za sada ne postoji u MySQL bazi.

Tablica 34. Pravila po vrstama ključeva – dvije transakcije paralelno pokušavaju postaviti ključeve

	READ	PROMOTABLE	WRITE	BEZ ZAKLJUČAVANJA
READ	OK	OK	Greška	OK
PROMOTABLE	OK	Greška	Greška	OK
WRITE	Greška	Greška	Greška	OK

34.6. NEPOTPUNI I POTPUNI ZASTOJ

34.6.1. Nepotpuni zastoj

Na slici je prikazan primjer nepotpunog zastoja (engl. *live lock*). Moguće je da transakcija 2 čeka 'zauvijek' iako je mnogo puta imala priliku zaključati podatak A. Rješenje koje DBMS provodi je strategija FIRST-COME-FIRST-SERVED, pri čemu će prednost pri zaključavanju imati transakcija koje prva ušla u red za čekanje zaključavanja.



Slika 65. Nepotpuni zastoј

34.6.2. Potpuni zastoј

Do potpunog zastoја (engl. *deadlock*) dolazi u slučaju prema slici.



Slika 66. Potpuni zastoј

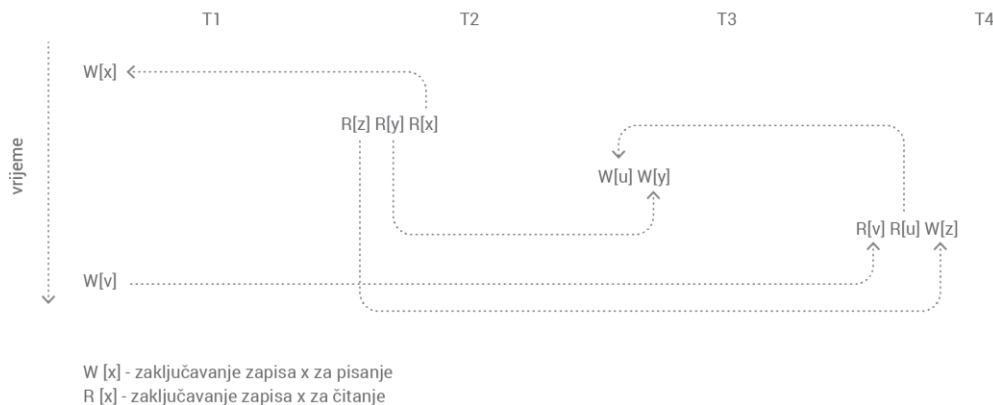
Navedeno se izbjegava time da transakcija zatraži sva zaključavanja odjednom na početku pri čemu uspije ili zaključati sve podatke koji su joj potrebni ili ne uspije zaključati ništa.

Transakcija čeka na dobivanje ključa, izvršava se prevencija na temelju **zadanog vremenskog odsječka** (engl. *timeout-based deadlock prevention*). Ako transakcija ne dobije ključ tijekom zadanog vremenskog odsječka, menadžer transakcija pretpostavlja da se ključ ne može postaviti upravo zbog pojave potpunog zastoја te poništava transakciju koja čeka na dobivanje ključa. U ovom je slučaju moguće da će se poništiti transakcije koje ne čekaju na dobivanje ključa.

Menadžer transakcija održava **wait-for graf** koji se sastoji od čvorova označenim identifikatorima transakcija (T_i , T_j , T_k ,...). luk usmjeren od čvora T_i prema čvoru T_j povlači se ako i samo ako transakcija T_i čeka na postavljanje ključa zbog nekog ključa kojeg je postavila transakcija. Potpuni zastoј detektira se otkrivanjem petlje u wait-for grafu. Petlja se iz grafa

uklanja izbacivanjem jednog ili više čvorova ($T_i, T_j, T_k, \dots$) i njima pripadnih lukova, uz istovremeno poništavanje pripadnih transakcija T_i, T_j, T_k ili ostalih.

Razmotrimo primjer četiri transakcije ($T_1 - T_4$) i pet podataka (x, y, z, u, v). Oznake W i R odnose se na pisanje i čitanje podataka.



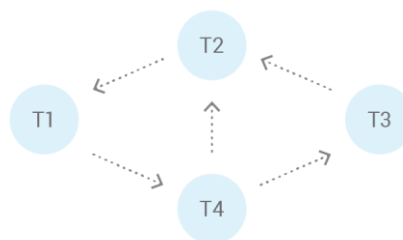
Slika 67. Wait-for graf

U wait-for grafu detetirane su tri petlje:

T_1, T_2, T_4

T_2, T_3, T_4

T_1, T_2, T_3, T_4



Slika 68. Petlje detektirane wait-for grafom

Ispitivanje postoje li petlje u wait-for grafu provodi se permanentno (kod dodavanja svakog novog luka u graf) i u određenim vremenskim razmacima. U slučaju da se dogodi potpuni zastoj, barem jedna transakcija mora se prekinuti čime se poništavaju njezini efekti. U ovom primjeru:

- poništavanjem T_3 eliminirala bi se samo jedna petlja,
- poništavanjem T_2 ili T_4 eliminirale bi se obje petlje u grafu i riješio potpuni zastoj.

Primjer:

Provjeriti i ispisati *timeout* vrijeme za trenutnu sesiju. Postaviti *timeout* na 30 sekundi.

```
SELECT * FROM information_schema.SESSION_VARIABLES
```

```
WHERE VARIABLE_NAME LIKE 'innodb_lock_wait_timeout';
```

Za verziju 5.7 ili veću:

```
SELECT @@innodb_lock_wait_timeout;
```

/* Default iznos za MySQL 5.7 je 50 sekundi.*/

@@innodb lock wait timeout
50

Postavljanje *timeout* vremena na 30 sekundi:

```
SET innodb_lock_wait_timeout = 30;
```

```
SELECT @@innodb_lock_wait_timeout;
```

@@innodb lock wait timeout
30

34.7. GRANULACIJA (ZRNATOST) ZAKLJUČAVANJA PODATAKA

Zaključavanje podataka i istodobnost pristupa podacima dva su povezana pojma, pri čemu dobar sustav zaključavanja omogućava visoku istodobnost. Jedan način da se poveća istodobnost je povećanje zrnatosti zaključavanja. Umjesto da se zaključaju cijele tablice, zaključava se samo dio koji sadrži potrebne podatke koji se koriste, te se na taj način dopušta više izmjena nad podacima istovremeno. Loša strana ovog pristupa je smanjenje performansi, te je potrebno pronaći ravnotežu između potrebnih performansi i potrebne zrnatosti zaključavanja. U nastavku je dan pregled tipova zrnatosti zaključavanja u MySQL-u. Prilikom definicije određene zrnatosti zaključavanja, važno je napomenuti da ne podržavaju svi tipovi mehanizma pohranjivanja podataka (engl. *storage engine*) sve zrnatosti zaključavanja.

34.7.1. Zaključavanje baze podataka

Zaključavanje baze podataka – osnovna sintaksa:

```
FLUSH TABLES WITH READ LOCK;
```

Otključavanje baze podataka odnosno zaključanih podataka – osnovna sintaksa:

```
UNLOCK TABLES;
```

Obzirom da postavljanje ovog ključa zaustavlja trenutni rad cijele baze, loše je za konkurentnost i ne preporuča se korištenje osim kod izrade backupa baze. Iako početak transakcije implicitno otključava zaključane podatke, obzirom da se u ovom slučaju radi o globalnom zaključavanju, početak transakcije neće otpustiti globalni ključ.

Primjer:

Zaključati bazu podataka sa ključem za čitanje. Provjeriti je li moguće izmijeniti bilo koji podatak u bazi podataka. Otključati podatke u bazi te ponoviti provjeru.

```
FLUSH TABLES WITH READ LOCK;
```

```
UPDATE ulogaizvrsitelja
```

```
    SET naziv = 'Dekan' WHERE id = 1; /*nije moguće izvršiti */
```

```
1 queries executed, 0 success, 1 errors, 0 warnings
```

```
Query: UPDATE ulogaizvrsitelja SET naziv = 'Dekan' WHERE id = 1
```

```
Error Code: 1223
```

```
Can't execute the query because you have a conflicting read lock
```

```
UNLOCK TABLES;
```

```
UPDATE ulogaizvrsitelja
```

```
    SET naziv = 'Dekan' WHERE id = 1;
```

```
2 queries executed, 2 success, 0 errors, 0 warnings
```

34.7.2. Zaključavanje tablica

Zaključavanje tablice – osnovna sintaksa:

```
LOCK TABLES imeTablice READ | [LOW PRIORITY] WRITE;
```

```
LOCK TABLES imeTablice1 WRITE, imeTablice2 READ;
```

Otključavanje tablice – osnovna sintaksa:

```
UNLOCK TABLES;
```

Najjednostavnije zaključavanje, koje ujedno najmanje opterećuje brzinu izvršavanja operacija, je zaključavanje na nivou tablice. Na ovaj način zaključava se čitava tablica. Moguće je postaviti ovakav ključ na privremenu tablicu ali ga *engine* ignorira. Obzirom da privremena tablica vrijedi samo u trenutnoj sesiji nije ju potrebno zaključavati jer je drugi korisnici ionako neće 'vidjeti'.

Primjer:

Potrebno je tablicu *ocjene* u bazi zaključati za čitanje. Provjeriti je li moguće u sesiji koja je zaključala podatke izvršiti naredbu čitanja podataka, te nakon toga naredbu izmjene podataka (pisanje).

Otvoriti novu sesiju te provjeriti mogućnost čitanja i pisanja u tablicu *ocjene* u novoj sesiji. Otključati tablicu u prvoj sesiji te ponoviti provjeru u drugoj sesiji.

Prva sesija:

```
LOCK TABLES ocjene READ;
```

```
SELECT * FROM ocjene;
```

Druga sesija:

```
/* uspješno čitanje */
UPDATE ocjene
SET ocjena = 5;
/* neuspješno ažuriranje */

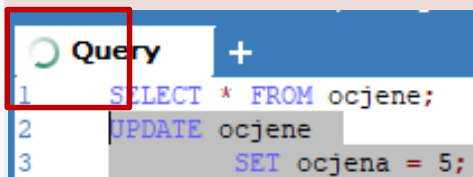
1 queries executed, 0 success,
1 errors, 0 warnings

Query: UPDATE ocjene SET ocjena
= 5

Error Code: 1099
Table 'ocjene' was locked with
a READ lock and can't be
updated
```

```
UNLOCK TABLES;
```

```
SELECT * FROM ocjene;
/* uspješno čitanje */
UPDATE ocjene
SET ocjena = 5;
```



```
Query +
1 SELECT * FROM ocjene;
2 UPDATE ocjene
3 SET ocjena = 5;
```

```
/* u drugoj sesiji uspješno je
izvršen update nakon
otključavanja */

1 queries executed, 1 success,
0 errors, 0 warnings

Query: UPDATE ocjene SET ocjena
= 5

182 row(s) affected
```

34.7.3. Zaključavanje n-torke

Zaključavanje n-torke – osnovna sintaksa:

```
SELECT selectIzjava
WHERE whereIzjava
```



```

LOCK IN SHARE MODE;
SELECT selectIzjava
WHERE whereIzjava
FOR UPDATE;

```

Otključavanje n-torke odvija se automatski nakon završetka naredbe / transakcije, za otključavanje n-torke ne postoji eksplicitna naredba.

Klauzula LOCK IN SHARE MODE odnosi se na ključ za čitanje, pri čemu je ostalim transakcijama dozvoljeno čitati ali ne i mijenjati podatke. Ako se zaključa podatak FOR UPDATE, podatak je zaključan s ekskluzivnim ključem pri čemu se očekuje da će prva transakcija podatke mijenjati odnosno drugim procesima nije dozvoljeno ni čitanje niti pisanje.

Zaključavanje na nivou retka pruža najbolju zrnatost zaključavanja, ali ujedno i najviše usporava izvršenje operacija.

Primjer:

Zaključati za čitanje sve n-torke u tablici *studenti* koje se odnose na studente prezimena Pavić.

```

SELECT * FROM studenti
WHERE prezime='Pavić' LOCK IN SHARE MODE;

```

Primjer:

Započeti transakciju i zaključati za pisanje n-torku u tablici *ustanove* koje se odnose na Prirodoslovno-matematički fakultet. Pokušati promijeniti navedenu n-torku u drugoj sesiji.

Prva sesija:

```

START TRANSACTION;
SELECT * FROM ustanove
WHERE naziv='Prirodoslovno-matematički fakultet' FOR UPDATE;

```

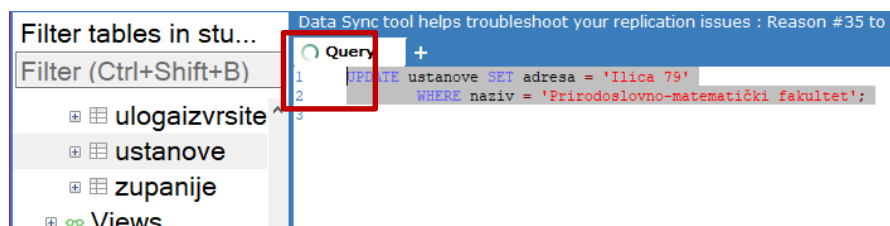
Druga sesija:

```

UPDATE ustanove SET adresa = 'Ilica 79'
WHERE naziv = 'Prirodoslovno-matematički fakultet';

```

U dugoj sesiji nije moguće provesti UPDATE naredbu jer je prva sesija zaključala n-torku za sebe. Pokušaj izvršavanja UPDATE naredbe u drugoj sesiji rezultira čekanjem dolaska na red (vrti se sat u tabu 'Query'):



Odnosno porukom o isteku vremena:

```
Error Code: 1205
```

```
Lock wait timeout exceeded; try restarting transaction
```

Prva sesija:

```
COMMIT WORK;
```

Tek bi se nakon završetka prve transakcije izvela UPDATE naredba u drugoj sesiji (ako u međuvremenu nije isteklo vrijeme – *timeout*).

34.7.4. Zaključavanje indeksa

Ovakva granulacija zaključavanja je pod nadzorom DBMS-a. Indeks je potrebno zaključati s ciljem sprječavanja sablasnih n-torki (čitanja n-torki koje nikad nisu postojale). Primjerice „čuvanje mjesta“ za ključ čiji je zapis obrisan za slučaj poništavanja transakcije.

34.8. RAZINE IZOLIRANOSTI

Moguće je definirati hoće li rezultati transakcije biti vidljivi drugim transakcijama sve dok transakcija nije izvršena. Za navedeno se brine svojstvo izoliranosti, pri čemu postoje četiri nivoa odnosno razine izoliranosti (engl. *isolation level*). Ove razine direktno su vezane uz ACID svojstvo izoliranosti. Navedeni nivoi određuju koje su promjene vidljive ostalim transakcijama odnosno koje nisu. Povećanjem razine izoliranosti sprečavaju se nepoželjne pojave nekonzistentnih čitanja podataka. U sljedećoj tablici prikazana je usporedba različitih razina izoliranosti.

Razinu izoliranosti moguće je postaviti globalno za cijeli poslužitelj ili za trenutnu sesiju. Definira se na početku transakcije odnosno nakon BEGIN WORK.

Osnovna sintaksa:

```
SET [GLOBAL | SESSION]
    TRANSACTION ISOLATION LEVEL razina
```

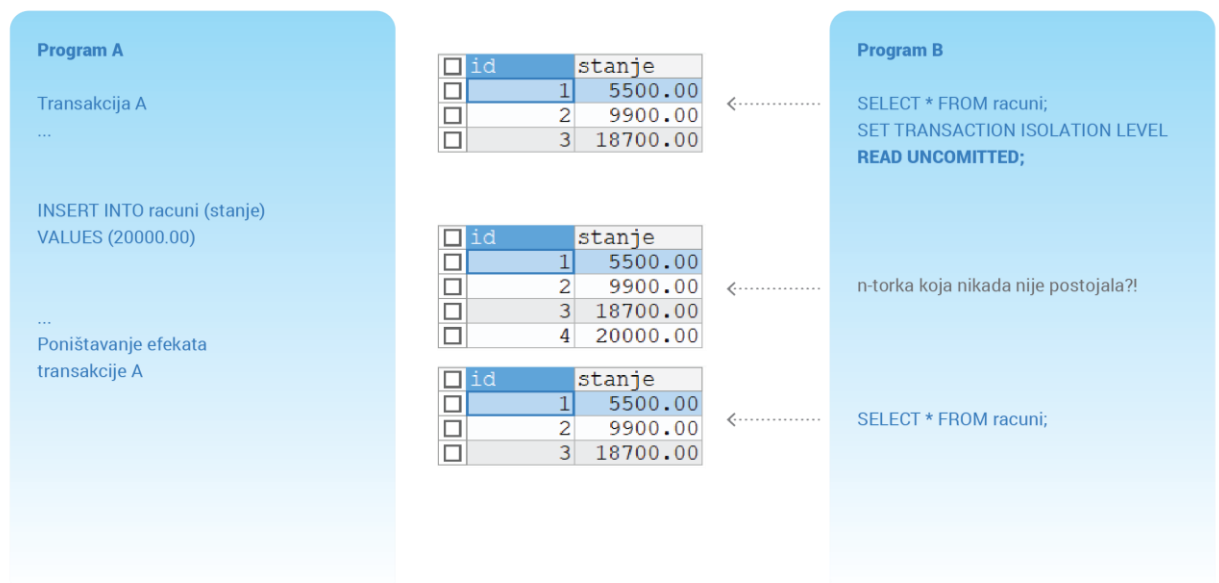
razina:

```
READ UNCOMMITTED
| READ COMMITTED
| REPEATABLE READ
| SERIALIZABLE
```

a) **READ UNCOMMITTED (DIRTY READ)**– prljavo čitanje / nezavršeno čitanje

- čitaju se svi traženi podaci **bez zaključavanja i bez provjere** jesu li možda zaključani
- mogu se pojaviti n-torke koje stvarno nikada nisu postojale u bazi podataka

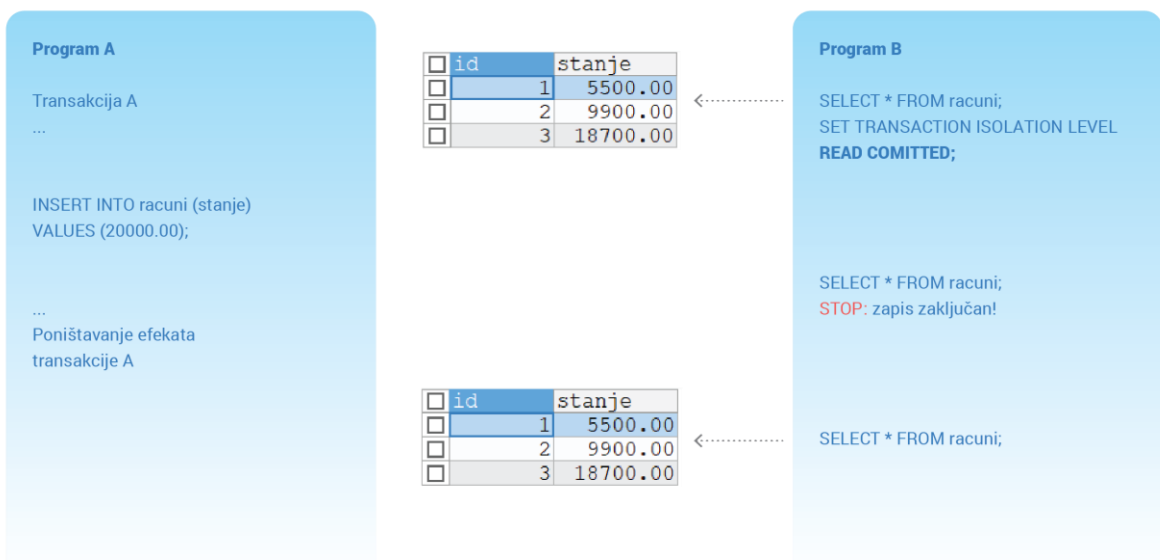
- koristi se kod pristupa statičnim podacima koji se rijetko mijenjaju



Slika 69. Razina izoliranosti - prljavo čitanje

b) READ COMMITTED - čitanje potvrđenih n-torki / izvršeno čitanje

- proces koji čita **provjerava** je li podatak koji se trenutno čita već zaključan za pisanje
- podatci se **ne zaključavaju**



Slika 70. Razina izoliranosti - čitanje potvrđenih n-torki

c) REPEATABLE READ - ponovljivo čitanje / ponavljano čitanje

- dohvat n-torke iz djelatnog skupa uzrokuje **zaključavanje** (za čitanje) odgovarajuće n-torke u bazi podataka
- dohvatom nove n-torke prethodna ostaje zaključana
- kod ponovnog dohvata istog djelatnog skupa unutar iste transakcije, prethodno zaključane n-torke i dalje su zaključane
- zapisi ostaju zaključani do kraja transakcije

d) **SERIALIZABLE** - ponovljivo čitanje / ponavljano čitanje

- isto kao i REPEATABLE READ
- ostao je radi kompatibilnosti sa InnoDB mehanizmom

Tablica 35. Usporedba razina izoliranosti

Razina izoliranosti	Vidljivi rezultati neizvršenih transakcija	Pojava neponavljanih redaka	Pojava fantomskih redaka
Prljavo čitanje	da	da	da
Čitanje potvrđenih n-torki	ne	da	da
Ponavljano čitanje	ne	ne	da
Serializacija	ne	ne	ne

Primjer:

Izraditi transakciju koja će promijeniti ime i prezime studentice Tanja Krog u Ivana Horvat. Nakon što se ažuriranje izvrši a transakcija još traje, provjerite u drugoj sesiji da li postoji studentica Ivana Horvat u slučaju:

- stupnja izolacije prljavog čitanja
- stupnja izolacije čitanja potvrđenih n-torki

Prva sesija:

```
BEGIN WORK;
UPDATE studenti
SET ime = 'Ivana', prezime =
'Horvat'
WHERE ime = 'Tanja' AND prezime
= 'Krog';
```

Druga sesija:

```
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;
```

```
SELECT * FROM studenti WHERE
ime = 'Ivana' AND prezime =
'Horvat';
```

```
/* druga sesija vidi Ivanu
Horvat iako prva sesija nije
potvrdila promjenu*/
```

☐ jmbag	ime	prezime	datumUpisa
☐ 0010081356	Ivana	Horvat	2015-09-28

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED;
```

```
SELECT * FROM studenti WHERE
ime = 'Ivana' AND prezime =
'Horvat';
```

```
/* druga sesija ne vidi Ivanu
Horvat jer prva sesija nije
potvrdila promjenu */
```

☐ jmbag	ime	prezime	datumUpisa
--------------------------------	-----	---------	------------

```
ROLLBACK WORK;
```

35. Izrada sigurnosne kopije i obnavljanje baze podataka

35.1. POTREBE ZA IZRADOM

Svaki bolji DBMS mora imati neki oblik kreiranja sigurnosne kopije (engl. *Backup*) i određeni mehanizam za obnavljanje istog (engl. *Recovery*). U ovom poglavlju biti će opisane generalne potrebe za kreiranjem sigurnosne kopije i obnavljanjem iste.

35.1.1. Razlozi kvarova u DBMS-u

35.1.1.1. Korisnikove greške

Korisnička greška je najčešći i najveći razlog kvara baze podataka. Korisnici nad podacima mogu prouzročiti štetu, gubitak i korupciju podataka unutar aplikacije pomoću vlastitog izbora. Najjednostavnije rješenje ovog problema je povratak baze podataka u neku određenu točku u vremenu prije nego što se greška dogodila. Zbog povratka podataka na stanje prije nego što se greška dogodila, sve ostale promjene koje su moguće bile ispravne će također biti izgubljene.

35.1.1.2. Kvar hardverskog sklopovlja

Hardversko sklopovlje može zakazati na više načina, a za baze podataka najproblematičniji je kvar tvrdog diska na kojemu DBMS pohranjuje podatke i transakcijske dnevnike (engl. *Transaction log*). Zbog činjenice da tvrdi diskovi nisu u potpunosti pouzdani, potrebno je iste češće mijenjati nego što to stoji u proizvođačevim specifikacijama.

35.1.1.3. Katastrofa

Katastrofa može biti prirodna poput vatre, potresa, poplava ili bilo koja prirodno uzrokovana katastrofa. Također, može biti i u obliku nestanka električne energije, računalni virus ili namjerno hakiranje. Svi navedeni događaji mogu biti uzročnici gubitku ili korumpiranosti podataka. Nedostatak sigurnosne kopije i plana za povrat sigurnosne kopije kod ovakvih događaja mogu biti još pogubnije za tvrtku koja je na ovaj način izgubila podatke. Povrat podataka je nemoguć bez sigurnosne kopije, a bez plana povratka podataka ne postoji jamstvo da će podaci uspješno proći proces povrata podataka.

35.1.2. Metode izrade sigurnosne kopije

35.1.2.1. Puna sigurnosna kopija (engl. *Full backup*)

Puna sigurnosna kopija sadrži sve podatke iz baze podataka i najsporija je metoda izrade sigurnosne kopije. Prilikom kopiranja svih podataka iz baze podataka, sigurnosna kopija može biti izuzetno velika i nezgrapna u slučaju da se radi o bazi podataka velike veličine kao npr. telekoma. Puna sigurnosna kopija se uvijek koristi kao baza za ostale tipove sigurnosnih kopija koje slijede u nadolazećem tekstu.



Slika 71. Izrada pune sigurnosne kopije svaki dan

35.1.2.2. Diferencijalna sigurnosna kopija (engl. *Differential backup*)

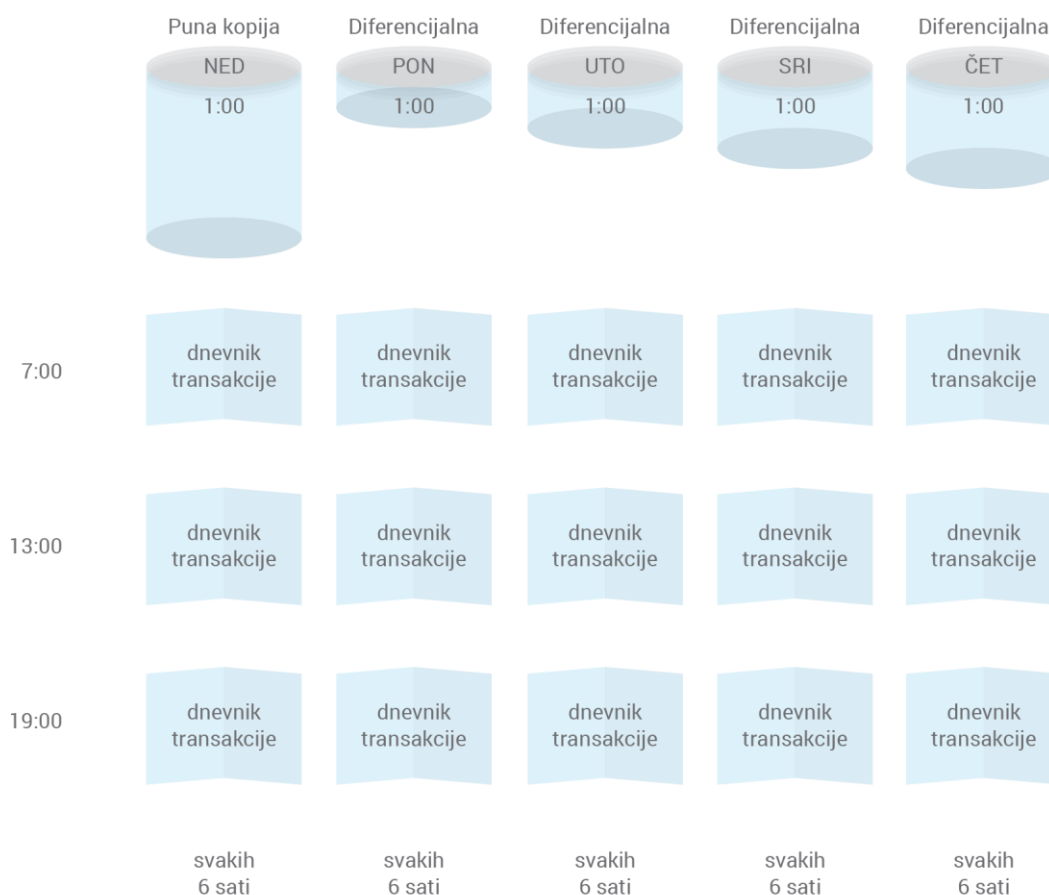
Diferencijalna sigurnosna kopija nije nezavisna i mora se bazirati na punoj sigurnosnoj kopiji baze podataka. Diferencijalna sigurnosna kopija sadrži samo podatke koji su bili promijenjeni od posljednje izrade pune ili diferencijalne sigurnosne kopije. Diferencijalne sigurnosne kopije su manje i brže se kreiraju nego puna sigurnosna kopija. Trebale bi se koristiti zbog manjeg zauzeća diskovnog prostora i veće brzine prilikom same izrade sigurnosne kopije. Zbog povećanja brzine izrade, diferencijalne sigurnosne kopije mogu se izvoditi mnogo češće od punih kako bi se smanjio rizik od nestanka podataka. Prilikom povratka podataka, podaci se prvo vraćaju s pune sigurnosne kopije, a potom diferencijalne.



Slika 72. Izrada pune sigurnosne kopije jednom tjedno uz diferencijalnu kopiju

35.1.2.3. Sigurnosna kopija dnevnika transakcija (engl. *Transaction log backup*)

Prilikom kreiranja sigurnosne kopije dnevnika transakcija, aktivnosti koje su se dogodile od posljednje pune ili diferencijalne sigurnosne kopije se pohranjuju. Dnevnik transakcija je zapravo lista svih promjena nakon pune ili diferencijalne sigurnosne kopije koje su se izvele na bazi podataka a uzrokuju promjenu podataka. Dnevnik transakcija je potreban kako bi se bazu podataka moglo vratiti u određenu točku u vremenu (npr. prije unosa neželjenih podataka). O DBMS-u uvelike ovisi interna implementacija dnevnika transakcija. Primjerice u MySQL DBMS-u, ona se zove binarni dnevnik (engl. *binary log*), u Oracle i SQL Server DBMS-ovima ona se zove dnevnik transakcija, a implementacije na softverskoj razini su u potpunosti drugačije. Sigurnosne kopije dnevnika transakcija su vrijedne samo uz punu sigurnosnu kopiju ili odgovarajuću diferencijalnu sigurnosnu kopiju (koja opet mora biti izgrađena na punoj sigurnosnoj kopiji).



Slika 73. Izrada sigurnosne kopije s dnevnikom transakcija

35.2. MEHANIZMI POHRANJIVANJA NA MYSQL-U

Prije nego se ustanovi kako se izvlače podaci u slučaju nezgode u radu s MySQL DBMS-om i kako se radi sigurnosna kopija, prvo se potrebno upoznati s mehanizmima pohranjivanja u bazu podataka i mogućim oštećenjima koja mogu nastati kako bi se jednostavnije shvatilo što se zapravo događa.

35.2.1. MyISAM mehanizam pohranjivanja

MyISAM je standardni mehanizam pohranjivanja u MySQL-u. Nastao je na temelju starijeg koda ISAM, te je proširen s mnogim korisnim ekstenzijama. Ovaj mehanizam pohranjivanja sa svojom jednostavnom strukturom, pruža kompromis između performansi i korisnih dodataka. Svaka baza podataka je direktorij sa svakom tablicom pohranjenom u zaseban skup datoteka. Svaka MyISAM tablica je pohranjena na tvrdi disk u tri datoteke. Imena datoteka imaju isto ime kao i tablica, ali se razlikuju po ekstenzijama koje označavaju o kojem tipu podatka se radi. MySQL koristi „frm“ datoteku za pohranjivanje definicije tablice. Međutim, ova datoteka nije dio MyISAM mehanizma već servera. Podatkovna datoteka ima ekstenziju „MYD“ (MYData), dok datoteka „MYI“ (MYIndex) sadrži indekse koji pripadaju

tablici te pojedine statističke podatke za tablicu. Budući da je MyISAM jedan od najstarijih mehanizama pohranjivanja u MySQL-u, tijekom vremena je razvijeno nekoliko korisnih mogućnosti za zadovoljavanje potreba korisnika i ispravljanja nedostataka. Automatsko popravljavanje (engl. *automatic repair*) omogućuje pregled i popravak tablica. Ako je MySQL pokrenut sa „myisam-recover“ opcijom, pri prvom otvaranju MyISAM tablice, pregledava se tablica kako bi se provjerilo je li pri prijašnjem korištenju ispravno zatvorena. Ako nije, a razlog je vjerojatno problem sa sklopovljem ili napajanjem, MySQL skenira tablice i pri pronalasku pogreške automatski popravljiva tablicu. Nedostatak ove opcije je što je potrebno sačekati dok se tablica popravljiva. MyISAM također pruža mogućnost ručnog pokretanja popravka preko naredbi CHECK TABLE i REPAIR TABLE. U slučaju kada je poslužitelj isključen, za pronalazak i ispravak pogrešaka se može koristiti „myisamchk“. Sigurnosna kopija MyISAM tablica se kreira koristeći „mysqldump“ ili neki „mysqlhotcopy“ alat. U slučaju pada sustava, tablice s redcima fiksne duljine jednostavnije je vratiti nego tablice s redcima promjenjive duljine. Međutim, prilikom pada sustava najčešće su samo indeksi zahvaćeni, što se može riješiti jednostavnim i automatskim, iako ponekad dugotrajnim procesom.

35.2.2. InnoDB mehanizam pohranjivanja

InnoDB je jedan od najpopularnijih mehanizama pohranjivanja u MySQL-u. Njegova najveća prednost nad ostalim mehanizmima pohranjivanja je mogućnost ACID transakcije. ACID (atomarnost, konzistentnost, izoliranost, trajnost) predstavlja skup svojstava koja garantiraju da se transakcije baza podataka odvijaju pouzdano. InnoDB ima potpuno različitu arhitekturu od MyISAM-a, koja se temelji na konceptu tabličnih prostora u koje se spremaju sve strukture, podaci i indeksi. Tablični prostor može sadržavati jednu ili više datoteka, čak i (iako ne često) neobrađenu particiju diska. Korištenje disk particije otežava kreiranje sigurnosne kopije InnoDB tablica, i rezultira smanjenim performansama. InnoDB tablica razvrstava podatke na disku kako bi se optimizirala za najčešće upite na temelju primarnih ključeva. Svaka InnoDB tablica ima indeks primarnog ključa koji se zove klaster (engl. *clustered*) indeks. Zadaća ovog indeksa je organizacija podataka. S InnoDB mehanizmom pohranjivanja svaka se aktivnost vrši unutar transakcije, tako da svaka naredba kreira vlastitu transakciju. InnoDB vrši zaključavanje na razini retka te izvodi upite kao nepromjenjivo čitanje bez mogućnosti zaključavanja. Informacije o zaključavanju se spremaju s iznimnom efikasnošću korištenja prostora, tako da eskalacija zaključavanja nije potrebna. Ovo omogućava zaključavanje svakog retka od strane više korisnika bez opterećivanja memorije. Ovisno o razini izolacije, InnoDB ne zahtjeva nikakvo zaključavanje za naredbu „select“. Za ažuriranje se koristi zaključavanje na razini redaka. Ovo omogućava iznimno visoku razinu istodobnosti, međutim zahtjeva tri puta više memorijskog prostora nego što je to slučaj kod MyISAM mehanizma pohranjivanja, te veliki broj RAM memorije za InnoDB međuspremnik (engl. *buffer*). InnoDB se oporavlja od pada sustava ponavljajući operacije iz evidencije.

35.2.3. Korupcija „.frm“ datoteke

Malo ljudi zna da MySQL zapravo nije u potpunosti „ACID“ čak ni sa InnoDB tablicama, bar ne za DDL upite. Kako je već navedeno ACID predstavlja skup svojstava koja garantiraju da

se transakcije baza podataka odvijaju pouzdano. Postoji mogućnost greške prilikom korištenja npr. CREATE ili RENAME TABLE naredbe u kojem se tablica kreira u InnoDB rječniku, ali „.frm“ datoteka nije kreirana ili u potpunosti napisana. Djelomično napisane „.frm“ datoteke ili „.frm“ datoteke koje nisu sinkronizirane s InnoDB rječnikom mogu uzrokovati rušenje MySQL DBMS-a s ispisom nekonvencionalnih grešaka. U MySQL-u 4.1 dodana je opcija *sync_frm* koja smanjuje mogućnost neke od tih grešaka, no one su i dalje moguće.

35.2.4. Korupcija dnevnika transakcija

Dnevnik transakcija, odnosno u MySQL-u zvani binarni dnevnik se mogu oštetiti ili biti van sinkronizacije s bazom podataka. To može oštetiti replikaciju baze podataka, ali ako je samo potrebno napraviti povratak podataka u određenu točku u vremenu, koji je detaljno opisan kasnije, to neće predstavljati problem. Za sinkronizaciju binarnih dnevnika koristi se *sync_binlog*, ali on uzrokuje gubitke u performansama. Ako se koristi InnoDB mehanizam pohranjivanja, bilo bi dobro koristiti i *innodb-safe-binlog* opciju od verzije MySQL-a 4.1 tako da su dnevnici u InnoDB i binarni dnevnik sinkronizirani.

35.3. SIGURNOSNE KOPIJE I OBNAVLJANJE NA MYSQL-U

Besplatna MySQL verzija DBMS-a omogućava kreiranje sigurnosnih kopija i obnavljanje na poprilično loš način te ne implementira sve funkcionalnosti i alate kao renomirani DBMS sustavi.

35.3.1. Izrada punih sigurnosnih kopija i obnavljanje

Fizička izrada sigurnosnih kopija se sastoji od toga da se sadržaj direktorija jednostavno kopira na drugo mjesto. Rijetko koja baza podataka podržava ovakvo kreiranje sigurnosne kopije zbog čestih problema sa zaključavanjem pristupa tablicama koje su trenutno u procesu kreiranja sigurnosne kopije. Sigurnosna kopija je veličinom manja, sadrži apsolutno sve podatke i brže se izvodi od logičke, ali postoji mogućnost neispravnih podataka jer je baza podataka „živa“. Ovaj tip backupa nije pretjerano praktičan zbog problema sa zaključavanjem tablica unutar baze nad kojom se kreira sigurnosna kopija.

Logička izrada sigurnosne kopije na MySQL DBMS-u kreira skriptu logičke strukture i podataka baze podataka u datoteku koju je moguće jednostavno pokrenuti („sql“ datoteka). Logička izrada je sporija od fizičke izrade zato što je potrebno iz postojećih podataka i struktura u kojima se isti nalaze kreirati skripte što može biti zahtjevno. Logička izrada ima prednost da je u potpunosti neovisna o operacijskom sustavu i hardveru računala. Skripta sadrži naredbe za kreiranje tablica i unos vrijednosti pomoću SQL upita. Kako bi se kreirala logička kopija, potrebno je pozvati program „mysqldump“ koji se nalazi u MySQL „bin“ mapi. program podržava brojne parametre i mogućnosti koje je moguće ispisati pomoću sljedeće naredbe:

```
mysqldump --help
```

Ako je potrebno kreirati sigurnosnu kopiju svih baza podataka pod cijelom instancom MySQL DBMS-a, moguće je napisati naredbu kao u nastavku:

```
mysqldump --user=[ime] --password=[pass] --all-databases >dump.sql
```

Ako se želi izvoz jedne ili više baza, sintaksa je nešto drugačija:

```
mysqldump --user=[ime] --password=[lozinka] --databases studenti  
test > dump.sql
```

Nakon što postoji logička sigurnosna kopija, istu je izuzetno jednostavno vratiti jer se upiti iz iste jednostavno ponovo pokrenu i na taj način se kreiraju sve tablice i podaci.

```
mysql -u [ime] -p[pass] [bazaPodataka] < dump.sql
```

35.3.2. Obnavljanje baze podataka u određenu točku u vremenu

Obnavljanje baze podataka u određenu točku u vremenu (engl. *Point in time recovery*) je metoda pomoću koje se može vratiti promjene nad podacima napravljene od određene točke u vremenu. Ova vrsta obnavljanja izvodi se nakon što je već vraćena prethodno napravljena sigurnosna kopija koja vraća server u stanje u kojem je bio za vrijeme pravljenja sigurnosne kopije. Ovaj način obnavljanja se svodi na uključivanje vođenja binarnog dnevnika (engl. *binary logging*) i definiranje nekoliko sistemskih varijabli vezanih za replikaciju. Dnevnik promjena predstavlja proces u kojem MySQL svaku promjenu tablice (alter, insert, delete) pohranjuje u posebnu datoteku (binarni dnevnik) u vidu upita (query), a zatim te promjene primjenjuje nad bazom. Da bi se vođenje dnevnika omogućilo, MySQL poslužitelj mora biti pokrenut s *--log-bin* opcijom. Da bi se podaci vratili pomoću binarnog dnevnika, potrebno je znati točno ime i lokaciju na koju su ti podaci pohranjeni. Da bi se vidjeli zapisi svih binarnih logova (dnevnika) koristi se naredba:

```
mysql> SHOW BINARY LOGS;
```

Da bi se vidjela imena trenutne datoteke koja sadrži binarne logove koristi se naredba:

```
mysql> SHOW MASTER STATUS;
```

Funkcija *mysqlbinarylog* pretvara događaje iz binarnog formata u tekstualni, ljudima razumljiv format. Funkcija *mysqlbinarylog* ima opciju za odabir dijelova binarnih logova baziranu na poziciji određenog događaja u vremenu. Da bi se izvršili događaji iz binarnih logova potrebno je izvršiti sljedeću naredbu:

```
shell> mysqlbinlog binlog_files | mysql -u root -p
```

Gledanje sadržaja logova može biti korisno kada je potrebno odrediti poziciju ili vrijeme određenog događaja. Da bi se dobio uvid u logove potrebno je proslijediti *mysqlbinarylog-u* sljedeći kod:

```
shell> mysqlbinlog binlog_files | more
```

Nakon toga je potrebno pohraniti dobivene podatke u datoteku koja će se kasnije otvarati u nekom uređivaču teksta:

```
shell> mysqlbinlog binlog_files > tmpfile  
shell> ... edit tmpfile ...
```

Pohranjivanje tih podataka u datoteku je korisno ako je prethodno potrebno promijeniti određene događaje (na primjer ako je napravljen slučajan DROP DATABASE). Iz datoteke se može brisati bilo koji upit čiji sadržaj se ne želi izvršiti. Nakon što je datoteka promijenjena, potrebno je izvršiti sljedeći programski kod:

```
shell> mysql -u root -p < tmpfile
```

Ako postoji više od jednog binarnog loga koji je potrebno izvršiti na MySQL-u, sigurnija metoda bila bi da ih se sve izvrši koristeći jednu vezu na server. Ovdje je primjer naredbi koje bi mogle biti nesigurne:

```
shell> mysqlbinlog binlog.000001 | mysql -u root -p # DANGER!!
```

```
shell> mysqlbinlog binlog.000002 | mysql -u root -p # DANGER!!
```

Izvršavajući binarne logove na ovaj način koristeći različite veze prema serveru uzrokuje probleme ako prva log datoteka sadrži naredbu „CREATE TEMPORARY TABLE“, a druga datoteka sadrži naredbu koja koristi privremenu tablicu. Kada se izvrši prva SQL naredba, server odbacuje privremenu tablicu i kada je drugi proces pokuša iskoristiti, server proslijedi „unknown table“ grešku. Da bi se ovakvi problemi izbjegli treba koristiti jednu vezu za izvršavanje sadržaja svih binarnih logova na sljedeći način:

```
shell> mysqlbinlog binlog.000001 binlog.000002 | mysql -u root -p
```

Još jedan pristup bio bi da se sve datoteke kopiraju u jednu datoteku nakon čega se samo ona izvršava:

```
shell> mysqlbinlog binlog.000001 > /tmp/statements.sql
```

```
shell> mysqlbinlog binlog.000002 >> /tmp/statements.sql
```

```
shell> mysql -u root -p -e "source /tmp/statements.sql"
```

36. Optimizacija baze podataka

Performanse baze podataka ovise o mnogim čimbenicima. Svaki sustav za upravljanje bazama podataka ima svoje prednosti i mane i njima se knjiga neće baviti, već će biti prikazane opće smjernice optimizacije upita, kreiranja indeksa, statistike i razine normalizacije na koje razvojni inženjeri mogu direktno utjecati prilikom razvoja aplikacije i baze podataka.

Performanse baze podataka ovise o:

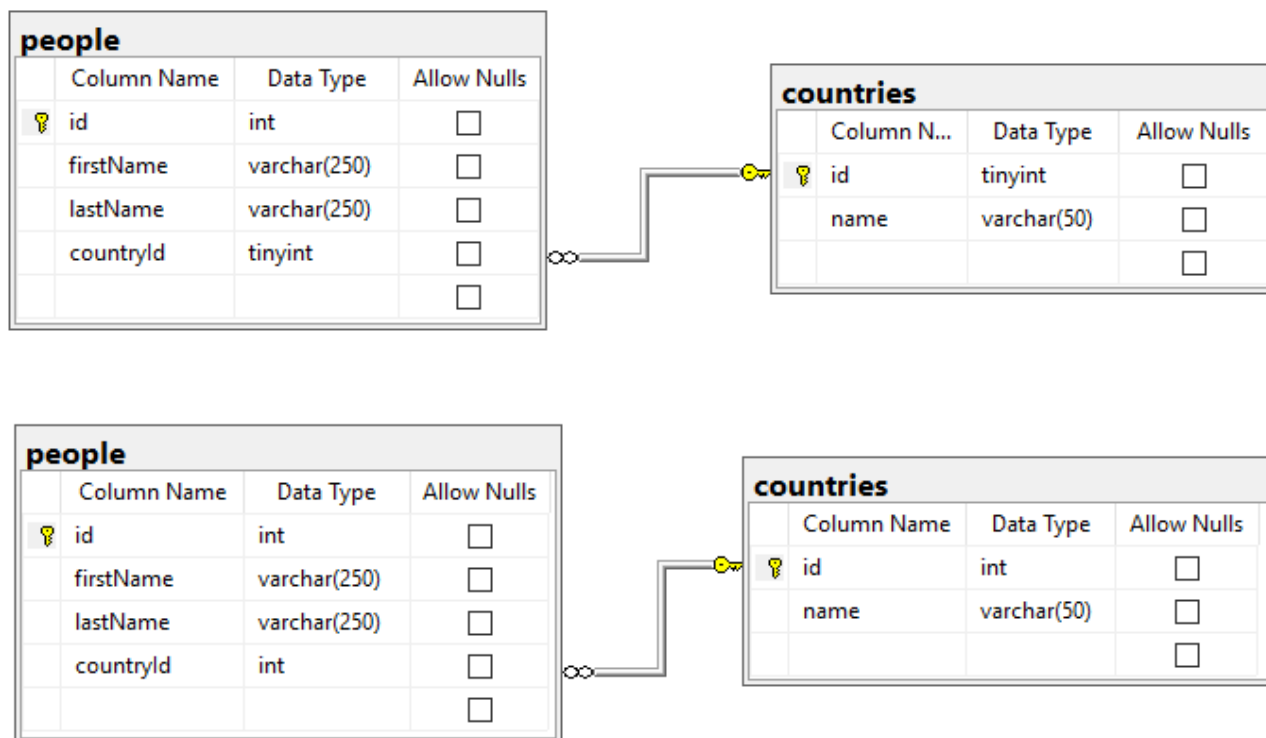
- Dizajnu (modelu) baze podataka
 - Tip podataka
 - Način na koji su kreirane tablice
 - Razina normalizacije
- SQL upitima
 - Kvalitetnije napisan upit će brže raditi
- Konfiguraciji sustava za upravljanje bazama podataka
 - O konfiguraciji ovisi kako će i koliko DBMS koristiti resursa računala
- Hardverskom sklopovlju
 - Brzina tvrdog diska
 - Brzina procesora
 - Količina i brzina radne memorije

36.1. OPTIMIZACIJA DIZAJNA BAZE PODATAKA

Ranije u udžbeniku je objašnjeno da se pojmovi izrade dizajna i izrade modela baze podataka svode na iste postupke. Dakle kada se u ovom poglavlju govori o optimizaciji dizajna, govori se o optimizaciji modela, i to relacijskog modela baze podataka.

Prilikom dizajna baze podatka potrebno je koristiti minimalnu potrebnu veličinu tipa podatka. Konkretno, ako broj redaka unutar tablice neće prelaziti maksimalnu veličinu minimalnog tipa podatka, taj tip podatka je potrebno upotrijebiti. Primjerice, poznato je da u svijetu ima otprilike dvjesto država (ne precizira se jer je podatak promjenjive prirode). U tom slučaju, moguće je za primarni ključ kojeg će baza podataka slijedno dodjeljivati koristiti „UNSIGNED TINYINT“ tip podatka jer zadovoljava sve potrebe za različitim podacima u tablici. Kao što je poznato „UNSIGNED TINYINT“ tip podatka može biti broj od 0 do 255. Zbog stranog ključa koji se veže na tablicu država potrebno je upotrijebiti isti tip podatka u obje tablice. Problem se dakle neće pretjerano manifestirati u tablici država zbog premalene količine podataka, već u svim tablicama koje su vezane na tablicu država preko stranog ključa.

Kao primjer i dokaz tvrdnje, na SQL Server DBMS-u kreirane su dvije identične baze podataka s jednom razlikom u tipu podatka kojeg kao primarni ključ koristi tablica država, a kao posljedicu isti tip podatka mora koristiti tablica ljudi koja je vezana na državu. Baze podataka koje su kreirane imaju samo dvije tablice i izgledaju kao na sljedećim slikama.



U obje baze podataka tablica osoba tj. „people“ je napunjena istim vrijednostima i kreirano je 10000000 redaka. Redci su uneseni u tablicu pomoću sljedećeg upita:

```

;with sequence as (
    select N = row_number() over (order by @@spid)
    from sys.all_columns c1, sys.all_columns c2
)
insert into people(firstname, lastname, countryId)
select
    'John' + right('000000' + cast(N as varchar(10)), 6),
    'Doe' + right('000000' + cast(N as varchar(10)), 6),
    ABS(Checksum(NewID())) % 200)+1
from sequence where N <= 10000000
    
```

Nakon unosa podataka u bazu podataka, izvršen je upit pomoću kojeg je na SQL Server DBMS moguće vratiti točno zauzeće prostora na tvrdom disku s pojedinom tablicom. Osim navedenog, upit vraća iskorišten i neiskorišten prostor. Upit pomoću kojeg je moguće na SQL Server DBMS-u dohvatiti zauzeće prostora tablica slijedi u nastavku:

```

SELECT
    
```

```

t.NAME AS NazivTablice,
p.rows AS BrojRedaka,
CAST(ROUND(((SUM(a.total_pages) * 8) / 1024.00), 2) AS
NUMERIC(36, 2)) AS VelicinaMB,
CAST(ROUND(((SUM(a.used_pages) * 8) / 1024.00), 2) AS
NUMERIC(36, 2)) AS IskoristenoMB,
CAST(ROUND(((SUM(a.total_pages) - SUM(a.used_pages)) * 8) /
1024.00, 2) AS NUMERIC(36, 2)) AS NeiskoristenoMB
FROM
    sys.tables t
INNER JOIN
    sys.indexes i ON t.OBJECT_ID = i.object_id
INNER JOIN
    sys.partitions p ON i.object_id = p.OBJECT_ID AND i.index_id =
p.index_id
INNER JOIN
    sys.allocation_units a ON p.partition_id = a.container_id
LEFT OUTER JOIN
    sys.schemas s ON t.schema_id = s.schema_id
WHERE
    t.NAME NOT LIKE 'dt%'
AND t.NAME <> 'sysdiagrams'
    AND t.is_ms_shipped = 0
    AND i.OBJECT_ID > 255
GROUP BY
    t.Name, s.Name, p.Rows
ORDER BY
    t.Name

```

U slučaju baze podataka s „TINYINT“ tipom podataka upit vraća sljedeće rezultate:

	NazivTablice	BrojRedaka	VelicinaMB	IskoristenoMB	NeiskoristenoMB
1	countries	201	0.07	0.02	0.05
2	people	10000000	378.95	378.84	0.10

U slučaju baze podataka s „INT“ tipom podatka upit vraća sljedeće rezultate:

	NazivTablice	BrojRedaka	VelicinaMB	IskoristenoMB	NeiskoristenoMB
1	countries	201	0.07	0.02	0.05
2	people	10000000	408.51	408.43	0.08

Razlika između dvaju tablica je na 10000000 redaka izuzetno velika i iznosi 29,56MB samo zbog tipa podatka atributa. Najgora činjenica je ta da veličina same tablice zapravo i nije najveći problem. Najveći problem je veličina indeksa, odnosno same stranice indeksa. U ovom slučaju kako bi JOIN naredba radila što brže u svrhu prebrojavanja osoba po državi, potrebno je kreirati indeks nad tablicom ljudi i stupcom „idcountry“. Naredba za kreiranje indeksa dana je u nastavku:

```
CREATE NONCLUSTERED INDEX ix_people_countries
ON people (countryId)
```

Veličinu indeksa nad tablicom osoba moguće je dobiti pomoću upita u nastavku:

```
SELECT
    i.name AS NazivIndeksa,
    CAST(ROUND((SUM(s.used_page_count) * 8) / 1024.00, 2) AS
    NUMERIC(36, 2)) AS VelicinaIndeksaMB
FROM sys.dm_db_partition_stats AS s
JOIN sys.indexes AS i
ON s.[object_id] = i.[object_id] AND s.index_id = i.index_id
WHERE s.[object_id] = object_id('dbo.people')
GROUP BY i.name
ORDER BY i.name
```

U slučaju baze podataka s „TINYINT“ tipom podataka upit vraća sljedeće rezultate:

	NazivIndeksa	VelicinaIndeksaMB
1	ix_people_countries	106.45
2	PK_people	378.84

U slučaju baze podataka s „INT“ tipom podatka upit vraća sljedeće rezultate:

	NazivIndeksa	VelicinaIndeksaMB
1	ix_people_countries	135.59
2	PK_people	408.43

Razlika između ukupne veličine indeksa dvaju tablica je na 10000000 redaka izuzetno velika i iznosi 58,73MB samo zbog tipa podataka jednog atributa. Ukupna razlika u veličini jedne tablice zajedno s indeksima dakle iznosi 88,29MB.

Osim navedenih problema s veličinom radnih tablica, dolazi i do velikih razlika u veličini sigurnosne kopije. U slučaju izrade pune sigurnosne kopije razlika će biti 88,29MB. Ako se kopija radi svaki dan, dolazi se do razlike od preko 2,5GB mjesečno što nije zanemarivo. Sigurnosna kopija će se također dulje izrađivati, biti će potrebno više prostora za pohranu iste i sve te podatke potrebno je poslati preko mreže.

Problem čak nije ako se radi o malom broju pogrešno kreiranih atributa, problem dolazi ako se uopće nije vodilo računa o veličini atributa u bazi podataka. Na takav način, baza podataka može biti višestruko veća nego što bi mogla biti ako se vodilo računa o veličini atributa prilikom dizajna baze podataka.

Prilikom kreiranja baze podataka potrebno je obratiti pozornost i na indekse, a pogotovo na bazi podataka koje služe prvenstveno za pohranjivanje nekih određenih događaja. Takve baze podataka uobičajeno imaju jako malen broj indeksa jer indeksi usporavaju upite i

zahtijevaju češće održavanje baze podataka. Osim navedenog, kao što je vidljivo u ranijem primjeru, zauzimaju određeni prostor na tvrdom disku.

Što je veći stupanj normalizacije, u bazi podataka postoji veći broj tablica. Baza podataka sa smanjenjem redundancije postaje manja, ali kako bi upiti bili učinkoviti na većem broju tablica, na strane ključeve je potrebno dodati indekse koji zauzimaju prostor, a što je još veći problem, upiti u vidu ispisa podataka postaju sporiji zbog spajanja. Bazu podataka je potrebno normalizirati, ali do granice iskoristivosti aplikativne podrške i baze podataka. Treća normalna forma smatra se dovoljno dobro normaliziranom bazom podataka i većina dobro kreiranih aplikacija koristi bazu podataka normaliziranu između druge i treće normalne forme.

36.2. OPTIMIZACIJA NA RAZINI UPITA

Kako bi baza podataka bila što optimiziranija potrebno je svakako optimizirati upite. To se pogotovo odnosi na baze podataka koje služe za pohranu podataka klasične poslovne aplikacije koja ima podjednak omjer unosa, izmjena i ispisa podataka. Upiti se najčešće šalju iz aplikacije. Upite je moguće podijeliti u nekoliko grupa s obzirom na način na koji se šalju DBMS sustavu:

- Generatori upita (Hibernate, Entity framework) – nemoguće je u potpunosti kontrolirati upite koji dolaze na bazu
- Pozivanje pohranjenih procedura iz aplikacije (kompilirani upiti na bazi)
- Upiti su pohranjeni u aplikaciji i kao takvi se šalju bazi podataka
- Upiti se ručno pišu u alatu za rad s bazom (npr. SQLyog, SQL Server management studio)

Najvažnija je optimizacija SELECT naredbe jer ona odnosi najviše vremena za izvršavanje. SELECT naredba pretražuje bazu podataka i ona je najveći prioritet kod optimizacije. Prilikom optimizacije SELECT naredbe, potrebno je voditi računa da se performanse INSERT, UPDATE i DELETE naredbi ne smanje do razine neprihvatljivosti.

Prvo je potrebno ustanoviti je li pomoću indeksa moguće ubrzati upit. Indeksi ubrzavaju evaluaciju, filtriranje i dohvat rezultata. Osim dobrih strana, valja biti svjestan negativnih nuspojava dodavanja indeksa, a to je da povećavaju veličinu baze podataka i smanjuju performanse unosa podataka i promjene ako se izvršavaju nad indeksiranim poljem. Brzina izvođenja SELECT naredbe ovisi o:

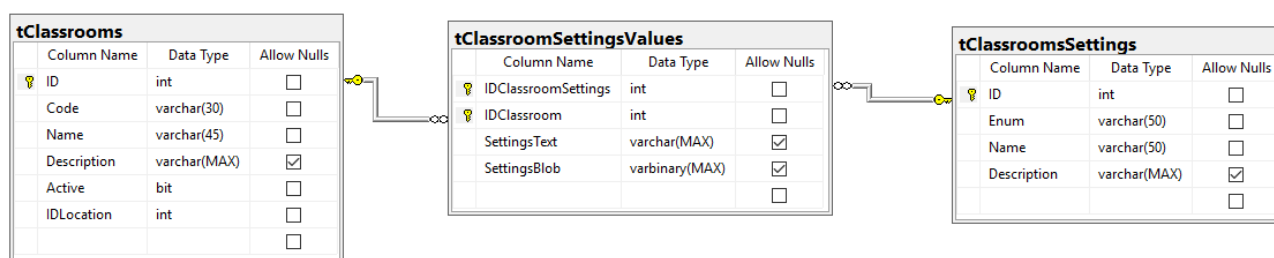
- Indeksu
 - Ubrzava evaluaciju, filtriranje i dohvat rezultata
 - Povećava veličinu baze podataka
 - Smanjuje performanse unosa podataka i promjene ako se izvršavaju nad indeksiranim poljem
- Korištenju funkcija

- Ako su polaganije, pokušati ih optimizirati
- S obzirom na upit, funkcije se zovu jednom ili za svaki redak
- Traženjima po tablici
 - Kreirati indeks kako bi se uklonila pretraga po cijeloj tablici

36.2.1. Korištenje funkcija i podupita

Unutar SELECT klauzule i WHERE klauzule potrebno je smanjiti broj funkcija i podupita koji se izvršavaju, a pogotovo ako koriste kursore kako bi dohvatili neke određene podatke (kursori će biti objašnjeni dalje u knjizi). S obzirom na upit, funkcije i podupiti će se pozivati jednom na razini cijelog upita ili za svaki redak posebno. Ako je nužno pozivanje funkcija ili podupita, iste je potrebno pokušati optimizirati kako bi se izvodili što brže.

Kao primjer, kreirane su tri tablice na SQL Server DBMS-u. Prva tablica prikazivati će entitet učionice na koju su vezana druga dva entiteta koja označavaju postavke i vrijednosti postavki. To je česta pojava kod kreiranja baza podataka za neke određene postavke ili vrijednosti za koje je poznato da će biti pohranjene u bazi podataka, ali još uvijek nije poznato koje će biti ni kako će se zvati. Shema baze izgleda kao na slici u nastavku.



Često je potrebno sve navedene postavke prikazati korisniku na jednom ispisu odvojene zarezom. Neke baze podataka posjeduju konkatenciju stringova uz grupiranje (npr. MySQL), dok neke istu ne posjeduju (npr. SQL Server). U slučaju da DBMS ne posjeduje konkatenciju stringova po grupi, potrebno je istu kreirati samostalno pomoću podupita ili funkcije. Funkcije se više preferiraju zbog preglednosti samog upita. U primjeru su korištena dva upita: jedan koji koristi funkciju i drugi koji istu ne koristi. Broj redaka u tablici učionica je 1000000 redaka. Prvi upit izgleda kao u nastavku:

```
select tClassrooms.*
from tClassrooms
```

Drugi upit izgleda kao u nastavku:

```
select tClassrooms.*, dbo.ClassroomsSettings(tClassrooms.ID) as
Postavke
from tClassrooms
```

Funkciju „ClassroomsSettings“ moguće je napisati na dva različita načina kako bi izbacila isti rezultat. Prvi način je pomoću kursora, a drugi je pomoću podupita.

Funkcija koja koristi kursore:

```
ALTER FUNCTION [dbo].[ClassroomsSettings] (@ClassroomId int)
    RETURNS varchar(MAX)
AS
BEGIN

declare @ret varchar(MAX) = '';
declare @name varchar(50);

DECLARE cur CURSOR FOR
select Name + ': ' + SettingsText
from tClassroomsSettings inner join
    tClassroomSettingsValues on
tClassroomSettingsValues.IDClassroomSettings =
tClassroomsSettings.ID
where tClassroomSettingsValues.IDClassroom = @ClassroomId

open cur;

FETCH NEXT FROM cur
into @name;
WHILE @@FETCH_STATUS = 0
BEGIN
SET @ret = @name + ', ' + @ret
FETCH NEXT FROM cur
    INTO @name;
END
CLOSE cur;
DEALLOCATE cur;

return SUBSTRING(@ret, 0, LEN(@ret))
END
```

Funkcija koja koristi podupite:

```
CREATE FUNCTION [dbo].[ClassroomsSettings] (@ClassroomId int)
    RETURNS varchar(MAX)
AS
BEGIN

declare @ret varchar(MAX) = '';

select @ret += Name + ': ' + SettingsText + ', '
from tClassroomsSettings inner join
    tClassroomSettingsValues on
tClassroomSettingsValues.IDClassroomSettings =
tClassroomsSettings.ID
where tClassroomSettingsValues.IDClassroom = @ClassroomId

return SUBSTRING(@ret, 0, LEN(@ret))
```

END

Rezultati izvođenja upita na 1000000 redaka unutar tablice učionica koristeći prvi upit i drugi upit su sljedeći:

- Klasičan SELECT upit izvodi se 2300ms
- Upit koji koristi funkciju s kursorima izvodi se 45000ms
- Upit koji koristi funkciju bez kursora izvodi se 28000ms

Pomoću ovog jednostavnog testa moguće je zaključiti da su kursori izuzetno „skupi“ za izvođenje, a podupiti također. Idealno bi bilo dizajnirati bazu podataka koja ne mora koristiti ni jednu od navedene tehnologije, ali to vrlo često nije moguće. Jedini način da razvojni inženjer ima apsolutnu kontrolu nad upitima koji se izvode jest da ih piše samostalno u obliku procedura ili upita koji se šalju na bazu podataka. Problem kod Entity framework, Hibernate i sličnih tehnologija jest da razvojni inženjeri nemaju potpunu kontrolu nad upitima koji dolaze i izvršavaju se na bazi podataka i često nisu dovoljno optimizirani zbog čega dolazi do loših performansa baze podataka koje su vrlo teško rješive.

36.2.2. JOIN klauzula

Prilikom spajanja tablica potrebno je izbjegavati nepotrebna spajanja, odnosno ukloniti višak spojenih tablica koje se neće koristiti kako bi se upit ubrzao. Brzina spajanja tj. izvođenje JOIN klauzule direktno ovisi o indeksima koji su kreirani. Svaki DBMS ima određene algoritme pomoću kojih će izvesti spajanje pojedine tablice. Odabir algoritma uvelike ovisi o veličini tablice i činjenici postoji li indeks nad poljem po kojem se tablica pretražuje. Algoritmi koji se koriste za spajanje tablica su:

- Nested loops – ako je ulaz manji (manje od 10 redaka), a druga tablica velika i indeksirana
- Merge join – ako su ulazi već sortirani po join stupcu i otprilike jednakih veličina po broju redaka
- Hash join – slične performanse kao i kod merge joina, bolji je kod ne indeksiranih stupaca i kod spajanja dvije tablice koje se uvelike razlikuju po broju redaka

Bolji DBMS sustavi posjeduju plan izvođenja upita unutar kojeg je točno moguće vidjeti koji algoritam DBMS koristi prilikom izvođenja nekog pojedinog upita.

36.2.2.1. Nested loop join algoritam

Nested loop join algoritam je najjednostavniji algoritam za spajanje i radi na principu dvije for petlje koje provjeravaju je li uvjet spajanja zadovoljen. Pseudo kod algoritma sljedi u nastavku:

```
For each tuple r in R do
    For each tuple s in S do
        If r and s satisfy the join condition
            Then output the tuple <r,s>
```

Osim ovog najjednostavnijeg algoritma, postoji i block nested loop algoritam koji koristi dodatnu memoriju kao predmemoriju (engl. *Buffer*) kako bi umanjio broj traženja po unutarnjoj petlji.

36.2.2.2. Hash join algoritam

Hash JOIN algoritam prvo kreira hash tablicu manje tablice do koje je brže moguće doći do redaka jer je dohvatljiva pomoću hash-a. Nakon što je kreirana hash tablica potrebno je pretražiti veliku tablicu i doći do relevantnih redaka iz manje tablice koja se nalazi u hash tablici u memoriji. Algoritam je poprilično jednostavan, ali zahtjeva da manja tablica stane u radnu memoriju što je koji puta nemoguće. Pseudo kod pojednostavljenog prikaza algoritma nalazi se u nastavku:

```
Za svaki redak r u ulazu R
    Dodaj r u hash tablicu
    Ako je veličina hash tablice jednaka maksimalnoj veličini u
    memoriji
        Pretraži tablicu S i dodaj odgovarajući redak u izlaz
        Resetiraj hash tablicu
Napravi posljednju pretragu tablice S i dodaj spojene retke u
izlaz
```

36.2.2.3. Merge join algoritam

Merge JOIN algoritam prvo sortira retke u obje tablice po atributu spajanja kako bi prilikom operacije bili na istom mjestu. Merge join moguće je upotrijebiti samo na tablicama približno iste veličine. Pseudo kod pojednostavljenog prikaza algoritma nalazi se u nastavku:

```
var relation output
    var list left_sorted := sort(left, a) //Sortiraj
    var list right_sorted := sort(right, a)
    var attribute left_key, right_key
    var set left_subset, right_subset
    advance(left_subset, left_sorted, left_key, a)
    advance(right_subset, right_sorted, right_key, a)
    while not empty(left_subset) and not empty(right_subset)
        if left_key = right_key // Join zadovoljio
            Dodaj kartezijski produkt lijeve i desne u izlaz
            advance(left_subset, left_sorted, left_key, a) //
dodaje poziciju ++
```

```

        advance(right_subset, right_sorted, right_key, a)
    else if left_key < right_key
        advance(left_subset, left_sorted, left_key, a)
    else
        advance(right_subset, right_sorted, right_key, a)
return output

```

36.2.3. Plan izvođenja upita

Plan izvođenja upita grafički prikazuje kako se točno koji upit izvršava i koji se algoritam koristi u kojoj situaciji. Koji će algoritam spajanja biti odabran od strane DBMS-a prvenstveno ovisi o situaciji, dizajnu baze podataka, dostupnim indeksima i veličini same tablice u vidu broja redaka u pojedinoj tablici. Naravno, određene algoritme moguće je poprilično jednostavno isprovocirati uklanjanjem i dodavanjem indeksa i statistika na određene atribute i tablice u bazi podataka. Koji će se algoritam upotrijebiti moguće je vidjeti u SQL Server execution planu kojeg je moguće prikazati prilikom pozivanja upita.

Kao primjer različitih planova izvršavanja upita, korištene su dvije identične baze podataka studenata preseljene na SQL Server. Jedna baza podataka posjeduje indekse, dok druga baza podataka iste ne posjeduje. Na te dvije baze podataka moguće je vidjeti različite situacije i planove izvođenja identičnog upita.

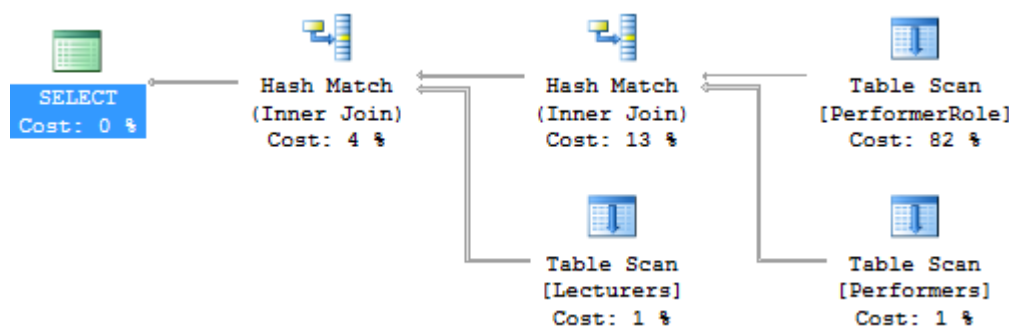
Prvi upit koji će se izvršiti nad obje baze podataka slijedi u nastavku:

```

SELECT      Lecturers.*
FROM        PerformerRole INNER JOIN
            Performers ON PerformerRole.Id =
Performers.Id PerformerRole INNER JOIN
            Lecturers ON Performers.IdLecturer = Lecturers.Id
where PerformerRole.Name = 'asistent'

```

U slučaju baze podataka koja ne posjeduje indekse, plan izvođenja izgleda kao na slici u nastavku.

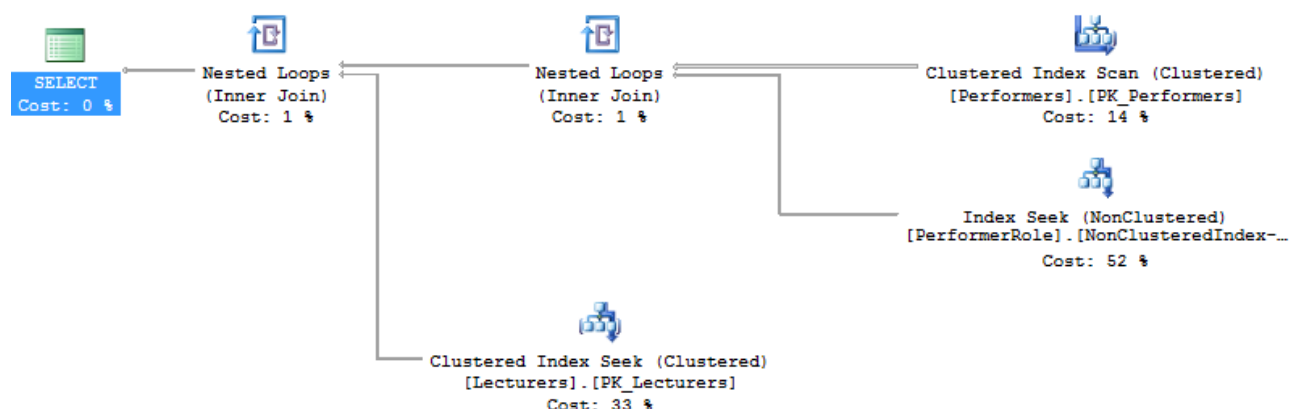


Slika 74. Plan izvođenja upita bez korištenja indeksa

Na planu izvođenja moguće je vidjeti da DBMS mora slijedno pretraživati sve tablice (engl. *Table scan*) zbog nedostatka indeksa, nakon čega kreira hash tablicu unutar radne memorije

i spaja navedene tablice. Sve navedene radnje su izuzetno skupe i zahtijevaju mnogo vremena, radne memorije i ostalih resursa računala.

U slučaju baze podataka koja posjeduje indekse, plan izvođenja izgleda kao na slici u nastavku.



Slika 75. Plan izvođenja upita uz korištenje indeksa

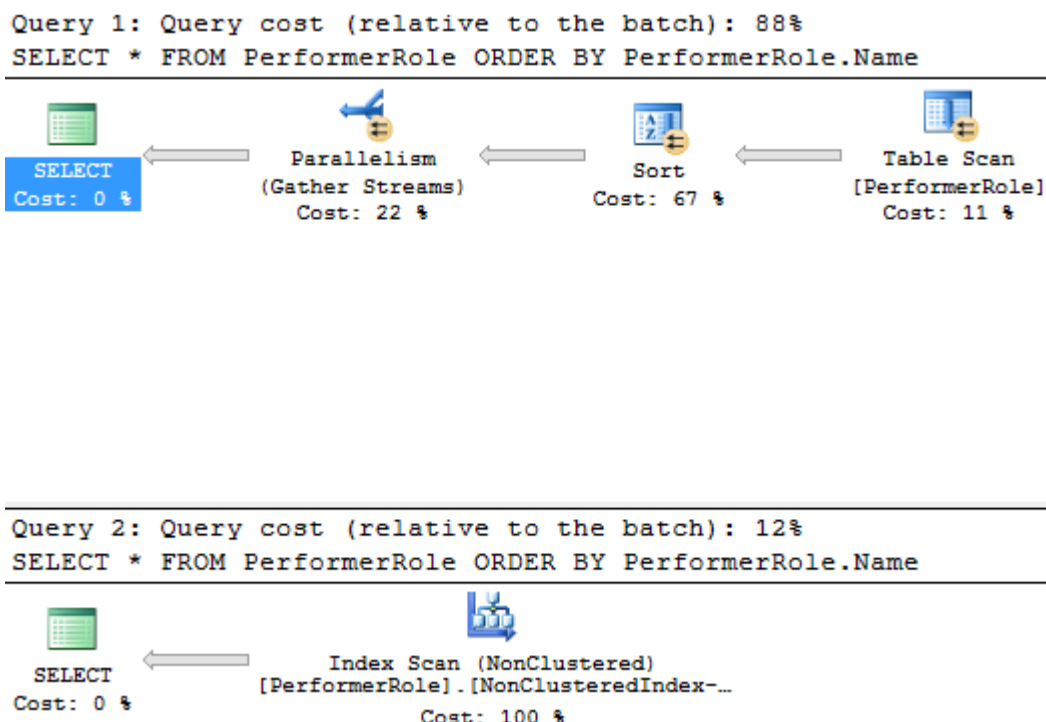
Na potonjem planu izvođenja moguće je vidjeti da DBMS sve tablice pretražuje koristeći indekse i da se spajanje tablica obavlja pomoću mnogo jednostavnijeg i resursno manje zahtjevnog algoritama petlje unutar petlje, odnosno nested loop join algoritma.

Prilikom izvođenja upita na obje baze, prvi upit traje znatno dulje i cijena izvođenja navedenog upita je 95%, što znači da je cijena upita s indeksima svega 5% od ukupne cijene.

Drugi upit koji će prikazati razliku prilikom sortiranja podataka i biti će uspoređen ponovo na obje baze podataka nalazi se u nastavku:

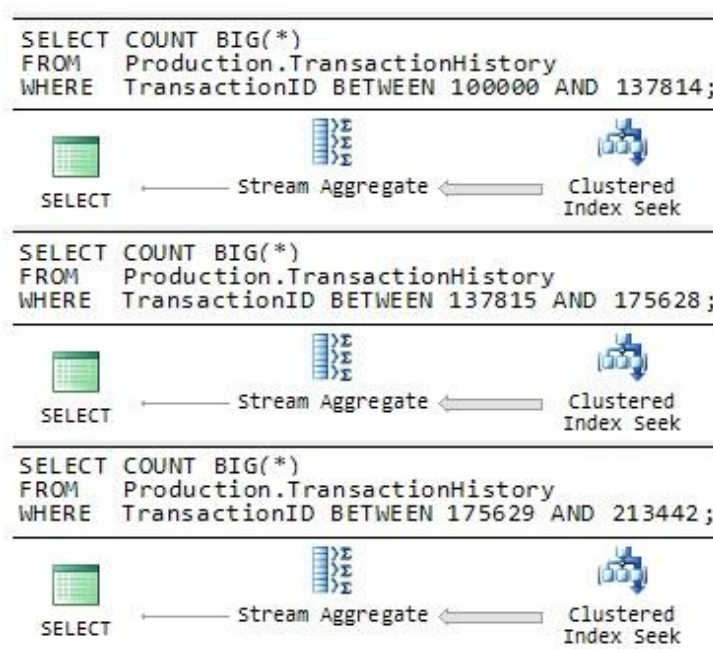
```
SELECT *
FROM PerformerRole
ORDER BY PerformerRole.Name
```

Planovi izvođenja oba upita nalaze se u nastavku.



Slika 76. Planovi izvođenja upita sa sortiranjem

Moguće je primjetiti da upit koji se izvodi na bazi podataka bez indeksa košta 88% od ukupne cijene oba upita. Moguće je primjetiti da kako bi se sortirali rezultati DBMS mora pretražiti sve retke i napraviti pretragu cijele tablice. Navedene radnje se izvode na više jezgara računala i zato postoji prikupljanje podataka prije izlaza. Paralelizam znači zapravo da svaka jezgra dobije svoju zadaću. Upit se podjeli na podjednak broj redaka koji se potom obrađuju zasebno u svakoj jezgri procesora.



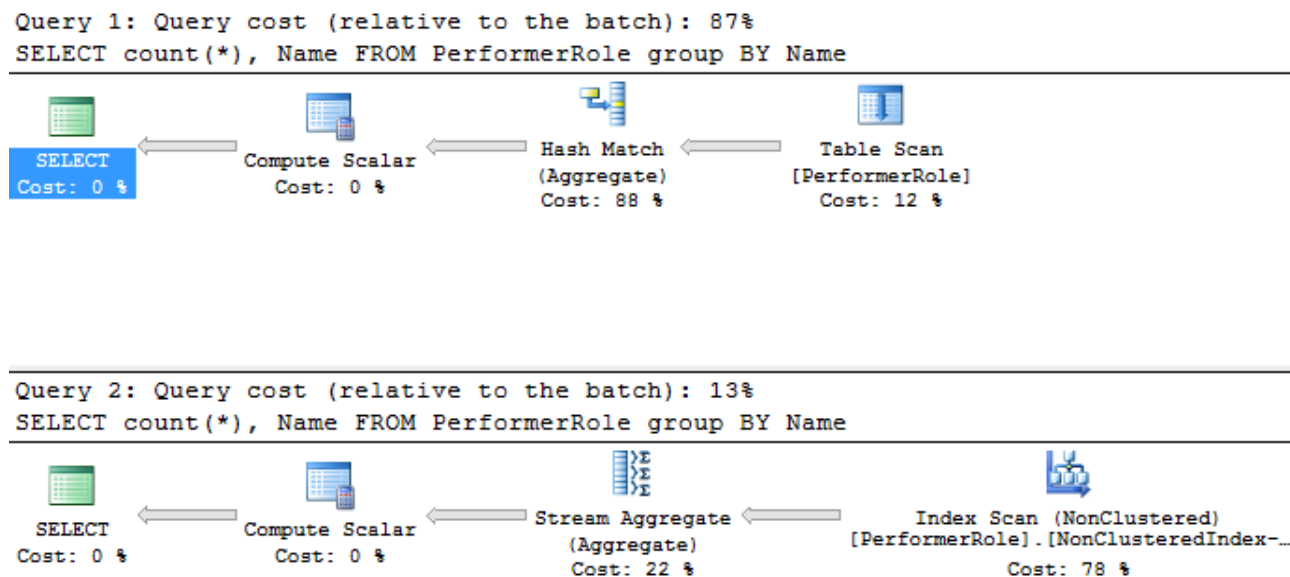
Slika 77. Paralelizam

Upit na bazi podataka s indeksima se izvodi na izuzetno jednostavan način jer se sortiranje vrši pomoću indeksa koji je unaprijed sortiran i nije potrebno obavljati dodatne radnje.

Treći upit koji će prikazati razliku prilikom grupiranja podataka i biti će uspoređen ponovo na obje baze podataka nalazi se u nastavku:

```
SELECT count(*), Name
FROM PerformerRole
GROUP BY Name
```

Planovi izvođenja oba upita nalaze se u nastavku:



Slika 78. Planovi izvođenja upita s grupiranjem

Slično kao i u upitima do sada, indeksi ubrzavaju rad upita na način da se pretraga vrši pomoću indeksa, a ne slijednim traženjem podataka po tablici.

37. NoSQL

Iako je relacijski model baza podataka godinama bio dominantno (ako ne i jedino) rješenje za pohranu podataka, u novije vrijeme se pojavljuju nove mogućnosti u vidu baza podataka koje bi ih mogle zamijeniti u određenim situacijama. Nemoguće je očekivati da će relacijske baze podataka uskoro nestati, ali se često traže rješenja u alternativnim opcijama kao što je NoSQL. NoSQL je također poznat kao i „ne samo SQL“ i „ne relacijska baza“. NoSQL nije baza podataka i nije čak ni tip baze podataka. Naziv NoSQL obuhvaća bilo koji katalog podataka koji ne prati klasičan relacijski model sustava za upravljanje bazama podataka (engl. *RDBMS* - *Relational Database Management System*). Točnije, podaci nisu relacijski i ne koristi se SQL jezik za kreiranje upita. On obuhvaća nove generacije baza podataka koje spadaju pod jednu od idućih kategorija: nisu relacijske, distribuirane, open-source i vodoravno su skalabilne.

U idućim poglavljima će biti pokriven princip rada NoSQL baza podataka, klasifikacija po modelu podataka i općeniti način rada najčešćih tipova NoSQL baza podataka.

37.1. CAP TEOREM

Za razliku od tradicionalnih RDBMS sustava koji se oslanjaju na ACID transakcije, NoSQL se oslanja na CAP teorem znanstvenika Erica Brewera i BASE transakcije.

CAP teorem se pojavio prvi puta 1998. godine, a 2000. godine ga je znanstvenik Eric Brewer predstavio dok je držao govor kao glavni govornik na simpoziju o načelima distribuiranih sustava (engl. *Symposium on Principles of Distributed Computing*). Teorem nalaže da u svakom distribuiranom sustavu za pohranu podataka nije moguće istodobno osigurati dosljednost, dostupnost i toleranciju na particioniranje. Smatra se da je moguće osigurati maksimalno dvije od ove tri stavke. Svrha ovog teorema je bila otvoriti umove dizajnerima baza podataka i opravdati potrebu za daljnjim istraživanjima kako bi se razvio što veći raspon različitih skladišta podataka.

CAP predstavlja skraćenicu za :

- Konzistentnost (engl. *Consistency*) predstavlja jednaku ideju kao i u ACID transakcijama.
- Dostupnost (engl. *Availability*) označava dostupnost servisa ili sustava kada god je on tražen.
- Tolerancija na particioniranje (engl. *Partition Tolerance*) ili robusnost označava mogućnost sustava da nastavi s radom čak i kod gubitka podataka ili pada sustava (engl. *System failure*). Dodatnu definiciju tolerancije na particioniranje donose Gilbert i Lynch: „Nijedan skup kvarova manji od pada cijelog mrežnog sustava ne smije dopustiti da sustav reagira na pogrešan način.“



Slika 79. CAP

Unatoč tome što su u praksi latencija i particioniranje usko povezani, CAP zanemaruje latenciju. Sama esencija CAP-a se odvija tijekom isteka vremena (engl. *Time-out*). To je period unutar kojega program mora donijeti partijsku odluku: hoće li će poništiti operaciju i smanjiti dostupnost, ili će nastaviti s radom i riskirati nekonzistentnost u radu. Jedan od rješenja za ovaj problem je ponovni pokušaj slanja poruke čime se zapravo bira između jednog od iduća dva bita CAP-a: konzistentnost i dostupnost.

Eric Brewer je 12 godina nakon izlaganja svoje ideje o CAP teoremu izdao članak gdje navodi kako se aspekti ovog teorema često pogrešno shvaćaju, a naročito opseg dostupnosti i konzistentnosti što dovodi do nepoželjnih rezultata. Primjer pogrešnog shvaćanja je kada korisnici uopće ne mogu doći do usluge (nema ni C ni A) pri čemu se dio usluge pokreće na klijentskoj strani. Ova metoda rada, izvanmrežni način, postaje sve važnija pa tako neke HTML 5 značajke omogućuju lakši rad u ovakvim slučajevima.

37.1.1. BASE

ACID i BASE predstavljaju dvije filozofije na suprotnim krajevima spektra raspoloživosti i konzistencije. Osobine ACID-a se usredotočuju na konzistentnost i tradicionalni pristup baza podataka. Eric Brewer je s kolegama stvorio BASE krajem 1990-ih kako bi zaokružio nove dizajnerske pristupe za visoku raspoloživost i novi pristup prema razvoju skladišta podataka. Suvremeni veliki širokopojasni (engl. *Large-scale wide-area*) sistemi, uključujući oblak, koriste mješavinu oba pristupa.

BASE je akronim za:

- Osnovna dostupnost (engl. *Basic Availability*) – Svaki zahtjev mora dobiti odgovor, bez obzira bio on uspješan ili ne.
- Soft-state (engl. *Soft-state*) – Stanje sustava se može mijenjati s vremenom čak i u vremenskom periodu kada nije primio nikakve ulazne podatke.
- Eventualna dosljednost (engl. *Eventual consistency*) – Sustav će eventualno postati dosljedan kada prestane primati ulazne podatke.

37.1.2. ACID ili BASE

Kako je Brewer naveo u svojem predavanju, glavne značajke ACID i BASE principa su:

Tablica 1. Usporedba ACID i BASE principa

ACID	BASE
Jaka dosljednost	Slaba dosljednost (neodređeni podaci su u redu)
Izolacija	Dostupnost na prvom mjestu
Usredotočenost na „commit“	Najbolji napor
Ugniježdene transakcije	Približni odgovori su OK
Dostupnost	Agresivne
Konzervativnost	Jednostavnost
Teška evolucija (na primjer Schema)	Brzina
	Jednostavnija evolucija

37.2. KLASIFIKACIJA NOSQL BAZA PODATAKA

Najčešće se NoSQL baze podataka razlikuju od onih relacijskih s obzirom na njihov model podataka. S obzirom na razlike u modelima, NoSQL baze se može podijeliti na iduće oblike:

37.2.1. Stupac baze podataka

Za razliku od relacijskih baza podataka gdje se stupac koristi kao dio tablice koji je vidljiv u svakom retku iste, u distribuiranim bazama podataka to nije slučaj. U ovom tipu baza podataka, pojam tablice ne postoji. Svaki pojedini stupac se koristi kao spremište za podatke. Sastoji se od jedinstvenog imena, vrijednosti i vremenske oznake.

Tablica 36. Stupac baza podataka

Stupac baze podataka

Ime	Vrijednost	Vremenska oznaka
-----	------------	------------------

Za primjer se može uzeti tablica "mjesta" iz baze "studenti". U klasičnoj relacijskoj bazi podataka pohrana podataka izgleda na idući način:

postbr	nazivMjesto	idZupanija
10000	Zagreb	1
10290	Zaprešić	1
10295	Kupljenovo	2
10310	Ivanić-Grad	2
10315	Novoselec	2
10340	Vrbovec	2
10345	Gradec	2
10410	Velika Gorica	2
10430	Samobor	2
20000	Dubrovnik	20
21000	Split	18

Odnosno, podaci će biti formirani u tablici na idući način:

```
10000, Zagreb, 1
10290, Zaprešić, 1
10295, Kupljenovo, 2
10310, Ivanić-Grad, 2
10315, Novoselec, 2
```

Pojednostavljeno, u ovom tipu NoSQL baze podataka ti isti podaci će biti pohranjeni u stupcima na sljedeći način:

```
(postbr) 10000, 10290, 10295, 10310, 10315
(nazivMjesto) Zagreb, Zaprešić, Kupljenovo, Ivanić-Grad, Novoselec
(idZupanija) 1, 1, 2, 2, 2
```

U NoSQL stupac bazama podataka podaci nisu vezani jedan s drugim pomoću stranog ključa, već se oni uzastopce upisuju u retke, a ovisno o mjestu koji zauzimaju, s tim su podacima vezani u sljedećem retku. Na primjer, ako su potrebni podaci za grad Zaprešić, moraju se gledati vrijednosti unesene na drugom mjestu u svakom retku.

Ovakav način pisanja podataka je „jeftiniji“ i čini verzioniranje podataka jednostavnije, što ih čini pogodnim za rad sa „Big Data“ problemima te sa sadržajem web stranica.

Za razliku od većine NoSQL baza podataka, ovaj tip ima shemu tako da je svakako potrebno unaprijed znati kako će izgledati struktura podataka koja se planira koristiti u sustavu. Ako nije moguće unaprijed definirati strukturu podataka, ovakav tip NoSQL baze podataka nije pogodan za korištenje.

Primjeri: Accumulo, Cassandra, SAP HANA...

37.2.2. Dokument baze podataka

Dokument baze podataka su posebno dizajnirane za pohranu, preuzimanje i upravljanje informacijama koje su pohranjene u dokumentima. One su takozvana skladišta dokumenata. Podaci pohranjeni u ovakvim bazama se također nazivaju i polustrukturirani podaci. Često su pohranjivani u obliku XML (EXtensible Markup Language), JSON (Javascript Object Notation) ili BSON (Binary JSON) datoteka.

U nastavku slijedi primjer napisan kao XML dokument:

```
<ustanova>
  <oib>08814003451</oib>
  <naziv>Tehničko Veleučilište u Zagrebu</naziv>
  <zRacun>2360000-1101350801</zRacun>
  <adresa>
    <ulica>Vrbik 8</ulica>
    <postbr>10000</postbr>
    <nazivMjesto>Zagreb</nazivMjesto>
    <nazivZupanija>Grad Zagreb</nazivZupanija>
  </adresa>
  <datumOsnutka>1998-05-27</datumOsnutka>
</ustanova>
```

U nastavku slijedi primjer napisan kao JSON dokument:

```
{
  "ustanova": {
    "oib": "08814003451",
    "naziv": "Tehničko Veleučilište u Zagrebu",
    "zRacun": "2360000-1101350801",
    "adresa": {
      "ulica": "Vrbik 8",
      "postbr": "10000",
      "nazivMjesto": "Zagreb",
      "nazivZupanija": "Grad Zagreb"
    },
    "datumOsnutka": "1998-05-27"
  }
}
```

Za razliku od JSON i XML dokumenata, BSON dokumenti najčešće nisu čitljivi ljudima. Oni su binarno kodirane serijalizacije dokumenata sličnih JSON dokumentima.

S obzirom na to da dokumenti nisu međusobno povezani kao što bi bile tablice unutar relacijske baze podataka, oni mogu biti pohranjeni na više različitih poslužitelja što omogućuje distribuirane implementacije. Svaki dokument sadrži svoj jedinstveni identifikacijski ključ (ID) koji ga eksplicitno identificira. Zbog manjka relacija, velika mana ovakvih baza podataka je činjenica da svaki dokument mora sadržavati sve relevantne podatke potrebne za njegovu upotrebu. Za razliku od relacijskih baza podataka gdje je poželjna normalizacija, u dokument bazama podataka je uobičajeno da se podaci ponavljaju kroz više dokumenata.

Ovakve baze podataka su pogodne za velike distribuirane sustave i varijabilne domene. Kada se unaprijed ne može točno definirati shema podataka, tada dokument baze predstavljaju prikladan izbor. Upravo iz ovog razloga, moguće je na jednostavan način mapirati dokumente na modele u objektno orijentiranom načinu programiranja. Mapiranje objekata može znatno ubrzati proces razvoja sustava, te umanjiti greške prilikom dohvaćanja podataka iz baze u model i obrnuto.

Primjeri: Apache CouchDB, IBM Domino, MongoDB...

37.2.3. Ključ-vrijednost baze podataka

Relacijske baze podataka unaprijed definiraju strukturu podataka u bazi kao skup tablica koje sadrže polja s vrstama podataka. Suprotno relacijskom pristupu, ključ-vrijednost baze podataka ne definiraju koji će tip podataka biti pohranjen, pa tako svaki unos u bazu može biti različit. Ovakva struktura podataka se u programiranju još naziva i rječnik ili hash. Rječnici sadrže zbirke objekata ili zapisa, koji pak sadrže mnogo različitih polja unutar sebe, od kojih svaki sadrži podatke. Ovakvi zapisi se pohranjuju i dohvaćaju pomoću ključa koji jedinstveno identificira svaki zapis. To znači da nije moguće pretraživati pomoću vrijednosti, već samo pomoću ključa.

Tablica 37. Ključ-vrijednost baza podataka

Ključ - vrijednost	
Ključ	Vrijednost

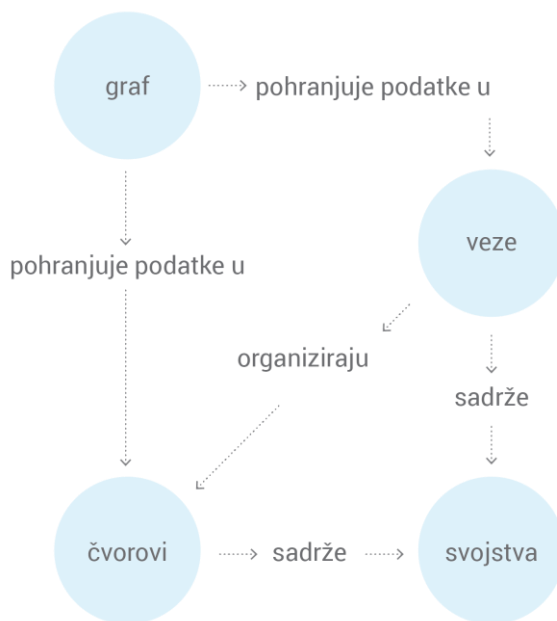
Iako se nije moguće pretraživati pomoću vrijednosti, tip vrijednosti se može definirati. U ključ-vrijednost bazama podataka, mogu se razlikovati vrijednosti kao što su string, lista, set i slično.

Zapisi su međusobno neovisni i stoga se njihovim odnosima mora upravljati putem aplikacijske logike. Ovakve baze podataka nemaju shemu zbog njihove jednostavnosti i najčešće služe za operacije koje se temelje samo na ključnim atributima. Primjerice, mogla bi koristiti za pohranjivanje sesija korisnika na web stranicama. Zbog svoje jednostavnosti i brzine, na ovaj način se može ubrzati rad same stranice.

Primjeri: ArangoDB, InfinityDB, Oracle NoSQL Database, Riak...

37.2.4. Graf baze podataka

Kod relacijskih baza podataka relacije među podacima se pohranjuju unutar relacijske sheme baze podataka, te se kod njih do podataka i povezanih podataka dolazi pomoću upita i JOIN klauzule. Graf baze podataka su dizajnirane kako bi omogućivale brz pristup kompleksnim hijerarhijskim strukturama. One koriste strukture grafova za semantičke upite s čvorovima (engl. *Nodes*), rubovima (engl. *Edges*) i svojstvima za prikaz i pohranu podataka. Ključni koncept je grafikon koji izravno povezuje podatke u bazi podataka.



Slika 80. Dijagram graf baze podataka

Graf baze podataka su pogodne za rad s mrežnim aplikacijama (engl. Networking applications). Često se koriste za rad društvenih mreža kada je potrebno povezivati korisnike s obzirom na neke njihove preferencije, osobine, poznanstva i slično.

Upravo zbog svoje međusobne povezanosti između čvorova, graf baze podataka nisu najpogodnije za sustave kojima je potrebno mrežno particioniranje.

Primjeri: ArangoDB, Apache Giraph, Neo4J...

37.2.5. Multi-model baze podataka

Dok je većina sustava za upravljanje bazom podataka organizirana oko jednog modela upravljanja, postoje i multi-model baze podataka koje koriste njih više. Takve baze podataka mogu podržavati višestruke podatkovne modele, pa tako mogu kombinirati sve gore navedene modele podataka. Ovakve baze podataka daju veću fleksibilnost sustavima u kojima se koriste.

Primjeri: ArangoDB, Couchbase, InfinityDB...

37.3. ODABIR POGODNE BAZE PODATAKA

NoSQL baze podataka mogu biti brže, imati veću dostupnost i veću toleranciju na particioniranje od klasičnih relacijskih baza podataka. Međutim, to ne znači da su one ujedno i bolji izbor za određeno rješenje. Prilikom odabira baze podataka koja će biti korištena u projektu bitno je unaprijed dobro procijeniti koje su potrebe samog sustava i u početku odabrati najbolje rješenje. Odabir baze podataka iziskuje pomno planiranje i analizu potreba sustava. Sve činjenice, od potreba za dostupnošću, izgleda podataka pohranjenih na poslužitelju, brzine izvršavanja upita, pa čak i mogućnost povrata podataka u slučaju kvara moraju biti uzete u obzir. Kada se svi zahtjevi uzmu u obzir, potrebno je pomno odvagati koje rješenje bi mogli biti najoptimalnije za određeni problem odnosno sustav. Iako je u nekim slučajevima bolje koristiti NoSQL baze podataka, i dalje se često odabiru klasične baze podataka upravo zbog njihovog ACID svojstava i općenite robusnosti.

38. Literatura

1. Atzeni, P., and De Antonellis, V., Relational Database Theory, Benjamin-Cummings, Menlo Park, CA, 1993.
2. Balling, D. J. ; Zawodny, J.: High Performance MySQL, O'Reilly, 2004.
3. Baranović, M.: Baze podataka, materijali za predavanja, Sveučilište u Zagrebu, Fakultet elektrotehnike i računarstva, 2008.
4. Baranović, M.; Zakošek, S.: Baze podataka, materijali za predavanja, Sveučilište u Zagrebu, Fakultet elektrotehnike i računarstva, 2007.
5. Brewer, E.; "CAP Twelve Years Later: How the "Rules" Have Changed"; Computer (Volume: 45, Issue: 2, Feb. 2012); URL: <http://ieeexplore.ieee.org/document/6133253/>
6. Cabral, S.; Murphy, K.: MySQL Administrator's Bible, Wiley Publishing, Inc., Indianapolis, Indiana, 2009.
7. Captain, F.A.: Six-Step Relational Database Design™: A step by step approach to relational database design and development Second Edition, CreateSpace Independent Publishing Platform, 2013.
8. Celko, J.; "Joe Celko's Complete Guide to NoSQL"; Morgan Kaufmann; 2014.
9. Chamberlin, D.D.; Astrahan, M.M.; Eswaran, P.P.; Lorie, R.A.; Mehl, J.W.; Reisner, P.; and Wade, B.W.: SEQUEL 2: A Unified Approach to Data Definition, Manipulation and Control, IBM Journal of Research and Development, 20:6, 97–137, 1976.
10. Chamberlin, D.D.; Boyce, R.F.: SEQUEL: A Structured English Query Language, Proceedings of ACM Sigmoid Workshop, 1, 249–264, 1974.
11. Chamberlin, D.D.; Boyce, R.F.: SEQUEL: A Structured English Query Language, Proceedings of ACM Sigmoid Workshop, 1, 249–264, 1974.
12. Chen, P.P.: The Entity-Relationship Model: Toward a Unified View of Data, ACM Transactions on Database Systems, 1:1, 9–36, 1976.
13. Codd, E.F.: A Relational Model of Data for Large Shared Data Banks (Reprint). Commun. ACM, 26(1), 1983: 64–69.
14. Codd, E.F.: A Relational Model of Data for Large Shared Data Banks. Commun. ACM, 13(6), 1970: 377–387.
15. Codd, E.F.: Extending the Data Base Relational Model to Capture More Meaning. SIGMOD Conference, 1979, p. 161.
16. Codd, E.F.: Recent Investigations into Relational Data Base Systems, Proc. IFIP Congress, Stockholm, Sweden, 1974.
17. Codd, E.F.: The Relational Model for Database Management, 2nd ed. Reading, MA: Addison-Wesley, 1990.
18. Connolly, T., Connolly, T. M.; Begg, C. E.: Database Systems, Addison-Wesley, 2014.

19. Coronel, C.; Morris, S.: Database Systems: Design, Implementation, & Management (13th edition), Course Technology, 2018.
20. Darwen, H.: An Introduction to Relational Database Theory, Ventus Publishing ApS, 2010.
21. Date, C.J., and Darwen, H., A Guide to the SQL Standard, 3rd edition, Addison-Wesley, Reading, MA, 1993.
22. Date, C.J.: An Introduction to Database Systems, 8th Edition, Addison Wesley, 2004.
23. Date, C.J.: SQL and Relational Theory: How to Write Accurate SQL Code (2nd edition), O'Reilly, 2012.
24. DuBois, P.: MySQL Cookbook, Solutions for Database Developers and Administrators, treće izdanje, O'Reilly, 2014.
25. Dyer, R.: MySQL in a Nutshell, O'Reilly, 2008.
26. Feuerstein, S.; Harrison, G.: MySQL Stored Procedure Programming, O'Reilly, 2006.
27. Fritchey, G.: SQL Server 2012 Query Performance Tuning, Apress, 2012.
28. Harrington, J.L.: Relational Database Design and Implementation (4th Edition), Morgan Kaufmann, 2016.
29. Hecht, R., Jablonski, S.; NoSQL Evaluation, A Use Case Oriented Survey; 2011 International Conference on Cloud and Service Computing
30. Jarger, R. J.; Reese, G.; King, T; Williams, H. E.: Managing and Using MySQL, 2nd Edition, O'Rilley, 2002.
31. Lightstone, S.; Teorey, T.J.; Nadeau, T.: Physical Database Design, Morgan Kaufmann, 2007.
32. Moncur, M.: MySQL Your visual blueprint to open source database management, Wiley Publishing, 2003.
33. MySQL Documentation: MySQL Reference Manuals, URL: <https://dev.mysql.com/doc/>
34. Pachev, S.: Understanding MySQL Internals, O'Reilly, prvo izdanje, 2007.
35. Predavanja iz kolegija „Baze podataka“, Fakultet elektrotehnike i računarstva, Sveučilište u Zagrebu, www.unizg.fer.hr, 2016.
36. Powell, G.: Beginning Database Design, Wrox, 2005.
37. Ramakrishnan, R.; Gehrke, J.: Database Management Systems, treće izdanje, McGraw-Hill, New York, 2003.
38. Reese, G.: MySQL Pocket Reference, drugo izdanje, O'Rilley, Kanada, 2007.
39. Schneider, R.D.: MySQL Database Design and Tuning, Pearson Education, 2005.
40. Schwartz, B.; Zaitsev, P.; Tkachenko, J.; Zawodny, J.D.; Lentz, A.; Balling, D.J.: High Performance MySQL, Second Edition, O'Reilly, 2008.

41. Silberschatz, A.; Korth, H.; Sudarshan, S.: Database System Concepts, McGraw-Hill, 2011.
42. Skočir, Z.; Matasić, I.; Vrdoljak, B.: Organizacija obrade podataka, Fakultet elektrotehnike i računarstva, Sveučilište u Zagrebu, verzija 1.1., 2000.
43. SQL Server documentation; URL: <https://docs.microsoft.com>
44. SQL Server technical documentation; URL: <https://technet.microsoft.com>
45. Strate, J.; Krueger, T.: Expert Performance Indexing for SQL Server 2012, Apress, 2012
46. Sumathi, S.; Esakkirajan, S.: Fundamentals of Relational Database Management Systems, Verlag Berlin Heidelberg, Springer, 2007.
47. Teorey, T., Lightstone, S., Nadeau, T.; Jagadish H.V.: Database Modeling and Design: Logical Design (5th Edition), Morgan Kaufmann, 2011.
48. Tiwari, S.; "Professional NoSQL"; John Wiley & Sons, Inc.; 2011.
49. Ullman, J. D.; Widom, J.: A First Course in Database Systems, Prentice-Hall, 2008.
50. Varga, M.: Upravljanje podacima, Element, 2012.
51. Vaswani, V.: MySQL Database Usage & Administration, McGraw-Hill, 2010.
52. W3resource MySQL tutorials; URL: <https://www.w3resource.com/mysql/mysql-tutorials.php>